



EMPLOYEES' ENGAGEMENT AND LEARNING HUB
ON WEBSITE AND MOBILE APPLICATION

MS. NICHAPA LEESAKUL
MS. SIRAPART TEPWONG
MS. JIRAPHAT RUTTANABUMRUNGSIN

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)
FACULTY OF ENGINEERING
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2025

Employees' Engagement and Learning Hub
on Website and Mobile Application

Ms. Nichapa Leesakul
Ms. Sirapart Tepwong
Ms. Jiraphat Ruttanabumrungsin

A Project Submitted in Partial Fulfillment
of the Requirements for
the Degree of Bachelor of Engineering (Computer Engineering)
Faculty of Engineering
King Mongkut's University of Technology Thonburi
2025

Project Committee



.....
(Piyanit Ua-areemit, Ph.D.)

Project Advisor

.....
(Suthathip Maneewongvatana, Ph.D.)

Committee Member

.....
(Naruemon Wattanapongsakorn, Ph.D.)

Committee Member

.....
(Mr. Naveed Sultan)

Committee Member

Copyright reserved

Project Title	Employees' Engagement and Learning Hub on Website and Mobile Application
Credits	18
Member(s)	Ms. Nichapa Leesakul Ms. Sirapart Tepwong Ms. Jiraphat Ruttanabumrungsin
Project Advisor	Piyanit Ua-areemit, Ph.D.
Program	Bachelor of Engineering
Field of Study	Computer Engineering
Department	Computer Engineering
Faculty	Engineering
Academic Year	2025

Abstract

Human resource management at SCGC's affiliated startup faces challenges due to the lack of integrated platforms for managing employee satisfaction and learning resources. The existing platforms do not meet user experience standards and are missing essential functionalities, making it difficult for HR employees to efficiently access morale data and manage learning initiatives. This situation increases the workload for HR, reducing their ability to provide support. Without these tools, employee engagement may decline, resulting in lower job satisfaction. Developing a user-friendly learning hub would address these challenges by offering personalized development opportunities, improving HR processes, and enhancing employee growth and organizational performance.

The project focuses on two key areas across two platforms: Human Resource Management (HRM) on UniCulture and Human Resource Development (HRD) on UniLearning. UniCulture allows employees to participate in daily and yearly morale surveys, as well as 360-degree assessments, while providing managers and admins with dashboards to track and analyze satisfaction and feedback. Additionally, a machine learning model is integrated to analyze satisfaction from open-ended survey responses.

Meanwhile, UniLearning concentrates on providing employees access to various learning resources, courses, and quizzes. Machine learning is employed to personalize video course recommendations based on individual preferences, enhancing the learning experience. Furthermore, another machine learning model supports career aspiration insights, allowing managers to explore career paths based on skills and interests, facilitating better promotion and development decisions. The system supports four primary roles: Admins, Managers, Employees, and Outsiders.

The results from implementing both platforms include improved efficiency in employee management by the HR department, enhanced employee engagement, more accurate strategic decision-making in human resource management, as well as the development of employee competencies and the ability to measure learning outcomes. By addressing current challenges and adding interactive tools, the platform will improve Ant HR's ability to support employee development and organizational growth.

Keywords: Human Resource Management (HRM) / Human Resource Development (HRD) / Employee Engagement / Learning Hub / Machine Learning / Dashboards

หัวข้อปริญญานิพนธ์	แพลตฟอร์มส่งเสริมการมีส่วนร่วมและการเรียนรู้ของพนักงานบนเว็บไซต์และแอปพลิเคชันมือถือ
หน่วยกิต	18
ผู้เขียน	นางสาวสิริภัทร เทพวงษ์ นางสาวนิชาภา ลีสกุล นางสาวจิรภัทร รัตนบำรุงศิลป์
อาจารย์ที่ปรึกษา	ดร.ปิยนิตย์ เอื้ออารีมิตร
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมคอมพิวเตอร์
ภาควิชา	วิศวกรรมคอมพิวเตอร์
คณะ	วิศวกรรมศาสตร์
ปีการศึกษา	2568

บทคัดย่อ

การบริหารจัดการทรัพยากรบุคคลของบริษัทในเครือบริษัท เอสซีจี เคมิคอลส์ จำกัด (มหาชน) กำลังเผชิญความท้าทายในเรื่องของการขาดแพลตฟอร์มในการจัดการความพึงพอใจและพัฒนาบุคคลของพนักงาน แพลตฟอร์มในปัจจุบันที่มีการใช้งานอยู่นั้นยังไม่สามารถตอบสนองความต้องการและประสบการณ์ของผู้ใช้งานได้ ทำให้พนักงานฝ่ายทรัพยากรต้องใช้เวลามากขึ้นในการจัดการข้อมูล และไม่สามารถให้การสนับสนุนพนักงานได้อย่างเต็มที่ เนื่องจากการขาดข้อมูลของพนักงานในการวิเคราะห์ ส่งผลให้ความพึงพอใจของพนักงานในบริษัทลดลง รวมถึงอาจทำให้อัตราของพนักงานลาออกมีมากขึ้นซึ่งจะส่งผลเสียต่อบริษัท นอกจากนี้แพลตฟอร์มของการพัฒนาบุคคลยังไม่ครอบคลุมถึงการให้บริการ เช่น พนักงานต้องไปหาแหล่งเรียนรู้ด้วยตัวเองซึ่งอาจจะไม่ตรงความประสงค์ของตำแหน่งงาน และไม่สามารถทดสอบประสิทธิภาพในการพัฒนานี้ได้

เพื่อตอบสนองต่อปัญหาเหล่านี้ โครงการนี้จึงพัฒนาแพลตฟอร์มดิจิทัลที่มีอยู่เพื่อให้ครอบคลุมทั้งด้านการบริหารและการพัฒนาทรัพยากรบุคคลผ่านระบบหลัก 2 ส่วน ได้แก่ การจัดการทรัพยากรมนุษย์บนแพลตฟอร์ม UniCulture และ การพัฒนาทรัพยากรมนุษย์บนแพลตฟอร์ม UniLearning โดยที่แพลตฟอร์ม UniCulture จะมุ่งเน้นในเรื่องของการมีส่วนร่วมของพนักงาน โดยการเก็บข้อมูลของพนักงาน ทั้งแบบรายวันและรายปีเพื่อนำมาวิเคราะห์ถึงสุขภาพใจของพนักงาน รวมถึงการประเมินแบบ 360 องศาที่จะสามารถให้คนรอบตัวของพนักงานประเมินตัวพนักงานได้ การมีแดชบอร์ดเพื่อแสดงผลสุขภาพใจของพนักงานและมีการประยุกต์ใช้การเรียนรู้ของเครื่อง (Machine Learning) เพื่อวิเคราะห์ข้อความจากแบบสอบถาม

ขณะที่แพลตฟอร์ม UniLearning จะทำหน้าที่เป็นศูนย์รวมแหล่งเรียนรู้ที่พนักงานสามารถเข้าถึงข้อมูลการเรียนรู้ หลักสูตร และแบบทดสอบต่าง ๆ ได้ในหนึ่งแพลตฟอร์ม โดยจะมีระบบแนะนำหลักสูตรที่เหมาะสมกับความสนใจของพนักงานผ่านการเรียนรู้ของเครื่อง (Machine Learning) และยังมีโมเดลวิเคราะห์ความสนใจในสายอาชีพเพื่อช่วยให้ผู้จัดการสามารถวางแผนการพัฒนาและเติบโตในหน้าที่การงานของพนักงานได้อย่างมีประสิทธิภาพ โดยที่ทั้งสองแพลตฟอร์มจะมีผู้ใช้งานหลักอยู่ 4 บทบาทได้แก่ ผู้ดูแลระบบ (Admin) ผู้จัดการ (Manager) พนักงาน (Employee) และบุคคลภายนอก (Outsider)

ผลลัพธ์จากการนำทั้งสองแพลตฟอร์มไปใช้งาน ได้แก่ การเพิ่มประสิทธิภาพในการจัดการพนักงานของฝ่ายทรัพยากรบุคคล การส่งเสริมการมีส่วนร่วมของพนักงานและการตัดสินใจเชิงกลยุทธ์ด้านทรัพยากรบุคคลที่แม่นยำมากขึ้น รวมถึงการพัฒนาความสามารถของพนักงานและการวัดผลของการเรียนรู้ของพนักงาน โดยรวมแล้ว ทั้งสองแพลตฟอร์มสามารถสนับสนุนการเติบโตส่วนบุคคลของพนักงานในเครือบริษัท เอสซีจี เคมิคอลส์ จำกัด (มหาชน) ได้อย่างมีประสิทธิภาพ และมีส่วนช่วยต่อความสำเร็จขององค์กรในระยะยาว

คำสำคัญ: การจัดการทรัพยากรมนุษย์ (HRM)/ การพัฒนาทรัพยากรมนุษย์ (HRD)/ การมีส่วนร่วมของพนักงาน/ ศูนย์รวมแหล่งเรียนรู้/ การเรียนรู้ของเครื่อง (Machine Learning)/ แดชบอร์ด

ACKNOWLEDGMENTS

First and foremost, we would like to express our deepest gratitude to our advisor, Piyanit Ua-areemitr, Ph.D., for her invaluable guidance, support, and encouragement throughout this project. Her insights, feedback, and expertise were essential in shaping both the technical development and strategic direction of the Learning Hub platform. We are especially thankful for her patience and mentorship, which greatly contributed to our learning and growth during this journey.

We would also like to thank our mentors from the Ant HR who provided foundational knowledge and advice that helped us approach the challenges of this project with confidence and clarity.

A special thanks goes to our friends who supported us throughout this journey. Whether it was offering feedback on our designs, helping with testing, or just cheering us on during stressful times, your support meant a lot and helped us stay motivated.

Additionally, We would like to acknowledge the team at SCGC and the Ant HR initiative for providing a real-world context that inspired this project. Their vision to improve employee engagement and development through innovation was a key driver in creating a meaningful and impactful solution.

Lastly, we would like to thank everyone involved in the Ant HR project, especially those who inspired and guided us with real industry insights. Working on this platform has been a valuable experience that not only helped us grow as developers but also taught us the importance of collaboration, user experience, and creating impact through technology.

Thank you all for being a part of our journey.

CONTENTS

	PAGE
ABSTRACT	ii
THAI ABSTRACT	iii
ACKNOWLEDGMENTS	iv
CONTENTS	v
LIST OF TABLES	ix
LIST OF FIGURES	x
LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS	xiv
 CHAPTER	
1. INTRODUCTION	1
1.1 Problem Statement, Motivation, and Potential Benefits	1
1.2 Proposed Method	1
1.2.1 Objectives	1
1.2.2 Scope	1
1.3 Original Engineering Content	2
1.3.1 Database System	2
1.3.2 Software Engineering	2
1.3.3 User Interface Design	2
1.3.4 Dashboard	3
1.3.5 Web Application	3
1.3.6 Mobile Application	3
1.3.7 Data Analysis	3
1.4 Deliverables	3
1.4.1 First Semester	3
1.4.2 Second Semester	4
1.5 Tasks Breakdown and Schedule	5
1.5.1 Task List	5
1.5.2 Schedule	6
1.5.2.1 First Semester 2024	7
1.5.2.2 Second Semester 2024	8
 2. BACKGROUND THEORY AND RELATED RESEARCH	9
2.1 Theory and Core Concept	9
2.1.1 Human Resources	9
2.1.2 Human Resources Management	9
2.1.3 Human Resources Development	9
2.1.4 Database System	10
2.1.5 Data Analysis	10
2.1.6 Machine Learning	10
2.1.7 Performance Dashboard	10
2.1.8 User Interface Design	10
2.1.9 Websites	10
2.1.10 Mobile Applications	11

2.2	Language and Technology	11
2.2.1	React.js	11
2.2.2	React hook form + yup	11
2.2.3	Flutter	11
2.2.4	SQL	11
2.2.5	MySQL Workbench	12
2.2.6	node.js	12
2.2.7	express.js	12
2.2.8	Google Cloud	12
2.2.9	GitHub	13
2.2.10	Figma	13
2.2.11	Jira	13
2.3	Related Research / Competing Solutions	13
2.3.1	UniCulture and UniHR Platform	13
2.3.1.1	Happily	13
2.3.1.2	Bonusly	14
2.3.1.3	Motivosity	14
2.3.1.4	Summary	15
2.3.2	UniLearning Platform	16
2.3.2.1	Kahoot!	16
2.3.2.2	Skooldio	16
2.3.2.3	Summary	17
3.	SYSTEM DESIGN	18
3.1	System Overview	18
3.1.1	Human Resource Management	18
3.1.1.1	Morale Daily and Yearly Question Forms	19
3.1.1.2	Morale Daily and Yearly Dashboard	19
3.1.1.3	Assessment 360	19
3.1.2	Human Resource Development	20
3.1.2.1	Quiz Management	20
3.1.2.2	Video Learning	20
3.1.2.3	Video Course Suggested by AI	21
3.1.2.4	Competency and Skill	21
3.1.2.5	Skill Mapping	21
3.2	System Requirement and List	21
3.2.1	Human Resource Management (HRM)	21
3.2.2	Human Resource Development (HRD)	25
3.3	Workflow	27
3.4	Use Cases Diagram and Narrative	33
3.5	Sequence Diagram	43
3.6	Database Design	59
3.6.1	Entity Relationship Diagram	60
3.6.2	Data Dictionary	61
3.7	Products of The Design Process	66
3.7.1	Navigation Map	66
3.7.2	System Architecture	70
3.7.3	User Interface	70
3.8	Evaluation Plans	110
3.8.1	User Acceptance Testing (UAT)	110
3.8.2	Survey	111

4. RESULTS AND DISCUSSION	112
4.1 Team Overview	112
4.2 Working Process	113
4.2.1 Meeting stand ups	113
4.2.2 Deploying and get the feedback	114
4.3 UniCulture Implementation	114
4.3.1 Front-End Implementation	114
4.3.2 Back-End Implementation	120
4.3.3 Machine Learning	121
4.3.3.1 Sentiment Analysis for UniCulture	121
4.3.3.1.1 Data Preparation	121
4.3.3.1.2 Implementation	122
4.3.3.1.3 Thai Sentiment Analysis	122
4.3.3.1.4 Model Training	122
4.3.3.1.5 Model Input and Result	123
4.3.3.1.6 Model Evaluation	123
4.4 UniLearning Implementation	124
4.4.1 Front-End	124
4.4.2 Back-End	124
4.4.3 Machine Learning	125
4.4.3.1 Video Course Recommendation	125
4.4.3.2 Data Preparation	126
4.4.3.3 Implementation	126
4.4.3.4 Model Input and Result	126
4.4.3.5 Model Evaluation	127
4.5 UniHR Implementation	127
4.5.1 Machine Learning	127
4.5.1.1 Career Aspiration for UniHR	127
4.5.1.1.1 Data Preparation	127
4.5.1.1.2 Feature Extraction Using TF-IDF	128
4.5.1.1.3 Similarity Computation Using Cosine Similarity	128
4.5.1.1.4 Weakness Penalty	128
4.5.1.1.5 Model Input and Result	129
4.5.1.1.6 Model Evaluation	129
4.6 User Acceptance Testing (UAT)	129
4.6.1 UniCulture	131
4.6.2 UniLearning	134
4.7 Challenges and Resolutions	141
4.7.1 Uploading Excel Files and Automating Email Sending to Outsiders	141
4.7.2 Limited Thai Dataset for Model Training	141
4.7.3 Dependency Conflicts Preventing Library Installation	141
4.7.4 Frequent Changes Due to Client Testing	141
4.7.5 Error Bugs	141
4.7.6 Understanding New Code and Database Structure after Transition to UniCulture	142
4.7.7 System Failures Due to High Concurrent Users in the Rush Quiz Function	142
4.8 Conclusions	142
5. CONCLUSIONS	143
5.1 Conclusion	143
5.2 Completion Status Table	143
5.3 Discussion of Problems and Solutions	144
5.4 Lessons Learned	144
5.5 Future Extensions and Enhancements	145

REFERENCES	146
APPENDIX	148
A IMPLEMENTATIONS OF PLATFORMS	149
B USER RATING ON MODULE	174

LIST OF TABLES

TABLE	PAGE
1.1 Task list for semester one	5
1.2 Task list for semester two	6
2.1 The comparison between Happily.ai, Motivosity, Bonusly and UniHR	15
2.2 The comparison between Kahoot!, Skooldio, and UniLearning	17
3.1 Use case narrative for create question forms	37
3.2 Use case narrative for configure and assign morale daily question forms	37
3.3 Use case narrative for publish forms for morale yearly	38
3.4 Use case narrative for view assigned question forms	38
3.5 Use case narrative for answer and submit forms	39
3.6 Use case narrative for view morale yearly dashboard	39
3.7 Use case narrative for view morale yearly dashboard	39
3.8 Use case narrative for view morale daily dashboard	40
3.9 Use case narrative for creating the assessment	40
3.10 Use case narrative for duplicating the assessment	41
3.11 Use case narrative for assigning the assessment	41
3.12 Use case narrative for responding the assessment	42
3.13 Use case narrative for video course learning	42
3.14 Morale table	61
3.15 Morale question table	62
3.16 Question table	62
3.17 Question topic table	62
3.18 Morale answer table	62
3.19 Morale participant table	63
3.20 Morale daily question table	63
3.21 Morale daily table	63
3.22 Morale daily participant table	63
3.23 UniCulture_Appraisee	64
3.24 UniCulture_Appraisee_SampleSize	64
3.25 UniCulture_Assessment	64
3.26 UniCulture_Assessment_Answer	65
3.27 UniCulture_Assessment_Question	65
3.28 Training_OnlineVideoTotalWatching table	65
3.29 Training_Competency table	65
5.1 Completion Status Table	144

LIST OF FIGURES

FIGURE	PAGE
1.1 Project Schedule of Semester 1	7
1.2 Project Schedule of Semester 2	8
3.1 System workflow for morale yearly question form	27
3.2 System workflow for morale daily question form	28
3.3 System workflow for morale yearly dashboard	29
3.4 System workflow for morale daily dashboard	29
3.5 System workflow for assessment 360	30
3.6 System workflow for rush quiz	31
3.7 System workflow for Video Course Learning	31
3.8 System workflow for Competency and Skill	32
3.9 Use case diagram of morale daily and yearly question form	33
3.10 Use case diagram of morale daily and yearly dashboard	34
3.11 Use case diagram of Assessment 360	35
3.12 Use case diagram of Video Course Learning	35
3.13 Sequence diagram of creating morale dairy question forms	43
3.14 Sequence diagram of creating morale yearly question forms	44
3.15 Sequence diagram of configure and assign morale daily question forms	45
3.16 Sequence diagram of publish morale yearly question forms	46
3.17 Sequence diagram of answer and submit forms	47
3.18 Sequence diagram of viewing morale daily dashboard	48
3.19 Sequence diagram of filtering parameters on morale daily dashboard	49
3.20 Sequence diagram of exporting employee table on morale daily dashboard	50
3.21 Sequence diagram of viewing morale yearly dashboard	51
3.22 Sequence diagram of filtering on morale yearly dashboard	52
3.23 Sequence diagram of creating assessment	53
3.24 Sequence diagram of duplicating assessment	54
3.25 Sequence diagram of assigning assessment	55
3.26 Sequence diagram of responding assessment	57
3.27 Sequence diagram of video course learning	58
3.28 Entity relationship diagram of morale daily and yearly	60
3.29 Entity relationship diagram of Assessment 360	61
3.30 Navigation map of admin role on UniCulture	67
3.31 Navigation map of manager role on UniCulture	68
3.32 Navigation map of user role on UniCulture	68
3.33 Navigation map of user role on UniLearning	70
3.34 System architecture of our website	70
3.35 Question management of morale daily question form user interface before redesign	71
3.36 Question management of morale daily question form user interface after redesign	71
3.37 Create morale daily question form	72
3.38 Publish and deactivate morale daily question form	73
3.39 Morale daily answer page	74
3.40 User responds to morale daily question form	75
3.41 Question management of morale yearly question form user interface before redesign	76
3.42 Question management of morale yearly question form user interface after redesign	76
3.43 Create morale yearly question form	77
3.44 Publish morale yearly question form	78

3.45 Edit morale yearly question form	79
3.46 Morale yearly answer page	80
3.47 User's morale yearly question form list	81
3.48 User responds to morale yearly question form	82
3.49 admin's morale daily dashboard before enhancement	83
3.50 admin's morale daily dashboard filter before enhancement	83
3.51 admin's morale daily dashboard filter before enhancement	84
3.52 Admin's morale daily dashboard after enhancement	84
3.53 Admin's morale daily dashboard after enhancement	85
3.54 Admin's morale daily dashboard after enhancement	86
3.55 Admin's morale daily dashboard after enhancement	86
3.56 Admin's morale yearly dashboard before enhancement	87
3.57 Admin's morale yearly dashboard before enhancement	87
3.58 Admin's morale yearly dashboard before enhancement	88
3.59 Admin's morale yearly dashboard after enhancement	88
3.60 Admin's morale yearly dashboard after enhancement	89
3.61 Admin's morale yearly dashboard after enhancement	89
3.62 Admin's morale yearly dashboard after enhancement	90
3.63 Manager's morale yearly dashboard before enhancement	90
3.64 Manager's morale yearly dashboard after enhancement	91
3.65 Manager's morale yearly dashboard after enhancement	91
3.66 Manager's morale yearly dashboard after enhancement	92
3.67 Manager's morale yearly dashboard after enhancement	92
3.68 User responds to morale daily question form (Mobile)	93
3.69 Question management of assessment 360 user interface before redesign	94
3.70 Question management of assessment 360 user interface after redesign	94
3.71 Create assessment of assessment 360 user interface before redesign	95
3.72 Create assessment of assessment 360 user interface after redesign	96
3.73 Duplicate assessment of assessment 360 user interface	97
3.74 Assessment details user interface when the assessment has not been used	98
3.75 Assessment details user interface when the assessment has been used	98
3.76 Assign the assessment page on the appraiser list	99
3.77 Assign the assessment page on the appraiser list when outsider is selected	99
3.78 Assign the Assessment Page by Uploading the Excel File	100
3.79 Answer overview of assessment 360 user interface before redesign	101
3.80 Answer overview of assessment 360 user interface after redesign	101
3.81 Welcome page for outsider	102
3.82 List of assessments for outsider	103
3.83 Answering the Choice Question	103
3.84 Answering the Text Question	104
3.85 List of Question assessment 360 user interface	104
3.86 Answer assessment 360 user interface	105
3.87 User Interface of Online Lesson	106
3.88 User Interface of Video course suggested by AI	106
3.89 User Interface of Rush Quiz	107
3.90 Rush Quiz Creation and Management	108
3.91 Rush Quiz Lobby Interface	109
3.92 User Interface of Competency	110
4.1 Admin Morale Daily Dashboard – Sentiment Analysis Component	119
4.2 Input and output of the Thai sentiment analysis model	123
4.3 Input and output of the video course recommendation model	126

4.4	Input of the career aspiration model	129
4.5	Text output of the career aspiration model	129
4.6	Chart output of the career aspiration model	129
4.7	Overall User Ratings on the Morale Daily and Yearly Question Forms	131
4.8	Overall User Ratings on the Morale Daily and Yearly Dashboard	132
4.9	Overall User Ratings on the 360 Assessment	133
4.10	Overall User Ratings on UniCulture Platforms	134
4.11	User Suggestions on UniCulture Platforms	134
4.12	Overall User Ratings on the Quiz Management	135
4.13	Overall User Ratings on the Video Learning	136
4.14	Overall User Ratings on the AI-Suggested Video Courses	137
4.15	Overall User Ratings on the Competency and Skill	138
4.16	Overall User Ratings on the Career Aspiration	139
4.17	Overall User Ratings on UniLearning Platforms	139
4.18	User Suggestions on UniLearning Platforms	140
4.19	Overall User Ratings on Platforms	140
A.1	Survey Navigation System	149
A.2	Answer Management	149
A.3	Fetching Card Style of Pending Assessment 360 Questions	150
A.4	Assessment 360 updateAnswer Function	150
A.5	Assessment 360 handleSubmit Function	151
A.6	Form State Management (React Hook Form)	151
A.7	Dynamic Question Rendering Code	152
A.8	Publishing Workflow Code	152
A.9	Derived State (Date Validation)	153
A.10	Tab State Management	153
A.11	The totalAverageScore function	153
A.12	Data preparation	154
A.13	Top3StrengthDailyTable data preparation function	154
A.14	Top3WeaknessDailyTable Data Preparation Function	155
A.15	Department Average Score Table Data Preparation Function	155
A.16	Section Average Score Table Data Preparation Function	156
A.17	calculateTotalAverageScore Function	156
A.18	avgScore Function	156
A.19	compareAverageScore Function	157
A.20	Bar Chart of Total Average Score According to Question Topic Data Preparation	157
A.21	Bar Chart of Total Average Score According to Section Data Preparation	157
A.22	getTopStrengthsAndWeaknesses Function	158
A.23	prepareChartData Function	158
A.24	calculateAverageScore Function	159
A.25	The getMoraleDailyKPI Function	160
A.26	The dogetQuestionDailyAnswerText Function	161
A.27	The getMoraleKPI function	162
A.28	The addMoraleDaily Function	163
A.29	The addMorale Function	164
A.30	The updateMorale Controller	165
A.31	The analyzeSentiment Function	166
A.32	Thai Sentiment Analysis Machine Learning	167
A.33	Thai Sentiment Analysis Machine Learning	168
A.34	Fetching The Video Progress	168
A.35	Question Navigation Panel	169

A.36	Question Editing System	169
A.37	Data Saving Function	170
A.38	Preview Function	170
A.39	getFinishVideoCourse Function	170
A.40	updateDuration Function	170
A.41	doGetFinishVideoCourse Function	171
A.42	Request Validation & Initial Setup	172
A.43	Database Connection & Transaction Management	172
A.44	Inserting Into Database	172
B.1	User Ratings on the Morale Daily and Yearly Question Forms	174
B.2	User Ratings on the Morale Daily and Yearly Dashboard	174
B.3	User Ratings on the 360 Assessment	175
B.4	User Ratings on the Quiz Management	175
B.5	User Ratings on the Video Learning	176
B.6	User Ratings on the AI-Suggested Video Courses	176
B.7	User Ratings on the Competency and Skill	177
B.8	User Ratings on the Career Aspiration	177

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS

HRM	=	Human Resource Management
HRD	=	Human Resource Development
ML	=	Machine Learning
UniCulture	=	Platform involves managing employee-related operations.
UniLearning	=	Platform is designed to help employees improve by providing access to courses, quizzes, and learning resources.

CHAPTER 1 INTRODUCTION

1.1 Problem Statement, Motivation, and Potential Benefits

Ant HR is a platform that manages outsourced and subcontractor employees under the ZERO TO ONE by SCG project. In Thailand, millions of outsourced employees are hired through third-party companies and sent to work for various businesses in Thailand and abroad. However, there hasn't been a program to manage important data like employee information and time tracking. Ant HR saw this problem and developed different HR tech solutions, leading to the creation of its first service.

Ant HR is now launching the Ant HR Learning Hub, a project designed to help employees improve their skills. This platform will provide various functions, including recruitment, Human Resource Development, Human Resource Management, and resources for learning and development, aiming to build a culture of growth and innovation within the company. By making learning opportunities accessible, Ant HR hopes to support employees in their professional growth and make learning an enjoyable part of their work experience.

This project will be a part of Ant HR learning hub and focus on two main areas: HRD and HRM. In HRD, we will create a learning hub for all employees to easily access various learning resources, courses, assessments, and interactive tools through both a website and a mobile application. The video content will include both internal and external resources, encouraging continuous learning and growth. Employees will have personalised learning paths and opportunities to explore topics related to their roles and interests.

The HRM part will focus on rewarding and recognizing employees. Every employee can earn rewards by participating in activities. The rewarding module includes receiving "boost-up coin" and sending "hearts" to recognize others, which will help promote strong teamwork and a positive culture. The platform will also include surveys for checking morale and mental health, along with data analysis to monitor employee engagement. This will help ensure that employees' mental well-being is in good condition and identify when they might need help. Machine learning will assist in predicting and improving the employee experience within the company.

1.2 Proposed Method

1.2.1 Objectives

- Design and develop both website and mobile applications to ensure employees can engage in learning activities anytime, anywhere.
- Integrate features in the (HRD) section, including an adaptive learning hub and quizzes section, with customised pathways to individual employees' needs to enhance their skills using a machine learning model.
- Integrate features in the (HRM) section according to the company's culture, including rewarding employee engagement and satisfaction using a machine learning model.
- Integrate interactive features and competency assessments to make learning engaging and measurable.

1.2.2 Scope

- Create wireframes for UX/UI to ensure a user-friendly experience on both the web and mobile applications both IOS and Android.

- Set up a secure database to store learning materials and track employee progress across both HRD and HRM sections.
- Develop interactive learning features including quizzes and assessments within the HRD section to increase employee engagement and track learning outcomes.
- Build a model for HRM aspects to suggest career progression paths for employees, evaluate compensation and reward structures, and detect employee engagement and satisfaction levels.
- Develop machine learning (ML) components to both the HRD and HRM sections to suggest appropriate employee learning pathways, monitor employee mental health and their satisfaction levels, and utilise current data to find pertinent ML use cases.
- Redesign existing platform to improve the user experience

1.3 Original Engineering Content

The original engineering content covered in the project includes the following:

1.3.1 Database System

The database system is crucial for our project, as it is responsible for storing and retrieving data. In this project, MySQL Workbench serves as the database management tool, allowing us to interact with the pre-existing database without needing to make any modifications. The database system was already designed by AntHR, so we are not required to make any changes to it.

1.3.2 Software Engineering

Developing the Employees' Engagement and Learning Hub for Ant HR involves a structured approach based on software engineering principles, including requirements analysis, system design, development, and testing. The process begins with a thorough examination of user expectations and specific requirements for the website and mobile application, ensuring alignment with business goals. These requirements are then translated into a comprehensive design that outlines the system's architecture, user interface, and technical specifications. The coding and development phase follows, focusing on building a reliable, functional, and user-friendly platform. Finally, the testing phase includes unit testing, integration testing, and user acceptance testing to ensure the platform meets its intended purpose and is free of defects before launch.

1.3.3 User Interface Design

The development of the user interface for the Employees' Engagement and Learning Hub Website and Mobile Application for Ant HR company on creating an intuitive and user-friendly experience that supports HRM and HRD activities. The design focuses on easy navigation and smooth interactions, tailored specifically for different roles like admins, managers, and general users.

The UI will be carefully crafted to ensure intuitive functionality, matching the company's established colour palette and typography to maintain a consistent look and feel. By following these design guidelines, the goal is to deliver a seamless and cohesive user experience across the platform.

1.3.4 Dashboard

The dashboard in this project will display the computed and monitored performance of employee engagement within the website and application. It provides users with a visual representation to help them better understand their activities and engagement within the platform.

1.3.5 Web Application

The HRM and HRD website will use React.js for the front-end, Node.js for the back-end, and Figma for UI design and wireframing, creating an efficient and user-friendly platform. React.js will build a dynamic and responsive interface for features like employee management, performance tracking, and learning hub. Node.js will handle server-side tasks like data processing and user authentication, while Figma will help design the interface and layout. This combination of tools ensures a smooth development process and a web application dedicated to the organisation's needs.

1.3.6 Mobile Application

The development of the Employees' Engagement and Learning Hub will focus on the mobile app using Flutter, which allows building apps for both iOS and Android from one codebase, making it efficient and cost-effective. The app will include key features like user login, personalised dashboards and learning modules, all integrated with backend services for smooth data flow. Flutter's real-time updates and customizable widgets ensure a smooth, responsive user experience. Performance, accessibility, and compatibility will be prioritised through thorough testing, ensuring the app meets Ant HR's needs and enhances employee engagement and learning.

1.3.7 Data Analysis

The data analysis section plays a critical role in transforming raw data from both the application and the website into actionable insights. Our analysis process involves gathering, cleaning, and processing data from user interactions, system logs, and other relevant sources. By using statistical methods and machine learning algorithms, we are able to uncover patterns and trends that inform decision-making.

1.4 Deliverables

1.4.1 First Semester

- Wireframes
- User journey maps
- Navigation map
- Wireframes for mobile application and website redesign
- User interface for mobile application and website redesign
- UI design survey results
- Backend setup and database integration
- Frontend & Backend integration
- Machine learning algorithm design
- Progress report

1.4.2 Second Semester

- HRD interactive learning features
- HRM engagement tools
- Competency assessments and quizzes
- Employee engagement features
- Machine learning models
- Implementation of the dashboard feature
- User Acceptance Testing (UAT)
- Project delivery
- Final report

1.5 Tasks Breakdown and Schedule

1.5.1 Task List

Table 1.1 Task list for semester one

Task description	Dependencies
1. Project planning & requirements	
1.1 Project scope & requirement	
1.2 Idea document	1.1 Project scope & requirement
1.3 Draft proposal	1.2 Idea document
2. Requirement Gathering & Objective Finalisation	
3. Tool & Platform Research and Learning	
3.1 Learn tools and platforms	
3.2 UI Design Evaluation Survey	3.1 Learn tools and platforms
4. Redesign Phase: User Experience & Interfaces	
4.1 User Journey Mapping	
4.2 Navigation map creation	4.1 User Journey Mapping
4.3 Website redesign wireframes	4.2 Navigation map creation
4.4 Wireframe design for mobile	4.2 Navigation map creation
5. Final proposal	
6. Proposal presentation	
7. Initial Development Phase	
7.1 Setup backend services & database	
7.2 Initial frontend & backend integration for website	
7.3 Initial frontend & backend integration for mobile	
8. Machine Learning Design	
8.1 Design machine learning model	
8.2 Data collection and preprocessing	
8.3 Integration with backend	
9. Term 1 report	
10. Team 1 presentation	

Table 1.2 Task list for semester two

Task description	Dependencies
1. Implement: HRD Features	
1.1 Develop learning modules	
1.2 Interactive video integration	1.1 Develop learning modules
1.3 Course tracking system	1.2 Interactive video integration
2. Implement: HRM Features	
2.1 Reward system implementation	
2.2 Engagement tracking	2.1 Reward system implementation
2.3 Employee Satisfaction Surveys	2.2 Engagement tracking
3. Competency Assessments	
3.1 Competency Assessment Framework	
3.2 Tracking and reporting	
4. Machine Learning Model Implementation	
4.1 Machine learning for personalised learning paths	
4.2 Model for Employee Satisfaction Prediction	
5. Dashboard Design & Development	
5.1 Design dashboard wireframes	
5.2 Integrate learning and engagement metrics	5.1 Design dashboard wireframes
5.3 Real-time data visualisations	
6. User Acceptance Testing (UAT)	1-5 All implement tasks
7. Project Delivery	6. User Acceptance Testing (UAT)
8. Project Submission	7. Project Delivery
9. Final Project Presentation and Report	8. Project Submission

The tables provided present a detailed breakdown of all the key tasks required to implement and complete the project, organised across two tables: Table 1.1 for the first semester and Table 1.2 for the second semester. Each table outlines the tasks along with their dependencies, offering a clear and structured view of the project's timeline, progression, and requirements over the course of both semesters.

1.5.2 Schedule

Table 1.1 shows, in August, the primary goal was to develop a detailed project proposal. The team worked on drafting the core objectives, methodologies, and scope of work for both the HRD and HRM components. By September, the project officially started with the gathering of user requirements and finalisation of the project's objectives for the Ant HR Learning Hub. The team began learning about the tools and platforms required for the development of both the web and mobile applications. This included technologies for machine learning integration, the backend database, and frontend technologies. At the same time, a UI design evaluation survey was conducted to gather feedback and improve the usability of the platform.

In October, team responsibilities were distributed, and a User Journey Map and Navigation Map were developed to plan user interaction flows. Initial wireframe designs for mobile applications were created, and additional redesign of the website.

From November to December focused on early-stage implementation of improved front-end and back-end systems. This included setting up database structures, back-end services, and establishing initial integrations between the web and mobile platforms. During this period, the team also started developing machine learning algorithms to customise learning paths based on user data and track employee well-being.

The second semester began in January with the full-scale development of both the HRD and HRM sections. The team prioritised creating interactive learning features, personalised learning pathways, and implementing the reward and recognition system for employee engagement. The timeline is shown in Table 1.2

In February, progress continued with the development of competency assessments and quizzes for the HRD section. At the same time, the HRM section's engagement features, including “boost-up coins” and “hearts” for recognition, were integrated into the platform.

By March, the focus was on completing machine learning models for both HRD and HRM functionalities. These models were designed to track employee progress, suggest personalised learning materials, monitor morale, and predict employee satisfaction. Additionally, The platform's dashboard was refined to display key insights, such as employee engagement metrics and learning progress.

In April, the team began User Acceptance Testing (UAT) to ensure all features met the necessary requirements. The project was completed with the final presentation and delivery, showcasing the development of the Ant HR Learning Hub and its capabilities.

1.5.2.1 First Semester 2024

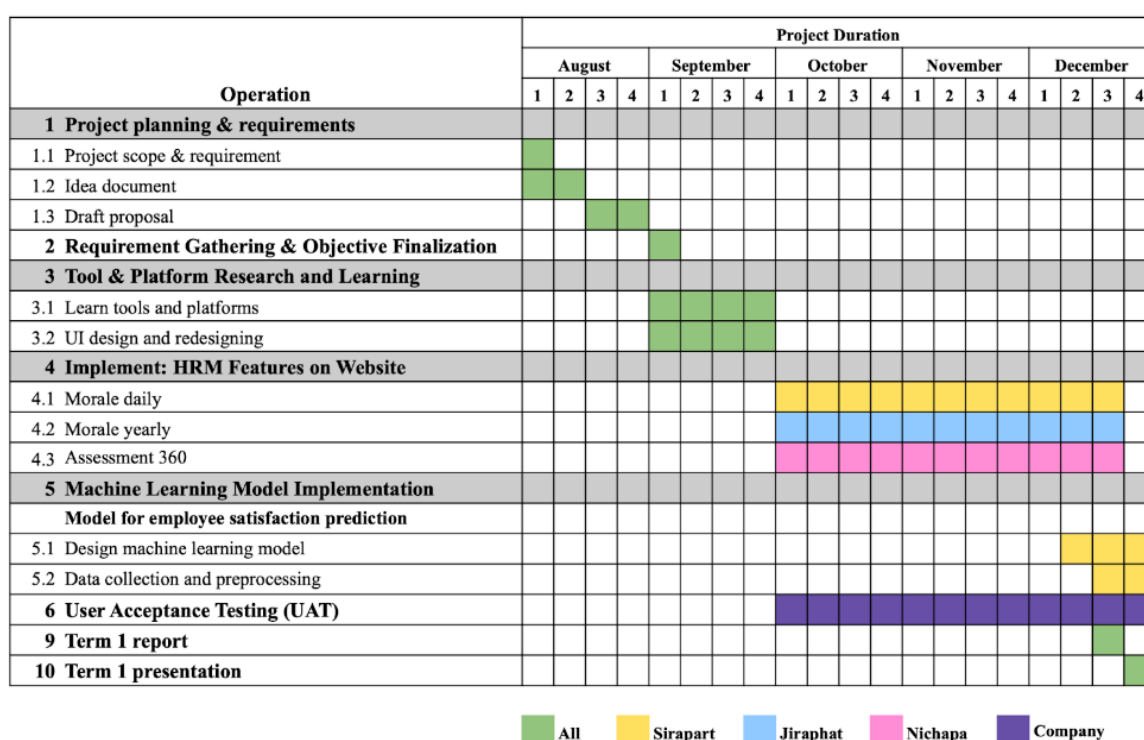


Figure 1.1 Project Schedule of Semester 1

1.5.2.2 Second Semester 2024

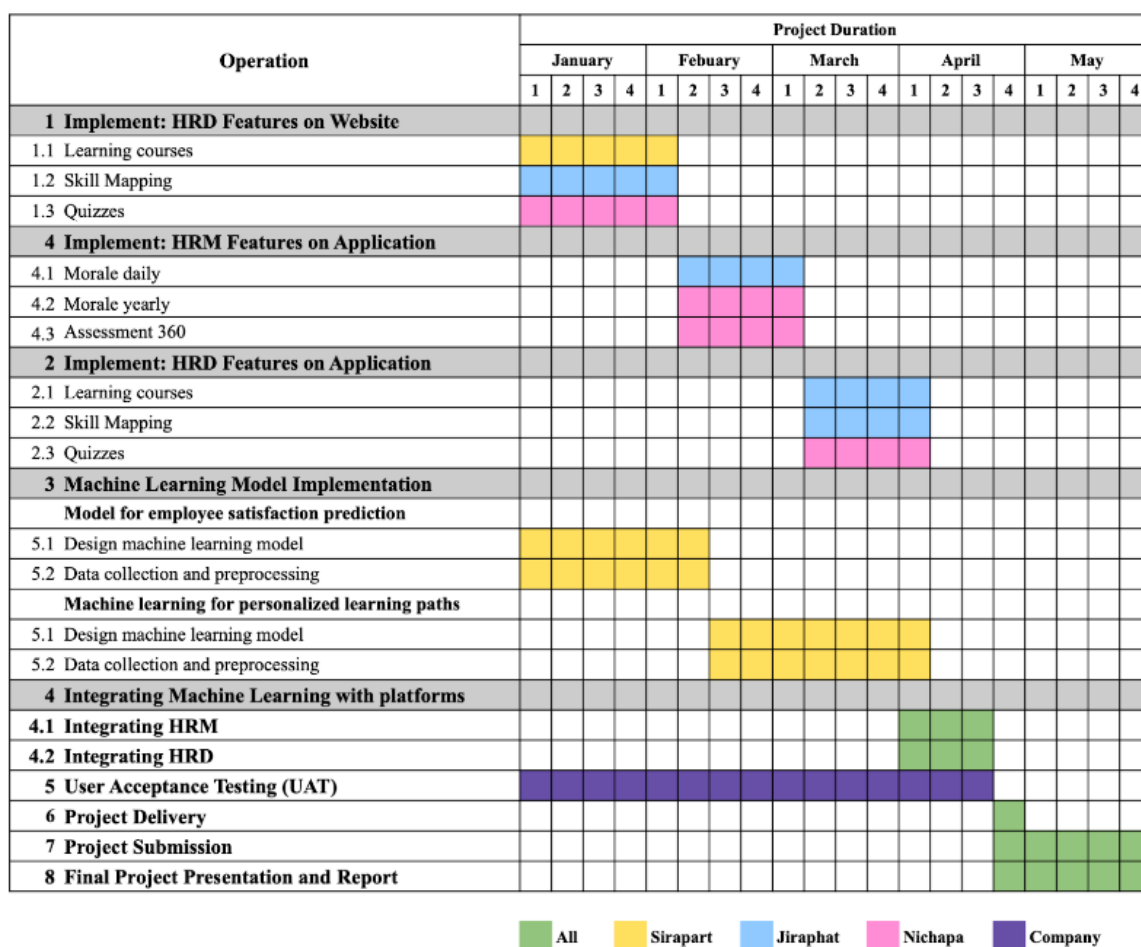


Figure 1.2 Project Schedule of Semester 2

CHAPTER 2 BACKGROUND THEORY AND RELATED RESEARCH

Chapter two provides an in-depth examination of the background, theory, and related research behind the development of the website and application for employee engagement and learning platforms. It situates the project within existing knowledge, outlining the platform's distinctive contributions.

The first section, Background, sets the foundation by providing context, supporting the identified problem with external information, and explaining the key challenges faced by HR departments. The Theories and Concepts section establishes the theoretical basis by exploring the key concepts behind the tool's development and offering a detailed explanation of relevant theories. In the Tools and Technologies section, the specific programming languages, frameworks, and databases used in creating the tool are introduced.

The Potential Solutions section reviews existing products and solutions for sales data management, describing their features and capabilities, identifying gaps in current solutions, and showing how the new platform can address those gaps with improvements.

2.1 Theory and Core Concept

2.1.1 Human Resources

Human resources is responsible for recruiting, onboarding, training and managing an employee from their job application to final severance [1]. This end-to-end management of employees includes strategies for payroll and benefits administration, upskilling, establishing positive workplace culture, optimising employee productivity, handling employer-employee relationships, employee termination and more.

Human resource management [2] should not be confused with workforce management, which focuses on strategic planning for the direction of an organisation's growth and not on the day-to-day supervision of employees.

2.1.2 Human Resources Management

Organisational Lifecycle Theory views organisations as going through stages similar to biological life cycles - birth, growth, maturity, decline, and death. It recognizes that different stages require different approaches to manage human resources [3]. Understanding an organisation's current stage in its lifecycle informs HR strategies and practices.

This theory is closely related to HR culture platforms, as UniCulture platforms must evolve adjacent to an organisation's growth and changing needs. In the early stages of a company's life cycle (birth and growth), HR culture platforms often focus on attracting talent, building a strong foundational culture, and encouraging innovation. As the organisation matures, the focus shifts to maintaining employee engagement, supporting career development, and formalising HR processes [3]. During periods of decline, the HR culture platform may prioritise managing change, retaining key employees, and preparing for potential restructuring. Understanding where the organisation is in its life cycle allows HR to tailor the platform's features to align with the current needs, ensuring a supportive and adaptable work environment.

2.1.3 Human Resources Development

Human Resources Development (HRD) refers to the process of improving the capabilities, skills, and knowledge of employees within an organisation to enhance their performance and contribute to organisational suc-

cess [4]. In essence, HRD aims to unlock the full potential of human resources within an organisation, leading to enhanced individual and collective capabilities that drive organisational success.

2.1.4 Database System

A database is a collection of interrelated data that helps in the efficient retrieval, insertion, and deletion of data from the database and organises the data in the form of tables, views, schemas, reports, etc [5]. For example, a university database organises the data about students, faculty, admin staff, etc. which helps in the efficient retrieval, insertion, and deletion of data from it.

2.1.5 Data Analysis

Data analysis is the practice of working with data to glean useful information [6], which can then be used to make informed decisions.

2.1.6 Machine Learning

A subfield of computer science and artificial intelligence called machine learning (ML) [7] is concerned with using data and algorithms to help AI mimic human learning processes and progressively become more accurate.

2.1.7 Performance Dashboard

A performance dashboard is a type of data visualisation tool that helps management and employees measure, control, and keep an eye on internal business operations in order to achieve organisational objectives [8]. The employees' results will be visualised in this project.

2.1.8 User Interface Design

UI design, also known as user interface design, focuses on the visual and aesthetic aspects of a product's interface, such as apps or websites. It involves crafting interfaces with an emphasis on appearance and style [9]. While primarily associated with digital interfaces, it also covers voice-controlled interfaces (VUI).

A UI designer is responsible for designing all screens within a product and creating visual elements that enable user interaction. They utilise elements like patterns, spacing, text, and colour to guide the user's experience.

In addition to being highly creative, UI designers consider the user's perspective, anticipating their expectations at every step. We'll explore more about their role later on.

2.1.9 Websites

A website is an electronic platform consisting of interconnected web pages that are accessed via the internet using HTTP protocols. Websites serve various purposes, including informational, commercial, educational, and entertainment content delivery [10]. They are composed of HTML, CSS, and JavaScript code that define the structure, layout, and interactive functionality of web pages. Websites can contain text, images, videos, and other multimedia content, linked together through hyperlinks. Users access websites using web browsers, which interpret the HTML, CSS, and JavaScript code to display the content on their devices. Websites can be static (pre-built content) or dynamic (content generated on-the-fly), and they may incorporate databases, APIs, and server-side technologies to manage and deliver complex information and services.

2.1.10 Mobile Applications

Mobile applications, commonly referred to as apps, are software programs designed to run on mobile devices such as smartphones and tablets [11]. These applications are developed to perform specific functions or provide services tailored for mobile users.

2.2 Language and Technology

2.2.1 React.js

React.js is a JavaScript library used for building dynamic, single-page user interfaces. In this project, React.js was chosen for the web interface due to its component-based architecture, which allows for the development of reusable UI elements. This was particularly useful for creating modular components such as dashboards, course listings, and employee engagement tools within the Ant HR Learning Hub. React's Virtual DOM significantly improved rendering performance, ensuring a smooth user experience even when handling large datasets like employee records and learning materials [12]. Additionally, React's ecosystem, including state management tools like Redux, helped manage the application's state efficiently across different components, allowing real-time updates to the user interface as employees engaged with the platform.

2.2.2 React hook form + yup

React Hook Form was used for handling user input within React components, offering a performance-optimised solution for managing form states. Forms are central to many interactions on the platform, such as quiz submissions, and feedback forms. React Hook Form provided an efficient way to handle these interactions by minimising the number of re-renders and improving validation speed [13]. Paired with Yup, a schema builder, form validation became more robust and declarative. Yup, it allowed for complex validation rules, such as conditional field requirements [14]. The combination of these tools ensured that only valid data was submitted to the backend, reducing errors and improving data integrity.

2.2.3 Flutter

Flutter, Google's open-source UI toolkit, was selected to develop the mobile version of the Ant HR Learning Hub. Flutter's key advantage lies in its ability to create cross-platform applications for both iOS and Android from a single codebase, reducing development time and effort. Flutter's hot reload feature enabled rapid testing and iteration, allowing developers to see changes in real time without restarting the application [15]. This was particularly useful during the implementation of interactive features like quizzes, video content, and push notifications for employee reminders. Additionally, Flutter's rich set of customizable widgets enabled the team to design an intuitive and responsive interface that mirrored the web version, ensuring a seamless user experience across devices.

2.2.4 SQL

Structured Query Language (SQL) was employed for interacting with the project's relational database, specifically managing the learning hub's structured data such as employee records, quiz results, course materials, and engagement metrics. SQL allowed the team to efficiently query, update, and manage large datasets. Through JOIN operations, the team could link related tables such as employee data and learning outcomes, enabling detailed reports on user progress. SQL's transactional control features ensure data consistency, especially during operations like updating quiz scores or issuing rewards, where multiple tables are affected [16]. Moreover, the use of stored procedures streamlined complex database operations, improving performance by reducing network traffic between the application and database server.

2.2.5 MySQL Workbench

MySQL Workbench served as the graphical user interface (GUI) tool for managing the MySQL database, which was hosted on Google Cloud. This tool was instrumental in designing the database schema, which included tables for storing user profiles, learning materials, quiz results, and employee engagement data. MySQL Workbench facilitated database design through its visual entity-relationship diagrams (ERDs), making it easy to map out relationships between different data entities [17]. Additionally, the tool provided SQL query development, debugging, and performance tuning features, ensuring efficient database operations. MySQL Workbench's migration tools were also leveraged to seamlessly apply database schema updates during the development process, ensuring that the database structure evolved as new features were added.

2.2.6 node.js

Node.js, a JavaScript runtime built on Chrome's V8 engine, was the backend environment for the Ant HR Learning Hub. Node.js was chosen for its asynchronous, event-driven architecture, making it highly scalable and efficient for handling a large number of requests, especially in real-time applications like the learning hub. Node.js allowed the development team to use JavaScript on both the frontend and backend, creating a uniform codebase across the entire application stack [18]. Its non-blocking I/O model ensured that the application could handle multiple database requests simultaneously, a crucial feature when many employees were accessing learning materials or submitting quizzes concurrently. Additionally, Node.js's npm ecosystem provided access to numerous libraries and tools that streamlined backend development, such as authentication, logging, and data validation middleware.

2.2.7 express.js

Express.js, a minimal and flexible Node.js web application framework, was used to build the backend services that powered the Ant HR Learning Hub. Express was critical in setting up RESTful APIs that connected the frontend (React.js and Flutter) with the backend services. These APIs handled user authentication, course content delivery, quiz submissions, and data retrieval for employee engagement metrics. Express.js middleware such as body-parser and cors were utilised to parse incoming request bodies and handle cross-origin requests, ensuring secure and seamless data transmission between the client and server [19]. Additionally, the Express Router enabled the team to modularize different parts of the application (HRD, HRM, machine learning services), making the backend easier to maintain and scale.

2.2.8 Google Cloud

Google Cloud provided the infrastructure for hosting both the application and the MySQL database. The Google Cloud SQL service ensured that the database was scalable and secure, with automated backups and high availability features. This was crucial for the Ant HR Learning Hub, where data integrity and uptime were top priorities due to the sensitive nature of employee data and the importance of continuous access to learning resources. Google Cloud Storage was used for storing large learning materials, such as video content and documents, while Google Cloud Functions were considered for handling certain serverless tasks, such as processing quiz results and running periodic engagement reports [20]. Google Cloud's IAM (Identity and Access Management) services provided secure access control, ensuring that only authorised personnel could access and modify critical infrastructure and data.

2.2.9 GitHub

GitHub was the version control platform used to manage the project's codebase. GitHub allowed the development team to collaborate efficiently by tracking changes, reviewing code, and merging updates through pull requests [21]. This was especially important in a multi-developer environment where both the web and mobile applications were being developed in parallel. GitHub Actions were set up to automate tests and deploy the application to Google Cloud upon each code merge, ensuring continuous integration and delivery (CI/CD). Branching strategies like feature branching helped keep the main codebase stable while allowing for new features and bug fixes to be worked on independently.

2.2.10 Figma

Figma was the design tool used during the UI/UX design phase. Figma's cloud-based, collaborative platform allowed multiple team members to work simultaneously on designing wireframes, user flows, and high-fidelity prototypes for both the web and mobile applications. Figma's design systems feature enabled the team to create reusable components and styles, ensuring consistency across different parts of the application [22]. The interactive prototyping capabilities allowed the team to simulate user interactions with the platform, which helped gather feedback and make necessary improvements to the design before actual development began. Figma also facilitated handoff between designers and developers by providing specifications and CSS snippets directly from the designs.

2.2.11 Jira

Jira was the project management tool used to organise tasks and track progress throughout the development lifecycle. The team followed agile methodologies, using Jira's Scrum boards to manage sprints and assign tasks. Each task in Jira was linked to a specific feature or bug, ensuring accountability and transparency within the team. Jira's epics and user stories helped break down complex features into manageable tasks, such as implementing machine learning models or creating the reward system for the HRM component [23]. Burndown charts and other reports provided insights into team velocity, helping the project stay on track and meet deadlines. Additionally, Jira integrated with GitHub, automatically updating tasks when code was committed or pull requests were merged.

2.3 Related Research / Competing Solutions

2.3.1 UniCulture and UniHR Platform

Sections 2.3.1.1 to 2.3.1.4 look at research and comparisons related to the UniCulture and UniHR platforms. These parts review other employee engagement and HR management platforms—Happily, Bonusly, and Motivosity—which help show what features and ideas are commonly used in today's workplaces. By studying these platforms, we can better understand what helps improve employee happiness, performance, and overall company success.

This comparison also helps us see what UniCulture and UniHR do well and where they can improve. Learning about the strengths and weaknesses of each platform can guide how UniCulture and UniHR grow and become more useful for both employees and leaders in the future.

2.3.1.1 Happily

Happily is a cloud-based employee engagement and learning platform designed to streamline how organisations manage their relationship with employees. It covers everything from tracking employee progress,

feedback, and training to managing team communication and performance goals. In addition to employee-focused data, Happily also offers tools for managing information related to departments and team leaders overseeing employee development. This fosters stronger employee relationships and improves overall organisational performance.

Key features of the Happily platform include Engagement Tracking, which allows managers to monitor employee participation in tasks, training sessions, and internal discussions [24]. The Learning Hub provides access to personalised learning materials and tracks individual progress in skill development. Performance Management offers tools to set clear objectives, measure outcomes, and give continuous feedback to employees. The Feedback System encourages open communication by facilitating feedback between peers and supervisors. Additionally, the Analytics Dashboard offers real-time insights into employee engagement and performance, allowing for easy creation of detailed reports that can be accessed from anywhere. Together, these features help enhance the overall employee experience and support the organisation's growth.

2.3.1.2 Bonusly

Bonusly is an innovative employee recognition and rewards platform that enhances workplace culture and boosts employee engagement. By enabling peer-to-peer recognition and microbonuses, Bonusly fosters a positive work environment where employees feel valued and appreciated [25]. Its customizable features allow companies to tailor rewards and recognition programs to suit their unique needs. With integrations for various HRIS systems, communication tools, and mobile apps, Bonusly supports remote teams and global organisations. By leveraging social media integration and providing insights into team connections, participation trends, and recognition frequency, Bonusly helps companies build stronger, more cohesive teams. Overall, Bonusly serves as a powerful tool for improving employee satisfaction, retention, and productivity in medium to large corporations across various industries.

2.3.1.3 Motivosity

Motivosity is a cloud-based employee engagement and recognition platform designed to create a positive workplace culture by fostering peer-to-peer recognition, promoting employee well-being, and enhancing communication across teams [26]. The platform emphasises recognition and reward as central drivers of employee satisfaction and retention, providing tools that allow employees and managers to acknowledge and celebrate achievements regularly.

Key features of Motivosity include the Recognition Feed, where employees can give and receive kudos for completing tasks, reaching milestones, or demonstrating company values. This peer-to-peer recognition is reinforced by a point-based reward system, allowing employees to accumulate points for each recognition they receive. These points can then be redeemed for tangible rewards, motivating employees to consistently perform at their best.

The Leadership and Engagement Tools focus on strengthening connections between team members and management. Motivosity enables leaders to track engagement levels, identify disengaged employees, and offer support where necessary. Additionally, it provides tools for Team Communication and Feedback, encouraging transparency and fostering a culture of open communication. This feedback loop helps address concerns early and improves overall workplace morale.

The Insights Dashboard in Motivosity provides managers with real-time data on employee engagement, recognition trends, and well-being metrics. These insights allow organisations to take proactive steps in improving employee satisfaction and creating a more supportive work environment. By offering detailed analytics and

reporting, Motivosity helps organisations measure the effectiveness of their engagement and recognition programs, ultimately contributing to employee retention and productivity.

2.3.1.4 Summary

Table 2.1 The comparison between Happily.ai, Motivosity, Bonusly and UniHR

Functions	Happily.ai	Motivosity	Bonusly	UniHR
Daily Check-Ins	✓	✗	✓	✓
Start Essential Conversations	✓	✗	✗	✓
Daily Emotional Intelligence	✓	✓	✓	✓
Automated team reports every morning	✓	✗	✗	✗
Strategic HR & Leadership	✗	✓	✓	✗
Customization Options	✓	✓	✓	✓
Peer-to-Peer Recognition	✓	✓	✓	✓
Recognition coin	✓	✗	✓	✓
Social Media Integration	✗	✓	✗	✗
Peer Recognition	✗	✓	✓	✓
Custom Awards	✓	✓	✓	✓
Rewards System	✓	✓	✓	✓
Thank You Note	✗	✗	✓	✗
Automated Celebrations	✗	✓	✗	✗

From Table 2.1, it shows the comparison between Happily.ai, Motivosity, Bonusly, and UniHR highlights the distinct strengths and focus areas of each platform. Happily.ai focuses with its emphasis on employee well-being and emotional engagement, offering features like Daily Check-Ins, Daily Emotional Intelligence, and Automated Team Reports. These functions are designed to nurture emotional intelligence and provide consistent insights into team morale.

Meanwhile, Motivosity and Bonusly focus on fostering a social and recognition-driven culture. They are outstanding in features like Peer-to-Peer Recognition, Recognition Coins, and Rewards Systems, which encourage employee engagement through peer acknowledgment and gamified incentives. Their inclusion of Social Media Integration and Automated Celebrations further caters to companies that value a vibrant, socially connected workforce.

In contrast, UniHR takes a more strategic, HR-centric approach. It includes core features such as Daily Check-Ins, Essential Conversations, Emotional Intelligence Tracking, and Automated Reports, aligning with Happily.ai in terms of promoting employee well-being. However, UniHR distinguishes itself by incorporating Strategic HR & Leadership tools, which are absent in the other platforms. This makes UniHR a better fit for organisations that emphasise leadership development and structured HR management over peer-driven recognition.

However, UniHR lacks some of the customization and engagement features seen in Motivosity and Bonusly, such as Peer-to-Peer Recognition, Rewards Systems, and Social Media Integration. The absence of these features reflects UniHR's focus on leadership-driven tools and essential HR functions rather than social engagement or gamification. Moreover, all of these functions could be enhanced and improved on our platforms to be better in the future.

Overall, UniHR is ideal for organisations that prioritise leadership development and professional growth over informal recognition systems, making it a more structured, leadership-oriented solution compared to its more engagement-focused counterparts.

2.3.2 UniLearning Platform

Sections 2.3.2.1 to 2.3.2.3 look at research and comparisons related to the UniLearning platform. The platform is designed to support employee learning and development through a combination of modern educational tools, intelligent recommendations, and engaging user experiences. Built with both learners and HR professionals in mind, UniLearning aims to bridge the gap between skill acquisition and employee engagement in the workplace.

This section compares UniLearning with two widely used learning platforms—Kahoot! and Skooldio—to understand the current landscape of digital learning tools. Each platform offers unique strengths: Kahoot! focuses on interactive, gamified learning, while Skooldio provides structured professional development through expert-led courses. By examining their key features, limitations, and use cases, we can better understand how UniLearning brings together the best aspects of both systems while adding new capabilities such as machine learning-driven recommendations, skill mapping, and employee progress dashboards.

Through this analysis, UniLearning is positioned as a comprehensive solution that not only delivers engaging learning content but also supports long-term growth, motivation, and well-being within organisations.

2.3.2.1 Kahoot!

Kahoot! is a game-based learning platform that transforms traditional training and education sessions into interactive experiences. Widely used in both educational and corporate settings, Kahoot! engages users through live quizzes, surveys, and polls, encouraging active participation and knowledge retention. In a corporate environment, it is often used for onboarding, team-building activities, and microlearning sessions, making learning more enjoyable and impactful.

Kahoot! allows facilitators to create custom quizzes or use pre-existing templates to deliver content in an engaging format. Its live gameplay, immediate feedback, and leaderboard features make it effective for maintaining learner motivation. In addition, the platform offers reporting and analytics tools that enable trainers and HR professionals to assess knowledge gaps and participation levels, helping them refine future training content.

Despite its strong gamification aspect, Kahoot! is limited in scope when it comes to broader HR functions such as performance management or employee engagement tracking. It primarily focuses on making learning fun and engaging rather than offering a comprehensive employee development or feedback system. Nevertheless, its effectiveness in boosting short-term engagement and learning outcomes makes it a valuable tool for knowledge reinforcement and interactive learning.

2.3.2.2 Skooldio

Skooldio is a Thai-based digital learning platform that provides upskilling and reskilling solutions for professionals in areas such as technology, business, and design. It caters to both individuals and enterprises by offering curated courses, workshops, and learning programs led by industry experts. Skooldio's platform supports self-paced learning as well as live sessions, combining flexibility with expert guidance.

For organisations, Skooldio provides tailored learning paths and employee development programs aligned with company goals. This makes it an effective solution for internal training, helping businesses close skill

gaps and boost workforce capabilities. Key features include progress tracking, certification, and collaborative learning experiences that foster continuous development.

Unlike platforms focused solely on employee engagement or rewards, Skooldio's strength lies in skill development and structured learning. It lacks social recognition tools or emotional engagement features, but excels at delivering high-quality content and actionable skills for career growth. When integrated with employee development plans, Skooldio can be a powerful complement to broader HR and learning platforms.

2.3.2.3 Summary

Table 2.2 The comparison between Kahoot!, Skooldio, and UniLearning

Functions	Kahoot!	Skooldio	UniLearning
Gamified Learning Experience	✓	✗	✓
Video Course Library	✗	✓	✓
Live Quizzes and Polls	✓	✗	✓
Personalised Learning Paths	✗	✓	✓
Skill Mapping and Competency Tracking	✗	✓	✓
Quiz Variety (Timed, Topic-Based)	✓	✗	✓
Machine Learning Recommendations	✗	✗	✓
Admin & Manager Progress Dashboard	✗	✓	✓
Multi-Platform Support (Web & Mobile)	✓	✓	✓
Custom Course Upload (Admin Role)	✗	✓	✓

Table 2.2 presents a comparative analysis of three learning platforms: Kahoot!, Skooldio, and UniLearning, focusing on their functionalities and how well they support employee learning and development in organisational contexts.

Kahoot! is known for its gamified learning experience, primarily through live quizzes, polls, and interactive competitions. It effectively boosts short-term engagement but lacks deeper features such as skill tracking, personalisation, and management tools. It does not support structured video learning or personalised development paths, limiting its usefulness in long-term employee development strategies.

Skooldio, on the other hand, offers high-quality video courses and structured learning paths, making it well-suited for professional upskilling and reskilling. Its strength lies in content quality and learner progress tracking, particularly useful for technical and business training. However, it does not provide gamified engagement, mental health tracking, or built-in reward systems, which can affect motivation and sustained participation over time.

UniLearning, as designed in this project, combines the strengths of both platforms while addressing their limitations. It supports gamified learning through quizzes and reward systems (such as coins and badges), a personalised video course recommendation engine powered by machine learning, and comprehensive dashboards for both admins and managers. Additionally, UniLearning introduces skill mapping, competency tracking, and morale assessment tools, making it not only a learning platform but also a strategic HRD tool that supports employee well-being and long-term development.

Overall, UniLearning offers a more integrated and employee-centered approach, suitable for organisations looking to foster both skill growth and positive workplace culture through a combination of learning, engagement, and intelligent analytics.

CHAPTER 3 SYSTEM DESIGN

This chapter provides a detailed explanation of the methodology and design for developing the website and application for Employees' Engagement and Learning Hub. This comprehensive system architecture will be explained in detail, with a particular emphasis on the key components and their functionalities. Furthermore, a detailed discussion will be provided regarding the functions related to Human Resource Management and Human Resource Development.

3.1 System Overview

This project is focusing on the improvement of Human Resource Management (HRM) and Human Resource Development (HRD) which deliver users to function on the usability of these functions. On the mobile application there will be 2 functionalities available, there are morale daily and yearly (user side) and 360 assessment (user side).

3.1.1 Human Resource Management

The system has three primary user roles: Admin, Manager, and Employee. Admins have full control over the organisation, with the ability to view and manage all activities, teams, and processes, including user management and system settings. Managers oversee specific teams or departments, allowing them to view team activities, track performance, and access relevant reports. Employees, as end-users, utilise the system's core functions to complete their tasks, interact with team members, and receive work-related updates and notifications. There are 3 websites including Uniculture, UniHR and Unilearning.

The Human Resource Management (HRM) module covers Morale Daily, Morale Yearly, and Assessment 360, and is integrated into the Uniculture platforms. These platforms cater to three distinct user roles, each with tailored access levels and functionalities aligned with their specific responsibilities.

1. **Admin:** Administrators manage the assessment process, including setting up and managing assessments, configuring questions, managing system information, and exporting reports. They have access to comprehensive administrative features specific to their role.
2. **Manager:** Managers, like employees, can answer and view assessments. Additionally, they have access to Morale Daily, Morale Yearly, and Assessment 360 on the Uniculture website, enabling them to oversee and analyze team performance.
3. **Employee:** Employees can answer and view assessments, participating in Morale Daily, Morale Yearly, and 360 assessment evaluations on the Uniculture website.

Moreover, the Human Resource Management (HRM) module will also include the Career pathway features within the UniHR platform. The function will be available exclusively to the Admin role. On the mobile application, only the functionalities of employees will be available.

- **Career Pathway:** This feature helps the admin identify users with the potential to grow in specific pathways by analyzing both their strengths and weaknesses. It includes the required skills, the estimated time needed for development, and career path suggestions provided by their leader.

In summary, the Human Resource Management system will ensure efficient tracking and analysis of morale trends and employee performance across the organization. The three main feature include managing morale daily and yearly question form, dashboard, and the assessment 360.

3.1.1.1 Morale Daily and Yearly Question Forms

The system is designed to manage daily and yearly morale question forms for organizations, enabling admins, managers, and users to interact with the forms effectively. Its main features include:

- **Question Form Management:** The system allows admins to create, delete, edit, duplicate, and assign both daily and yearly question forms to employees or specific departments. These forms can be customized with detailed attributes, including answer types, topics, text, morale type, pictures, and descriptions. Admins can assign start and end dates to control form availability, and forms can be saved in draft or published states. A preview feature lets admins view the forms from the employee perspective, ensuring clarity and usability. Additionally, the system enables admins to filter and view question forms based on their state, type, or topics, facilitating efficient management.
- **Response Tracking and Analytics:** Admins can track participation and analyze responses through detailed progress displays for each question form. Progress bars show the completion rate, while participant counts offer a clear view of engagement levels. The system provides key metrics such as average scores, best ratios, and overall satisfaction levels to evaluate performance. Admins can also access and analyze responses, organized by questions and participants, for in-depth insights into morale trends and feedback.
- **User and Manager Interaction:** The system empowers users to answer and submit question forms within the specified timeframe, ensuring timely feedback. For yearly forms, users can add additional comments to provide richer insights. Users are rewarded with coins for successfully completing daily question forms, fostering consistent engagement. Managers can review form details, and assess the performance of their teams, enhancing their ability to address morale-related concerns effectively.

3.1.1.2 Morale Daily and Yearly Dashboard

The dashboard provides a comprehensive platform for admins and managers to monitor and evaluate work satisfaction across multiple companies. Its main features include:

- **Visualization of Work Satisfaction Metrics:** Offers clear, data-driven insights into the overall happiness and satisfaction levels within each company, using visual elements like charts, graphs, and heatmaps.
- **Company Happiness Indicator:** Displays a summarized status for each company, highlighting whether the organization is classified as “happy” or “unhappy” based on satisfaction scores.
- **Detailed Analytics:** Allows admins to dive deeper into specific factors, such as Brand, Supervision, and Empowerment / Autonomy.

3.1.1.3 Assessment 360

- **Assessment Management:** The system allows users to perform various actions related to the assessment including creating, deleting, duplicating and assigning assessments to employees. It supports managing appraisers, including internal staff and external contributors, ensuring evaluations are fair and relevant. Appraiser assignments are tracked and organized for clarity. Moreover, there will be a function to only show the assessment follows the logic; All, Active, In-active.
- **Assessment 360 Answer Display:** The system provides a detailed view of assigned assessments, displaying each question’s score and the overall average results. Data is visually represented through

charts and graphs, offering an easy-to-understand summary of performance. Additionally, a progress bar tracks completion rates, showing how many appraisers have responded and who still needs to provide feedback. The display includes a question list and a participant list, making it easier for admins to manage and review responses effectively. This organized layout ensures efficient tracking and analysis of assessment progress.

On the mobile application, the assessment 360 will be available for only the employees or user role only and can only do the assessment.

3.1.2 Human Resource Development

The system is designed to support three primary user roles: Admin, Manager, and Employee. Each role is equipped with specific functionalities and access tailored to their development-related responsibilities. The HRD module will be implemented across the Unilearning platforms to facilitate employee growth, training, and skill enhancement.

1. **Admin:** Administrators oversee the entire HRD process, including designing and managing training programs, tracking employee progress, and configuring development goals. They can manage system information, generate detailed reports, and ensure alignment with organizational objectives.
2. **Manager:** Managers can monitor the development progress of their teams, assign training sessions, and review performance metrics. They are responsible for encouraging team participation in development activities and accessing reports to track team improvement.
3. **Employee:** Employees utilize the system to engage in learning opportunities, track their own development progress, and complete assigned training programs. They can access resources, provide feedback, and participate in organizational growth initiatives.

In summary, the Human Resource Development system support employee growth through modules for collectively enhance employee learning, facilitate role-specific development, and ensure alignment with organizational goals. The primary features include quiz management, video learning, video course suggested by AI, competency, and skill mapping.

3.1.2.1 Quiz Management

- **Quiz Management:** Admins can create, edit, and delete quizzes, assign them to specific employees or groups, and set time limits or other parameters. The system should allow categorization of quizzes based on skills, topics, or roles, ensuring relevance.
- **Quiz Display:** Employees and Managers can view quizzes assigned to them through an intuitive interface. This includes options to take quizzes, view progress, and access results upon completion. The display should be user-friendly and accessible on multiple devices.

3.1.2.2 Video Learning

- **Video Learning:** Users can access a variety of video content through the learning platform to enhance their knowledge and abilities. Admins can upload and categorize videos based on topics, skills, or roles to ensure content relevance. The system should support video playback on multiple devices and allow users to track their viewing progress.

3.1.2.3 Video Course Suggested by AI

- **Video Course Suggestion:** The platform provides users with personalized video course recommendations designed to address individual skill gaps. Leveraging machine learning algorithms, the system analyzes skill gaps. Based on this analysis, it suggests targeted courses that align with the user's current role, career goals, and learning preferences. This machine learning recommendation system ensures a more effective and relevant learning experience, helping users grow professionally and stay competitive in their fields.

3.1.2.4 Competency and Skill

- **Competency:** Users can view their current level for each defined competency. The system should present competency ratings in a clear and visual format, based on assessments or completed learning activities. Admins and Managers can also track team competency trends for development planning.
- **Skill:** Users can view their proficiency level for each listed skill. The platform should allow filtering by skill categories and display progress over time. This helps users identify areas for improvement and align their learning with required job roles or career growth paths.

3.1.2.5 Skill Mapping

- **Skill Map Calculation:** The system calculates skill levels based on quizzes, performance data, and self-assessments. It compares these against required skill benchmarks for roles or projects, identifying strengths and gaps.
- **Skill Mapping Display:** A graphical or tabular interface provides a clear overview of skill alignment, highlighting areas of proficiency and development needs. This feature should allow Admins and Managers to filter by team, individual, or skill category for detailed insights.

3.2 System Requirement and List

The system requirements are structured to provide role-specific functionalities, ensuring that each user has access to features aligned with their responsibilities. The Human Resource Management System includes three primary features: Morale Daily, Morale Yearly, and Assessment 360. These features are categorized and tailored to the needs of three distinct roles: Admin, Manager, and Employee. Each role is granted specific access privileges based on its scope of responsibilities, enhancing usability and security.

3.2.1 Human Resource Management (HRM)

Morale Daily

- **Daily Question Form (Admin role)**
 - The system must allow users to create question forms, including the answer type, question topic, and question text.
 - The system must allow users to assign a specific number of coins to each question form.
 - The system provides a preview of question forms from the employee's perspective.
 - The system must allow users to save question forms to the activated state.
 - The system must allow users to publish question forms.
 - The system must allow users to set start and end dates for question forms.

- The system must allow users to assign question forms to all employees or specific departments.
- The system must allow users to store and retrieve question forms for later use.
- The system must allow users to view and filter question forms by state and question topics.
- The system displays question form details, including question text and a progress bar showing the number of participants for each form.
- The system must allow users to view all answers by respondents for each question form.
- The system displays the correct average score, best ratio for each question form, and the overall score.

- **Daily Question Form (Manager and User role)**

- The system must allow users to answer question forms.
- The system must allow users to submit answers while the question form is still within the available date range.
- The system must allow users to submit completed answers.
- The system must allow users to view detailed information in each question form.
- The system gives the coins to users after successfully completing a question form.

- **Daily Dashboard Enhancement (Admin role)**

- The system allows the admin to select a specific year and view corresponding daily data.
- The system must be ensured that the filter dynamically updates all dashboard components within the page.
- The system allows identifying and rectifying any inaccuracies in the existing data calculations, as well as validating and standardizing the formulas used for metrics to ensure consistency and accuracy across all visualizations.
- The system allows adding a gauge chart to display the percentage of employees who gave a satisfactory level.
- The system allows the use of clear visual indicators, such as feeling icons (e.g., low, medium, and high satisfaction levels), to enhance the clarity of the chart.
- The system provides a top 3 factors table to highlight the three highest-scoring factors contributing to employee satisfaction.
- The system provides a bottom 3 factors table to display the three lowest-scoring factors that need attention, such as issues in specific teams or recurring complaints.
- The system must provide the average satisfaction scores or performance metrics for each department through Department-Level Scores.
- The system must provide the average satisfaction scores or performance metrics for each section through section-Level Scores.
- The system ensures that the data funnel is organized in a logical flow to maintain coherence.
- The system ensures the readability of the data funnel, making it easy for users to interpret and understand.
- The system must display the open-ended answers for users.

- The system must be able to determine the percentage of satisfaction in open-ended answers.

Morale Yearly

- **Yearly Question Form (Admin role)**

- The system must allow users to create question forms, including picture, name, description, code, start date, end date, morale type, question type, question topic, and question text.
- The system must allow users to assign question forms to all employees in the company or specific departments.
- The system must allow users to set the status of question forms.
- The system must allow users to manage question forms by adding, editing, or removing them.
- The system must allow users to manage individual questions by adding, editing, duplicating, or deleting them.
- The system must allow users to view and filter question forms based on type and status.
- The system provides details and a progress bar to show the number of participants for each question form.
- The system must allow users to view all answers by question list and respondent list for each topic.

- **Yearly Question Form (Manager and User role)**

- The system must allow users to answer question forms.
- The system must allow users to submit answers while the question form is still within the available date range.
- The system must allow users to write additional comments for each question.
- The system must allow users to submit answers and additional comments.
- The system must allow users to view and filter question forms based on their status.
- The system must allow users to view details for each question form.

- **Yearly Dashboard Enhancement (Admin role)**

- The system provides a functional filter button to allow users to sort and customize the displayed data based on specified criteria (e.g., date range, category, region).
- The system's appealing dashboard design enhances user experience and accessibility.
- The system is going to optimize the layout for ease of navigation, ensuring all key elements are clearly visible and appropriately categorized.
- The system must verify and rectify inaccuracies in existing data visualizations.
- The system has to validate all calculations to ensure precision and consistency across the dashboard.
- The system provides a table to display the three highest-performing factors affecting the overall outcomes.
- The system provides a table to display the three lowest-performing factors affecting the overall outcomes.
- The system must display the open-ended answers for users.

- The system must be able to determine the percentage of satisfaction in open-ended answers.
- The system provides a manager's morale yearly dashboard: the manager's yearly dashboard will present relevant data and insights customized to the manager's scope of responsibility, focusing on a specific company. Unlike the admin's dashboard, the manager's dashboard will restrict visibility and functionality to ensure data privacy and relevance.

Assessment 360

- **Assessment 360 (Admin role)**

- The system must allow users to create assessments that include the name, description, question type, start date, end date, and assign to all the employees.
- The system must allow users to manage the appraisee and appraiser, including the outsider.
- The system must allow users to manage assessment questions, including adding, editing, duplicating, and removing questions.
- The system must allow users to view and filter assessments based on status.
- The system must allow users to set deadlines and reminders for completing assessments.
- The system must allow admins to view all the answers by the question list and people list.
- The system must allow admins to view the progress of the completed process.
- The system must provide a dashboard to display an overview of assessment progress.
- The assessment overview display must include details such as participant feedback, scores, completion rates, and graphical visualizations.
- The system must send the assessment to the outsider through email.

- **Assessment 360 (Manager and User role)**

- The system must allow participants to answer assessment questions.
- The system must allow users to view assessments and their progress through the dashboard.
- The system must allow the users to answer the question while still in the available date.
- The system must show the correct average score for each question and also the overall score.

- **Career Path (Admin role)**

- The system must be able to determine which career path the user can pursue.
- The system must allow admins to create forms for users to collect their interested career pathways.

- **Career Path (Manager role)**

- The system must provide a form for managers to fill out in order to suggest career paths for employees. For example, a manager may suggest that an employee has the potential to become a DevOps engineer rather than a web developer.
- The system must provide a form for managers to fill out in order to collect data on the desired career paths.
- The system must be able to determine which career path the user can pursue under the supervision of the responsible company.

- **Career Path (User role)**

- The system must provide a form for users to fill out in order to collect the data on the desired career paths.

3.2.2 Human Resource Development (HRD)

Quiz (Admin role)

- **Lesson Quiz**

- The system allow user to create quiz

- **Rush Quiz**

- The system must be able to create quizzes with multiple question types.
- The system must support customizable time limits and point settings for each question.

Skill Mapping (Admin role)

- The system must display which users are interested in each lesson.

Quiz (User role)

- **Lesson Quiz**

- The system must be engaging enough to keep users consistently using it.

- **Rush Quiz**

- The system must respond to users in real time.

Video Course Learning

- The system must allow users to view all available video courses on the platform.
- The system must allow users to filter and search video courses by category and name.
- The system must track and display the user's course progress, including videos started, in progress, and completed.
- The system must allow users to resume video courses from where they left off.
- The system must support video playback across multiple devices and browsers.
- The system must display recommended courses based on user interests, role, or skill gaps. (Machine Learning)
- The system must allow managers and admins to track video learning engagement and progress of their team members.

Competency and Skill

- The system must allow users to view all defined competencies within the organization.
- The system must display each competency with a description, related skills, and proficiency levels.
- The system must allow users to take competency tests to assess their current level.

- The system must update the user's competency level based on test results or completed learning activities.
- The system must allow admins to view competency data for individual users or teams.
- The system must display visual indicators (e.g., charts or progress bars) to show competency levels and gaps.
- The system must allow users to view all available skills relevant to their role or department.
- The system must display each skill with a description and related competencies or learning resources.
- The system must show the user's current skill level.
- The system must allow filtering or searching of skills by category.
- The system must allow admins to create, edit, and manage the skill list.
- The system must allow admins to track skill development and identify training needs.

Skill Mapping (Admin role)

- The system must be able to suggest to admins which career path users is suitable based on interest, weakness, strength of the users
- The system must apply a penalty (from weakness) to the similarity score based on how many user weaknesses overlap with required skills in the career.
- The system must identify and list matched interests between the user and the career's job category.
- The system must return the top N career recommendations based on the final suitability scores, where N is configurable (default is 3).

Skill Mapping (User role)

- The system must be able to map the interested skills to the provided lessons within the platform.

3.3 Workflow

According to our schedule, the implementation of the Human Resource Management (HRM) system will begin this semester and continue into the beginning of Semester 2. The Human Resource Development (HRD) system will be implemented in the following semester. The workflow for the HRM system is outlined as follows.

- **Morale Daily and Yearly**

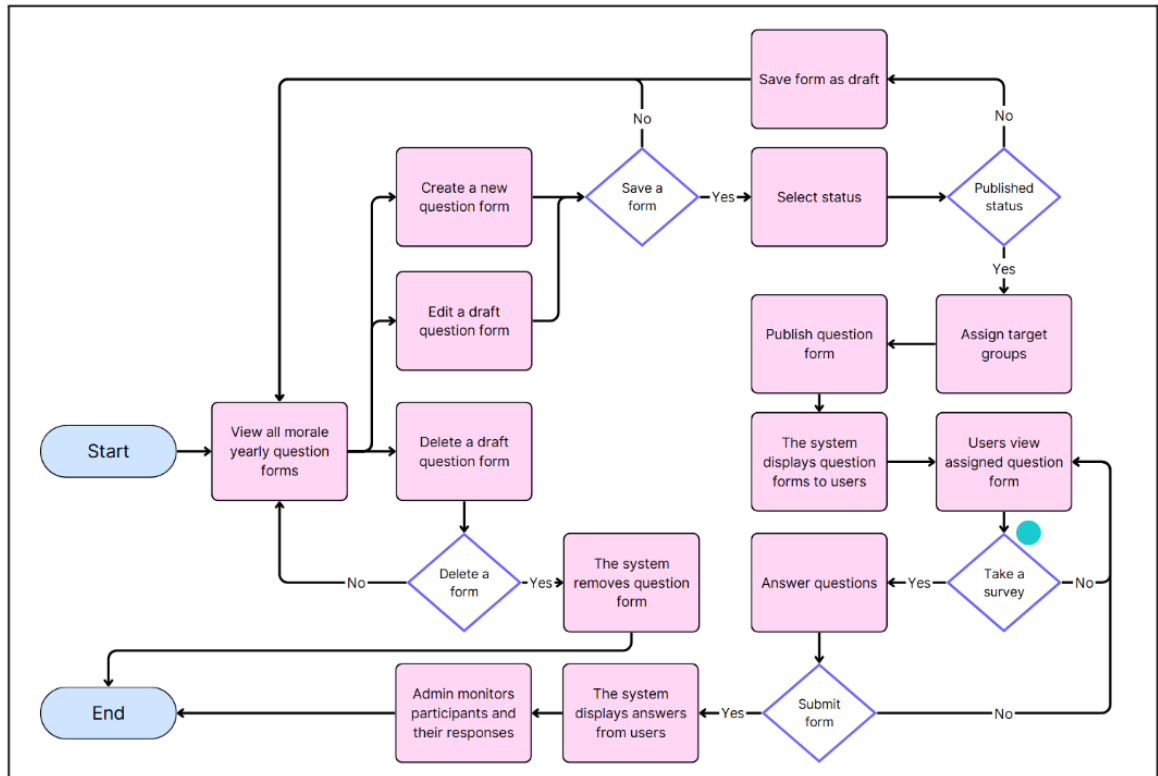


Figure 3.1 System workflow for morale yearly question form

In Figure 3.1, the system workflow for morale yearly question forms begins with the admin creating the form. In this step, the admin inputs key details such as the title, description, and associated topics, and may upload a relevant morale image. The admin then proceeds to add or modify questions, selecting question types and topics to align with organizational goals.

Next, once the form is created, the form enters the publishing phase, where the admin either publishes the form to make it live or saves it as a draft for further refinement. If selected publish, the admin specifies the target audience (e.g., departments or employees). After publishing, the system notifies participants about the assigned form and tracks their participation. Participants can accept or decline the form; if accepted, they proceed to complete and submit their responses. Finally, the system collects and analyzes responses, providing the admin with insights such as completion rates, individual scores, and overall trends. These results are available for export to support decision-making or presentations.

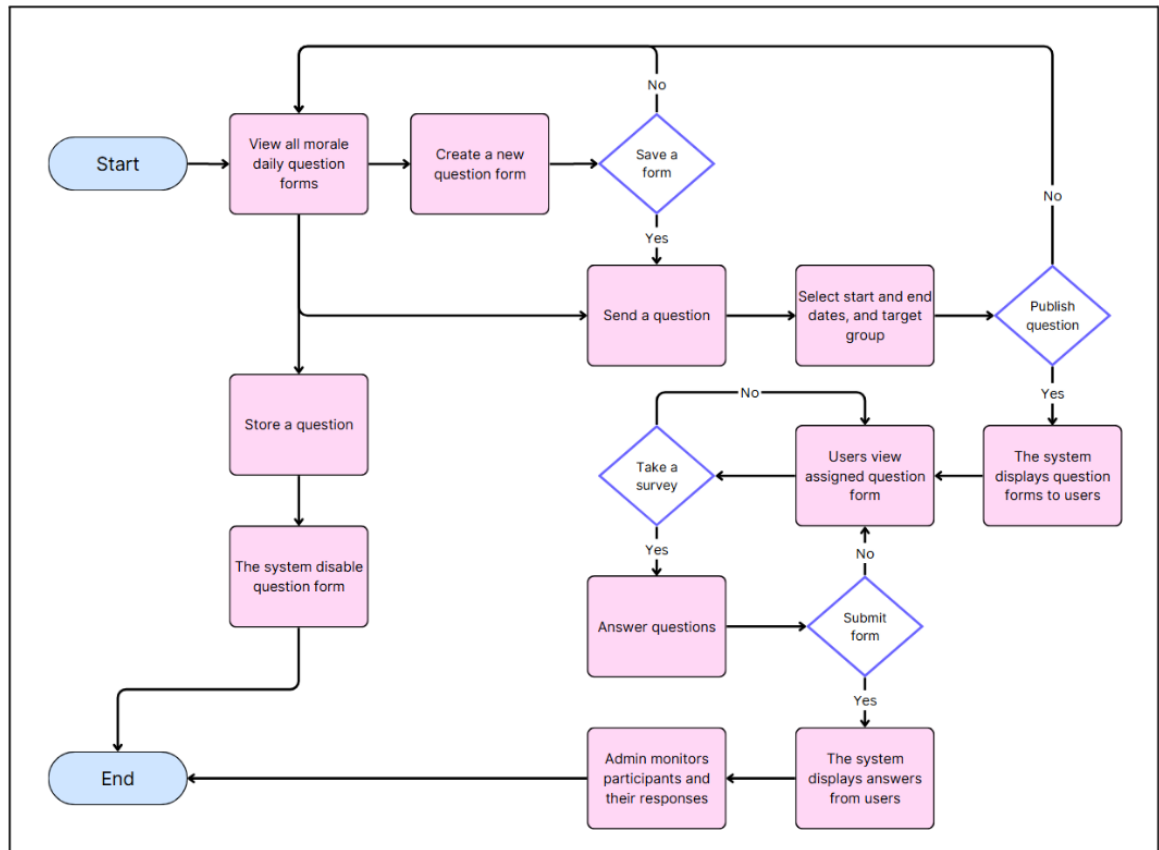


Figure 3.2 System workflow for morale daily question form

In Figure 3.2, the system workflow for morale daily question forms starts similarly with the admin creating a form. Daily forms focus on capturing quick morale checks through concise questions. The admin selects the question type, text, and topic, and assigns reward coins to incentivize participation.

Once the form is ready, the admin publishes or stores it. If a question is sent for publishing, the admin then configures the form, sets activation dates, and assigns it to the relevant audience. Published forms are accessible to participants during the specified timeframe. Participants receive notifications and can view the form on their dashboard. They have the option to accept or decline the form; if accepted, participants answer the questions and submit their responses. The system tracks these submissions and provides the admin with real-time progress tracking through visual progress bars and participation metrics. After the submission phase, the admin can access aggregated results and trends, supporting daily morale evaluations and engagement strategies.

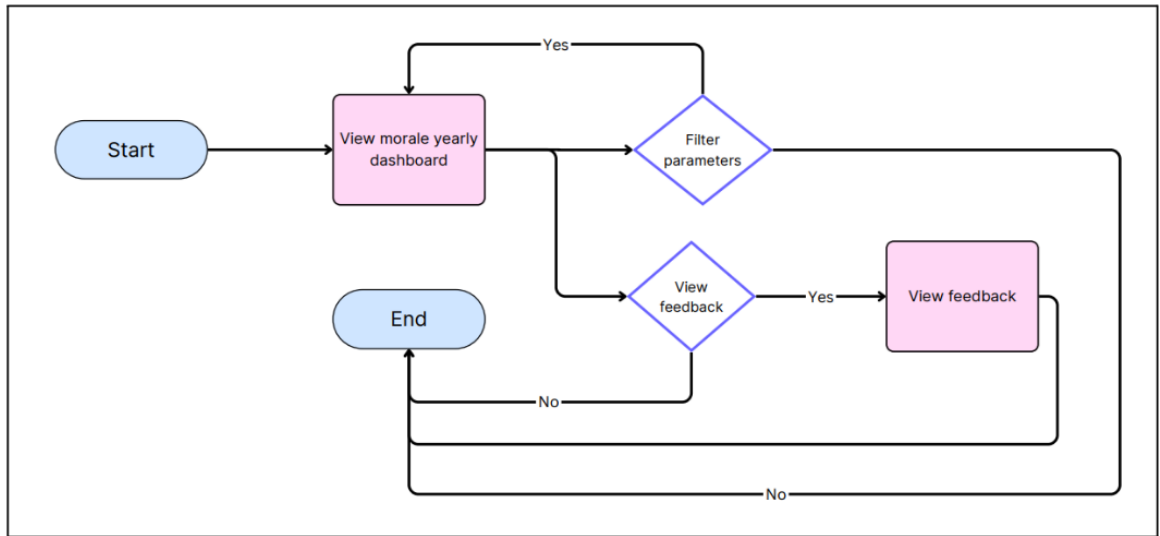


Figure 3.3 System workflow for morale yearly dashboard

In Figure 3.3, the system workflow for the “Morale Yearly Dashboard” begins with users accessing the main dashboard, which displays yearly morale data as the central hub for detailed insights. Users can refine the data by applying various filters, and the system updates the dashboard accordingly. At a decision point, users can choose whether to view specific feedback. If they opt to do so, they proceed to view the detailed feedback, after which they return to the dashboard to either continue exploring or apply further refinements. The workflow concludes when no additional actions are needed, allowing the user to exit the process.

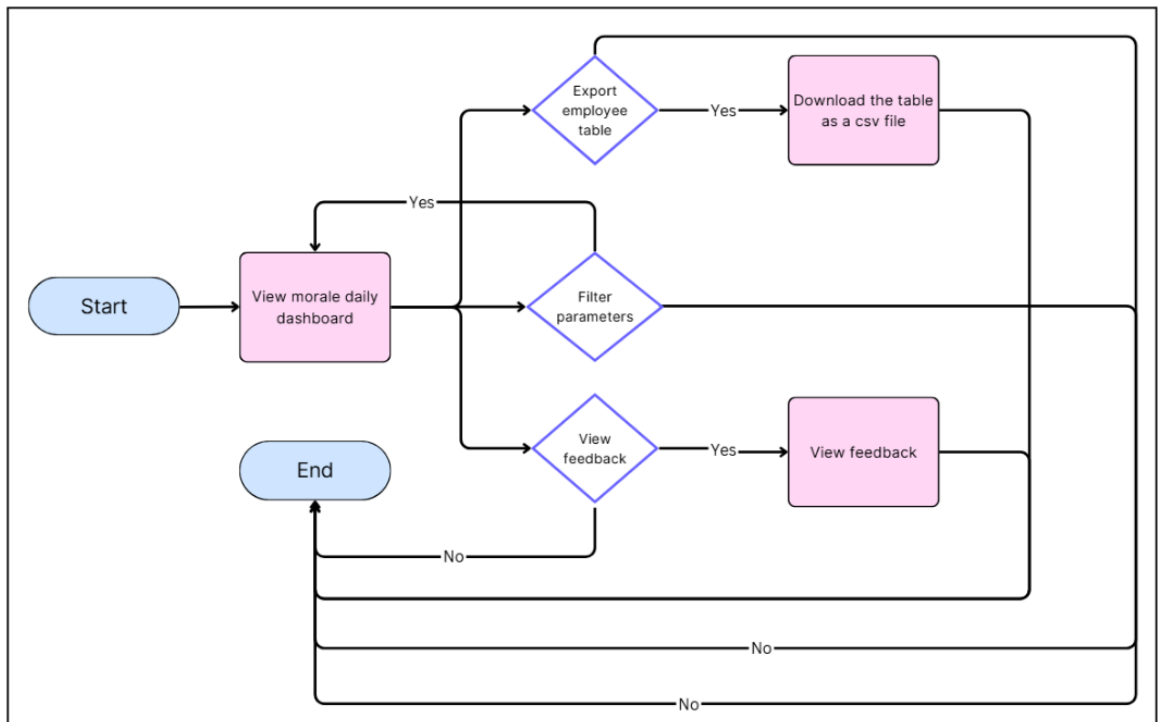


Figure 3.4 System workflow for morale daily dashboard

In Figure 3.4, the diagram illustrates the system workflow for the Morale Daily Dashboard, outlining the steps users follow to interact with the dashboard and gain insights. The process starts with users viewing the main dashboard, which displays daily morale data for employees. If users choose to export the data, they can download the employee table in CSV format before returning to the dashboard. Users can further refine the displayed data by applying various filters, prompting the system to update the dashboard accordingly. Additionally, users can view detailed feedback on daily morale reports, returning to the dashboard after reviewing it. The workflow concludes when the user decides to exit, having completed all desired actions.

- **Assessment 360**

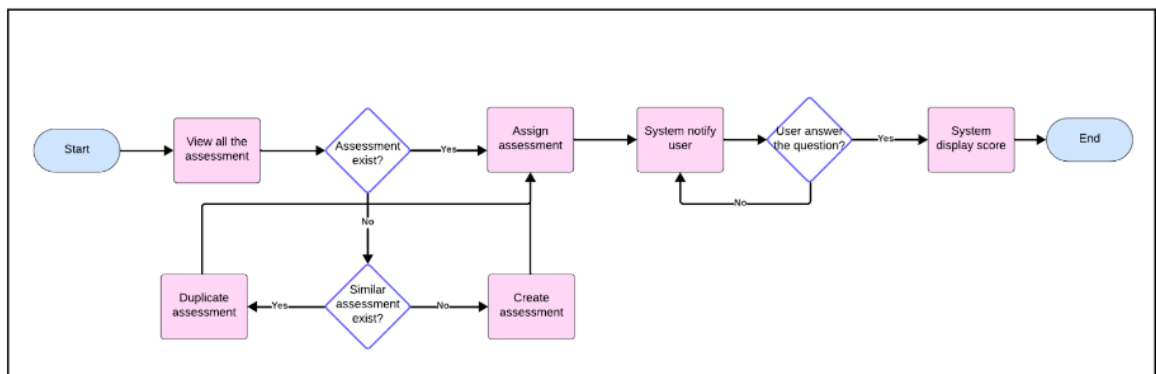


Figure 3.5 System workflow for assessment 360

In Figure 3.5, the system workflow illustrates the process flow within the Assessment 360 platform. The workflow begins by displaying all available assessments to the users. Next, the system allows for the creation or duplication of assessments before they are assigned to the relevant individuals. Once an assessment is assigned, the system notifies the users to complete it. After the users submit their responses, the system processes the answers and calculates the scores. Finally, the system displays the assessment results, completing the process. This streamlined workflow ensures that each step is clearly defined and facilitates a smooth user experience throughout the assessment cycle.

• Rush Quiz

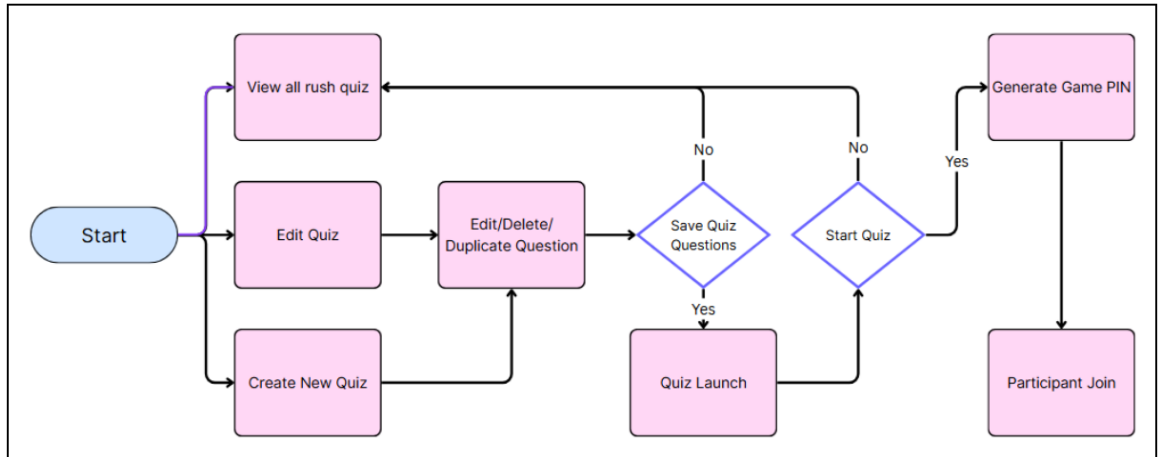


Figure 3.6 System workflow for rush quiz

In Figure 3.6, in the system workflow for Rush Quiz, represents the process of managing and conducting a quiz session. It has three main actions such as view all rush quizzes, edit an existing quiz, or create a new quiz. If the user chooses to create a new quiz or edit an existing one, they are directed to a stage where they can edit, delete, or duplicate questions within that quiz. After modifying the quiz questions, the system checks whether the user wants to save the quiz questions. If the user selects “No”, they are returned to the view all quizzes page. If “Yes”, the process moves to quiz launch, indicating that the quiz is ready to be activated.

Following the launch, the system prompts the user with another decision: whether to start the quiz. If the user chooses not to start it immediately, they are taken back to view all available quizzes. If they choose to proceed, the system will generate a game PIN, a unique code that allows participants to access the quiz. Once the PIN is generated, participants can use it to join the quiz session. This structured flow ensures that the quiz creator has full control over quiz preparation and that participants can easily join a live session through the generated game PIN.

• Video Course Learning

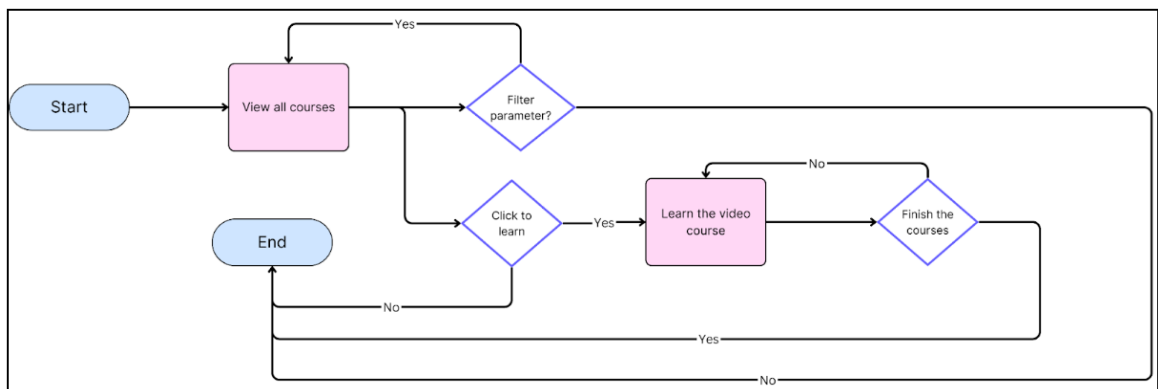


Figure 3.7 System workflow for Video Course Learning

In Figure 3.7, in the system workflow for video course learning, users can view all available courses and filter them based on parameters such as course name. Upon selecting a course, the system navigates them to the course page to start learning. As they progress, the system tracks their progress, displaying a progress bar to indicate how much of the course has been completed. If the user has not finished the course, they can continue watching until completion. Once the course is completed, the system updates the status to “Completed” and directs the user to the end of the course, offering additional steps like a certificate or feedback. If the course is not finished, users are prompted to continue learning until they complete it. This workflow ensures a smooth learning experience, guiding users from course selection to completion while encouraging continued engagement.

- **Competency and Skill**

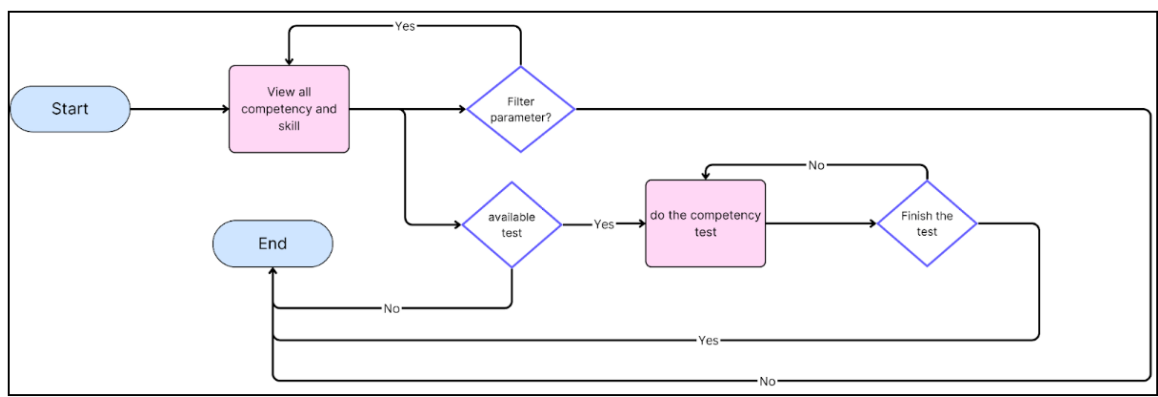


Figure 3.8 System workflow for Competency and Skill

In Figure 3.8, users can view a list of all available competencies and skills. If a competency test is available, the user can proceed to take the test. Upon completion of the tests, the system tracks the user’s progress and updates their competency status. Once the test is finished, the user can review their results or move on to the next available competency, ensuring a streamlined process for assessing and improving their skills. This workflow ensures users can easily navigate through the competency tests and track their progress toward skill development.

3.4 Use Cases Diagram and Narrative

• Morale Daily and Yearly Question Form

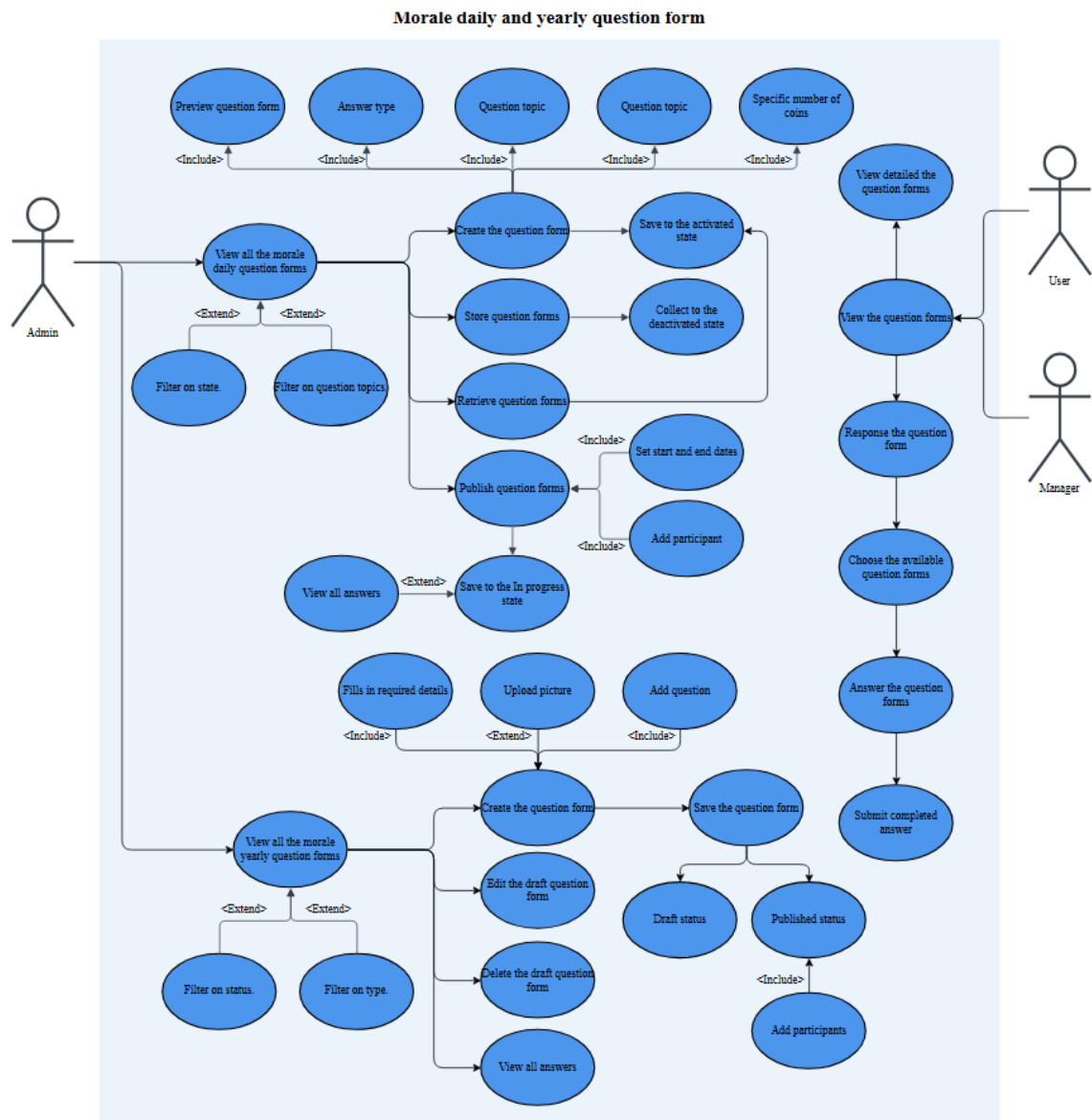


Figure 3.9 Use case diagram of morale daily and yearly question form

In Figure 3.9, a use case diagram for morale daily and yearly question form represents the interaction between different actors (Admin, User, and Manager) and the system's functionalities. The Admin plays a central role in managing the lifecycle of question forms, starting from creating, editing, and saving them. They can save the forms in different states, including draft or published, and manage the morale daily form to activate, deactivate. Admins are also responsible for setting start and end dates for the forms and assigning specific participants to ensure targeted distribution. On the other hand, users and managers interact with the system by viewing the available question forms that the admin has published. They can select forms, answer the questions, and submit their responses. Their engagement is an essential input for analyzing morale across the organization.

• Morale Daily and Yearly Dashboard

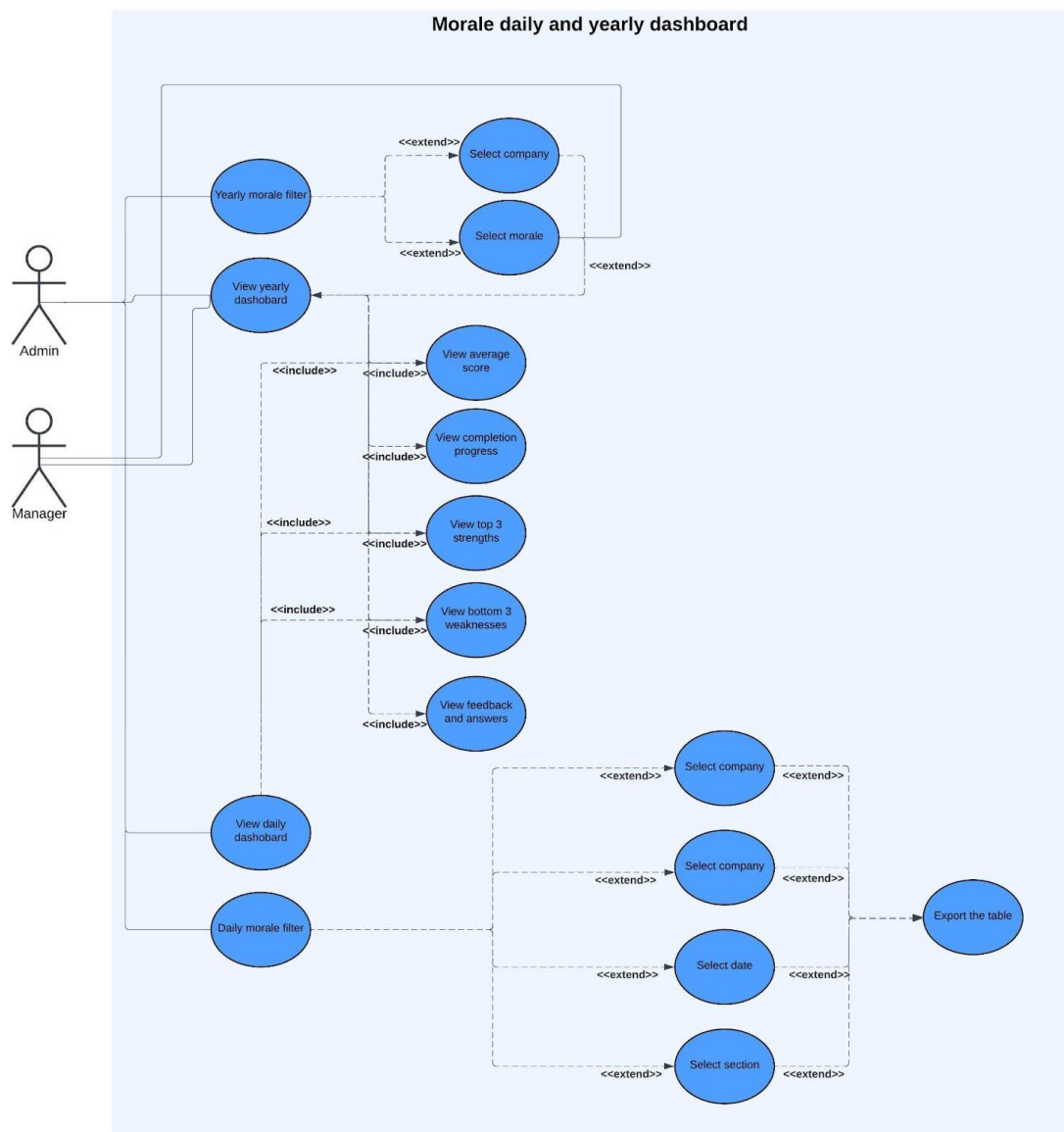


Figure 3.10 Use case diagram of morale daily and yearly dashboard

In Figure 3.10, the diagram illustrates the interactions between the Admin and Manager actors and the various use cases or features of the dashboard system. It showcases the different capabilities and analysis options available to the users, such as selecting companies, metrics, viewing performance metrics, trends, progress, and identifying anomalies.

• Assessment 360

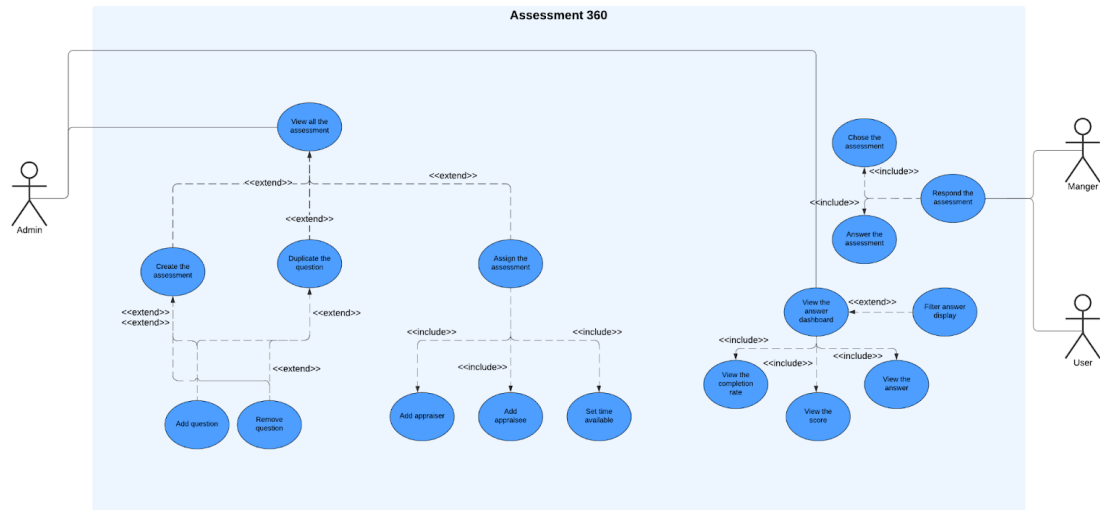


Figure 3.11 Use case diagram of Assessment 360

In Figure 3.11, a use case diagram for the assessment 360 is presented, providing a comprehensive overview of the key functionalities and interactions within the platform. The diagram showcases various actions and interactions between users and the system, illustrating the different roles and their specific tasks. Each use case in the diagram is paired with a detailed narrative description, offering a clear explanation of the actions and processes involved.

• Video Course Learning

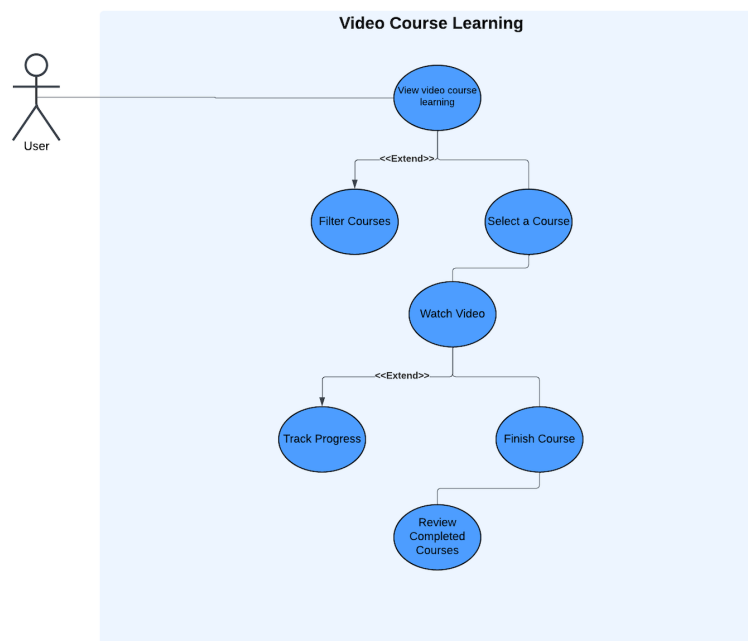


Figure 3.12 Use case diagram of Video Course Learning

In Figure 3.12, a use case diagram for the Video Course Learning is presented, providing a comprehensive overview of the key functionalities and interactions within the platform. The use case for video course learning where they can view a list of available video courses. The user has the ability to filter the courses by parameters such as name. After selecting a course, the user starts watching the videos, and their progress is tracked by the system. If the user hasn't completed the course, they can continue from where they left off. Upon finishing the course, the system marks it as completed, and the user can review their progress or receive certificates and feedback. If there are issues such as video playback errors or unavailable courses, the user is notified and may be directed to alternative actions. Overall, the system ensures that users can seamlessly access, learn, and track their progress through video courses.

Use Case Narrative

Tables 3.1 to 3.13 provide a use case narrative of the system's functionalities, encompassing daily and yearly question forms (Table 3.1 to 3.5), dashboards (Table 3.6 to 3.8), and 360 assessments (Table 3.9 to 3.12). These tables highlight how the system enables admins to create, edit, and publish morale forms, manage participant assignments, and analyze engagement through dashboards. The Assessment 360 assessments further enhance the system's capabilities by facilitating detailed performance evaluations through multi-rater feedback, providing a structured and holistic approach to assessing individual or team performance. Collectively, these tables demonstrate that empower organizations to track, measure, and enhance morale and performance in a streamlined and efficient manner.

• **Morale Daily and Yearly Question Form**

Table 3.1 Use case narrative for create question forms

Use case name	Create question forms
Actors	Admin
Goal	To design and save morale question forms for daily or yearly use.
Preconditions	1. The admin is logged into the system. 2. Relevant topics and types for the questions exist in the database.
Main success scenario	1. Admin navigates to the “Create Question Forms” page. 2. Admin fills in required details, such as question type, topic, text, and optional images. 3. For daily form, admin assigns a specific number of coins for participation. 4. For the daily form, admin previews the form from the user’s perspective. 5. Admin clicks “Save” to store the question form. For yearly form, admin can select status “Draft” or “Publish” to make it active. 6. System navigates admin to the recent question forms page.
Extension (a)	5a. If the admin attempts to save without required fields. 6a. The system displays an error and prompts completion. 7a. Return to step 2
Extension (b)	5b. If the admin closes the page before saving. 6b. The system prompts for confirmation to discard changes. 7b. Return to step 6
Postconditions	The question form is stored in the system in either draft or active status.

Table 3.2 Use case narrative for configure and assign morale daily question forms

Use case name	Configure and assign morale daily question forms
Actors	Admin
Goal	To configure and assign question forms to specific groups.
Preconditions	1. At least one question form is saved in the system. 2. Admin access to configure and assign forms.
Main success scenario	1. Admin selects a question form to configure. 2. Admin sets the start and end dates. 3. Admin selects the target group (company or departments). 4. Admin saves the configuration. 5. System navigates admin to the recent question forms page.
Extension (a)	2a. If the admin sets an invalid date range. 3a. The system prompts for correction. 4a. Return to step 2
Extension (b)	3b. If the admin selects no departments or groups. 4b. The system prompts for at least one target group. 5b. Return to step 3
Postconditions	The question form is configured and assigned to the selected groups with defined dates.

Table 3.3 Use case narrative for publish forms for morale yearly

Use case name	Publish morale yearly question forms
Actors	Admin
Goal	To make the question forms active and visible to users.
Preconditions	1. The question form is saved in draft status. 2. All required fields are completed.
Main success scenario	1. Admin selects a draft question form. 2. Admin reviews the form and clicks “Save” then select “Published.” 3. Admin selects the target group (company-wide or specific departments). 4. Admin clicks “Publish”. 5. System activates the form, making it visible to users. 6. System navigates admin to the recent question forms page.
Extension (a)	2a. If required fields are incomplete. 3a. The system prevents publishing and prompts the admin to complete them. 4a. Return to step 2
Extension (b)	3b. If the admin cancels 4b. The form remains in draft status. 5b. Return to step 6
Postconditions	The question form is published and accessible to users within the set date range.

Table 3.4 Use case narrative for view assigned question forms

Use case name	View assigned question forms
Actors	User and manager
Goal	To enable users and managers to view question forms assigned to them.
Preconditions	1. Users has an active login. 2. At least one question form is assigned and active.
Main success scenario	1. Users logs in to the system. 2. Users navigates to the “Question Forms” section. 3. The system displays a list of all assigned question forms. 4. Users selects a specific question form to view. 5. The system shows detailed information about the selected form, including the title, description, start and end dates, and progress status.
Extension (a)	3a. If no question forms are assigned 4a. The system displays a message “No assigned question forms available.”
Postconditions	Users has successfully viewed details of assigned question forms.

Table 3.5 Use case narrative for answer and submit forms

Use case name	Answer and submit forms
Actors	User and manager
Goal	To complete and submit answers for assigned question forms.
Preconditions	1. Users has an active login. 2. At least one question form is assigned and active.
Main success scenario	1. Users logs in and navigates to the “Question Forms” section. 2. Users selects an active question form. 3. Users answers the questions and provides optional comments. 4. Users submits the completed form. 5. The system confirms successful submission and rewards coins.
Extension (a)	4a. If the user submits incomplete answers. 5a. The system prompts to complete all questions. 6a. Return to step 3
Postconditions	User responses are saved, and coins are rewarded for participation.

• **Morale daily and yearly dashboard**

Table 3.6 Use case narrative for view morale yearly dashboard

Use case name	Morale yearly dashboard
Actors	Admin
Goal	To view and analyze the morale yearly data for the company over the course of a year.
Preconditions	1. The admin has to log into the system.
Main success scenario	1.The user selects the “View yearly dashboard”. 2.The system displays the morale data filtering options. 3.The user selects the desired filters to narrow down the data. 4.The user selects the morale name they want to view. 5.The user selects the company they want to view data for. 6.The system displays the morale data and feedback for the selected morale and company.
Postconditions	The user has successfully viewed and analyzed the morale data for the selected morale, and company.

Table 3.7 Use case narrative for view morale yearly dashboard

Use case name	Morale yearly dashboard
Actors	Manager
Goal	To view and analyze the morale yearly data for the company over the course of a year.
Preconditions	1. The manager has to log into the system.
Main success scenario	1.The user selects the “View yearly dashboard”. 2.The system displays the morale data filtering options. 3.The user selects the desired filters to narrow down the data. 4.The user selects the morale name they want to view. 6.The system displays the morale data and feedback for the selected morale and company.
Postconditions	The user has successfully viewed and analyzed the morale data for the selected morale.

Table 3.8 Use case narrative for view morale daily dashboard

Use case name	Morale daily dashboard
Actors	Admin
Goal	To view and analyze the daily morale data for the company.
Preconditions	1. The admin has to log into the system.
Main success scenario	1.The user selects the “View overview satisfaction dashboard”. 2.The user selects the company, date, and section they want to view. 3.The system displays the relevant morale data. 4.The user can select an employee to view their overview data. 5.The user can export the displayed morale data table.
Extension (a)	4a.If the user needs to export the displayed employee data, they can use the “Export” option.
Postconditions	The user has successfully viewed and analyzed the daily morale data for the selected criteria, and can export the data as needed.

• **Assessment 360**

Table 3.9 Use case narrative for creating the assessment

Use case name	Create the assessment
Actors	Admin
Goal	To create a new assessment
Preconditions	1. User is Admin role 2. User is on viewing the assessment managing page
Main success scenario	1. User clicks the “สร้างชุดคำถาม” button 2. System displays form to fill out 3. User fills out the assessment information 4. User clicks “บันทึก” button 5. System navigates user to the assessment managing page, also show the new assessment that was just created on the top of the assessment list
Extension (a)	4a. User clicks “ยกเลิก” button 5a. System navigates user to the assessment managing page
Extension (b)	4b. System found the required field were incomplete 5b. System show the error text 6b. Return to step 3
Extension (c)	4c. System found validation error 5c. System show the error text 6c. Return to step 3
Extension (d)	3d. User clicks “เพิ่มคำถาม” button 4d. System display another box for user to fill out 5d. Return to step 3
Extension (e)	3e. User clicks delete icon 4e. System delete the selected question 5e. Return to step 3
Postconditions	The newly created assessment is displayed in the assessment list on the assessment managing page.

Table 3.10 Use case narrative for duplicating the assessment

Use case name	Duplicate the assessment
Actors	Admin
Goal	To duplicate the assessment
Preconditions	1. User is Admin role 2. User is on viewing the assessment managing page
Main success scenario	1. User clicks duplicate icon on the assessment card 2. System displays form to fill out with the question from the selected assessment 3. User fills out the assessment information 4. User clicks “บันทึก” button 5. System navigates user to the assessment managing page, also show the new assessment that was just created on the top of the assessment list
Extension (a)	4a. User click “ยกเลิก” button 5a. System navigates user to the assessment managing page
Extension (b)	4b. System found the required field were incomplete 5b. System show the error text 6b. Return to step 3
Extension (c)	4c. System found validation error 5c. System show the error text 6c. Return to step 3
Extension (d)	3d. User clicks “เพิ่มคำถาม” button 4d. System display another box for user to fill out 5d. Return to step 3
Extension (e)	3e. User clicks delete icon 4e. System delete the selected question 5e. Return to step 3
Postconditions	The newly created assessment is displayed in the assessment list on the assessment managing page.

Table 3.11 Use case narrative for assigning the assessment

Use case name	Assign the assessment
Actors	Admin
Goal	To assign the assessment
Preconditions	1. User is Admin role 2. User is on viewing the assessment managing page
Main success scenario	1. User clicks on “ใช้ชุดคำถาม” button 2. System navigates user to the assignment page 3. User fills out the assignment information 4. User clicks “บันทึก” button 5. System navigates user to the assigned assessment managing page, also show the new assigned assessment that was just created on the top of the assigned assessment list
Extension (a)	4a. System found the required field were incomplete 5a. System show the error text 6a. Return to step 3
Postconditions	The newly assigned assessment is displayed in the assigned assessment list on the assigned assessment managing page.

Table 3.12 Use case narrative for responding the assessment

Use case name	Respond the assessment
Actors	Manager or User
Goal	To respond the assessment
Preconditions	1. The available assigned assessment is presented on the assessment list for user 2. User is on viewing the assessment page
Main success scenario	1. User selects the assessment to respond 2. User clicks “ตอบแบบปรเมิน” button 3. System navigates user to the ready to answer assessment page 4. User respond to the question 5. User clicks “ส่งคำตอบ” button 6. System navigates user to the assessment page
Extension (a)	5a. System found the required field were incomplete 6a. System show the error text 7a. Return to step 4
Extension (b)	5c. System found validation error 6c. System show the error text 7c. Return to step 3
Postconditions	The system navigates the user to the assessment page and displays all assessments that have not been responded to.

Table 3.13 Use case narrative for video course learning

Use case name	Respond the assessment
Actors	User
Goal	To learn a video course and track completion status
Preconditions	1. User is logged into the system 2. User is on the “Video Learning” page
Main success scenario	1. User navigates to the “Video Learning” page 2. System displays the list of available video courses 3. User filters or searches for the desired course (optional) 4. User clicks the “Click to learn” button on a course 5. System navigates user to the video player page 6. User watches the video 7. System tracks and updates the user’s video progress 8. Upon completion, system marks the course as “Completed” and updates the status on the video learning page
Extension (a)	3a. User does not use filter/search and scrolls manually 4a. System still allows course access 5a. Continue to step 5
Extension (b)	3a. User does not use filter/search and scrolls manually 4a. System still allows course access 5a. Continue to step 5
Extension (c)	6c. Video fails to load or an error occurs 7c. System displays an error message and suggests retry or contacting support 8c. User can return to course list
Postconditions	The user has watched the video course and the system updates the course as “Completed” with saved progress.

3.5 Sequence Diagram

Morale Daily and Yearly Question Form

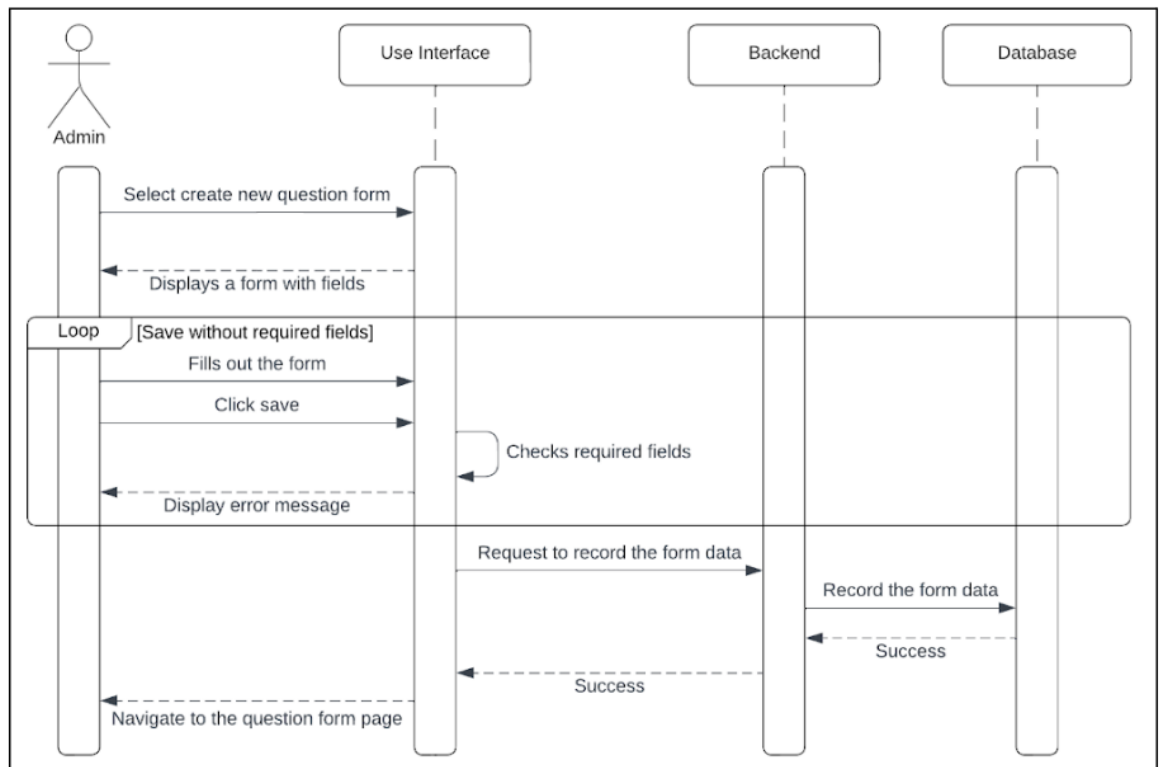


Figure 3.13 Sequence diagram of creating morale dairy question forms

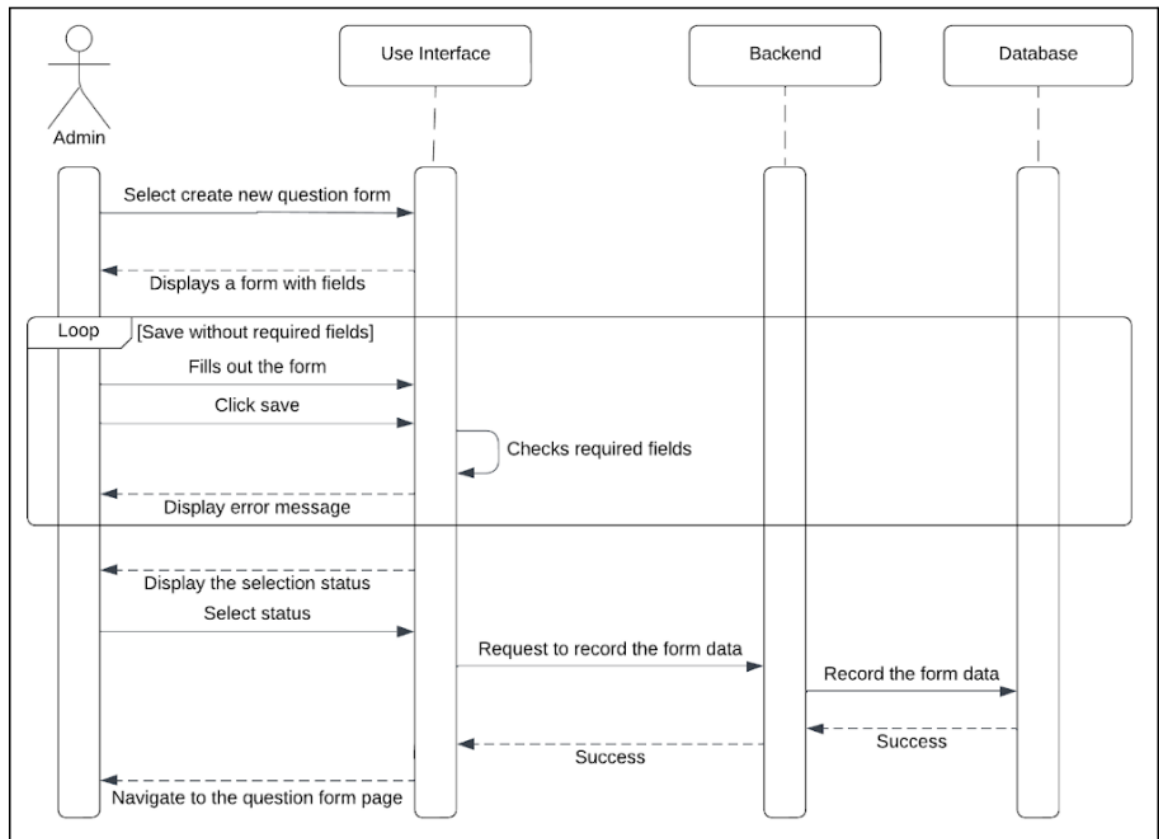


Figure 3.14 Sequence diagram of creating morale yearly question forms

The sequence diagram in Figure 3.13 and Figure 3.14 shows the interaction between the user (admin), the user interface, the back-end, and the database components during the process of creating morale daily and yearly question forms within the system. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the user initiating the action on the question forms list page, where the system displays all existing question forms. The user clicks the “Create New Question Form” button, which serves as the interface component for starting the creation process.
- Step 2: The user interface displays a form for creating a new question form. The user fills in the required fields and clicks the “Save” button. The front-end component validates the input fields. If validation fails, an error message is displayed on the front end. If validation succeeds, the front end sends a request to the back end to save the form data. For yearly forms, the front end also displays a status selection field, which the user must select before the request is sent to the back end.
- Step 3: The back-end component receives the request and processes the information. It communicates with the database, sending the required data to create the new question form.
- Step 4: The database processes the request and stores the question form information. Once the operation is completed successfully, it sends a confirmation response to the back end.
- Step 5: Upon receiving the success response, the back end sends a confirmation message to the front end, indicating that the question form has been created successfully.

- Step 6: The front-end component receives the confirmation and refreshes the question forms list page. The newly created question form is now visible in the list, ensuring the user can view it immediately.

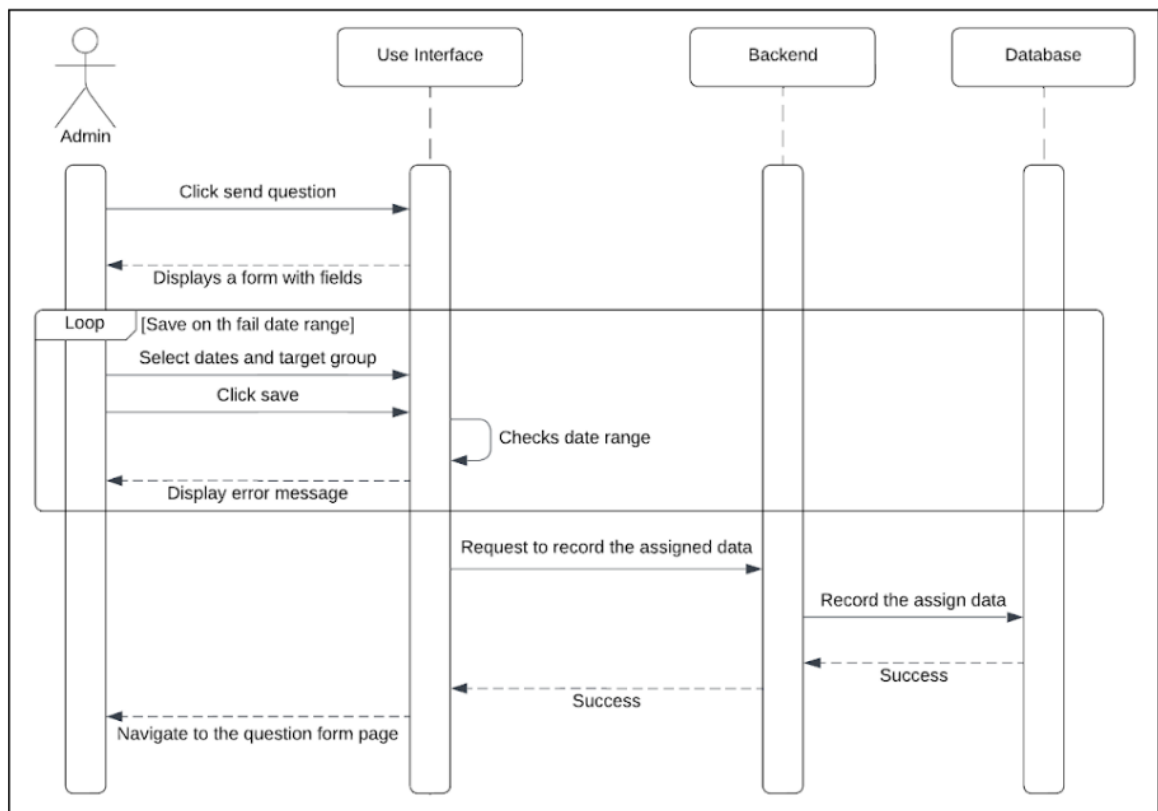


Figure 3.15 Sequence diagram of configure and assign morale daily question forms

The sequence diagram in Figure 3.15 shows the interaction between the user (admin), the user interface, the back-end, and the database components during the process of configure and assign morale daily question forms within the system. Here’s a breakdown of the sequence:

- Step 1: The sequence begins with the user initiating the action by clicking the “Send Question” button on the morale daily question forms list page under the activated tab, where the system displays all existing question forms.
- Step 2: The user interface displays a form for selecting the start and end dates and specifying the target group. The user sets the dates and selects the target group, then clicks the “Save” button. The front-end component validates the date range and target group. If validation fails, an error message is displayed on the front end. If validation succeeds, the front end sends a request to the back end to save the configuration details.
- Step 3: The back-end component receives the request and processes the information. It communicates with the database, sending the necessary data to update the database with the new assigned dates and target group for the question form.
- Step 4: The database processes the request and updates the question form information. Once the operation is completed successfully, it sends a confirmation response to the back end.

- Step 5: Upon receiving the success response, the back end sends a confirmation message to the front end, indicating that the question form has been successfully assigned with the new dates and target group.
- Step 6: The front-end component receives the confirmation and refreshes the morale daily question forms list page. The newly updated question form is now visible on the list page under the in-progress tab, ensuring the user can view it immediately.

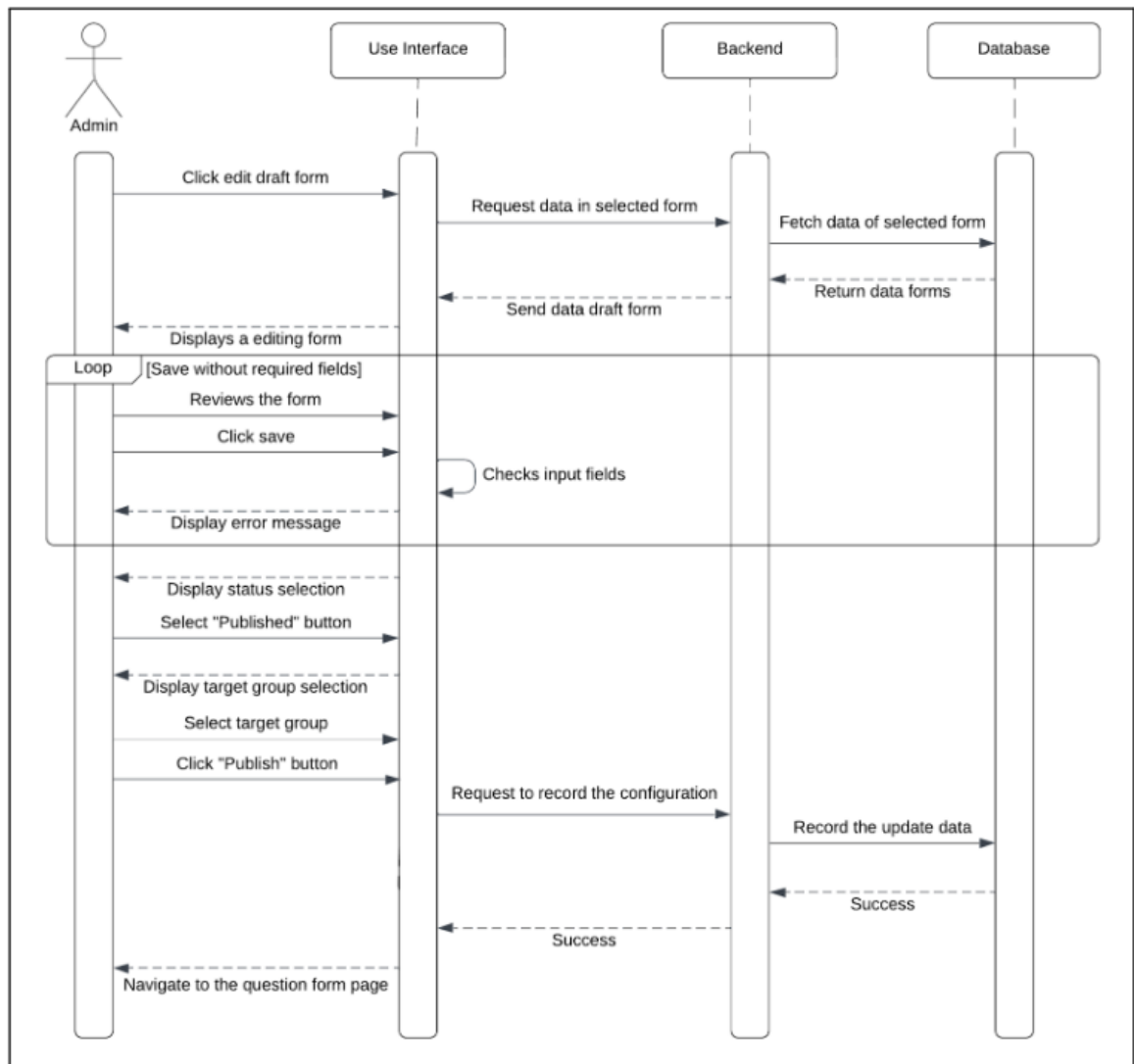


Figure 3.16 Sequence diagram of publish morale yearly question forms

The sequence diagram in Figure 3.16 shows the interaction between the user (admin), the user interface, the back-end, and the database components during the process of publish morale yearly question forms within the system. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the user initiating the action by clicking the "Edit" button at the draft status on the morale yearly question forms, where the system displays all existing question forms.
- Step 2: The user interface displays a form for editing details and managing questions. The user reviews the form, makes changes, and clicks the "Save" button. The front-end component validates the input

fields. If validation fails, an error message is displayed on the front end. If validation succeeds, the front end displays a status selection field.

- Step 3: The user selects a status by clicking the “Published” button. The front end then displays a target group selection field. After selecting the target groups and clicking the “Publish” button, the front end sends a request to the back end to save the configuration details.
- Step 4: The back-end component receives the request and processes the information. It communicates with the database, sending the required data to update the database with the new status and target groups of the question form.
- Step 5: The database processes the request and updates the question form information. Once the operation is completed successfully, it sends a confirmation response to the back end.
- Step 6: Upon receiving the success response, the back end sends a confirmation message to the front end, indicating that the question form has been successfully assigned to the target groups and its status updated.
- Step 7: The front-end component receives the confirmation and refreshes the morale yearly question forms list page. The updated question form with the published status is now visible on the list page, ensuring the user can view it immediately.

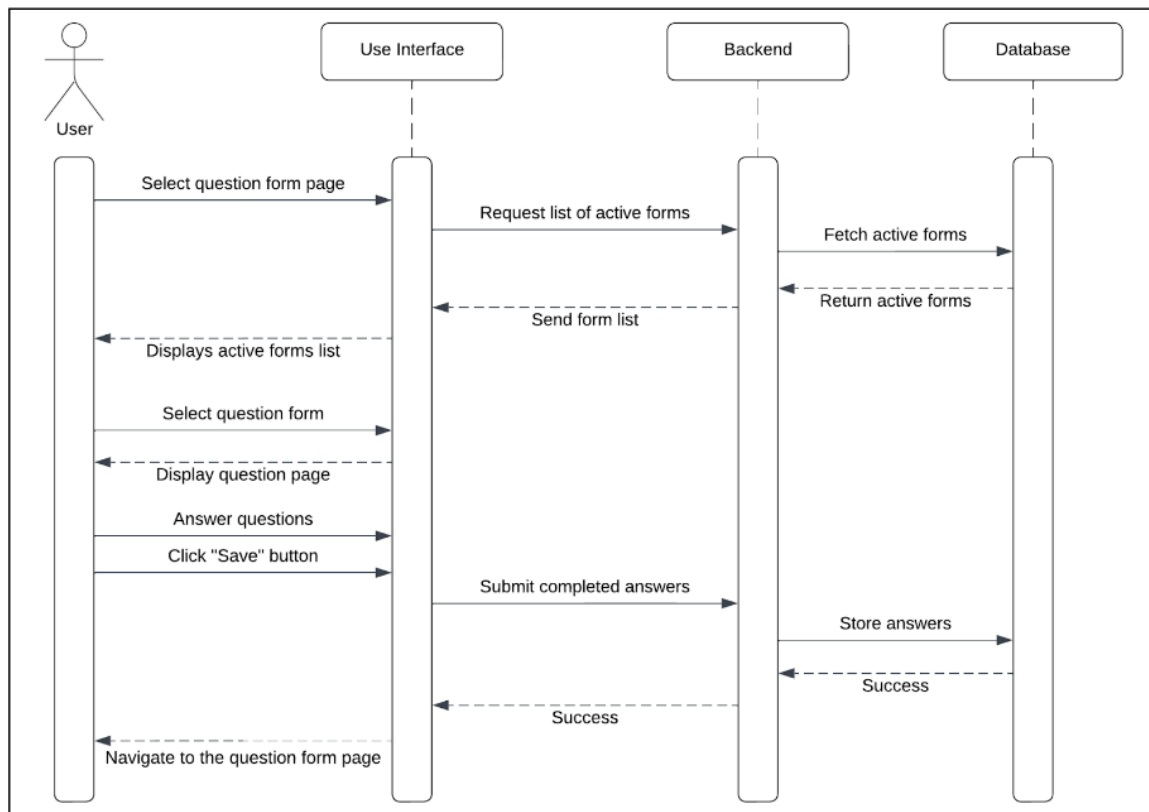


Figure 3.17 Sequence diagram of answer and submit forms

The sequence diagram in Figure 3.17 shows the interaction between the user (employee), the user interface, the back-end, and the database components during the process of answer and submit forms within the system. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the user accessing the morale question forms list page, where the system displays all available question forms within the valid date range.
- Step 2: The user selects a question form to answer by clicking the “Answer” button. The user interface displays the selected question form with all its questions and details.
- Step 3: Step 4: Once all questions are answered, the user clicks the “Save” button. The front end sends the completed responses to the back end for processing.
- Step 4: The back-end component receives the submitted answers and processes the information. It communicates with the database, storing the user's responses and updating the participant's completion status for the question form.
- Step 5: The database processes the request, storing the responses and updating the progress status. Once the operation is successfully completed, it sends a confirmation response to the back end.
- Step 6: Upon receiving the success response, the back end sends a confirmation message to the front end, indicating that the form has been successfully submitted.
- Step 7: The front-end component receives the confirmation and refreshes to remove the completed form from the question form list page. Additionally, for the daily question form, the user's earned coins are updated in the UI as a reward for completing the form.

Morale Daily Dashboard

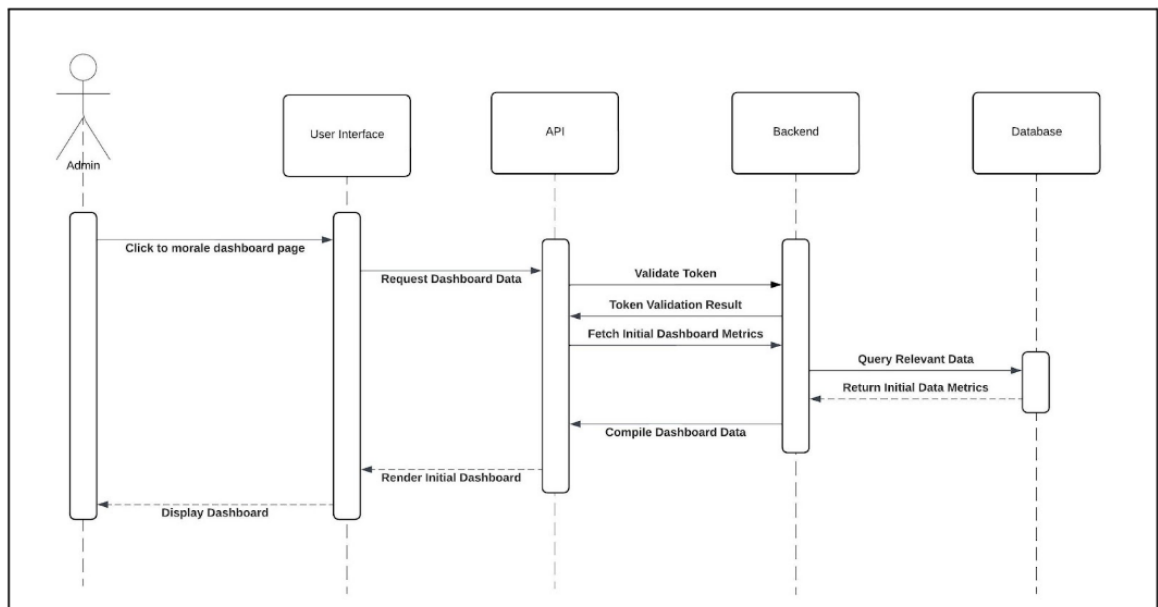


Figure 3.18 Sequence diagram of viewing morale daily dashboard

The sequence diagram in Figure 3.18 illustrates the interaction between the admin, the user interface, the API, the back-end, and the database components during the process of viewing the morale daily dashboard. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the admin clicking on the morale dashboard page in the user interface. The system initiates a request to load the dashboard data.
- Step 2: The user interface sends a request to the API to retrieve the required dashboard data.
- Step 3: The API validates the admin's token to ensure the request is authenticated and authorized. It sends the token validation result back to confirm whether the admin has the necessary permissions to access the dashboard.
- Step 4: Upon successful validation, the API forwards a request to the back-end to fetch the initial metrics needed for the dashboard.
- Step 5: The back-end queries the database to fetch relevant data for the morale daily dashboard.
- Step 6: The database processes the query and returns the requested data metrics to the back-end.
- Step 7: The back-end compiles the data into a structured format suitable for the dashboard and sends it back to the API.
- Step 8: The API returns the compiled data to the user interface.
- Step 9: The user interface processes the data and renders the initial view of the morale daily dashboard, displaying it to the admin.

This flow ensures secure and efficient delivery of morale data while maintaining modularity across system components.

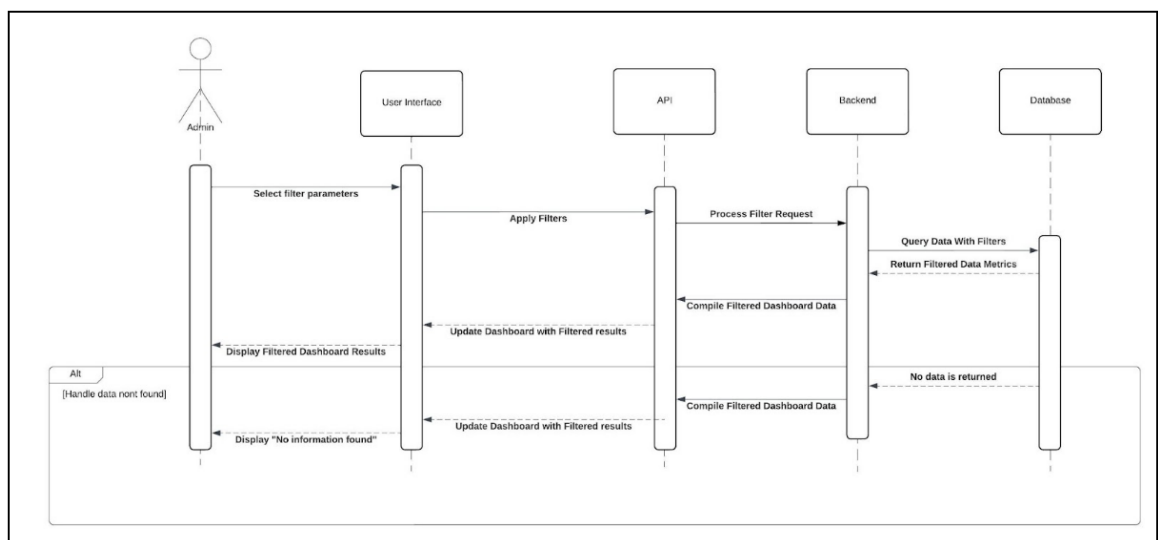


Figure 3.19 Sequence diagram of filtering parameters on morale daily dashboard

The sequence diagram in Figure 3.19 depicts the interaction between the admin, the user interface, the API, the back-end, and the database components during the process of filtering parameters on the morale daily dashboard. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the admin selecting filter parameters on the morale dashboard page. These parameters specify the desired criteria for filtering the displayed data.

- Step 2: The user interface sends the selected filter parameters to the API to apply the specified filters.
- Step 3: The API processes the filter request by validating the parameters and forwarding the request to the back-end.
- Step 4: The back-end queries the database with the specified filters to retrieve the relevant data metrics.
- Step 5 (Main Flow):
 - If the database returns filtered data metrics, the back-end compiles the data and sends it back to the API.
 - The API forwards the filtered dashboard data to the user interface.
 - The user interface updates the dashboard to display the filtered results to the admin.
- Step 5 (Alternate Flow):
 - If the database returns no data for the specified filters, the back-end notifies the API that no data was found.
 - The API forwards this information to the user interface.
 - The user interface updates the dashboard to display a message such as “No information found.”

This flow ensures that the admin can interactively apply filters to refine the displayed morale data while gracefully handling scenarios where no matching data is available.

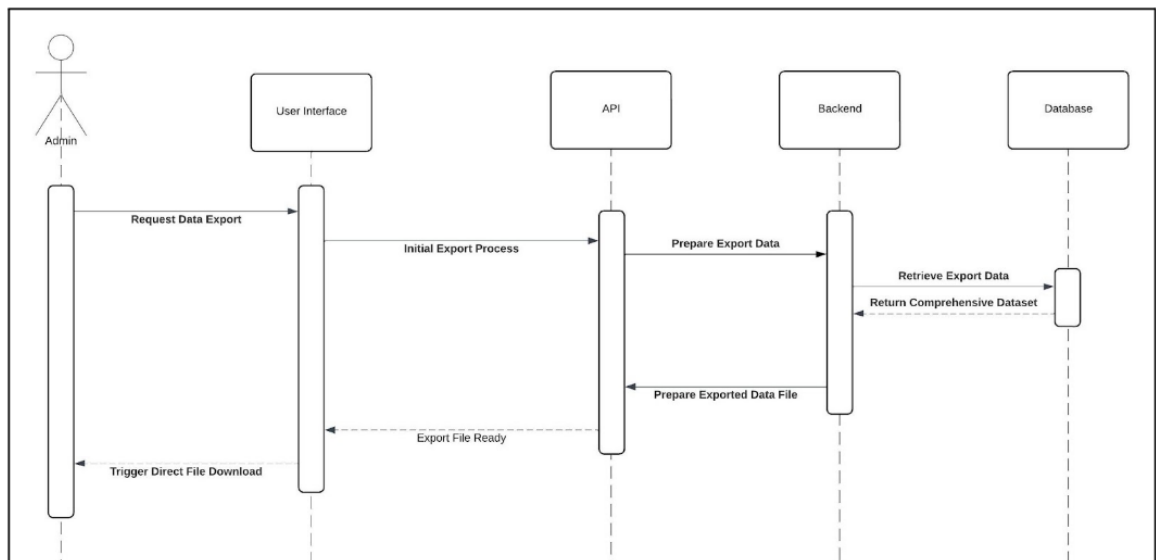


Figure 3.20 Sequence diagram of exporting employee table on morale daily dashboard

The sequence diagram in Figure 3.20 demonstrates the interaction between the admin, the user interface, the API, the back-end, and the database components during the process of exporting the employee table on the morale daily dashboard. Here’s a breakdown of the sequence:

- Step 1: The sequence begins with the admin initiating the export process by requesting to export the employee table data through the user interface.

- Step 2: The user interface sends the export request to the API, which triggers the initial export process.
- Step 3: The API forwards the request to the back-end to prepare the data for export.
- Step 4: The back-end communicates with the database to retrieve the comprehensive dataset required for the export.
- Step 5: The database processes the query and returns the requested dataset to the back-end.
- Step 6: The back-end prepares the exported data file in a suitable format (e.g., CSV, Excel, or JSON) and sends it back to the API.
- Step 7: The API notifies the user interface that the export file is ready for download.
- Step 8: The user interface provides a download link or triggers a direct file download for the admin to access the exported employee table.

This process ensures a seamless export of data with minimal interaction, providing the admin with a structured file for further analysis or record-keeping.

Morale Yearly Dashboard

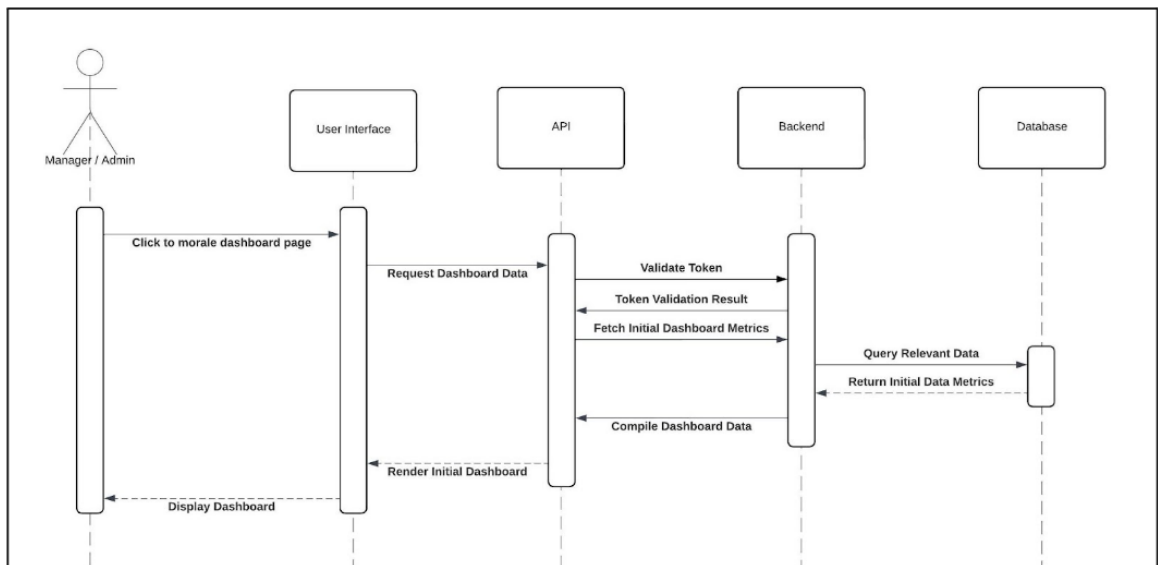


Figure 3.21 Sequence diagram of viewing morale yearly dashboard

The sequence diagram in Figure 3.21 demonstrates the interaction between the admin, the user interface, the API, the back-end, and the database components during the process of viewing the morale yearly dashboard. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the admin initiating the process by clicking to navigate to the morale yearly dashboard page through the user interface.
- Step 2: The user interface sends a request to the API to fetch the dashboard data.
- Step 3: The API validates the token to ensure the request is authenticated.
- Step 4: The API sends the token validation result back to the user interface.

- Step 5: Once validated, the API fetches the initial dashboard metrics by sending a request to the back-end.
- Step 6: The back-end queries the relevant data from the database based on the request.
- Step 7: The database processes the query and returns the initial data metrics to the back-end.
- Step 8: The back-end compiles the dashboard data and sends it back to the API.
- Step 9: The API returns the compiled data to the user interface.
- Step 10: The user interface renders the initial dashboard and displays it to the admin.

This process ensures that the morale yearly dashboard is securely fetched and displayed, providing the admin with comprehensive metrics for analysis.

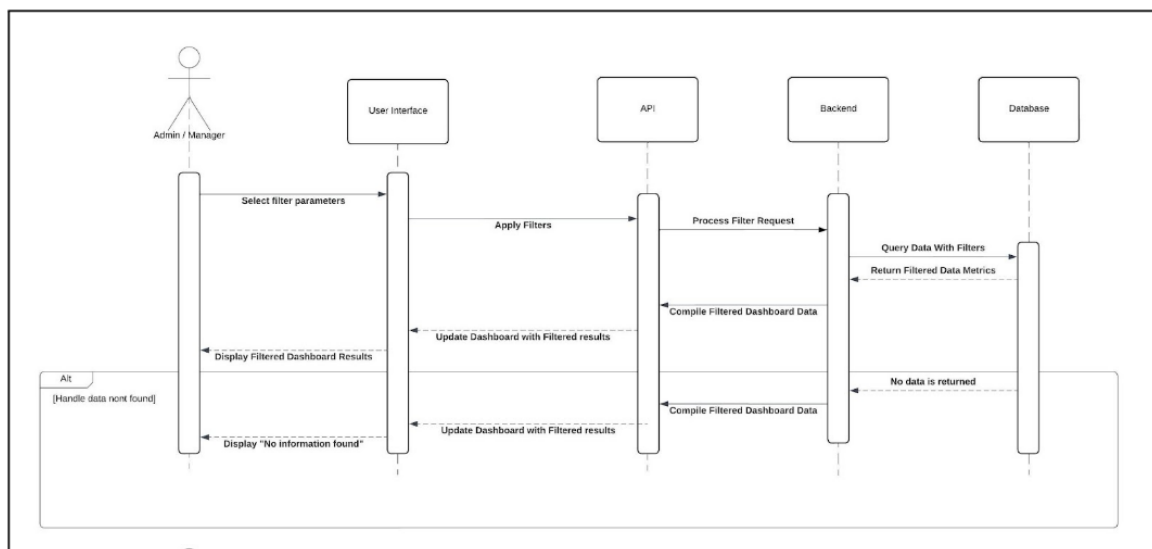


Figure 3.22 Sequence diagram of filtering on morale yearly dashboard

The sequence diagram in Figure 3.22 demonstrates the interaction between the admin, the user interface, the API, the back-end, and the database components during the process of filtering data on the morale yearly dashboard. Here's a breakdown of the sequence:

- Step 1: The sequence begins with the admin selecting filter parameters on the user interface.
- Step 2: The user interface sends the filter parameters to the API to apply the filters.
- Step 3: The API processes the filter request and forwards it to the back-end for data retrieval.
- Step 4: The back-end queries the relevant data from the database based on the applied filters.
- Step 5: The database processes the query and returns the filtered data metrics to the back-end.
- Step 6: The back-end compiles the filtered dashboard data and sends it back to the API.
- Step 7: The API returns the compiled filtered data to the user interface.
- Step 8: The user interface updates and displays the dashboard with the filtered results for the admin.

- Alternative Flow: No Data Found

If the database query returns no data, the back-end sends an empty result to the API. The API informs the user interface, and the user interface displays a message: “No information found.”

This process ensures that the morale yearly dashboard dynamically updates based on the applied filters, providing the admin with relevant and specific insights.

Assessment 360

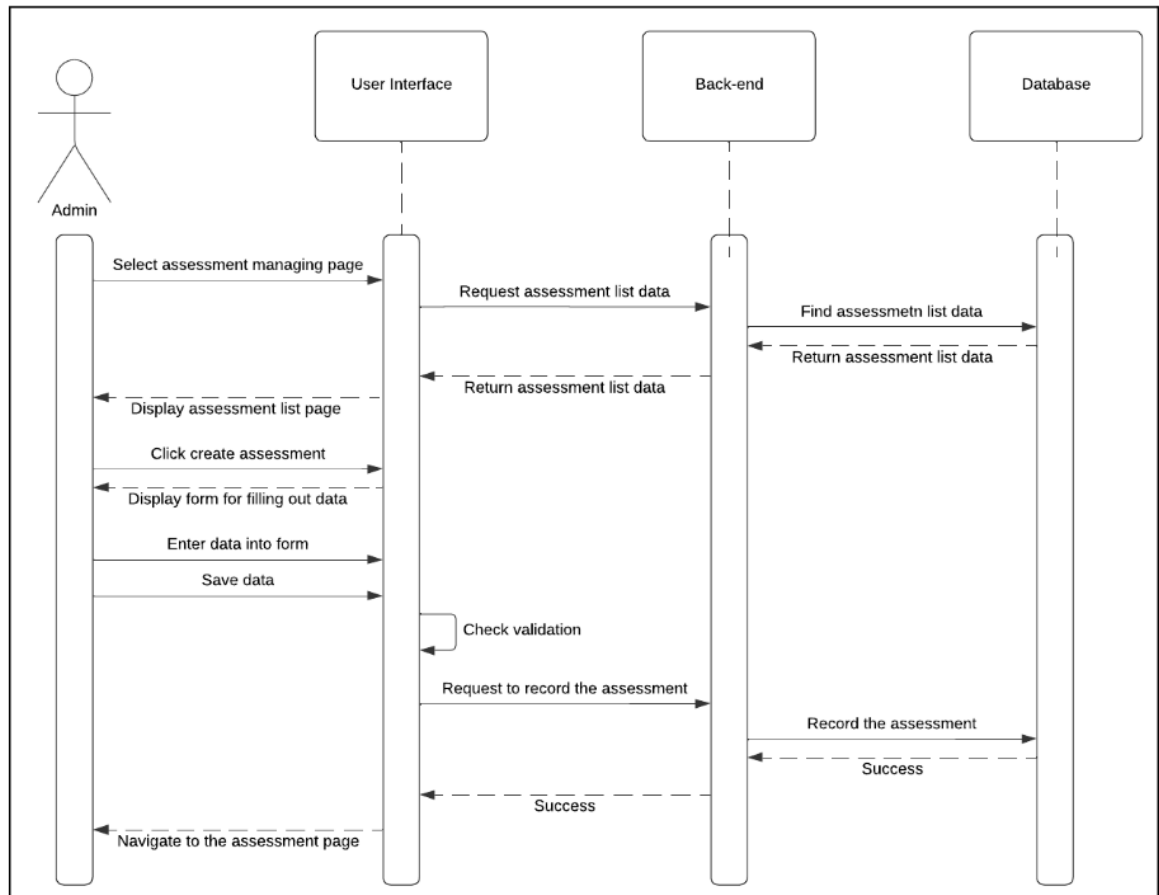


Figure 3.23 Sequence diagram of creating assessment

The sequence diagram in Figure 3.23 shows the interaction between the user (admin), the user interface, the back-end, and the database components during the process of creating assessment within the system. Here’s a breakdown of the sequence:

- Step 1: The sequence starts with the user initiating the action on the assessment list page, where the system displays all existing assessments. The user clicks the “สร้างชุดคำถามใหม่” button (Create New Assessment), which serves as the interface component for starting the creation process.
- Step 2: The user interface displays a form for creating a new assessment. After the user fills in the required fields and clicks the “Save” button, the front-end component validates the input. If the validation is successful, the front-end sends a request to the back-end to save the assessment details.

- Step 3: The back-end component receives the request and processes the information. It communicates with the database, sending the required data to create the new assessment.
- Step 4: The database processes the request and stores the assessment information. Once the operation is completed successfully, it sends a confirmation response to the back-end.
- Step 5: Upon receiving the success response, the back-end sends a confirmation message back to the front-end, confirming that the assessment has been created successfully.
- Step 6: The front-end component receives the confirmation and refreshes the assessment list page. The newly created assessment is now visible in the list, ensuring the user can view it immediately.

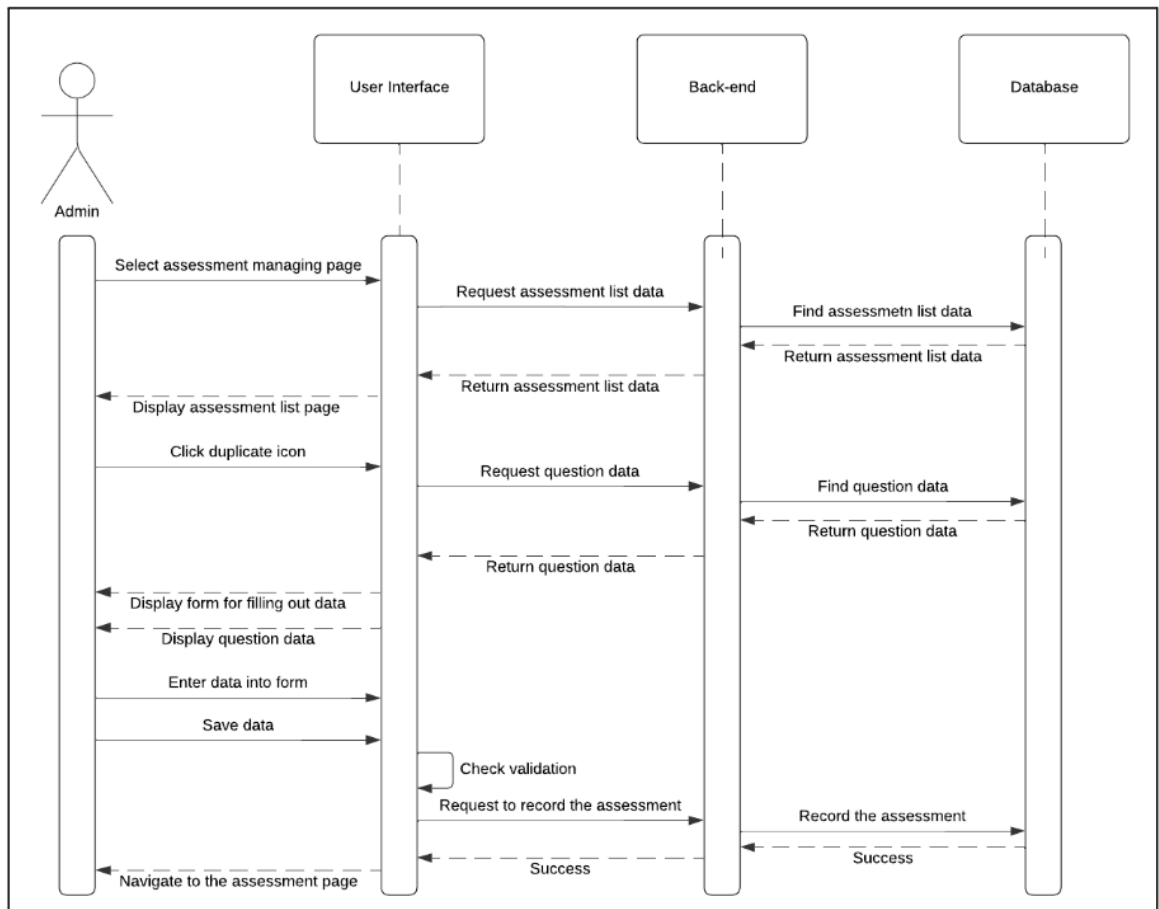


Figure 3.24 Sequence diagram of duplicating assessment

The sequence diagram in Figure 3.24 demonstrates the interaction between the user (admin), the user interface, the back-end, and the database components during the process of duplicating an assessment. Below is the detailed breakdown:

- Step 1: The sequence begins with the user (admin) initiating the duplication process on the assessment management page. The admin clicks the Duplicate icon on an assessment card, triggering the duplication process
- Step 2: The user interface displays a form pre-filled with the details and questions from the selected assessment. The admin can edit the form, add new questions by clicking the “เพิ่มคำถาม” (Add Question)

button, or remove existing questions by clicking the Delete icon. After making necessary adjustments, the admin clicks the “บันทึก” (Save) button to proceed.

- Step 3: The front-end validates the input. If all required fields are complete and no validation errors exist, the front-end sends a request to the back-end to duplicate the assessment. This includes sending the edited assessment details and associated questions.
- Step 4: The back-end receives the request and processes it by communicating with the database. It sends all required information to create the new, duplicated assessment.
- Step 5: The database processes the request and stores the duplicated assessment details. Upon successful completion, the database sends a confirmation response back to the back-end.
- Step 6: The back-end receives the confirmation and notifies the front-end that the assessment duplication was successful.
- Step 7: The user interface refreshes the assessment management page and updates the assessment list. The newly duplicated assessment appears at the top of the list, visible to the admin.

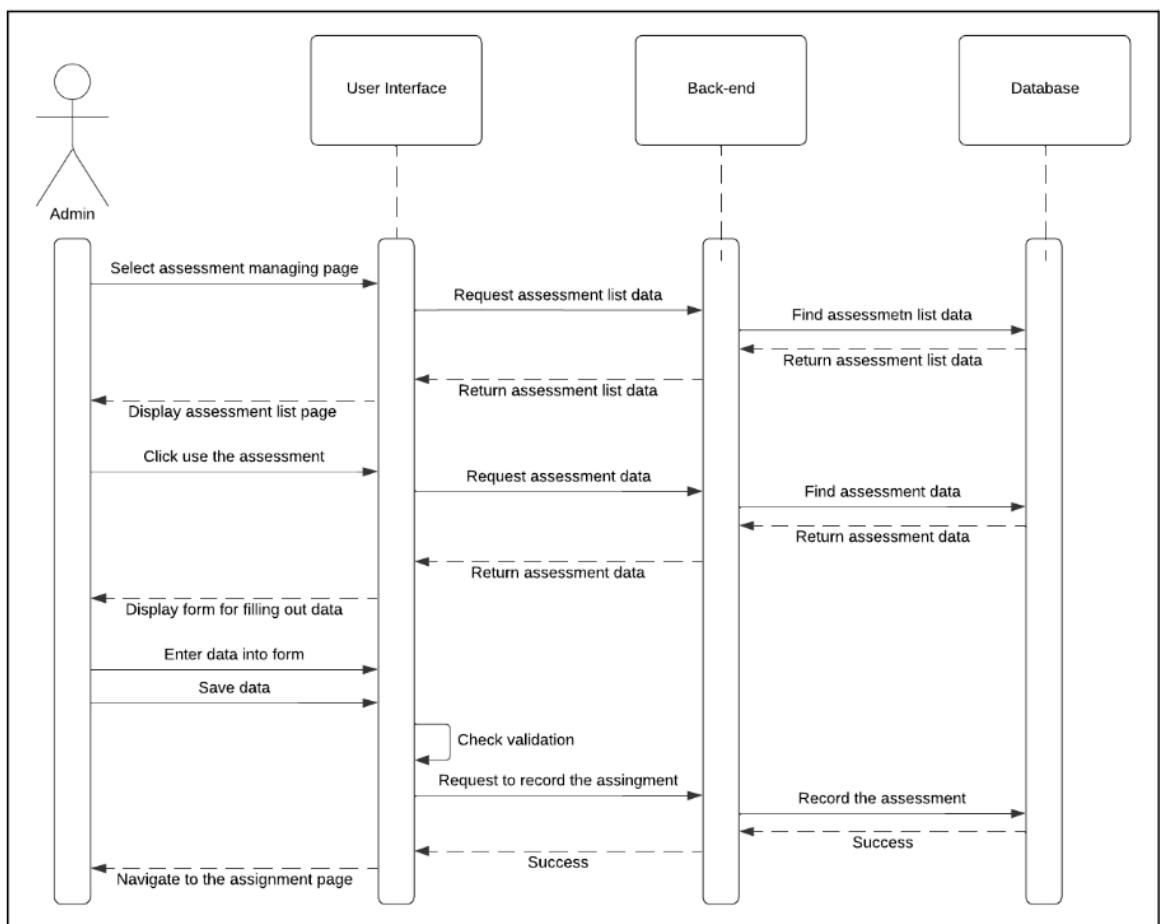


Figure 3.25 Sequence diagram of assigning assessment

The sequence diagram in Figure 3.25 illustrates the interaction between the admin, the user interface, the back-end, and the database components during the process of assigning an assessment. Below is the detailed breakdown:

- Step 1: The sequence starts with the admin initiating the assignment process on the assessment management page. The admin clicks the “ใช้ชุดคำถาม” (Use Assessment) button associated with the desired assessment, triggering the system to proceed.
- Step 2: The user interface navigates the admin to the assignment page. The form for assigning the assessment is displayed, where the admin can fill out the required assignment details, such as selecting employees.
- Step 3: Once the admin completes the form and clicks the “บันทึก” (Save) button, the front-end validates the input. If the validation is successful, the front-end sends a request to the back-end to save the assignment details.
- Step 4: The back-end receives the request and processes the information. It communicates with the database, sending all necessary data to create the new assignment record.
- Step 5: The database processes the request and stores the assignment details. Upon successful completion, it sends a confirmation response to the back-end.
- Step 6: The back-end receives the success response and sends a confirmation message to the front-end, confirming that the assignment has been successfully created.
- Step 7: The front-end navigates back to the assigned assessment management page. The newly assigned assessment is displayed at the top of the assigned assessment list, allowing the admin to view it immediately.

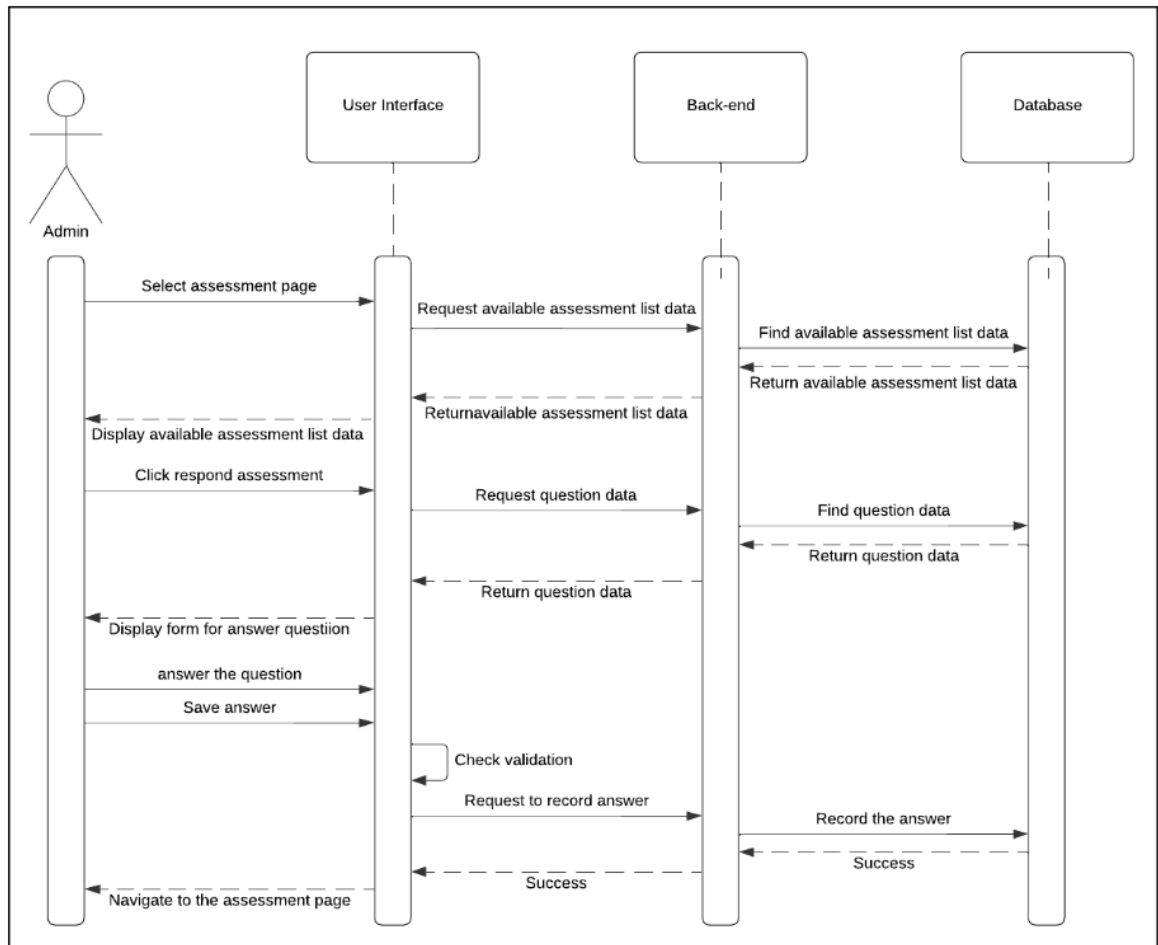


Figure 3.26 Sequence diagram of responding assessment

The sequence diagram in Figure 3.26 illustrates the interaction between the user (Manager or Employee), the user interface, the back-end, and the database components during the process of responding to an assessment. Below is the detailed breakdown:

- Step 1: The sequence begins with the user viewing the list of available assigned assessments on the assessment page. The user selects an assessment to respond to and clicks the “ตอบแบบประเมิน” (Respond to Assessment) button.
- Step 2: The system navigates the user to the assessment response page, which displays the assessment questions.
- Step 3: The user reviews the questions and provides responses by filling out the necessary fields on the form.
- Step 4: Once the user completes the responses, they click the “ส่งคำตอบ” (Submit Answers) button. The front-end validates the responses to ensure all required fields are filled and no errors exist.
- Step 5: If the input passes validation, the front-end sends a request to the back-end to save the user’s responses.
- Step 6: The back-end receives the request, processes the responses, and communicates with the database. It sends the user’s answers to be stored in the relevant assessment records.

- Step 7: The database processes the request, saving the responses. Upon successful completion, it sends a confirmation response back to the back-end.
- Step 8: The back-end receives the success response and notifies the front-end that the responses have been successfully submitted.
- Step 9: The user interface navigates back to the assessment page, displaying the updated list of available assessments. The submitted assessment is no longer listed as incomplete.

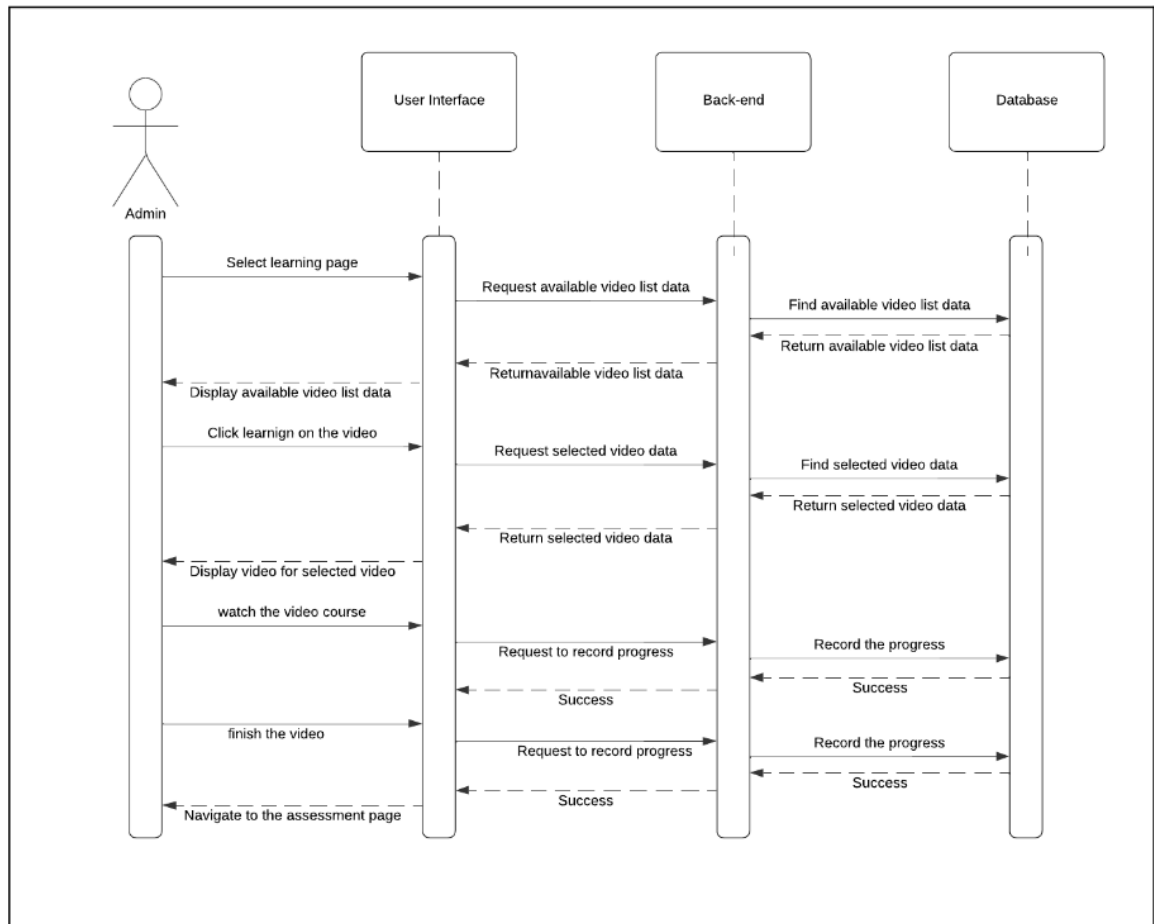


Figure 3.27 Sequence diagram of video course learning

The sequence diagram in Figure 3.27 illustrates the interaction between the user (Employee), the user interface, the back-end, and the database components during the process of learning a video course. Below is the detailed breakdown:

- Step 1: The sequence begins with the user accessing the “Video Learning” page, where the system displays a list of available courses. The user may use filters (e.g., course name, category) or browse directly to find a course of interest.
- Step 2: The user selects a course and clicks the “Click to Learn” button. The system navigates the user to the course learning page.
- Step 3: The course learning page loads the selected video. The user watches the video while the front-end tracks the viewing progress in real time.

- Step 4: As the user watches, the system continuously or periodically sends progress data to the back-end to ensure updates are recorded.
- Step 5: Once the video reaches 100% completion, the front-end sends a final update to the back-end to mark the course as completed.
- Step 6: The back-end processes the update and stores the course completion status in the database.
- Step 7: The database saves the new completion status for the user-course relationship and confirms the operation.
- Step 8: The back-end receives the confirmation and notifies the front-end that the course has been successfully marked as completed.
- Step 9: The user interface updates the course status on the “Video Learning” page to show that the course is completed (e.g., showing a “Completed” tag or progress bar at 100%).

3.6 Database Design

The Database Design is structured based on the Entity Relationship (ER) diagram and supported by a detailed data dictionary that defines each table’s attributes and relationships.

The ER diagram outlines the core entities for managing morale daily and yearly question forms and dashboards as well as the 360 Assessments system, including their connections to ensure seamless data flow and functionality. It illustrates the relationships between key entities and their attributes, providing a structured approach to data management.

A Data Dictionary is a centralized repository that defines and describes the structure, attributes, and relationships of the data stored in a database. It provides detailed information about each table, column, datatype, and data description, ensuring consistency and clarity across the system. The data dictionary also specifies the relationships between tables, including primary and foreign keys, helping to maintain data integrity and enforce referential constraints.

3.6.1 Entity Relationship Diagram

Morale Daily and Yearly

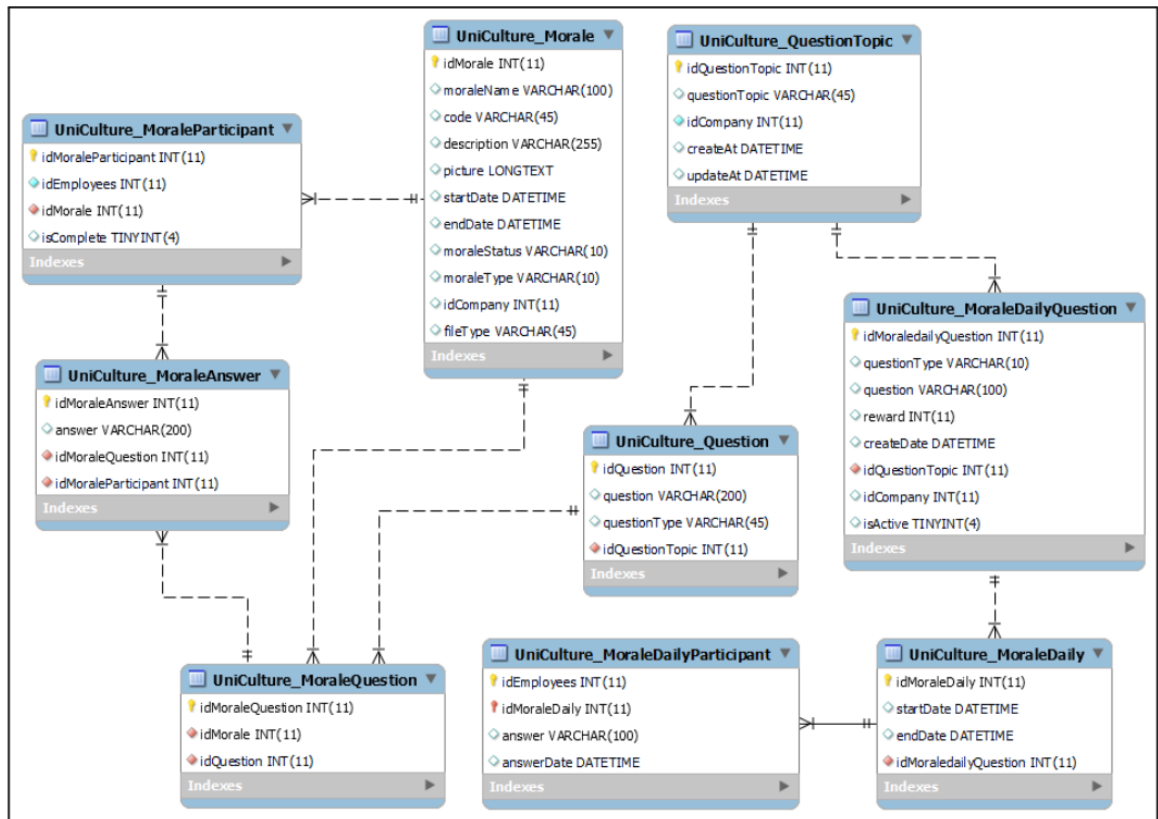


Figure 3.28 Entity relationship diagram of morale daily and yearly

In Figure 3.28, for morale question forms and dashboards, the diagram captures entities such as Morale, which represents initiatives with attributes like moraleName, startDate, and endDate, and Questions, which stores the associated questions categorized by type and topic. Additional entities include Participants, tracking employees assigned to specific initiatives and their completion status, and Answers, which record responses linked to individual questions and forms. These relationships ensure efficient tracking of participation, rewards management, and engagement analysis.

Assessment 360

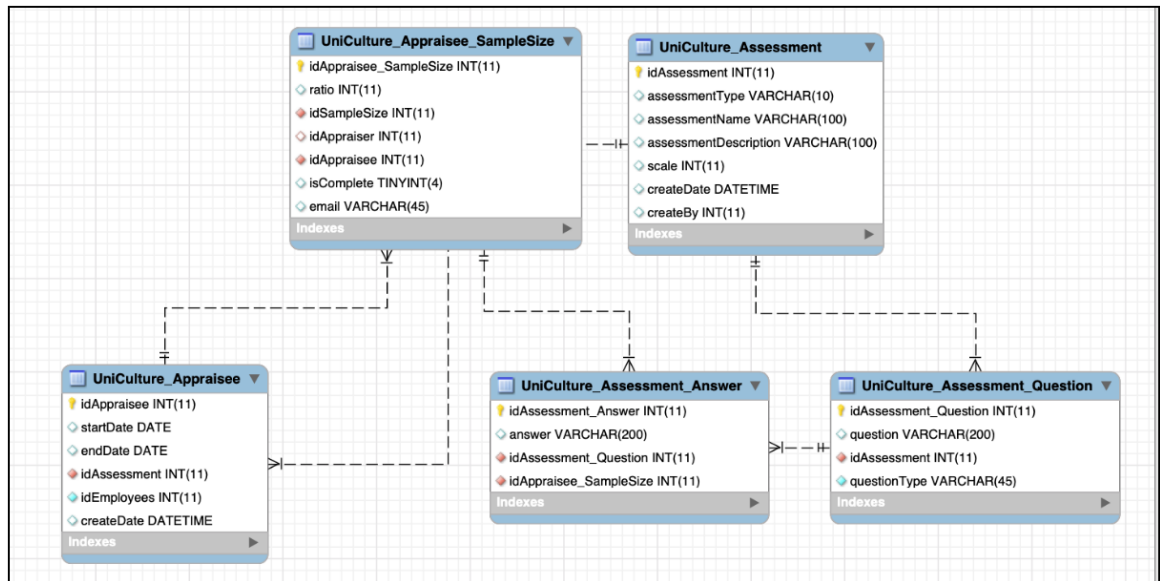


Figure 3.29 Entity relationship diagram of Assessment 360

In Figure 3.29, for the 360 Assessment, the ER diagram focuses on multi-rater evaluations, capturing entities like Assessments, which define evaluation sessions with details such as start and end dates, and Participants, tracking individuals being assessed and their appraisers. The Feedback entity stores responses from appraisers, linked to specific questions and participants, facilitating comprehensive performance reviews. The interconnected structure of these entities supports seamless data flow, enabling detailed analysis and insights into morale trends and performance evaluations across the organization.

3.6.2 Data Dictionary

Table name : UniCulture_Morale

Description : The 'UniCulture_Morale' table, as shown in Table 3.14, serves as a repository for storing the information about morale question form, including their details, status, and associations with companies.

Table 3.14 Morale table

Name	Data type	Key	Description
idMorale	INT	PK	Unique identifier for each morale entry.
moraleName	VARCHAR		Name of the morale initiative.
code	VARCHAR		Unique code to identify the morale entry.
description	VARCHAR		A brief description of the morale question.
picture	LONGTEXT		URL or path for an associated image.
startDate	DATETIME		The starting date of the morale question.
endDate	DATETIME		The ending date of the morale question.
moraleStatus	VARCHAR		Status of the morale question.
moraleType	VARCHAR		Type or category of the morale question.
idCompany	INT		The associated company.

Table name : UniCulture_MoraleQuestion

Description : The ‘UniCulture_MoraleQuestion’ table, as shown in Table 3.15, links morale entries to their associated questions, enabling a one-to-many relationship between morale and questions.

Table 3.15 Morale question table

Name	Data type	Key	Description
idMoraleQuestion	INT	PK	Unique identifier for each morale question link.
idMorale	INT	FK	Referencing the UniCulture_Morale table.
idQuestion	INT	FK	Referencing the UniCulture_Question table.

Table name : UniCulture_Question

Description : The ‘UniCulture_Question’ table, as shown in Table 3.16, serves as a repository for storing individual questions, including their text, type, and associated topics.

Table 3.16 Question table

Name	Data type	Key	Description
idQuestion	INT	PK	Unique identifier for each question.
question	VARCHAR		The text of the question.
questionType	VARCHAR		Type of the question.
idQuestionTopic	INT	FK	Referencing the UniCulture_QuestionTopic table.

Table name : UniCulture_QuestionTopic

Description : The ‘UniCulture_QuestionTopic’ table, as shown in Table 3.17, serves as a repository for storing the information about topics associated with questions, enabling categorization.

Table 3.17 Question topic table

Name	Data type	Key	Description
idQuestionTopic	INT	PK	Unique identifier for each topic.
questionTopic	VARCHAR		The name of the topic.
idCompany	INT		Referencing the associated company.
createAt	DATETIME		Timestamp indicating when the topic was created.
updateAt	DATETIME		Timestamp indicating the last update to the topic.

Table name : UniCulture_MoraleAnswer

Description : The ‘UniCulture_MoraleAnswer’ table, as shown in Table 3.18, serves as a repository for storing the answers provided by participants for specific morale questions.

Table 3.18 Morale answer table

Name	Data type	Key	Description
idMoraleAnswer	INT	PK	Unique identifier for each answer.
answer	VARCHAR		The text or value of the participant’s answer.
idMoraleQuestion	INT	FK	Referencing the UniCulture_MoraleQuestion table.
idMoraleParticipant	INT	FK	Referencing the UniCulture_MoraleParticipant table.

Table name : UniCulture_MoraleParticipant

Description : The ‘UniCulture_MoraleParticipant’ table, as shown in Table 3.19, serves as a repository for storing the information about tracks participants involved in morale yearly question, their progress, and completion status.

Table name : UniCulture_MoraleDailyQuestion

Table 3.19 Morale participant table

Name	Data type	Key	Description
idMoraleParticipant	INT	PK	Unique identifier for each participant entry.
idEmployees	INT		The employees participating in the morale yearly question.
idMorale	INT	FK	Referencing the UniCulture_Morale table.
isComplete	TINYINT		Boolean indicating whether the participant has completed the morale.

Description : The ‘UniCulture_MoraleDailyQuestion’ table, as shown in Table 3.20, serves as a repository for storing the information about daily morale questions and their metadata, including reward details.

Table 3.20 Morale daily question table

Name	Data type	Key	Description
idMoraledailyQuestion	INT	PK	Unique identifier for each daily question.
questionType	VARCHAR		Type of the daily question.
question	VARCHAR		The text of the daily question.
reward	INT		Points associated with answering the question.
createDate	DATETIME		Timestamp indicating when the question was created.
idQuestionTopic	INT	FK	Referencing the associated topic.
idCompany	INT		The associated company.
isActive	TINYINT		Boolean indicating if the question is currently active.

Table name : UniCulture_MoraleDaily

Description : The ‘UniCulture_MoraleDaily’ table, as shown in Table 3.21, serves as a repository for storing the information about tracks daily morale question, linking them to specific daily questions.

Table 3.21 Morale daily table

Name	Data type	Key	Description
idMoraleDaily	INT	PK	Unique identifier for each morale daily.
startDate	DATETIME		The starting date of the morale daily.
endDate	DATETIME		The ending date of the morale daily.
idMoraledailyQuestion	INT	FK	Referencing the UniCulture_MoraleDailyQuestion table.

Table name : UniCulture_MoraleDailyParticipant

Description : The ‘UniCulture_MoraleDailyParticipant’ table, as shown in Table 3.22, serves as a repository for storing the information about tracks participants’ responses to daily morale questions.

Table 3.22 Morale daily participant table

Name	Data type	Key	Description
idEmployees	INT	PK	The employees participating in the morale daily.
idMoraleDaily	INT	PK, FK	Referencing the UniCulture_MoraleDaily table.
answer	VARCHAR		The participant’s answer to the daily question.
answerDate	DATETIME		Timestamp indicating when the answer was submitted.

Table name : UniCulture_Appraisee

Description : The ‘UniCulture_Appraisee’ table, as shown in Table 3.23, serves as a repository for storing the information about assessment that has been assigned to the employee within the company.

Table 3.23 UniCulture_Appraisee

Name	Data type	Key	Description
idAppraisee	INT	PK	Unique identifier for the appraisee record.
startDate	DATE		Start date of the available period of assigned assessment.
endDate	DATE		Expiry date for the assigned assessment.
idAssessment	INT	FK	Identifier of the assessment.
idEmployees	INT		Identifier of the employees.
createDate	DATETIME		Date when the appraisee was created.

Table name : UniCulture_Appraisee_SampleSize

Description : The ‘UniCulture_Appraisee_SampleSize’ table, as shown in Table 3.24, serves as a repository for storing the information about who is assigned as the appraiser for the assessment.

Table 3.24 UniCulture_Appraisee_SampleSize

Name	Data type	Key	Description
idAppraisee_SampleSize	INT	PK	Unique identifier for the sample size record.
ratio	INT		Weight for each appraiser’s answer calculation.
idSampleSize	INT	FK	Identifier of sample size position.
idAppraiser	INT	FK	Identifier of the appraiser.
idAppraisee	INT	FK	Identifier of the appraisee.
isComplete	TINYINT		Status of the assigned assessment indicates whether the appraiser has answered the assessment. (enum value: 0/1)
email	VARCHAR		Email of the outsider appraiser.

Table name : UniCulture_Assessment

Description : The ‘UniCulture_Assessment’ table, as shown in Table 3.25, serves as a repository for storing the information about the general information for the each assessment.

Table 3.25 UniCulture_Assessment

Name	Data type	Key	Description
idAssessment	INT	PK	Unique identifier for the assessment.
assessmentType	VARCHAR		Type of the created assessment.
assessmentName	VARCHAR		Name of the created assessment.
assessmentDescription	VARCHAR		Description of the created assessment.
scale	INT		The scale of the assessment used for the answer score calculation.
createDate	DATETIME		Date when the appraisee was created.
createBy	INT		Identifier of the employee who created this assessment.

Table name : UniCulture_Assessment_Answer

Description : The ‘UniCulture_Assessment_Answer’ table, as shown in Table 3.26, serves as a repository for storing the information about answers from the appraiser in each assessment.

Table name : UniCulture_Assessment_Question

Table 3.26 UniCulture_Assessment_Answer

Name	Data type	Key	Description
idAssessment_Answer	INT	PK	Unique identifier for the answer.
answer	VARCHAR		Submitted answer from the appraiser.
idAssessment_Question	INT	FK	Identifier of the question in the assessment.
idAppraiser_SampleSize	INT	FK	Identifier of the appraiser.

Description : The ‘UniCulture_Assessment_Question’ table, as shown in Table 3.27, serves as a repository for storing the information about questions in the assessment.

Table 3.27 UniCulture_Assessment_Question

Name	Data type	Key	Description
idAssessment_Question	INT	PK	Unique identifier of the assessment.
question	VARCHAR		Question in the assessment.
idAssessment	INT	FK	Identifier of the assessment.
questionType	VARCHAR		Type of the question. (enum value: choice, text)

Table name : Training_OnlineVideoTotalWatching

Description : The ‘Training_OnlineVideoTotalWatching’ table, as shown in Table 3.28, serves as a repository for storing the information about total video time of learning.

Table 3.28 Training_OnlineVideoTotalWatching table

Name	Data type	Key	Description
idTraining_OnlineVideoTotalWatching	INT	PK	Unique identifier of video duration.
idEmpOnlineCourse	INT	FK	Identifier of the employee online course.
idOnlineDocuments	INT	FK	Identifier of the online document.
idEmployee	INT	FK	Identifier of the employee.
totalTime	INT		Tracking total time of watching video.
percentage	INT		Percentage of watching the video.
status	INT		Completed as 1 or not completed as 0.

Table name : Training_Competency Description : The ‘Training_Competency’ table, as shown in Table 3.29, serves as a repository for storing the information about competency.

Table 3.29 Training_Competency table

Name	Data type	Key	Description
idCompetency	INT	PK	Unique identifier of video duration.
competencyName	VARCHAR		Name of competency.
competencyAspect	VARCHAR		Aspect of competency.
competencyCode	VARCHAR		Code of competency.
idJobGroup	INT	FK	Identifier of job group.
competencyType	VARCHAR		Type of competency.
idCompany	INT	FK	Identifier of company.
img	VARCHAR		Image of the competency.
createBy	INT		Identifier of creation person.
createDate	DATETIME		Date of the competency creation.
updateBy	INT		Identifier of update person.
updateDate	DATETIME		Date of the competency update.
isActive	TINYINT		The status of the competency.

3.7 Products of The Design Process

The final product will be a user interface (UI) design that prioritizes an exceptional user experience while reflecting the company's identity. The design will feature a clear and intuitive navigation structure, allowing users to easily interact with the platform. The UI will integrate the company's color schemes, typography, and thematic elements to create a visually appealing and cohesive interface. This design will ensure both functionality and aesthetic alignment with the brand, resulting in a user-friendly experience that accurately represents the company's identity.

3.7.1 Navigation Map

- **Morale daily and yearly question form**

- **Morale daily questions management:** It enables admins to view, create, and assign daily morale questions in one streamlined interface. Admins can filter by question topic and status for specific questions, define the question type, topic, and rewards, and assign start and end dates for target groups such as departments or all companies. This integration ensures efficient management of daily morale questions.
- **Morale yearly questions management:** It provides admins with a unified platform to manage yearly morale forms. Admins can create, edit, delete, publish, and assign forms to specific groups, ensuring smooth deployment. This feature streamlines the entire process of long-term morale question management.
- **Morale daily and yearly questions respond:** This function allows users to view available daily and yearly morale questions assigned to them. Users can access the questions via their question form list, submit their responses, and submit their responses within the designated timeframe. For the daily forms, can receive rewards after completing the form. This feature enhances engagement by encouraging daily and yearly interaction with morale questions.
- **View daily and yearly question results:** Admins can analyze participant engagement, compare responses across departments or individuals, assess completion rates, and in daily form review structured morale data with insights into individual question scores and overall averages, either by question or respondent list.

- **Morale daily and yearly dashboard**

- **Morale daily dashboard:** This feature is accessible exclusively to administrators, who have the authority to view the dashboard. The dashboard provides an overview of employee work satisfaction across the company, indicating whether the company's overall average score reflects a positive or negative morale. Additionally, the dashboard presents detailed insights and analyses at various levels, including the company, departments, and specific sections. Users can interact with the dashboard by applying filters to focus on particular companies, departments, sections, and date ranges. Furthermore, the dashboard includes functionality to export an employee table that outlines the average scores provided by individual employees, as well as the scores for each contributing factor. This comprehensive tool facilitates informed decision-making by offering granular insights into organizational morale.
- **Morale yearly dashboard:** This feature is accessible to both administrators and managers, with differing levels of access. Managers are limited to viewing data for their assigned company, whereas administrators have access to all registered companies. The dashboard provides a visual

summary of the yearly morale survey, which can be conducted annually or quarterly. Users can interact with the dashboard by selecting specific morale metrics and applying filters to explore the data further. Additionally, users can review feedback related to individual survey questions, enabling deeper insights into employee sentiment and satisfaction.

- **Assessment 360**

- **Assessment management:** This section serves as the central hub for managing assessments. It is the initial page where admins can oversee and organize all assessments. The page provides access to buttons and tools for creating, editing, duplicating, assigning, or deleting assessments. Admins can efficiently manage all tasks related to assessments from this interface.
- **Assessment 360 respond:** This feature allows users (e.g., managers or employees) to view all assigned assessments available for completion. Users can select their desired assessment and respond to the questions, which can include both text-based and multiple-choice formats. Responses are calculated into scores, which contribute to the assessment results.
- **Assessment 360 answer display:** Once users have submitted their responses, this feature presents all the collected data in a clear and organized manner. It includes individual question scores and overall averages. Admins can view responses through two perspectives: by question list or by respondent list. This section provides essential insights into performance through a structured display of the results.

All functions are organized within a navigation map tailored to each user role. As shown in Figure 3.30, the navigation map for the Admin role provides access to administrative features and system management tools. Similarly, Figure 3.31 illustrates the navigation map for the Manager role, focusing on team oversight and performance tracking. Finally, Figure 3.32 depicts the navigation map for the User role, offering features designed for individual task completion and feedback. Each map is customized to ensure a role-specific and intuitive user experience.

- Admin role

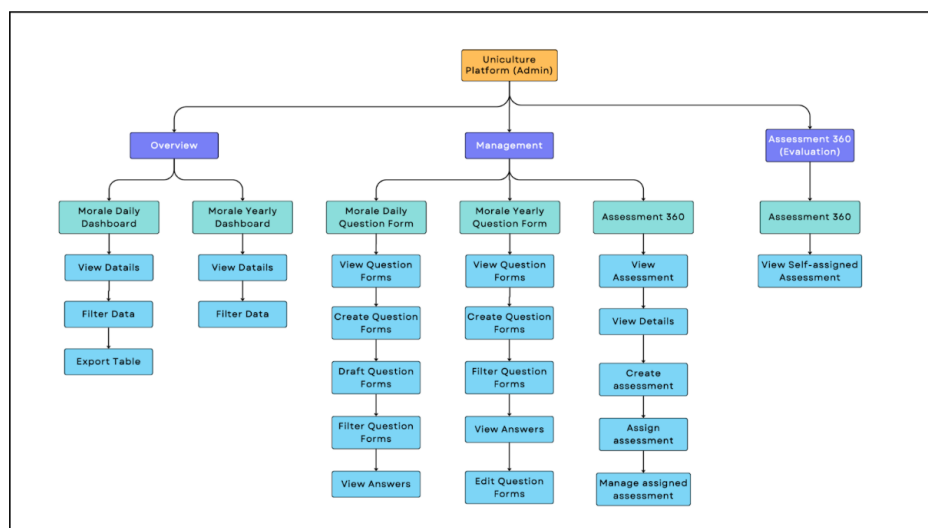


Figure 3.30 Navigation map of admin role on UniCulture

- Manager role

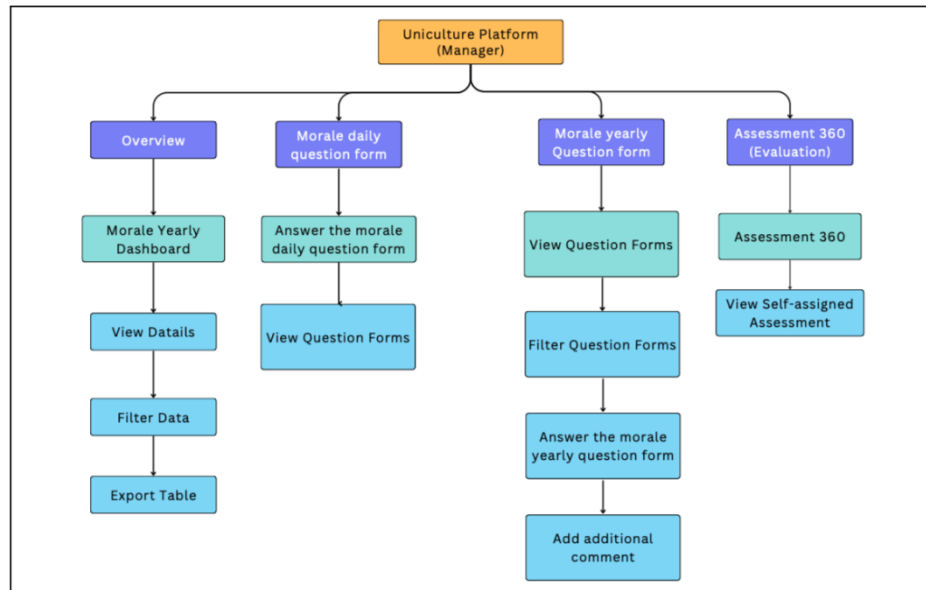


Figure 3.31 Navigation map of manager role on UniCulture

- User role

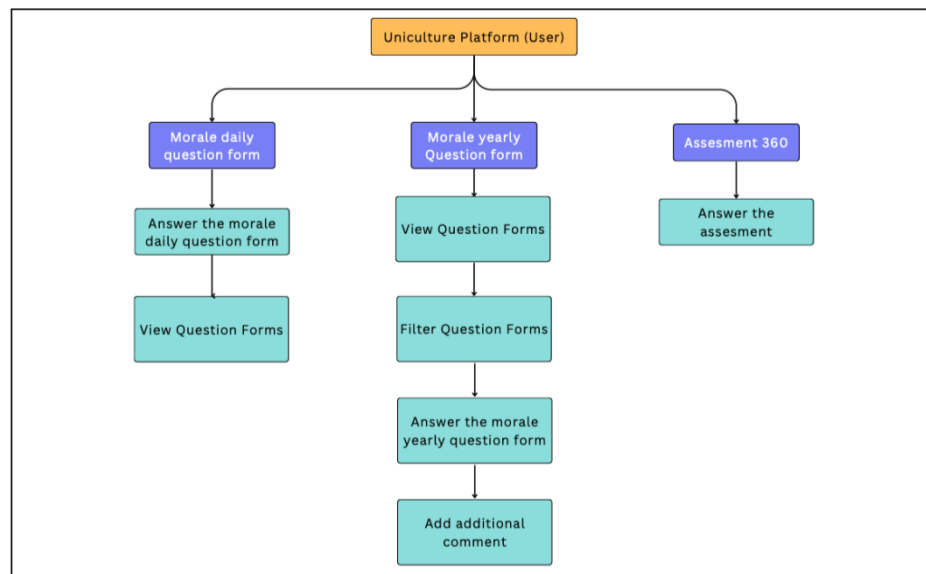


Figure 3.32 Navigation map of user role on UniCulture

- Rush Quiz

- **Game Joining Function:** Manages the critical first interaction where players enter the Rush Quiz game. It encompasses a comprehensive flow that begins with collecting and validating the game PIN entered by the player, ensuring it matches an active quiz session. The function then prompts for the player's name, validates it for appropriate length and content, and combines both pieces of information to send a registration request to the game server. Throughout this process, the

function provides immediate feedback to users if their PIN is invalid, if the game is already in progress or full, or if their chosen name is already taken by another player.

- **Lobby Management:** Creates a welcoming intermediate space where players wait for the quiz to begin. After successful registration, this function displays a confirmation screen showing the player’s registered name and the game PIN they’ve joined, serving as reassurance that they’re in the right place. It establishes a real-time connection to receive updates about other players joining the same session. The function maintains this waiting state while continuously checking for the host’s signal to start the game, at which point it transitions players smoothly to the first question. This intermediate step builds anticipation and ensures all players are properly synchronized before the competition begins.
- **Question Display:** The core interaction point where players engage with quiz content. When the server indicates a new question is available, this function retrieves the question text, possible answers, and time limit, then renders them on screen in an engaging format. It initializes and displays a visual countdown timer that decreases in real-time, creating a sense of urgency. The function arranges multiple choice answers as interactive, clearly distinguishable buttons that players can tap or click to make their selection. It handles different question types, including text-only questions, questions with images, and potentially other multimedia elements. Throughout each question’s duration, this function maintains the timer state and prepares to either receive the player’s answer or handle the time expiration event.

- **Video Course Learning**

- **Course Library:** This section serves as the main hub where users can browse through all available video courses. Users have access to filter tools that help narrow down course selections by keyword or category. The intuitive interface allows users to start learning by simply clicking on a course of interest. This area encourages self-paced learning and provides an overview of course topics and durations.
- **Video Learning Playback:** Once a course is selected, users are directed to the video learning page. This feature provides a smooth video playback experience. While the user watches the video, the system tracks their progress. Upon completion, the system automatically marks the course as “completed” and updates the user’s learning status. To enrich the experience, the system may suggest additional courses or provide a short quiz (rush quiz) to reinforce understanding.
- **Learning Progress and Recommendations:** Users can monitor their completed and in-progress courses through a structured learning dashboard. AI-powered recommendations suggest new courses based on user behavior, past learning, or role. This intelligent system promotes continuous learning by tailoring suggestions to individual development goals.

All functions are integrated within role-based navigation maps. Figure 3.33 shows the User role, enabling users to explore, learn, and track their own learning progress. Each navigation map ensures that users have access to relevant tools and features based on their roles.

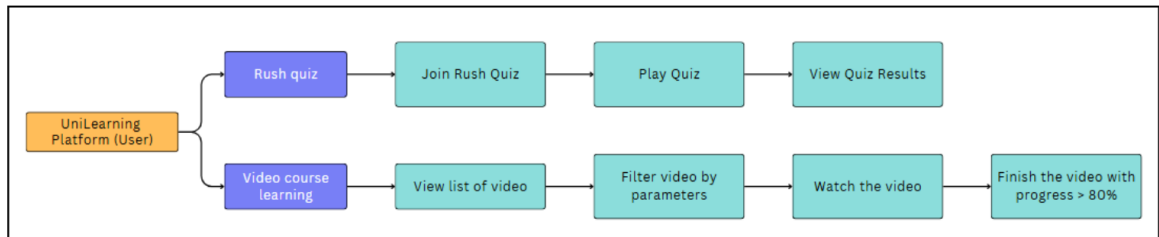


Figure 3.33 Navigation map of user role on UniLearning

3.7.2 System Architecture

In Figure 3.34, the architecture begins with the Web, where users interact with the front-end interface. The API Gateway receives requests from the front end and forwards them to the Back End. The Back End consists of two main components: the Business Logic layer, implemented using Express.js (Node.js), and the Machine Learning API, built using FastAPI (Python). Express.js handles the core application logic and workflows, while FastAPI manages machine learning-related operations such as predictions or recommendations. Both components can access and process data stored in the Local Database, which is managed using MySQL Workbench or hosted on Google Cloud for improved scalability and management. Once processing is complete, results are sent back through the API Gateway to the front end, allowing users to view the output. This structure ensures modular, scalable, and maintainable request handling across business and machine learning functions.

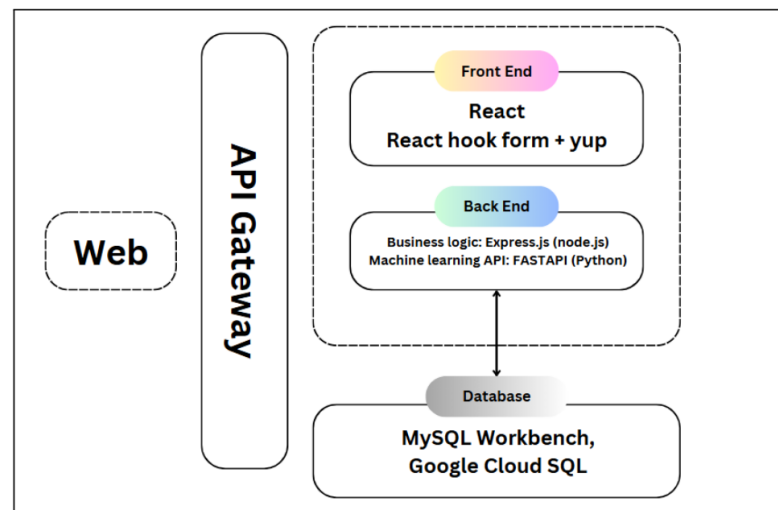


Figure 3.34 System architecture of our website

3.7.3 User Interface

UniCulture

The pages in the UniCulture module focus on the HRM features, including both newly designed pages and redesigned existing ones.

Morale Daily Question Form

• Question Management

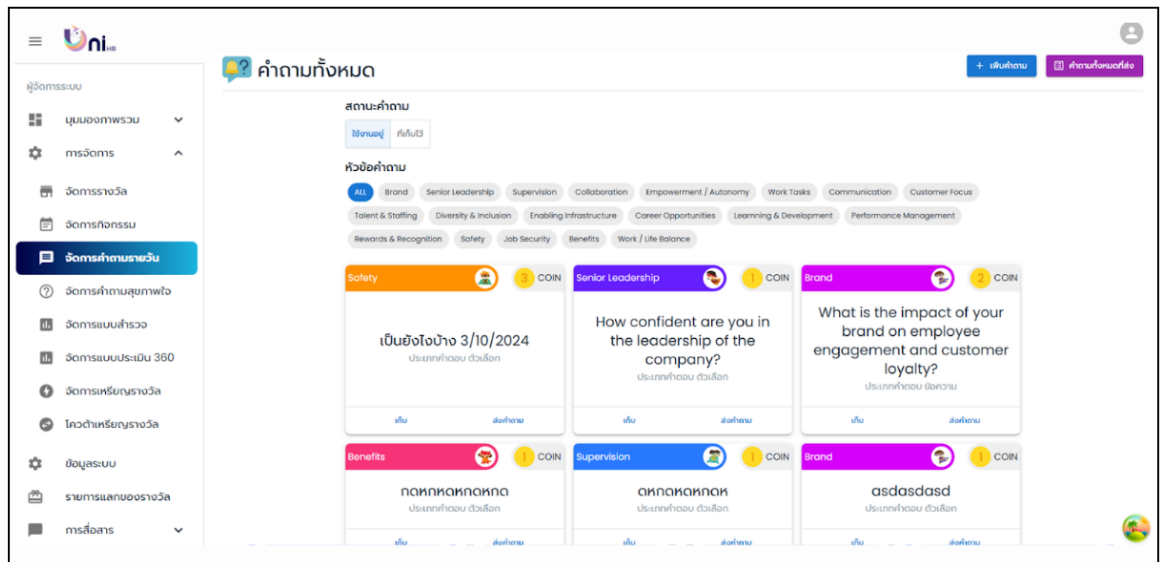


Figure 3.35 Question management of morale daily question form user interface before redesign

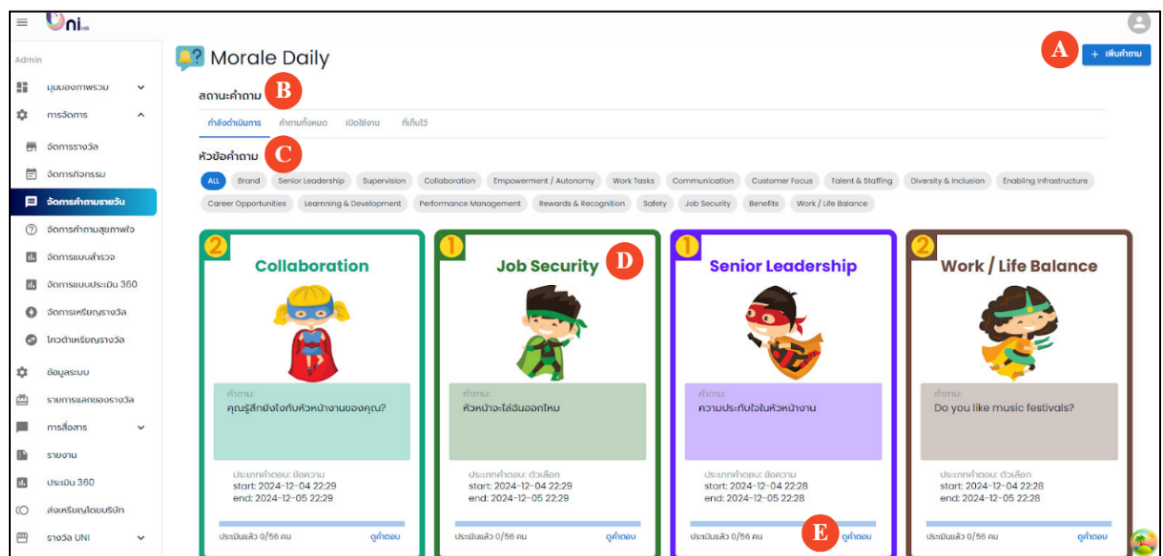


Figure 3.36 Question management of morale daily question form user interface after redesign

The initial design of the Question Management Interface in Figure 3.35 did not fully meet user requirements. To address these issues, the interface has been redesigned (see Figure 3.36). The updated interface enhances user experience by improving functionality and aligning with the system's requirements. It includes features related to the morale daily question form, such as creating questions, filtering question forms by status, and evaluating scores from users' responses. It makes it the central page for admins to manage the morale daily question form effectively.

Callout A - Create a new question form (เพิ่มคำถาม) button

This button allows admins to initiate the process of creating a new morale daily question form. Clicking it opens a modal where admins can define the question text, choose a question type and topic, and set the number of coins as a reward for completion.

Callout B - Filtering the status section

The filtering functionality helps admins categorize and view question forms based on their status, such as In progress, all questions, activated and stored. This allows for quick access to relevant question forms.

Callout C - Filtering the question topic section

Admins can filter questions by topic such as brand, supervision, collaboration, and communication making it easier to locate and manage forms that fall under specific categories or themes.

Callout D - Question form details

Displays key information about each question form, including the title, associated topics, start and end dates, and assigned rewards. This summary ensures admins can easily review and manage multiple forms.

Callout E - Participant progress bar

A visual progress bar that shows the participation rate for each question form, helping admins track engagement and response completion over time.

• Create Morale Daily Question Form

Figure 3.37 Create morale daily question form

The redesigned interface for creating morale daily question forms in Figure 3.37 focuses on ease of use and clarity. The modal includes fields to define the question type, topic, text, and reward coins. Admins can preview the form from the employee's perspective before saving, ensuring alignment with expectations. The save button finalizes the form, enabling admins to publish it or store it for later use. These features streamline the form creation process while providing flexibility and precision.

Callout A - Question type (ประเภทคำถาม)

Admins can select the format of responses, which are multiple-choice and open-ended. This ensures questions are tailored to collect the desired type of feedback.

Callout B - Question topic (หัวข้อคำถาม)

This dropdown allows admins to categorize questions by topic, enabling better organization and consistency across forms.

Callout C - Question text (คำถาม)

The text input field for defining the actual question to be answered by participants.

Callout D - Number of coins (จำนวนเหรียญรางวัล)

Admins can assign a specific number of coins as a reward for answering the question, and motivating users to participate.

Callout E - Preview of the form from the employee's perspective

Provides a real-time preview of the form, showing how it will appear to participants, and ensuring clarity and alignment with expectations.

Callout F - Save (บันทึก)

A button to save the question form, as an activated form ready for distribution or publishing.

- **Publish and Deactivate Question Forms**

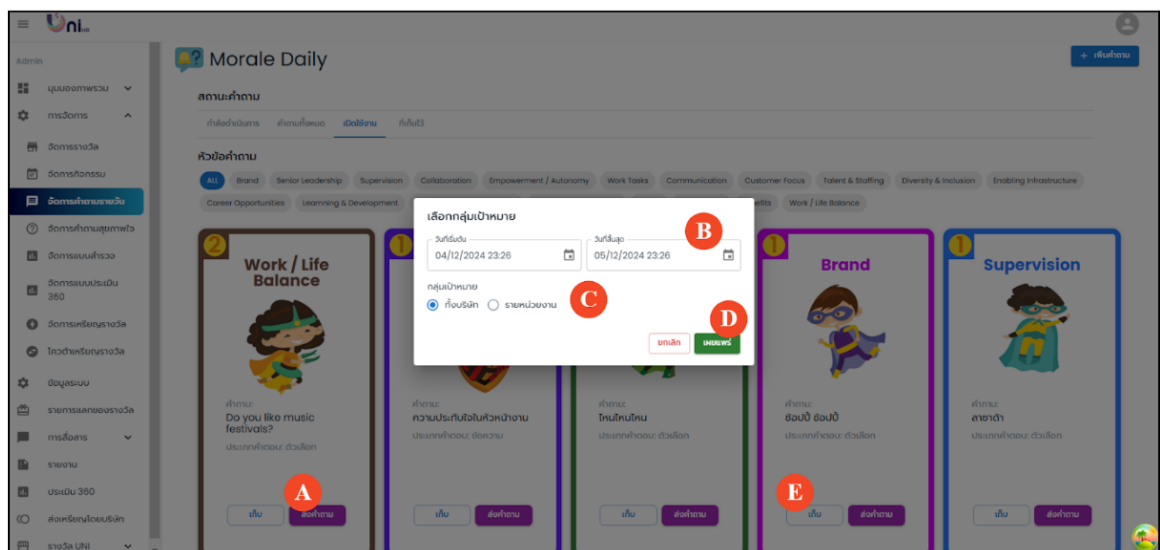


Figure 3.38 Publish and deactivate morale daily question form

The publish and deactivate section of the interface in Figure 3.38 provides options for activating or storing morale daily question forms. Admins can set start and end dates, define target groups, and monitor the status of each form. The “Publish” button makes the form live for participants, while the “Store” button archives the form for future use. These features ensure that question forms are deployed effectively and managed efficiently over time.

Callout A - Send question (ส่งคำตอบ) button

Admins can select the format of responses, which are multiple-choice and open-ended. This ensures questions are tailored to collect the desired type of feedback.

Callout B - Set start and end dates

This dropdown allows admins to categorize questions by topic, enabling better organization and consistency across forms.

Callout C - Specify target group

The text input field for defining the actual question to be answered by participants.

Callout D - Publish (เผยแพร่) button

Admins can assign a specific number of coins as a reward for answering the question, and motivating users to participate.

Callout E - Store (เก็บ) button

Provides a real-time preview of the form, showing how it will appear to participants, and ensuring clarity and alignment with expectations.

• Morale Daily Answer Page

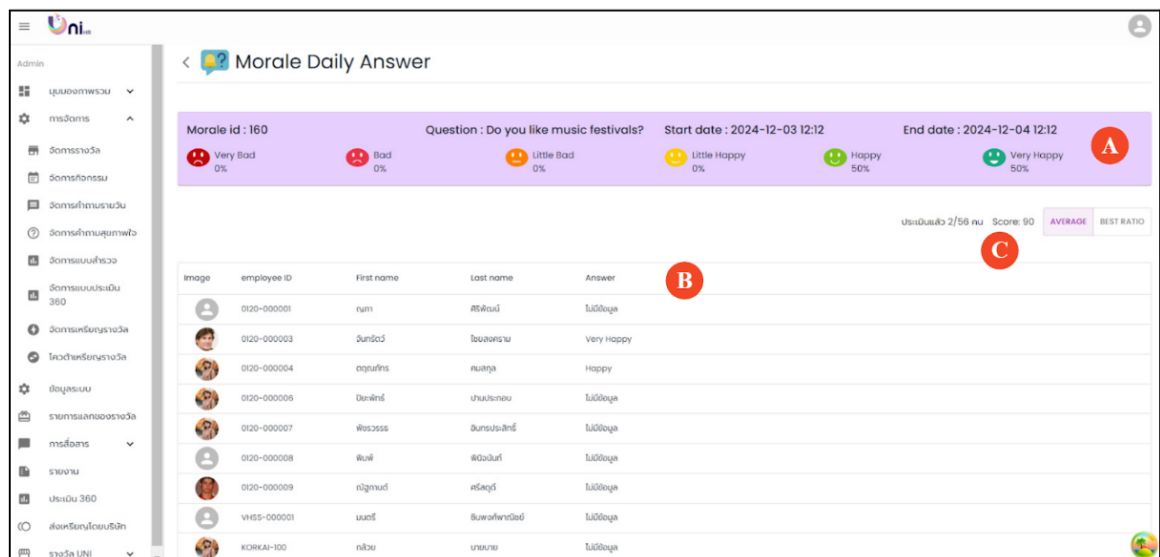


Figure 3.39 Morale daily answer page

This page in Figure 3.39, presents a detailed overview of responses to morale daily question forms. Admins can view the list of all participants, their answers, and individual completion statuses. A calculation section summarizes scores or metrics derived from the responses, providing insights into trends and overall performance. This functionality makes it easy to evaluate engagement and morale levels systematically.

Callout A - Question form details section

Displays comprehensive details of the form, including the ID, questions, start and end date, and response options.

Callout B - All target users with their answer

Shows a list of all participants and their submitted responses, making it easy for admins to track engagement.

Callout C - Calculation scores section

Provides summarized scores or metrics derived from the responses, offering a quick overview of results and trends.

- Respond Morale Daily Question Form

The screenshot displays a web application interface for responding to morale daily question forms. On the left, there is a sidebar menu with various navigation options. The main content area shows a list of questions, with the 'Job Security' question selected and its response form displayed in the center. The response form for 'Job Security' includes a title 'หัวหน้าจะไล่จับออกไหม' and five response options: VERY HAPPY, HAPPY, LITTLE HAPPY, LITTLE BAD, BAD, and VERY BAD, each with a yellow circle icon. The interface also shows a 'Senior Leadership' question on the right.

Figure 3.40 User responds to morale daily question form

The user interface for responding to morale daily question forms in Figure 3.40 displays a clean layout with a list of questions and response options or text fields for input answers. For multiple-choice, users can click the selected option button to submit the form. The interface ensures clarity and engagement, enabling users to complete their responses quickly and accurately.

Callout A - Answer question (ตอบคำถาม) button

Allows users to start responding to the selected question form.

Callout B - Question form

Displays the questions, including input fields for open questions, and response options for multiple-choice for users to complete the question form.

Morale Yearly Question Form

• Question Management

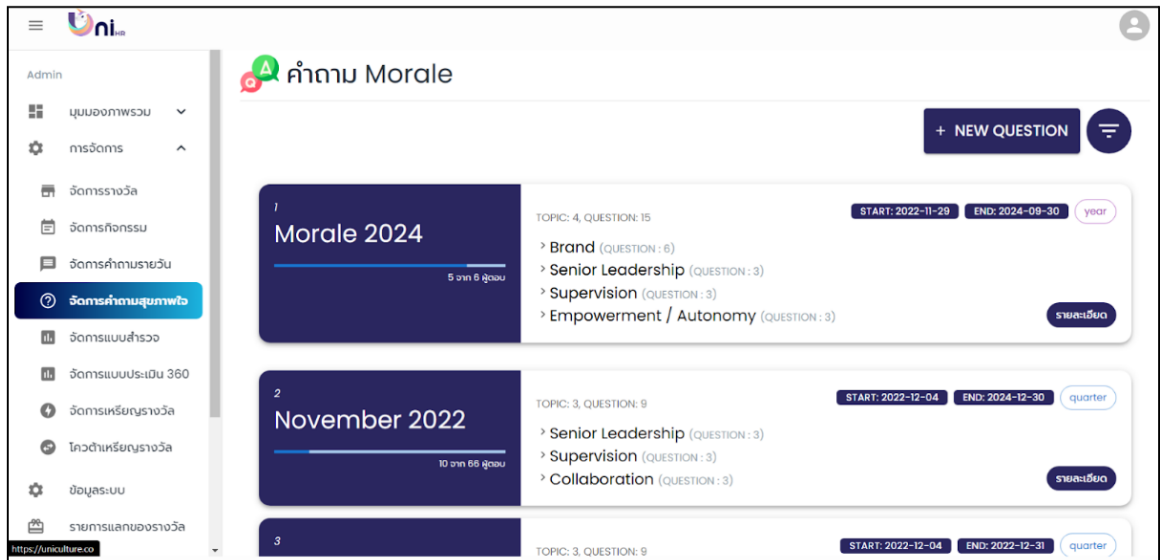


Figure 3.41 Question management of morale yearly question form user interface before redesign

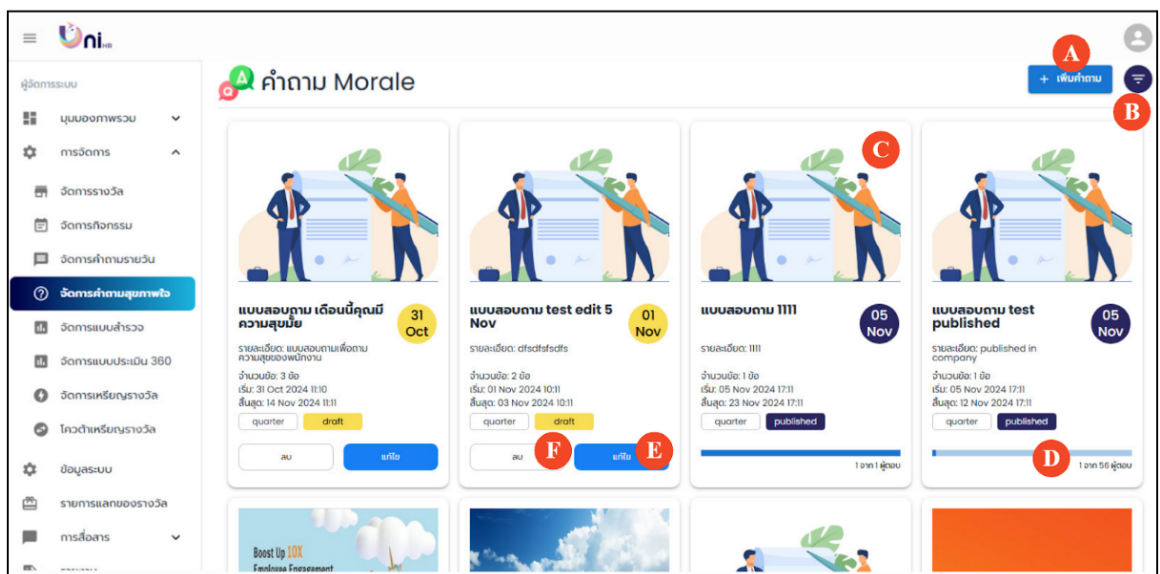


Figure 3.42 Question management of morale yearly question form user interface after redesign

The initial design of the Question Management Interface in Figure 3.41 did not fully meet user requirements. To address these issues, the interface has been redesigned (see Figure 3.42). The updated interface enhances user experience by improving functionality and aligning with the system's requirements. It includes tools for creating, editing, and deleting question forms. Admins can filter forms by topic or status, view details like associated questions and timelines, and monitor participant progress through visual progress bars. These

features centralize yearly question management, enabling admins to manage long-term morale initiatives effectively.

Callout A - Create question form (เพิ่มคำถาม) button

Admins can create a new yearly question form, similar to daily question forms but with extended scope and features.

Callout B - Question form filter section

Allows filtering by morale type and status form to help admins manage and locate forms effectively.

Callout C - Question form details

Shows an overview of each yearly question form, including titles, descriptions, number of questions, morale type, status, and deadlines.

Callout D - Participant progress bar

Indicates the number of participants who have completed the form.

Callout E - Edit question form (แก้ไข) button

Opens the selected form in edit mode, allowing admins to modify its contents.

Callout F - Delete question form (ลบ) button

Deletes a question form permanently from the system.

• Create Morale Yearly Question Form

Figure 3.43 Create morale yearly question form

The creation interface for morale yearly question forms in Figure 3.43 allows admins to upload a morale-related image, define form details, and add or edit questions. A question creation section lets admins specify question text, type, and topic. Duplicate and delete buttons enhance flexibility by enabling quick modifications. This user-friendly interface streamlines the creation of comprehensive yearly question forms.

Callout A - Upload morale picture

Admins can upload an image representing the form, such as a logo or morale campaign banner.

Callout B - Question form details section

Includes required fields for entering metadata, such as form title, description, morale type, and start and end dates.

Callout C - Question creating section

Allows admins to create individual questions with text, types, and topics. These are the fields that you have to fill out before saving the question form.

Callout D - Add question (เพิ่มคำถาม) button

Let admins include new questions in the form.

Callout E - Duplicate and delete question button

Enables duplicating existing questions with the same topic and type for quick creation or removing unwanted ones.

• Publish Morale Yearly Question Form

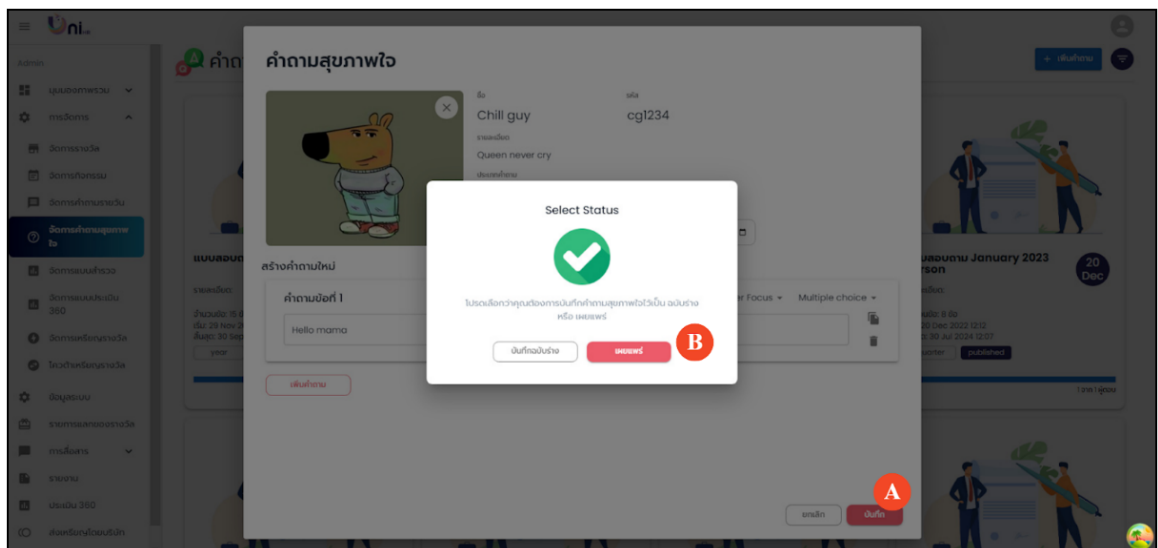


Figure 3.44 Publish morale yearly question form

In Figure 3.44, this section includes tools for saving as drafts and publishing morale yearly question forms. Admins can save forms in draft or published mode and ensure that forms are deployed accurately. The interface simplifies the transition from draft to active forms, ensuring that yearly morale initiatives are launched efficiently.

Callout A - Save question form (บันทึก) button

Saves the form in its current state and then displays a dialog for admins to select the status.

Callout B - Select status section

Allows admins to set the form's status as draft or published. If the published button is selected, displayed the dialog to specify the target group.

- Edit Morale Yearly Question Form

Figure 3.45 Edit morale yearly question form

The interface for editing morale yearly question forms in Figure 3.45 is designed to allow admins to refine existing forms efficiently. Admins can update form details, such as titles, descriptions, or associated dates, through the “Edit Form Details” section. They can also modify existing questions or add new ones in the “Edit Question Section.” A dedicated save button ensures that all changes are securely stored, while the interface remains user-friendly and intuitive, supporting admins in keeping morale assessments up-to-date and relevant.

Callout A - Edit question form details section

Enables admins to update general information about the form, such as the name, description, code, start and end date, and morale type.

Callout B - Edit question section

Allows admins to make changes to existing questions, including text, type, or topic.

Callout C - Add new questions

Provides a simple option for adding additional questions with the selected topic and type to the form.

Callout D - Save editing question form (บันทึก) button

Saves all modifications, ensuring that the updated form is ready for publishing or further use.

• **Morale Yearly Answer Page**

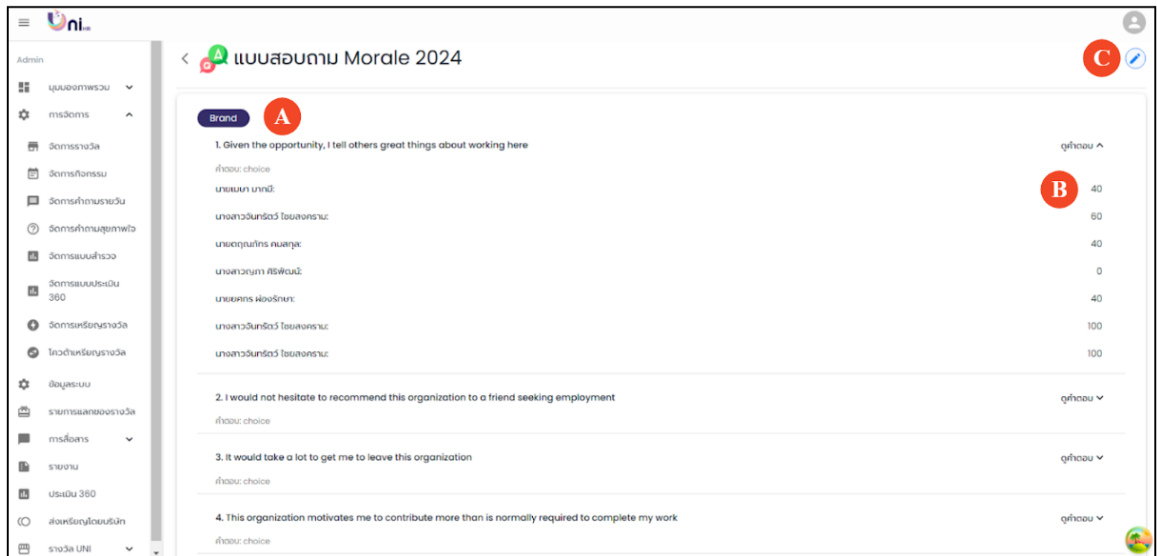


Figure 3.46 Morale yearly answer page

In Figure 3.46, this page offers admins a detailed view of participant responses to morale yearly question forms. The “Question List for Each Topic” section organizes questions by topics, providing clarity and structure. Admins can review all submitted answers in the “All Answers in Each Question” section, offering insights into individual and group performance. Additionally, admins can edit the morale name to reflect updated goals or initiatives, ensuring alignment with organizational objectives. This page is essential for tracking engagement and analyzing yearly morale trends.

Callout A - Question list for each topic

Displays all questions grouped by topic, providing an organized view for admins.

Callout B - All answers in each question

Lists all responses for each question, helping admins evaluate individual and collective feedback.

Callout C - Edit the morale name

Allows admins to update the morale initiative name to align with evolving objectives.

- Morale Yearly Question Form List of Users

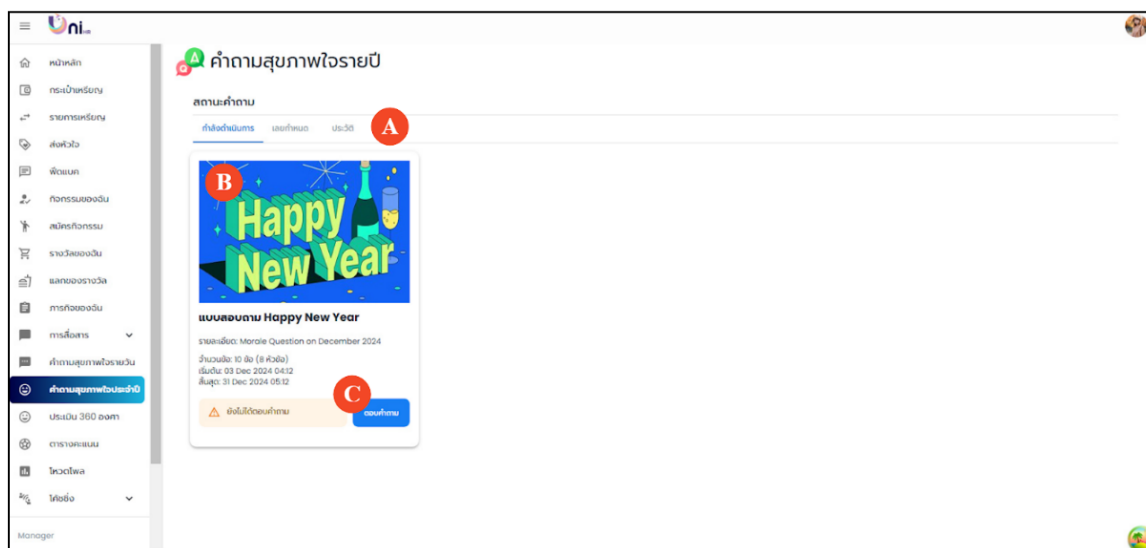


Figure 3.47 User’s morale yearly question form list

The morale yearly question form list in Figure 3.47 provides users with an organized view of all forms assigned to them. The filtering section allows users to sort forms by topic, status, or completion, ensuring easy navigation. Each form’s details, such as the name, timeline, and associated topics, are displayed for quick reference. Users can start responding to forms by clicking the “Answer Question” button, which redirects them to the form completion page. This interface ensures users can engage with morale initiatives efficiently and track their progress.

Callout A - Filter section

This enables users to sort and filter forms based on the specified time period and completed status. This is separated into in-progress, overdue, and history to store completed question forms.

Callout B - Question form details

Displays key information about each form, including its title, description, number of questions and topics, and timeline.

Callout C - Answer question (ตอบคำถาม) button

Allows users to begin responding to the selected form.

• Respond Morale Yearly Question Form

The screenshot shows a web application interface for a morale question form. On the left is a sidebar menu with various icons and text in Thai. The main content area has a header with a 'Happy New Year' banner and a timeline of question topics. Below the timeline is a 'Rewards & Recognition' section with a text input field and a 'Retrospective' button. Callouts A through E highlight specific features: A points to the 'Happy New Year' banner, B points to the timeline, C points to the 'Rewards & Recognition' section, D points to the 'Retrospective' button, and E points to the 'Submit Answers' button.

Figure 3.48 User responds to morale yearly question form

The interface for responding to morale yearly question forms in Figure 3.48 ensures a seamless and engaging experience for users. The “Question Form Details Section” presents all relevant information, including the form’s title, description, and timeline. Users can view the number of question topics and navigate through them easily. The “Answer Question Section” provides a clean layout for answering questions, ensuring clarity and accuracy in submissions. Users can use the “Retrospective” button to review their responses before submitting, while the “Submit Answers” button finalizes and records their input. This process supports active participation and accurate morale assessment.

Callout A - Question form details section

Displays the picture, title, description, and timeline of the form for user reference. Include explaining the meaning of each response option.

Callout B - Number of question topics

Shows the count of question topics with the number of questions in that topic in the form, helping users understand the scope.

Callout C - Answer question section

Provides input fields for users to complete their responses for open question type. Multiple choice has response options for users to select for answers to the question.

Callout D - Retrospective (ย้อนกลับ) button

Allows users to review their responses before submission.

Callout E - Submit answers (ส่งคำตอบ) button

Finalizes and records user responses.

Admin's Morale Daily Dashboard

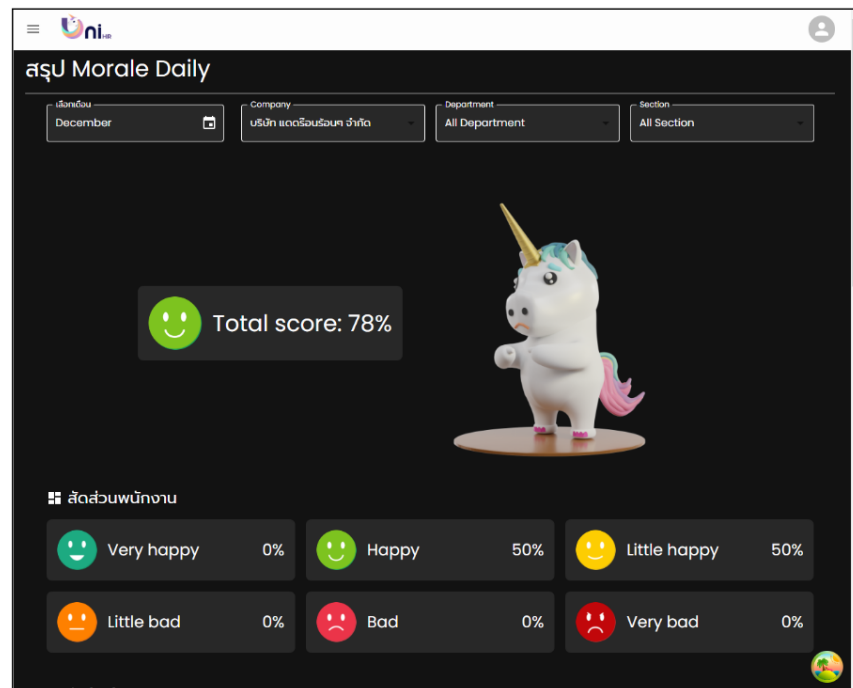


Figure 3.49 admin's morale daily dashboard before enhancement

In Figure 3.49, the total score calculation is incorrect; the feeling icon and the unicorn are not related.

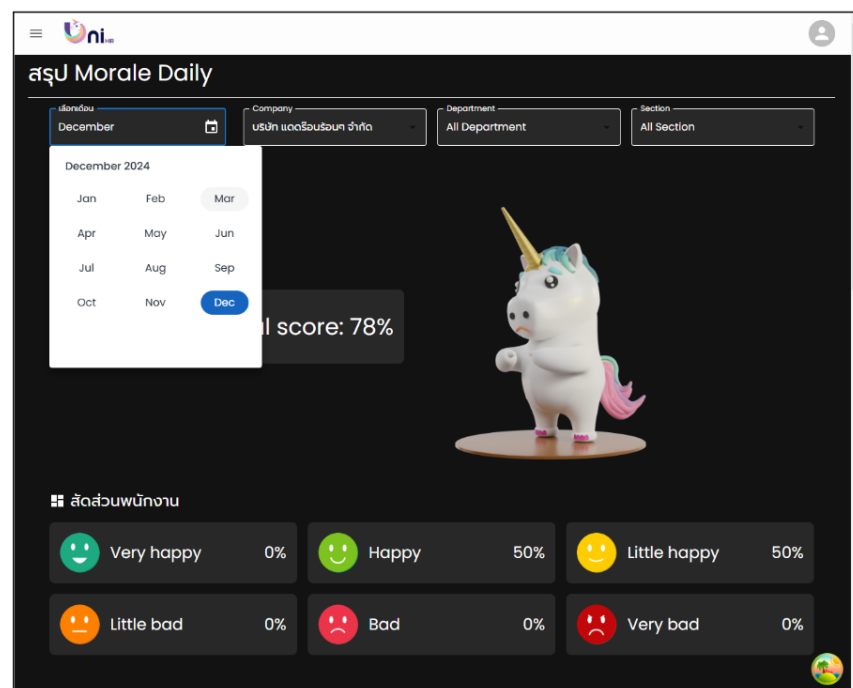
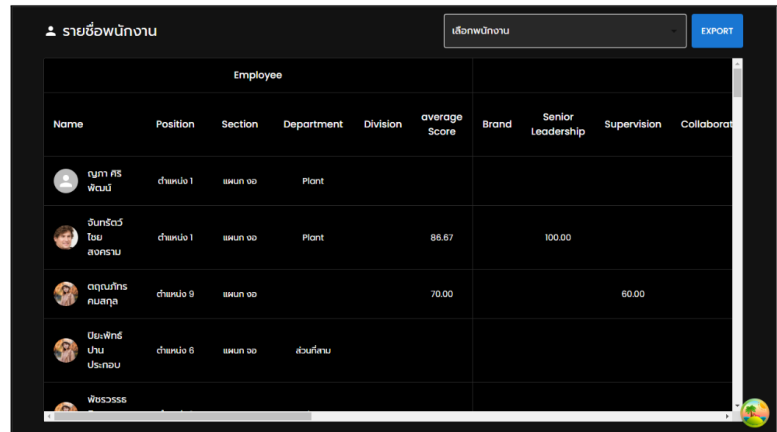


Figure 3.50 admin's morale daily dashboard filter before enhancement

In Figure 3.50, the original filter does not include an option for year selection, which may limit the ability to filter data based on specific time periods. This absence could affect the flexibility and accuracy of the data analysis.



Name	Position	Section	Department	Division	average Score	Brand	Senior Leadership	Supervision	Collaborator
คุณ อริส พันธุ์	ตำแหน่ง 1	แผนก ๑๐	Plant						
คุณธวัช โตย สอนกรณ	ตำแหน่ง 1	แผนก ๑๐	Plant		88.67		100.00		
คุณนพพร กลลฤา	ตำแหน่ง 9	แผนก ๑๐			70.00			60.00	
นายพริ้ง ปันย ปะรินท	ตำแหน่ง 6	แผนก ๑๐	ส่วนกลาง						
พริ้ง									

Figure 3.51 admin's morale daily dashboard filter before enhancement

In Figure 3.51, the original dashboard funnel is incorrect as it does not reflect the proper hierarchy. The correct sequence should be Division, Department, Section, and Position, in that order. This adjustment is crucial for accurately representing the organizational structure, ensuring that each level is properly categorized and aligned for better data analysis and reporting. The current structure could lead to confusion and misinterpretation of the data flow.

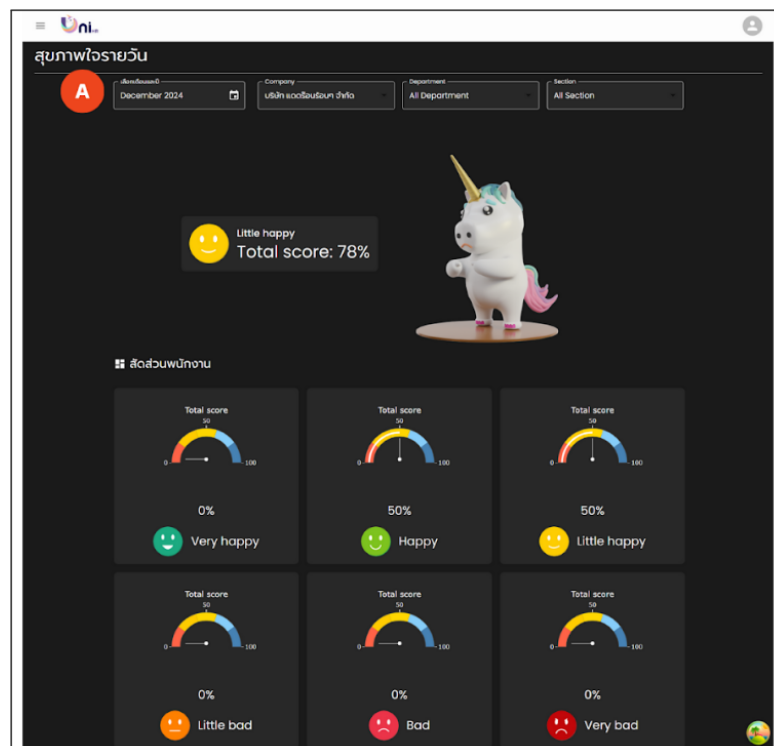


Figure 3.52 Admin's morale daily dashboard after enhancement

The interface displays the overall work satisfaction of the entire company based on the average scores collected from questionnaire responses. These responses are categorized into six levels of satisfaction: Very Bad, Bad, Slightly Bad, Slightly Happy, Happy, and Very Happy. Each question is associated with specific tags that indicate the factors to which the question belongs, such as communication, leadership, or work environment. The dashboard provides valuable insights by highlighting which factors are strengths or weaknesses for the company. By analyzing the aggregated data, the organization can identify areas that require improvement and their strengths.

In Figure 3.52, the enhancements include correcting the display of the total score and aligning the mood icons. Another enhancement is the addition of a gauge chart.

Callout A - Filter Button

Admins can use the filter button to refine the displayed data according to specific criteria, such as departments, sections, or time periods. This feature allows them to focus on particular aspects of employee satisfaction, enabling a more detailed and targeted analysis.

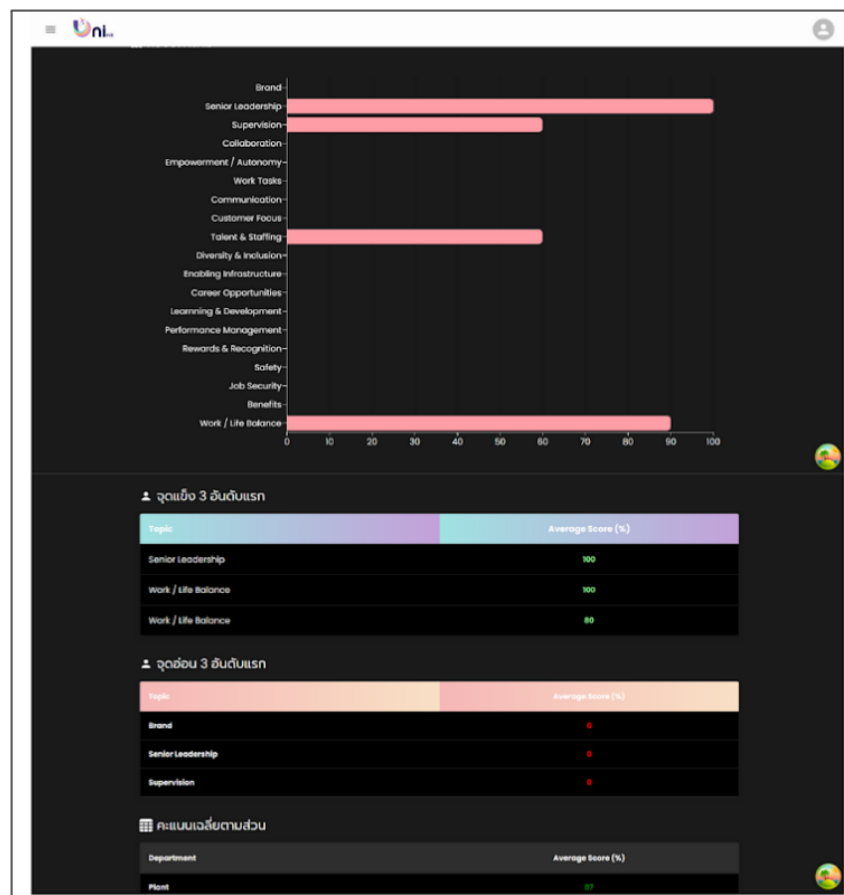


Figure 3.53 Admin's morale daily dashboard after enhancement

In Figure 3.53, the top 3 and bottom 3 factor tables have been added as enhancements.

คะแนนเฉลี่ยตามส่วน

Department	Average Score (%)
Plant	87
null	70
ส่วนโสม	0
ส่วนโศก	0
HRBP Manufacturing	0
ส่วนโศก	0
Operation	0
Y	0

คะแนนเฉลี่ยตามฝ่าย

Section	Average Score (%)
ส่วน ๑๑	0
ส่วน ๑๑	0
ส่วน ๑๑	0
null	0
Operation	0
Digital Capability & Innovation	0
ส่วน ๑๑	0
ส่วน ๑๑	0
Z	0

Figure 3.54 Admin's morale daily dashboard after enhancement

In Figure 3.54, an average score table by department and section has been added as an enhancement, following the requirements.

รายชื่อพนักงาน

Name	Division	Department	Section	Position	Average Score	Brand	Senior Leadership	Supervision
คุณ ศิรพัฒน์		Plant	ส่วน ๑๑	ตำแหน่ง 1	0	0	0	0
คุณวิวัฒน์ วัฒนศิริ		Plant	ส่วน ๑๑	ตำแหน่ง 1	87	0	100	0
คุณกฤษ งามหา			ส่วน ๑๑	ตำแหน่ง 9	70	0	0	60
คุณพิเชษฐ์ งามหา		ส่วนโสม	ส่วน ๑๑	ตำแหน่ง 8	0	0	0	0
คุณวิวัฒน์ วัฒนศิริ		Plant	ส่วน ๑๑	ตำแหน่ง 1	0	0	0	0
คุณวิวัฒน์ วัฒนศิริ		ส่วนโศก	ส่วน ๑๑	ตำแหน่ง 8	0	0	0	0
คุณวิวัฒน์ วัฒนศิริ					0	0	0	0

Figure 3.55 Admin's morale daily dashboard after enhancement

In Figure 3.55, the data has been rearranged to align with the data funnel structure. Colors have been added to improve clarity: the highest scores are highlighted in green, while the lowest scores are shown in red, making it easier to interpret the data at a glance.

Callout A

The admin can select either a specific employee or all employees and export the table for further analysis as a CSV file.

Admin's Morale Yearly Dashboard

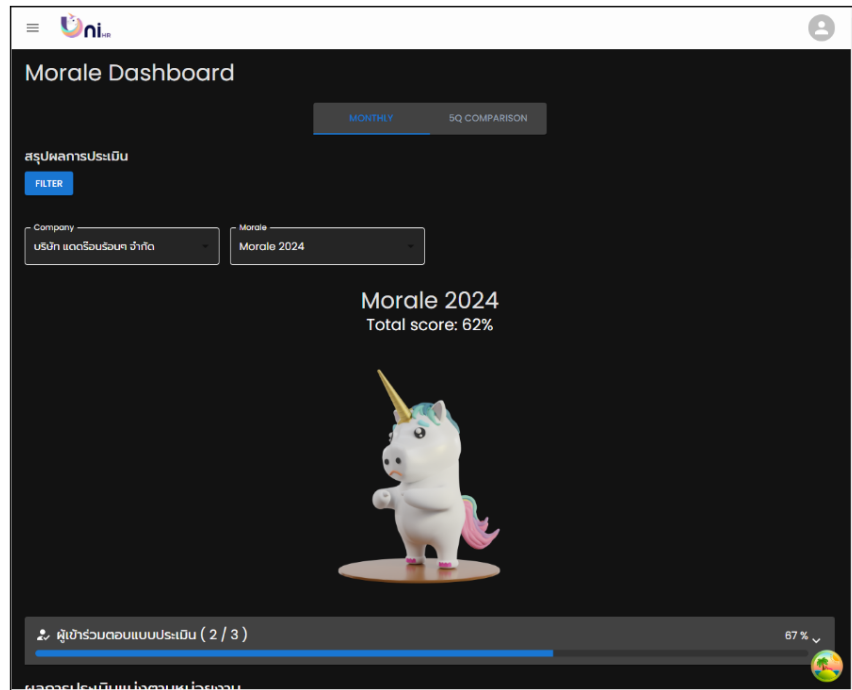


Figure 3.56 Admin's morale yearly dashboard before enhancement



Figure 3.57 Admin's morale yearly dashboard before enhancement

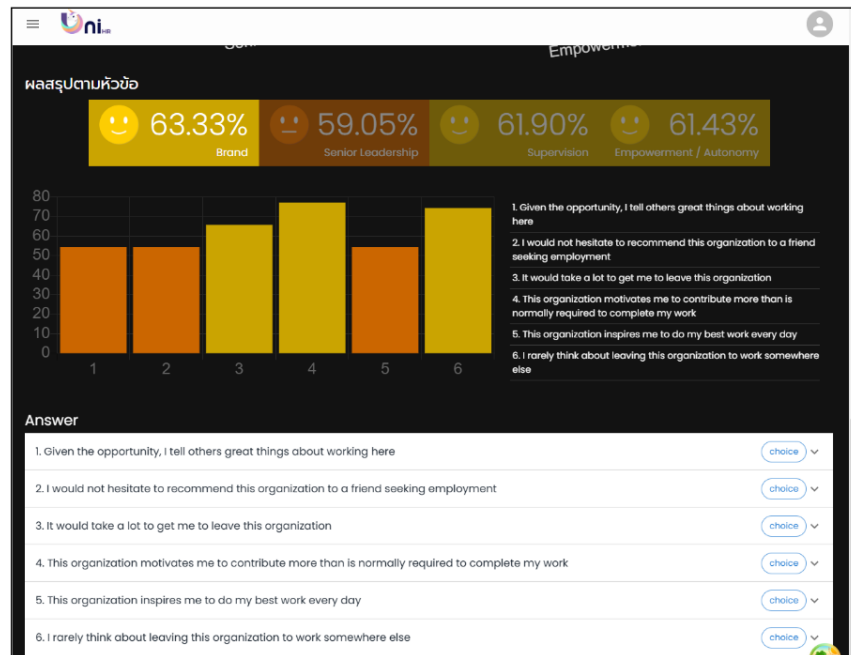


Figure 3.58 Admin's morale yearly dashboard before enhancement

In Figure 3.57 and Figure 3.58, the dashboards shown are the ones before the enhancement, which focuses on improving the clarity of the proportions. The original dashboards lacked clear visual differentiation between the various sections, making it difficult to interpret the data accurately. The enhancement aims to provide a more intuitive layout, ensuring that users can easily understand the proportionate relationships between different data categories.

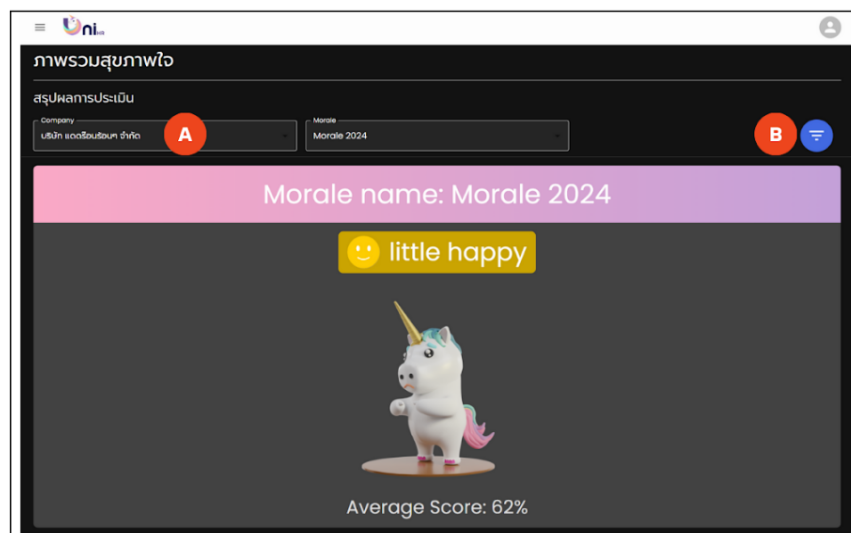


Figure 3.59 Admin's morale yearly dashboard after enhancement

In Figure 3.59, The filter button has been made functional by refactoring the frontend code. Additional enhancements include rearranging the page layout and reorganizing the content to create a clearer and more structured presentation.

Callout A

The admin can select a specific company and morale category to view an overview of the answer visualization.

Callout B

The admin can filter data for further analysis. Available filters include gender, generation, job level, and job type.

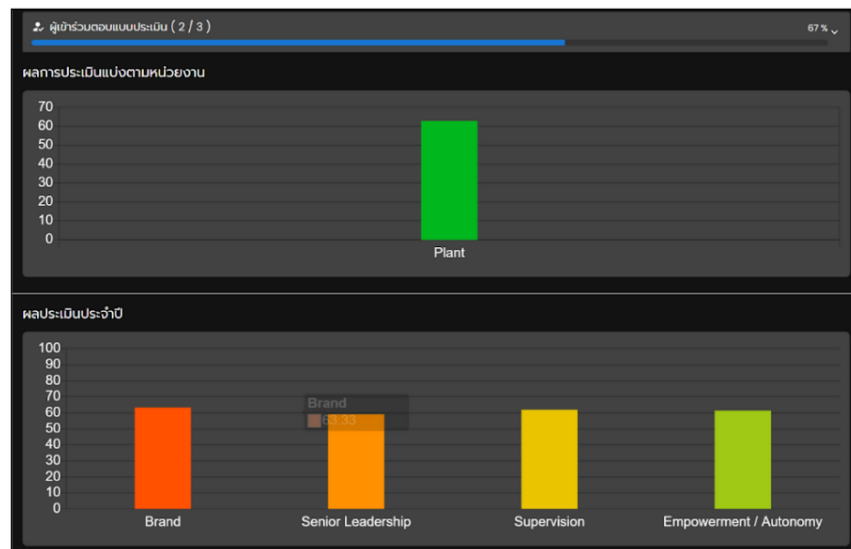


Figure 3.60 Admin's morale yearly dashboard after enhancement

In Figure 3.60, The enhancement is done by rearranging the page layout and reorganizing the content to create a clearer and more structured presentation.

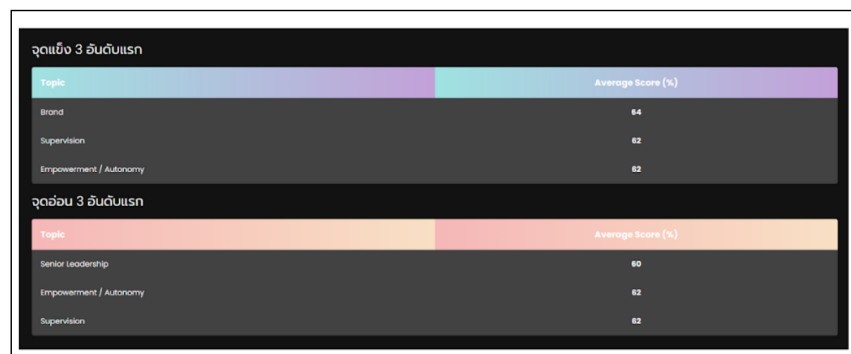


Figure 3.61 Admin's morale yearly dashboard after enhancement

In Figure 3.61, the enhancement includes the addition of the top 3 strength factors and the bottom 3 weakness factors for better insight into the data.



Figure 3.62 Admin's morale yearly dashboard after enhancement

In Figure 3.62, the admin's morale yearly dashboard is now improving the clarity of the proportions.

Callout A

The admin can click on a factor to view the average score of each question related to that factor.

Callout B

The admin can expand to see the feedback for each question.

Manager's Morale Yearly Dashboard

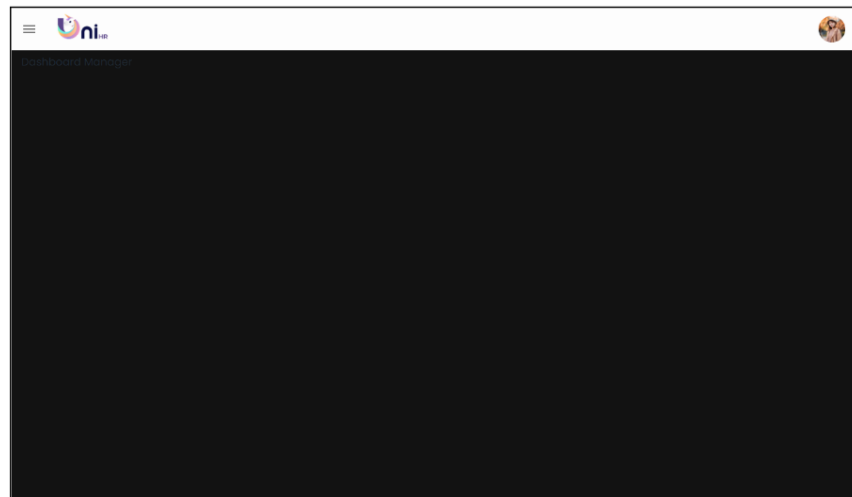


Figure 3.63 Manager's morale yearly dashboard before enhancement

In Figure 3.63, the manager's morale yearly dashboard page is empty, and there is no route leading to this page. This lack of content and navigation route creates a gap in the dashboard's functionality, as users are unable to access or view any relevant data. To improve usability, it's important to ensure that this page is populated with meaningful information and is easily accessible through the appropriate navigation links.

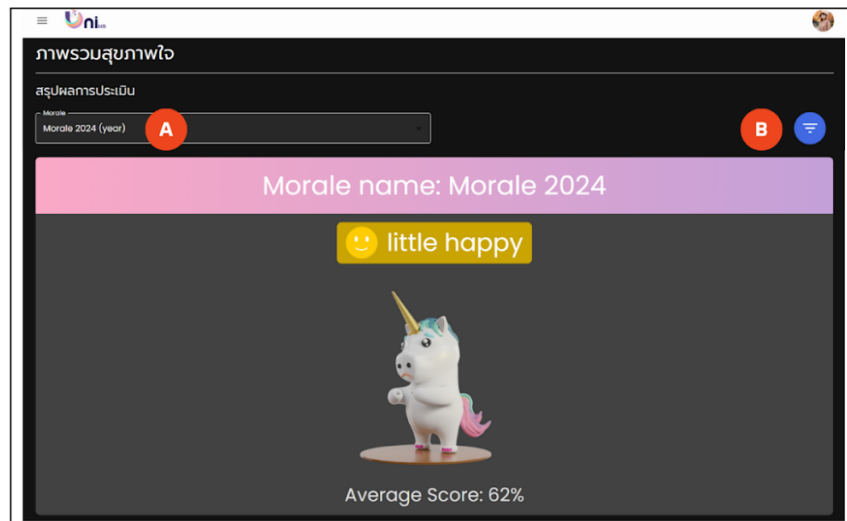


Figure 3.64 Manager's morale yearly dashboard after enhancement

In figure 3.64, The entire page has been newly created based on the existing admin morale yearly dashboard. Managers are only able to view the dashboard for their own company.

Callout A

The manager can select a specific morale category to view an overview of the answer visualization.

Callout B

The manager can filter data for further analysis. Available filters include gender, generation, job level, and job type.

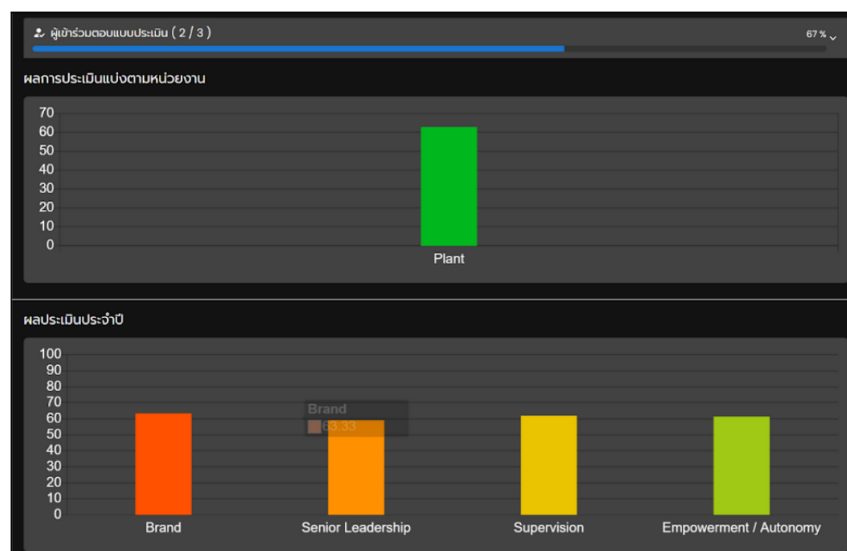


Figure 3.65 Manager's morale yearly dashboard after enhancement

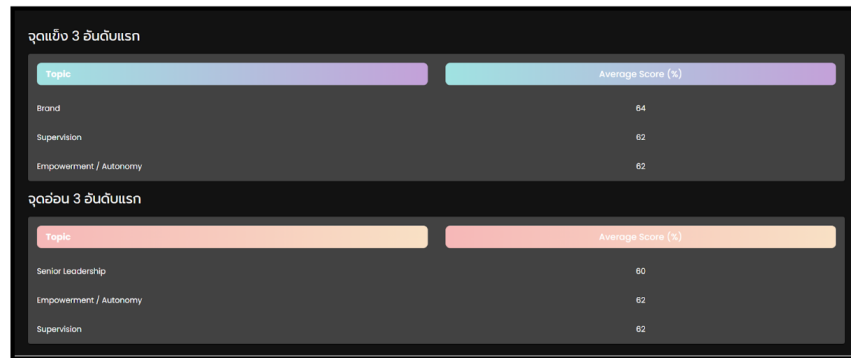


Figure 3.66 Manager's morale yearly dashboard after enhancement



Figure 3.67 Manager's morale yearly dashboard after enhancement

In Figures 3.65, 3.66, and 3.67, the manager's morale yearly dashboard after implementation is shown. These updates aim to align the dashboard more closely with the manager's specific needs and preferences, providing a clearer and more intuitive layout. The enhancements focus on improving data visibility and ensuring that the dashboard is tailored to effectively track and display relevant metrics for the manager's role.

Callout A (on Figure 3.67)

The admin can click on a factor to view the average score of each question related to that factor.

Callout B (on Figure 3.67)

The admin can expand to see the feedback for each question.

Morale Daily Question Form (Mobile Application)

- Respond Morale Daily Question Form

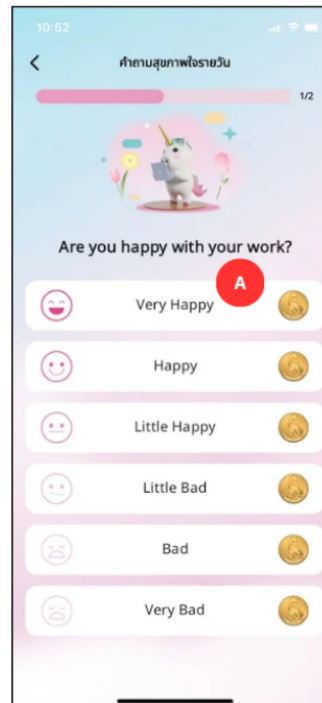


Figure 3.68 User responds to morale daily question form (Mobile)

The user interface for responding to morale daily question forms in Figure 3.68 displays a clean layout with a list of questions and response options input answers. For multiple-choice, users can click the selected option button to submit the form. The interface ensures clarity and engagement, enabling users to complete their responses quickly and accurately.

Callout A - Question form

Displays the questions, including input fields for open questions, and response options for multiple-choice for users to complete the question form.

Assessment 360

• Question Management

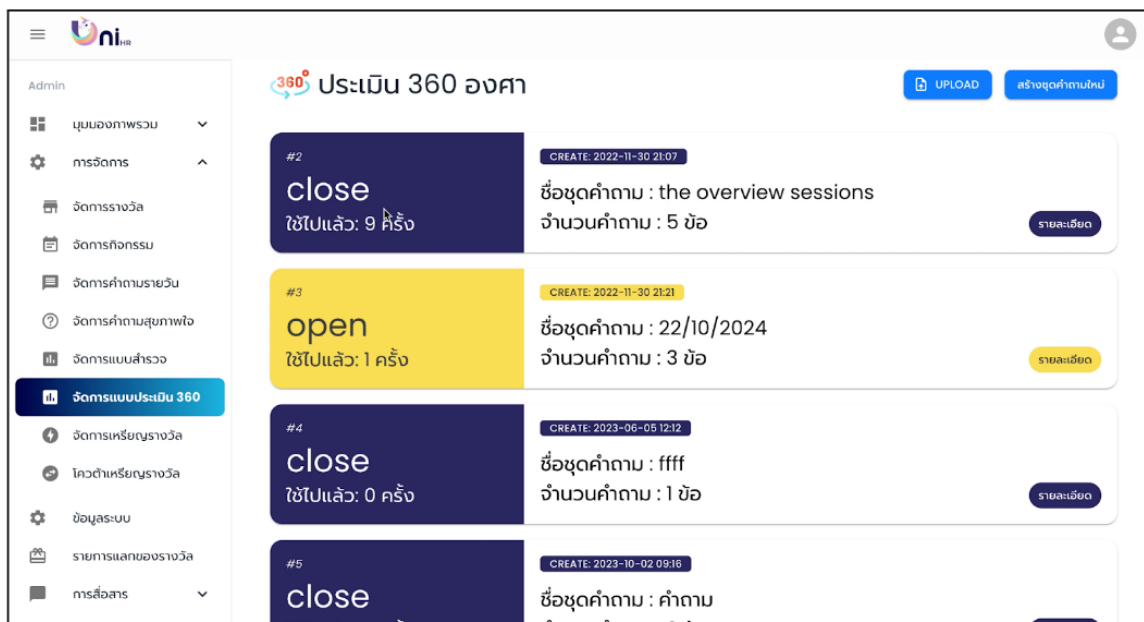


Figure 3.69 Question management of assessment 360 user interface before redesign

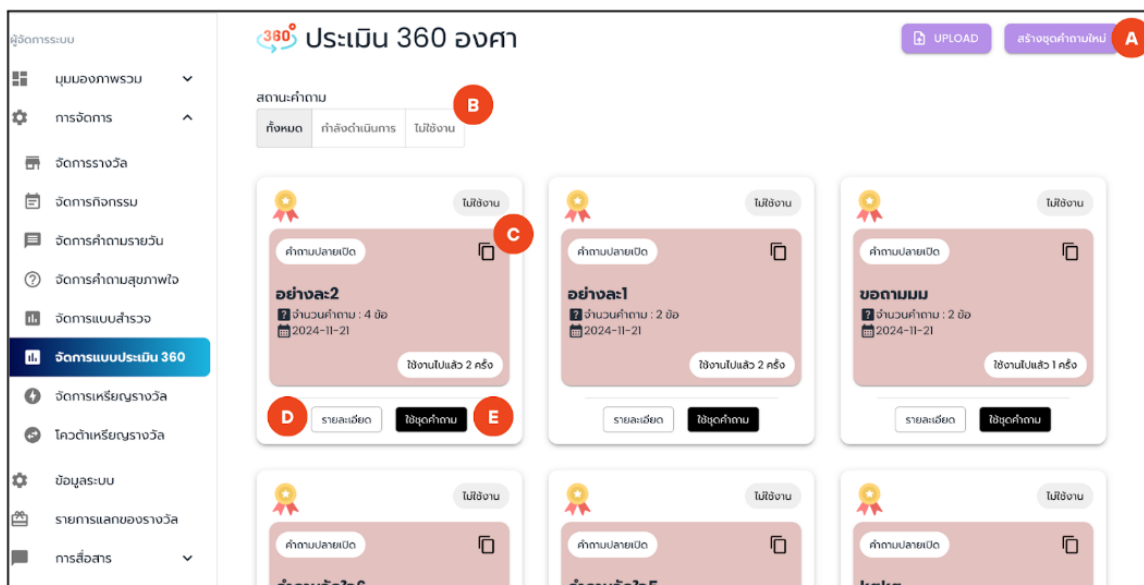


Figure 3.70 Question management of assessment 360 user interface after redesign

The initial design of the Question Management Interface in Figure 3.69 did not fully meet user requirements. To address these issues, the interface has been redesigned (see Figure 3.70). The updated interface enhances user experience by improving functionality and aligning with the system's requirements. It includes features related to Assessment 360, such as creating questions and using assessments, making it the central page for admins to manage assessments effectively.

Callout A - Create assessment (สร้างชุดคำถาม) button

This button serves as the entry point for creating new assessments. When clicked, it opens a form where admins can input the necessary details to build a customized assessment. Users can define individual questions, specify question types (e.g., multiple choice or open-ended), set scoring criteria, and organize questions into sections if needed.

Callout B - Assessment filter section

The filter section is a navigation through a potentially large list of assessments. Users can apply filters based on criteria; all, active or in-active assessment. This feature improves usability by reducing the time needed to locate specific assessments.

Callout C - Duplicate icon button

The duplicate button enables admins to quickly replicate an existing assessment. This is particularly useful for scenarios where similar assessments need to be created with only minor modifications. The duplicated assessment is prefilled with data from the original, reducing manual input and saving time.

Callout D - Details (รายละเอียด) button

This button provides in-depth access to selected assessment data. Clicking it reveals the assessment structure. This detailed view is essential for reviewing assessment content before making modifications or assigning it to new respondents.

Callout E - Use the assessment (ใช้ชุดคำถาม) button

This button facilitates the assignment of assessments to specific users. Upon clicking, the system navigates to the assignment page, where admins can configure respondent detail and deadlines. The feature ensures that assessments are appropriately distributed and includes options for previewing assignments before finalizing them. Additionally, admins can set rules for deadlines to track responses efficiently. This functionality is critical for integrating assessments into organizational workflows.

- Create Assessment

สร้างชุดคำถาม 360 องศา

คำถามปลายปิด

คำถามปลายเปิด

1 Assessment Name

2 Assessment Description

3 Create Questions

SAVE

Figure 3.71 Create assessment of assessment 360 user interface before redesign

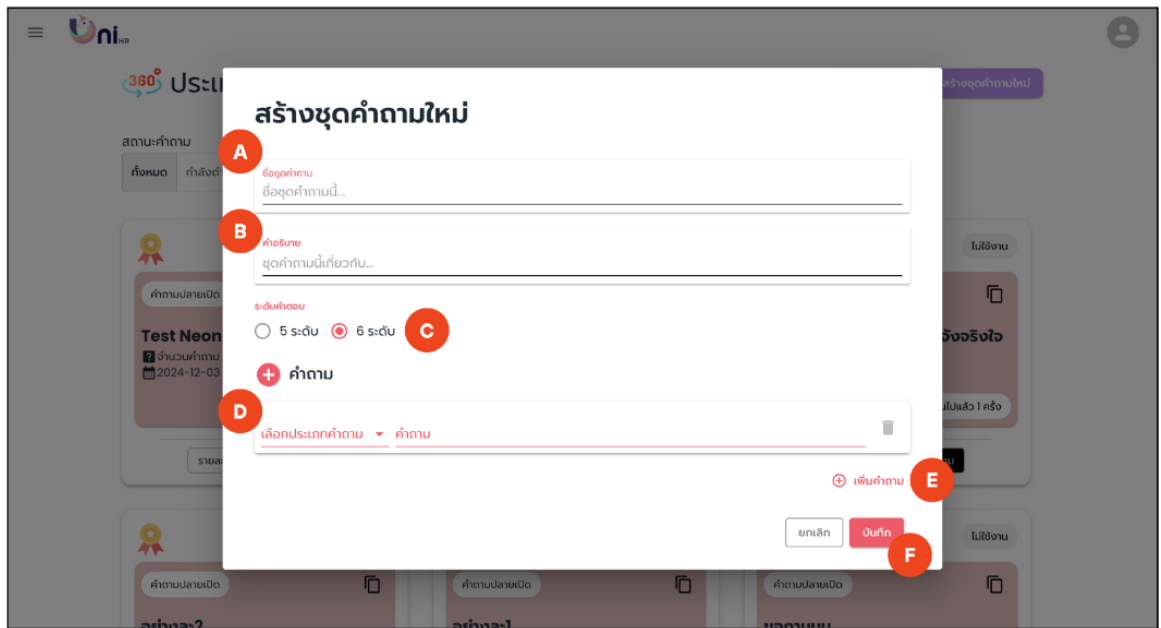


Figure 3.72 Create assessment of assessment 360 user interface after redesign

The Create Assessment interface in Figure 3.71 initially required users to navigate to a new page upon clicking Callout A (on Figure 3.72). However, with updated requirements, users now need the ability to select question types instead of assessment types. To accommodate these changes and improve the user experience, the interface has been redesigned, as shown in Figure 3.72 Create assessment of assessment 360 user interface after redesign, to use a modal instead of navigating to a separate page.

Callout A - Assessment name (ชื่อชุดคำถาม)

This section allows users to specify the name of the assessment. It serves as a key identifier and is required for organizing and managing assessments effectively.

Callout B - Assessment description (คำอธิบาย)

Users can provide a brief description of the assessment. This helps explain the purpose or context of the assessment to other users or respondents.

Callout C - Assessment scale (ระดับคำตอบ)

Users can choose between two predefined scale options, which influence how responses are scored. This flexibility ensures that assessments align with organizational scoring methodologies.

Callout D - Question section

This section enables users to define the questions within the assessment. Users can select the type of each question (text-based or multiple-choice) and enter the question content.

Callout E - Add question (เพิ่มคำถาม)

To add more questions, users can click this button. Each click generates a new input box, allowing them to continue adding questions without leaving the modal.

Callout F - save (บันทึก)

Once all questions are added, users click the Save button. The system validates all fields, checking any errors for correction before submitting the request. After successful validation, the assessment is recorded in the database, and users are returned to the question management page to view the new assessment.

- **Duplicate Assessment**

Figure 3.73 Duplicate assessment of assessment 360 user interface

The Duplicate Assessment feature (on Figure 3.73) is a new addition introduced based on updated requirements. This feature is accessible by clicking Callout C in Figure 3.67, which navigates users to a new page. It makes the creation of assessments by allowing users to replicate an existing assessment, complete with prefilled questions from the selected assessment.

While similar to the Create Assessment functionality, the Duplicate Assessment offers added convenience by preloading questions, reducing the need for manual input. Users can modify the duplicated assessment by adding new questions, deleting unnecessary ones, or editing existing ones. Additionally, users can update the assessment's name and description to differentiate it from the original. This functionality enhances flexibility and efficiency in managing assessments.

- Assessment Details

ni

this is love

สร้างแบบประเมิน

1. what is love?
คำถาม: 5 Scale

Figure 3.74 Assessment details user interface when the assessment has not been used

ni

Test Neon

สร้างแบบประเมิน

1. Q1
คำถาม: 5 Scale

2. Q2
คำถาม: 5 Scale

3. Q3
คำถาม: 5 Scale

Figure 3.75 Assessment details user interface when the assessment has been used

The user interfaces in Figure 3.74 and Figure 3.75 illustrate differences in functionality based on the logic behind their design. For assessments that have not yet been assigned, the Edit icon is displayed, as shown in Figure 3.74. This enables users to modify the assessment by editing its name and questions.

However, once an assessment has been assigned, the Edit icon is no longer available. This restriction ensures the integrity of the assigned assessment, preventing any changes that could affect its consistency or validity. This distinction in functionality enhances usability and ensures proper management of assessments depending on their status.

- **Assign the Assessment**

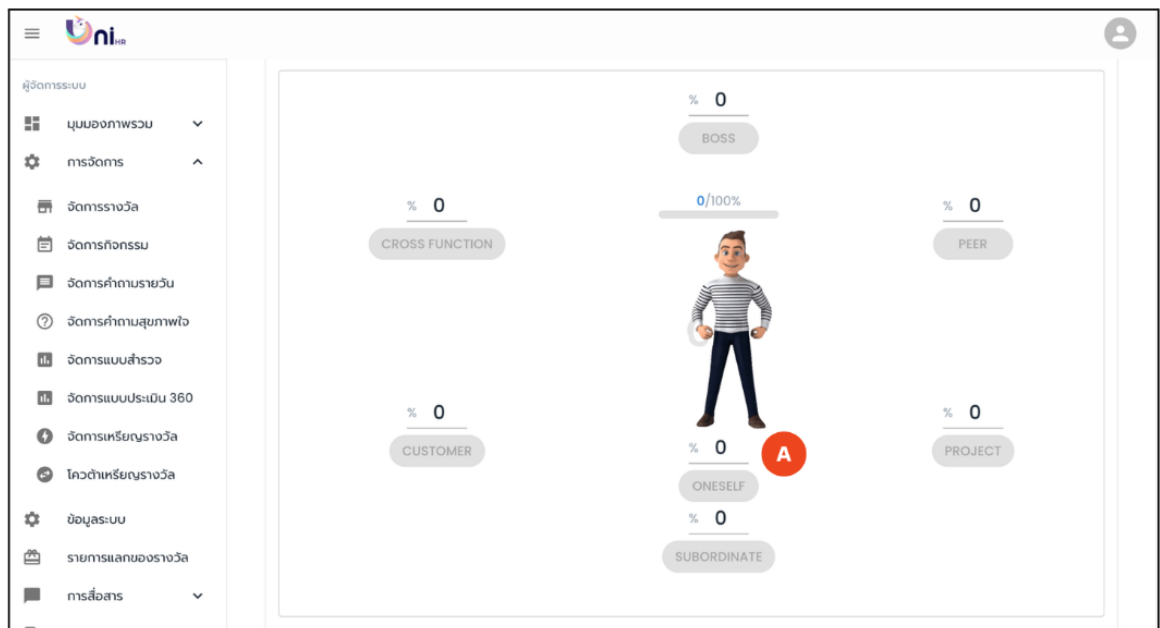


Figure 3.76 Assign the assessment page on the appraiser list

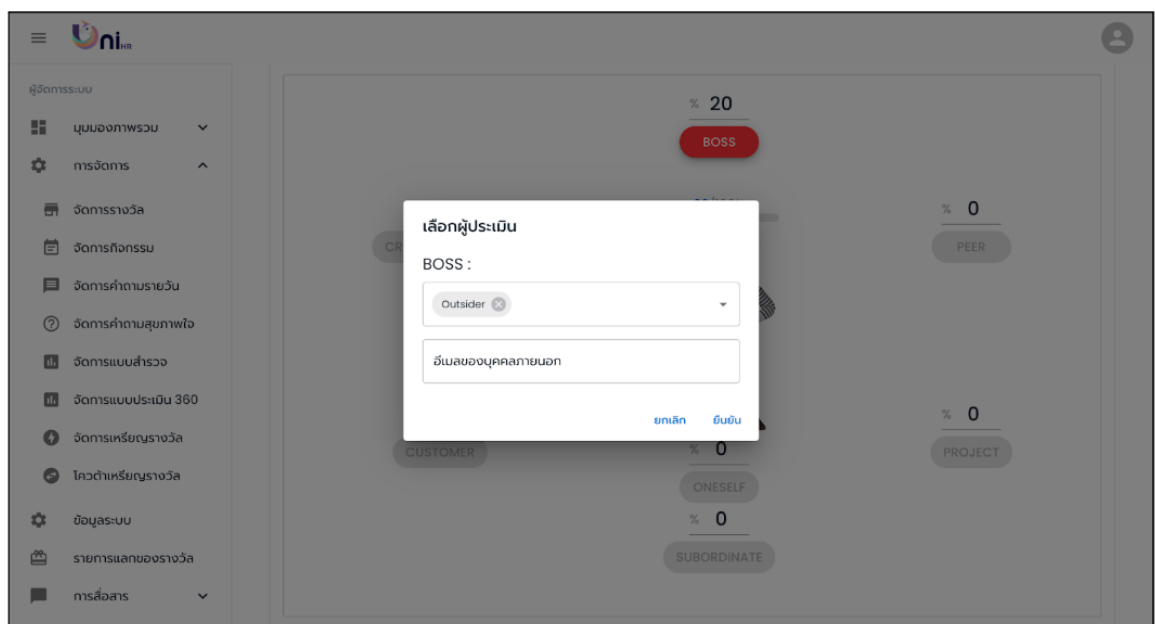


Figure 3.77 Assign the assessment page on the appraiser list when outsider is selected

The user interface shown in Figure 3.76 has been updated to accommodate new requirements for assigning appraisees. Previously, this interface allowed the admin to select groups of people for assigning appraisers. Now, additional features have been introduced to enhance functionality and flexibility:

Callout A - Self-Assessment Box (on Figure 3.76) A new option allows admins to assign appraisees to assess themselves. This self-assessment feature ensures a more comprehensive evaluation process by incorporating personal reflection into the appraisal.

Moreover, Admins can set ratios for each appraiser group. By clicking on the group name, they can select individuals from the list to act as appraisers, ensuring the ratio aligns with the organization's requirements.

On Figure 3.77, Outsider Appraiser Support, In response to the new requirement, outsiders can now participate as appraisers. When an outsider is selected as an appraiser, an additional input box appears for the admin to provide the outsider's email. This functionality allows the system to send the assessment directly to the outsider, enabling their participation seamlessly.

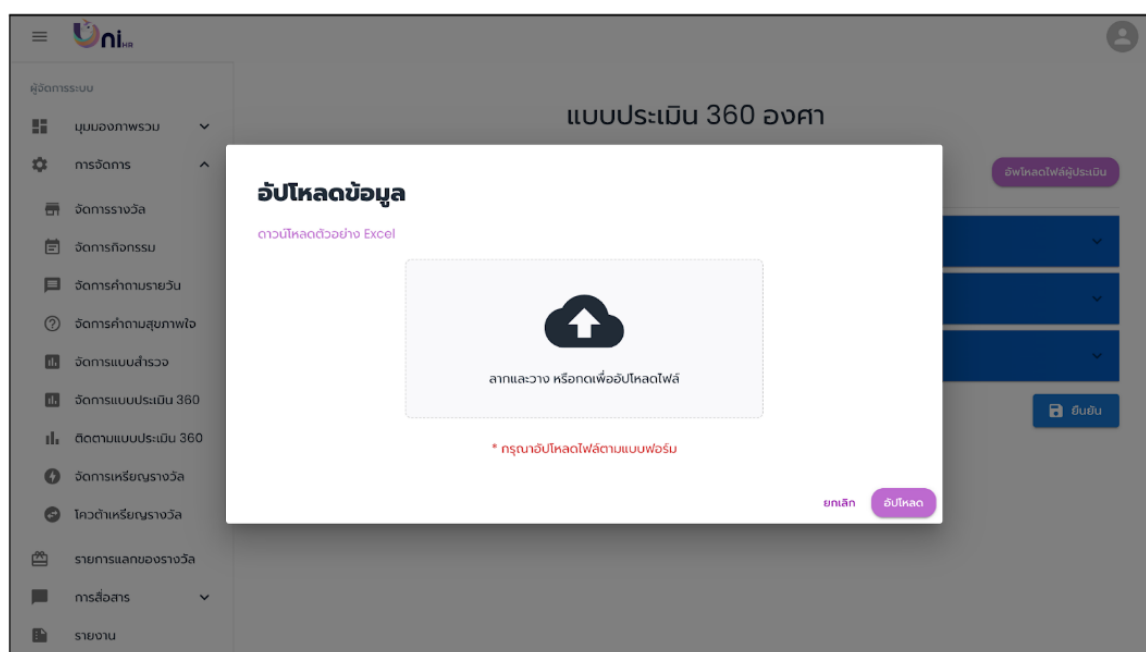


Figure 3.78 Assign the Assessment Page by Uploading the Excel File

The user interface shown in Figure 3.78 allows users to upload an Excel file to assign the assessment. Users can download the Excel template by clicking on the “Download Template File” (ดาวน์โหลดตัวอย่าง Excel) button. They can upload the file by dragging, pasting, or clicking on the designated area, and have the option to either upload or cancel the action.

- Answer Overview

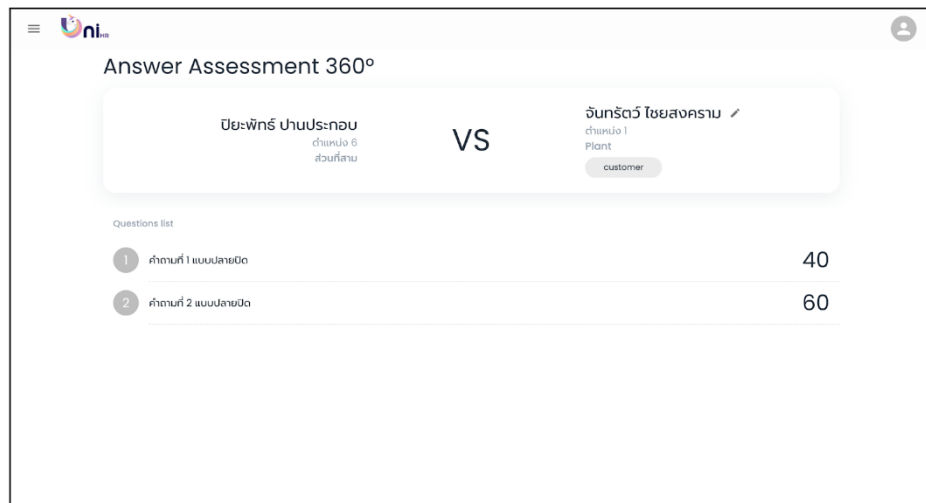


Figure 3.79 Answer overview of assessment 360 user interface before redesign

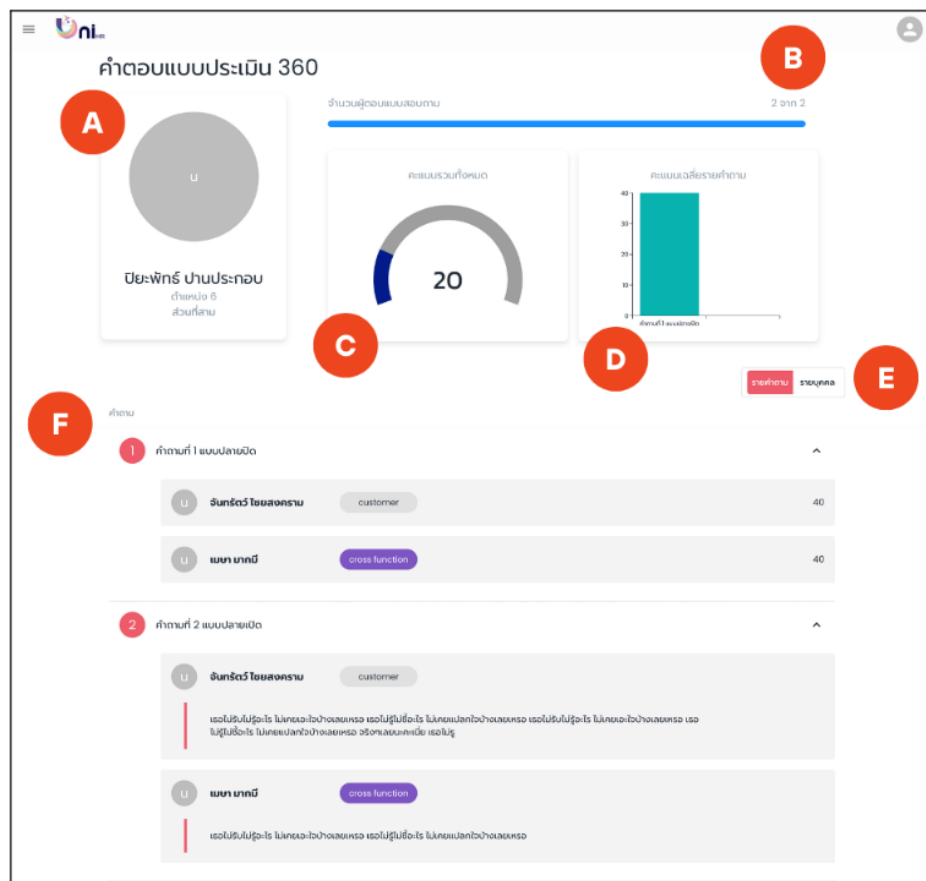


Figure 3.80 Answer overview of assessment 360 user interface after redesign

The Answer Overview page displays comprehensive details about the appraisees' responses, including their information, scores, and answers. In Figure 3.79, the old design faced usability issues, such as not showing

scores and making it difficult to track responses effectively. The redesigned layout in Figure 3.80 addresses these problems with the following features:

Callout A - Appraisee Information Card

Displays general details about the appraisee, such as name, position, and department, providing essential context at a glance.

Callout B - Progress Bar

A visual indicator of completion progress. Users can hover over the bar to view the exact percentage, making it easier to assess how far the appraisee has progressed.

Callout C - Average Score

Calculates and displays the average score based on appraisers' assigned weight ratios. A visual chart accompanies the score for better comprehension.

Callout D - Score by Question

Lists the score for each individual question, offering a detailed view of how the appraisee performed on specific aspects.

Callout E - Answer Display Filter

Allows users to filter the displayed answers either by question or appraisee. This functionality simplifies navigation through large datasets.

Callout F - Answer Display

Reflects the filtered selection, showing answers based on the chosen view (e.g., by question or by individual appraiser), enhancing clarity and tracking efficiency.

- **Outsider Answering Page**

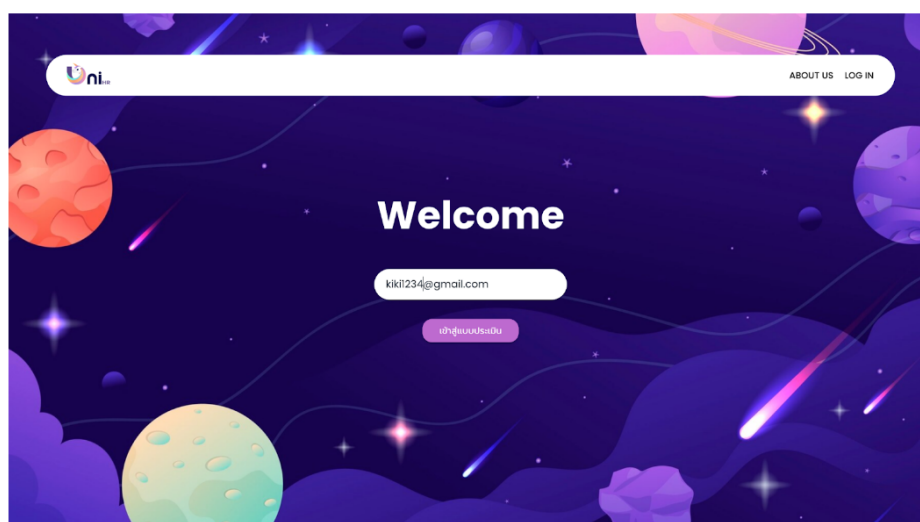


Figure 3.81 Welcome page for outsider

The Welcome page for outsiders (on Figure 3.81) displays a component that prompts users to enter their email in order to view the list of assessments assigned to them. The “Enter Assessment” (เข้าสู่แบบประเมิน) button will remain disabled until the user enters a correctly formatted email address.

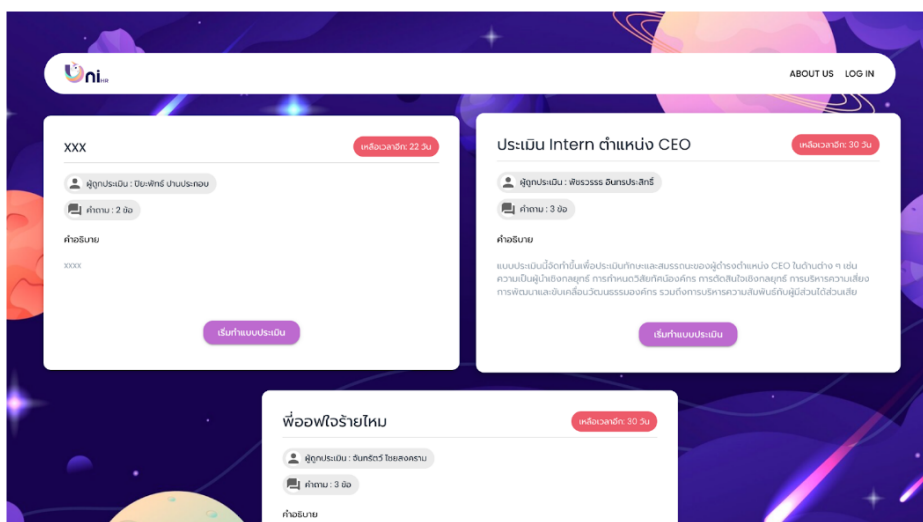


Figure 3.82 List of assessments for outsider

The list of assessments for outsiders (as shown in Figure 3.82) displays components that show details of each assigned assessment. These details include the assessment name, description, number of questions, appraiser's name, and the deadline date. There is also a button to start the assessment. If a user has multiple assigned assessments, this page will display multiple components by one for each assessment.

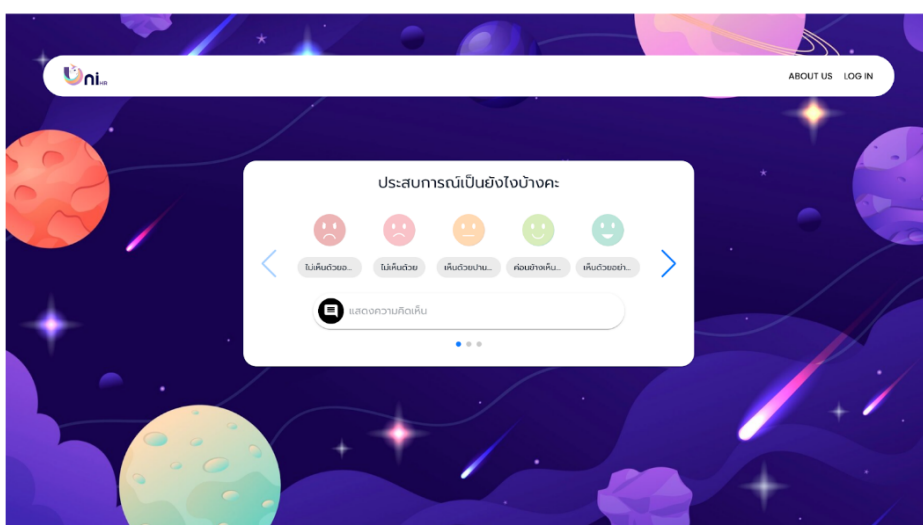


Figure 3.83 Answering the Choice Question

The “Answering the Choice Question” page (as shown in Figure 3.83) displays components that prompt users to respond by selecting an emotion icon, each accompanied by a descriptive label. Additionally, there is a text input field provided for users to add comments for each question.

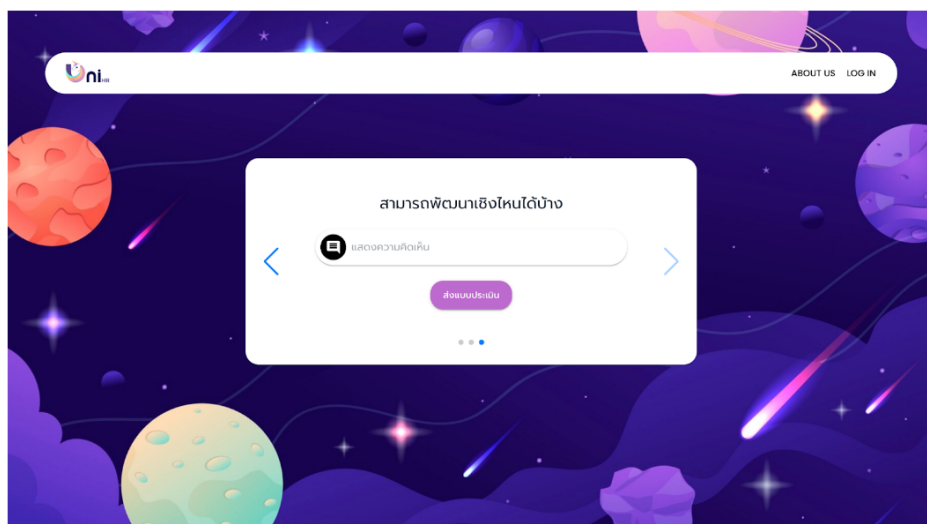


Figure 3.84 Answering the Text Question

The “Answering the Text Question” page (as shown in Figure 3.84) displays components that prompt users to type their responses for each question. After completing all the questions in the assessment, a “Submit Assessment” (ส่งแบบประเมิน) button will appear, allowing users to submit and finish the assessment.

Assessment 360 (Mobile Application)

- Quiz Management

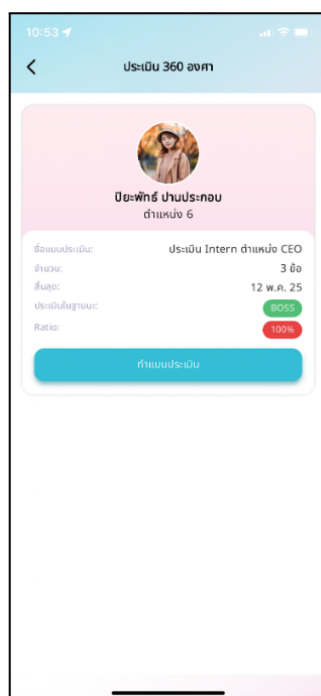


Figure 3.85 List of Question assessment 360 user interface

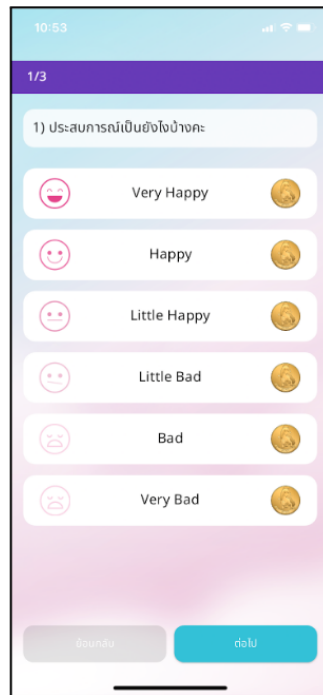


Figure 3.86 Answer assessment 360 user interface

Figure 3.85 illustrates a page where users can view a list of assigned assessments that are awaiting their responses. Each item in the list displays the following details:

- Name of the appraisee
- Position of the appraisee
- Assessment title
- Number of questions
- Current status and progress ratio of the assessment (as the appraiser)

When the user clicks the “Do the Assessment” (ทำแบบประเมิน) button, they will be navigated to the Answer Page, also shown in Figure 3.86 On this page, the user will see the questions along with multiple-choice options for selection

UniLearning

The pages in the UniLearning module focus on the HRD features, including both newly designed pages and redesigned existing ones.

- **Online Lesson**

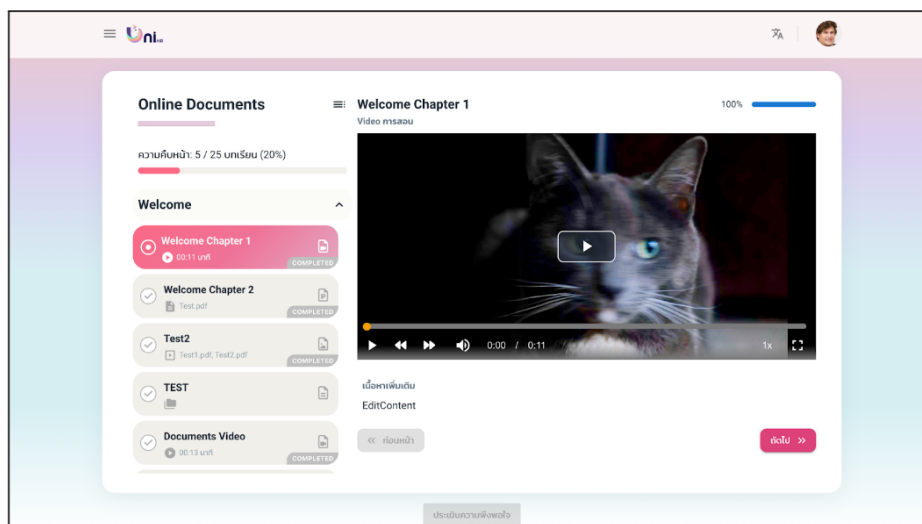


Figure 3.87 User Interface of Online Lesson

In Figure 3.87, the user can select which section and chapter they wish to focus on. The lesson content may include different formats such as video clips, PDFs, and slides. For video lessons, users are required to watch at least 80% of the clip's length before they can move on to the next video. This ensures that users engage thoroughly with the content before progressing.

- **Video Course Suggested by AI**

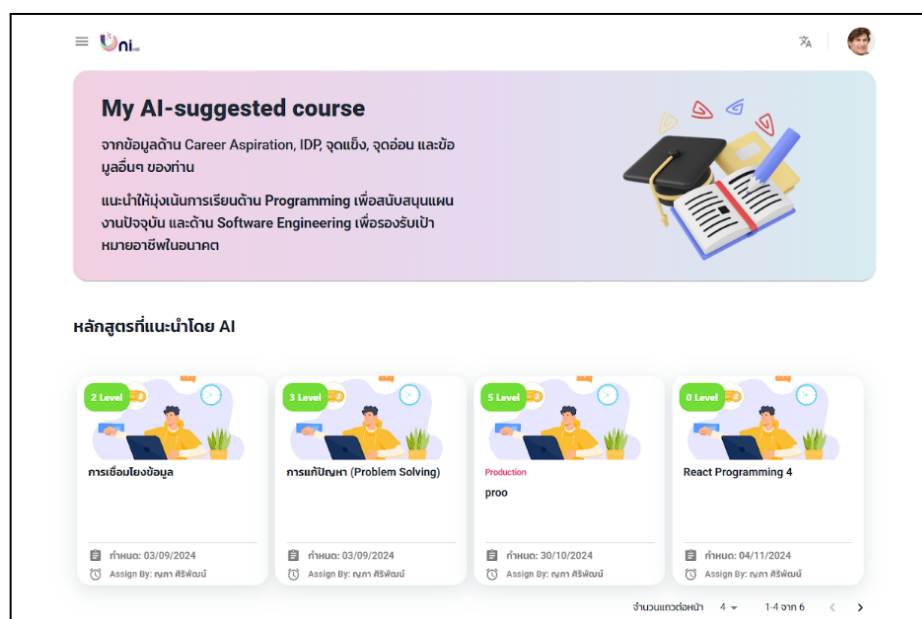


Figure 3.88 User Interface of Video course suggested by AI

In Figure 3.88, the platform presents a personalized learning course under the section “My AI-suggested course.” Based on the user’s individual development plan, and skill gaps, the system recommends a learning path focused on recommended topics. Below the banner, under the section titled หลักสูตรที่แนะนำโดย AI (Courses Recommended by AI), the user is shown suggested courses, each represented by a card.

- **Rush Quiz**

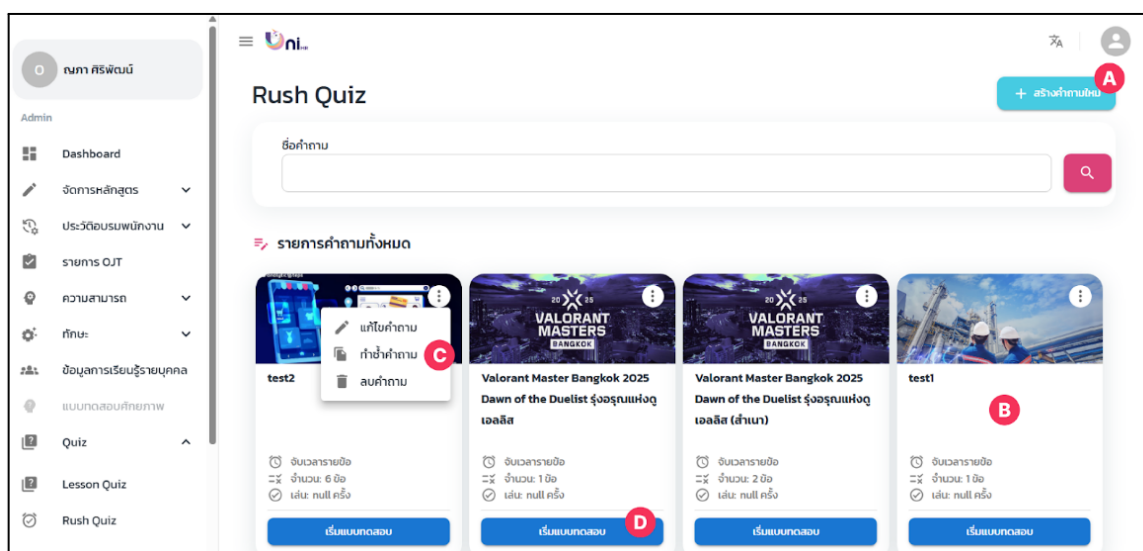


Figure 3.89 User Interface of Rush Quiz

In Figure 3.89, the rush quiz management system provides as an overview for admins to monitor and manage all quizzes. This interface displays a comprehensive view of existing quizzes in a card-based format, allowing admins to easily review and manage their quiz inventory. Each quiz card contains important details such as a relevant thumbnail image, title, question count, and use data. The top search bar allows for fast quiz filtering by name, and the “Create New Question” button makes it easy to add new information.

Callout A - Create new quiz button

The create new question button serves as the primary action point for users who need to add new quizzes to the system. When clicked, it likely directs users to a form where they can input all necessary details for creating a new quiz, including selected company, quiz title, and quiz image.

Callout B - Quiz card selection

When an admin clicks on a quiz card, it displays overview information about the selected quiz, such as related questions. Each card in the UI represents a distinct quiz, which admins can manage. The card contains a thumbnail image, the quiz title, and essential information such as the number of questions and participation rate.

Callout C - Quiz action menu

The action menu provides essential management options for the selected quiz. When clicked, this dropdown menu reveals three critical functions: “แก้ไขคำถาม” (Edit Question), “ทำซ้ำคำถาม” (Duplicate Question), and “ลบคำถาม” (Delete Question). The edit function enables admins to update quiz content, the duplicate function lets them create similar quizzes quickly without starting from scratch, and the delete function allows for the removal of outdated or unnecessary quizzes.

Callout D - Start quiz button

The “เริ่มแบบทดสอบ” (Start Quiz) button when activated, the system likely initiates the quiz session, making it available for participants to join and begin answering questions.

• Rush Quiz Creation and Management

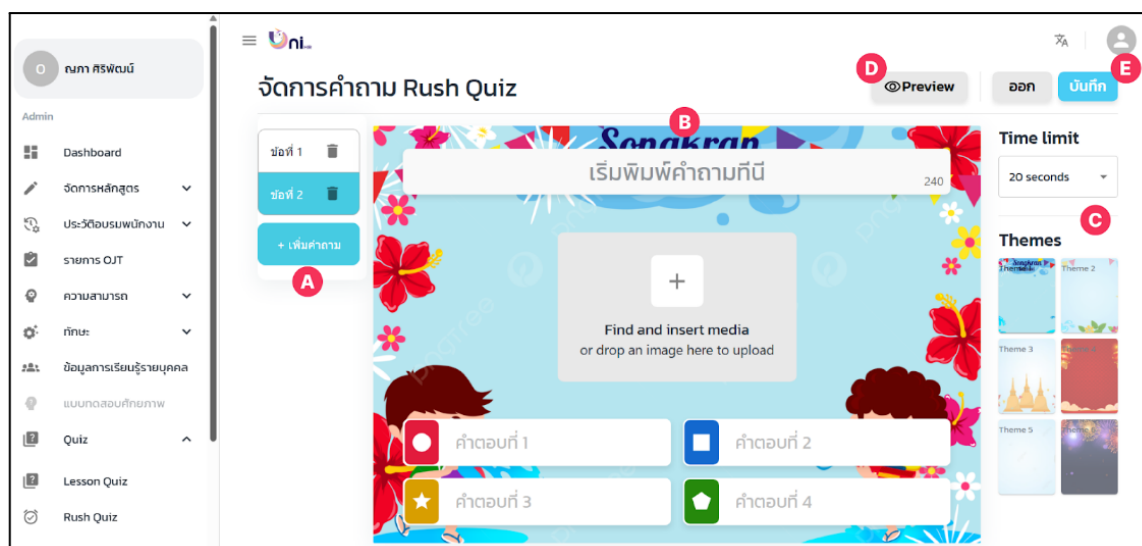


Figure 3.90 Rush Quiz Creation and Management

In Figure 3.90, the rush quiz creation and editing interface, which allows administrators to design quizzes. This workspace combines question management, content creation, and visual customization in a single cohesive screen. The left sidebar facilitates organized question management with numbered entries and an add question button for sequential creation. The central area functions as the primary workspace where administrators can craft question content, upload media, and configure multiple-choice answer options. The right panel offers quiz customization options including time limits and theme selection. The preview button enables quality control by showing how the quiz will appear to participants, while the save and exit actions for completing or abandoning the creation process.

Callout A - Question management panel

The question management panel on the left side of the interface provides admins to control quiz questions. Each question item includes a trash icon for quick deletion if needed. At the bottom of this panel, the Add Question button allows admins to add new questions to the quiz.

Callout B - Question content area

The question content area serves as the primary workspace for creating and editing quiz questions. The area includes a media upload section allowing administrators to enhance questions with visual elements. Below the media section are four answer options, each paired with a distinct colored icon.

Callout C - Time limit and theme selection panel

The time limit selection allows admins to set time-limit for each question ranging from 5 seconds to 4 minutes. The theme selection panel provides visual customization options for the quiz interface.

Callout D - Preview Button

The Preview button, allows administrators to view how the quiz will appear to participants before publishing it. This critical function enables quality control by letting quiz creators experience the quiz from a user's perspective, ensuring that all elements including questions, answer options, media, and the selected theme display correctly and function as intended.

Callout E - Action Buttons

The “บันทึก” (Save) button allows administrators to save all changes made to the quiz, while the gray “ออก” (Exit) button permits them to leave the quiz creation interface without saving if needed.

• Rush Quiz Lobby



Figure 3.91 Rush Quiz Lobby Interface

In Figure 3.91, the rush quiz lobby interface, which functions as the staging area between quiz creation and active participation. This interface serves as the critical connection point where administrators facilitate participant entry before launching the quiz session.

Callout A - Participant entry code

The participant entry code serves as the unique access code that participants must enter to join this specific Rush Quiz session. This six-digit code functions as the gateway to the quiz, allowing administrators to control access and participation.

Callout B - Waiting participants section

The waiting participants section displays “กำลังรอผู้เล่นอยู่...” (Waiting for players...), telling the administrator that the quiz session is live but waiting for participant connections. This status message gives real-time information regarding the status of the quiz session. The waiting room idea allows the administrator to check that all intended participants have successfully connected before starting the quiz.

Callout C - Current participant counter

The Current participant counter is a real-time counter that allows administrators to see exactly how many participants have successfully joined to the session using the specified entry code. As participants join, this number would increment accordingly, giving the administrator visual confirmation of successful connections.

Callout D - Start Quiz Button

The “เริ่ม” (Start) button allows the administrator to initiate the quiz once participants have joined the session. This action button transitions the quiz from the waiting room state to the active question-answering phase.

- Competency

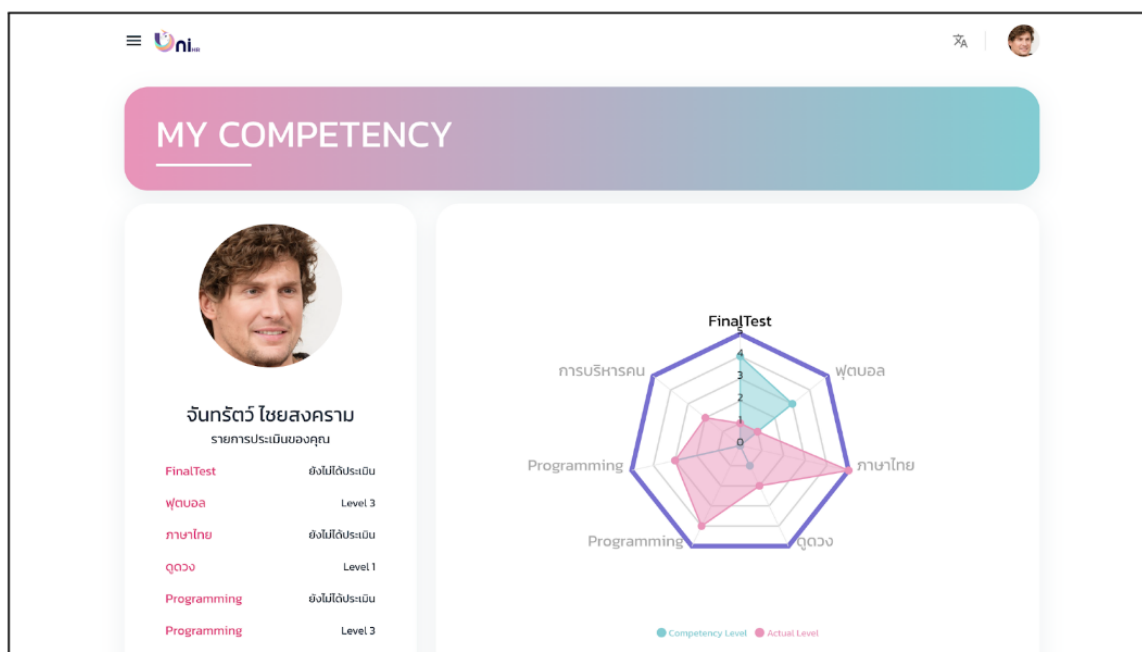


Figure 3.92 User Interface of Competency

In Figure 3.92, the user can view all the competencies they possess. A visual graph is displayed to illustrate the gap between the required competency levels and the user's current competency levels, allowing for easy identification of areas that need improvement.

3.8 Evaluation Plans

This evaluation plan is designed to thoroughly assess the platform's functionality, performance, and user satisfaction. The process includes conducting User Acceptance Testing (UAT) to verify that all features and capabilities align with the intended specifications and user needs. Additionally, surveys will be conducted with real users to gather direct feedback on their experiences, including ease of use, efficiency, and overall satisfaction. The evaluation aims to identify any potential issues or areas for improvement, ensuring the platform not only meets but exceeds user expectations. By combining technical testing and user feedback, this comprehensive approach ensures that the platform delivers a seamless, effective, and positive user experience.

3.8.1 User Acceptance Testing (UAT)

The User Acceptance Testing (UAT) will focus on evaluating the platform's functionality and performance to ensure it meets project requirements and delivers a seamless user experience. The key areas of evaluation include:

- **Functionality Assessment:** This assessment verifies whether the platform operates as specified in the project's functional requirements. Comprehensive test cases will be developed to validate each requirement, ensuring the platform behaves as expected across various scenarios and use cases.
- **Performance Evaluation:** This analysis examines the platform's overall performance, including its speed, responsiveness, and stability. It assesses how well the system performs under different conditions, user loads, and interactions, identifying potential bottlenecks or areas for optimization.

3.8.2 Survey

The survey will assess user satisfaction by gathering feedback from actual platform users. Its goal is to gain valuable insights into their interactions with the platform, focusing on overall satisfaction, ease of navigation, user experience, and the usefulness of features.

CHAPTER 4 RESULTS AND DISCUSSION

In this chapter, the working process of the Human Resource Management (HRM) and Human Resource Development (HRD) systems will be described, along with a discussion on the outcomes, including the user interface. The development of these systems primarily utilizes TypeScript and is intended for use for customers of Ant HR. Therefore, it is essential to develop these platforms according to the quality standards accepted by the company.

To meet these standards, the development environment includes tools such as Bitbucket for Git triggers, AWS Amplify for deployment and hosting, and AWS App Runner for managing backend services. The deployment process will be handled by senior staff within the company. Once all features are deployed, the project manager (CEO) will conduct a review meeting to evaluate the functionality and correctness of the features. Feedback gathered during this review will be used to refine the platform and enhance its efficiency.

In the first semester, we deployed the HRM features, including the morale yearly and daily question forms, morale yearly and daily dashboards, and Assessment 360, twice. After each deployment, we received valuable feedback from the project manager and mentor (developer), which helped us refine and improve the features.

This semester, we have completed the HRM features in the UniCulture module. According to our plan, we are now focusing on implementing HRD features in the UniLearning module. This includes designing new pages and redesigning existing ones to improve the user experience.

On the machine learning side, we have successfully developed two models for the HRM features: a sentiment analysis model and a career aspiration model. Currently, we are working on a video suggestion model for the HRD features.

However, we have encountered a challenge with the dataset used to train our models. Most of the available data is in English, while our website content is primarily in Thai. As a result, the models perform more accurately with English content than Thai. We are now exploring solutions to improve the model's performance with Thai language data.

4.1 Team Overview

Our team consists of five members, each with their own specific role. There are four developers responsible for implementing the system, and a founder who acts as the project manager, overseeing, directing, and testing the system. Below is a detailed description of each role.

- **Project Manager:** The project manager, also the founder, will be responsible for the overall delivery of the project. The main responsibilities entail monitoring the development team's progress, overseeing the project strategy and implementation, and ensuring timely completion of all tasks within the scope. The project manager ensures effective communication among the team members and stakeholders, prioritizes tasks, and resolves conflicts or issues arising. Also, they test the system to ensure that the quality of the system is up to par and that it addresses the original vision. Other responsibilities include the management of risk, resources, and readjustments in the project plan where necessary.
- **Developers:** The developers within the team are responsible for core implementation. This includes writing clean, efficient, and maintainable code, integrating various system components, and ensuring all technical requirements are met. They will work together to design features, develop and test them,

fix bugs, and work on the optimization of the performance of the system. Developers are also required to follow the standards of coding, use version control, and make sure the system is scalable, secure, and reliable. Each developer may specialize in a specific area, for example, front-end, back-end, or database, but all contribute to the overall development of the system.

4.2 Working Process

In line with the company's culture, we work onsite on Mondays and Fridays, and remotely the rest of the week. During remote days, we have online stand-up meetings, while onsite meetings take place on office days to maintain clear communication and alignment.

4.2.1 Meeting stand ups

Every Tuesday and Thursday, we will hold online meetings with the entire team, which consists of the three of us (the core team), the project manager, and the mentor, twice a day. In the morning session, we will outline our plans for the day, discussing the tasks we intend to complete and setting clear goals to ensure alignment within the team. Each task will be separated into individual jobs, allowing each team member to take ownership of specific tasks. This serves as an opportunity to clarify any uncertainties and confirm the priorities for the day.

During the meeting, each team member answers three standard questions:

- **What are the tasks I plan to accomplish today ?**

This question helps set clear goals and expectations for the day. It allows each team member to articulate what they plan to achieve, ensuring alignment with the overall project objectives.

- **What individual tasks have been assigned to me ?**

This question helps clarify each team member's responsibilities for the day, making sure that everyone knows their specific roles and duties, which contributes to efficient task management.

- **Is there anything I need from the team to help achieve my goals today ?**

This allows team members to request support, resources, or clarifications needed to proceed with their tasks, ensuring they have everything necessary to accomplish their work.

- **Are there any changes or updates to the project that I need to be aware of ?**

This question keeps everyone informed about any recent developments or adjustments in the project. It ensures that everyone is on the same page and can adjust their plans accordingly.

In the evening, we will reconvene for a follow-up meeting to provide an update on our progress, share any accomplishments, and discuss any challenges or obstacles we encountered. This session will also allow us to highlight any unresolved issues or concerns that may require additional input from the team, offering a chance to collaborate and seek solutions in real time.

- **What did I accomplish today ?**

This question provides an opportunity for each team member to reflect on their achievements and report on their progress, helping track the completion of tasks and overall project advancement.

- **Did I encounter any challenges or obstacles during the day ?**

This question helps identify any issues faced throughout the day, giving team members a chance to share roadblocks and seek advice or support to overcome them.

- **What is the plan for tomorrow based on the progress made today ?**

This question helps set the stage for the next day, allowing the team to review the progress made and plan the next steps, ensuring continuity and efficient task management.

The daily stand-up meetings are designed to foster open communication, giving developers a platform to raise any challenges or roadblocks they're facing and to seek assistance from their teammates. These meetings allow for quick problem identification and help ensure everyone stays on the same page. However, due to time constraints and the nature of the stand-up format, more complex issues may not be fully resolved during the meeting. In these cases, team members can address the problems more thoroughly outside of the stand-up, either by coordinating smaller group discussions or scheduling additional meetings, ensuring that the issues are effectively resolved without delaying progress. By maintaining this balance between quick updates and in-depth collaboration, we aim to streamline the development process while ensuring that all challenges are addressed efficiently.

4.2.2 Deploying and get the feedback

In this step, the developed features or updates are deployed to a testing platform. After deployment, key stakeholders, including the project manager and mentor, review the system. Feedback is collected to identify any issues or areas for improvement. This feedback helps the team understand how the system is performing and whether it meets expectations. Based on the feedback, the team can make necessary improvements, fix bugs, and enhance features to ensure the system meets its goals and provides a better user experience.

4.3 UniCulture Implementation

The implementation of the UniCulture module mainly focuses on redesigning existing pages, but also includes designing new pages. During this process, we worked on both the frontend and backend to ensure each page meets the functional and design requirements. In addition to the redesign, we also enhanced existing modules to improve overall system efficiency and added new features to support user needs. Also, with the machine learning that would be integrated on the platforms as well.

4.3.1 Front-End Implementation

The front-end of UniCulture has undergone significant updates, with both redesigns of existing pages and the creation of new ones to improve user experience and functionality. This approach ensures that all roles users, admins, and managers have a seamless experience while navigating through the platform.

User Role:

- **Redesigns:**
 - **Morale Daily Question:** The user-facing page for answering daily morale questions has been redesigned to offer a cleaner, more user-friendly experience, ensuring that users can easily respond to daily questions.
 - **Morale Yearly Question:** This page has been updated for better visual clarity and accessibility, making it easier for users to navigate and complete the questions.
 - **Answering Questions in Assessment 360:** A new page has been created for users to easily interact with and answer questions in the Assessment 360. The design focuses on simplicity, allowing users to respond to questions quickly and complete assessments without any hassle.

Admin Role:

- **Redesigns:**

- **Morale Daily Question Form:** The page for admins to create and manage daily morale questions has been redesigned with improved form controls and a more intuitive layout. This update makes it easier to add, edit, and track questions.
- **Morale Yearly Question Form:** The yearly morale question form has been redesigned for better usability, allowing admins to create and manage yearly questions more efficiently.
- **Morale Daily Dashboard:** The daily morale dashboard has been updated with better data visualization and a more user-friendly interface, making it easier for admins to track and analyze daily responses.
- **Morale Yearly Dashboard:** The yearly morale dashboard has been redesigned to present data in a more cohesive and organized manner, helping admins analyze trends and insights effectively.
- **Assigning Questions for Assessment 360:** This page has been redesigned to allow admins to easily assign specific questions to users for the Assessment 360 process.
- **Assessment 360 Dashboard:** The Assessment 360 dashboard has been revamped to provide admins with a comprehensive view of user responses, allowing them to track and manage data efficiently.

- **New page:**

- **Morale Daily Answer:** This page has been created for the admin to observe the daily answers of individual users to the published questions.

Manager Role:

- **Redesigns:**

- **Morale Daily Question:** The page for answering daily morale questions has been redesigned with a cleaner, more user-friendly interface, similar to the user role. This makes it easy for managers to view and answer daily questions.
- **Morale Yearly Question:** This page has also been revamped to improve accessibility and visual clarity, ensuring managers can easily navigate and respond to the questions.
- **Answering Questions in Assessment 360:** The page for managers to answer questions in the Assessment 360 has been redesigned for greater efficiency. The new interface is easy to use, enabling managers to quickly complete their assessments.

- **New page:**

- **Morale Yearly Dashboard:** The Morale Yearly Dashboard for managers has been newly developed and is now aligned with the administrator's version. However, unlike administrators, managers do not have the authorization to access data from other companies, as they lack the necessary permissions to select a company.

Outsider Role:

- **New page:**
 - **Answering Questions in Assessment 360:** A new page has been created specifically for outsiders, allowing them to interact with and answer questions in the Assessment 360 without needing to create an account or log in. This enables external users to participate easily.

The following section outlines the key features of our platform, while the code implementation details are available in Appendix A.

User’s Morale Daily Question

To enhance engagement, the morale daily system integrates a reward mechanism that grants coin incentives and displays a celebration popup upon successful submission. The submission process includes real-time validation, ensuring accurate data entry, and employs visually engaging features such as animated notifications, emoji-based feedback, and clear visual scales. The component enforces a one-time submission constraint to maintain data integrity.

User’s Morale Yearly Question

The implementation highlights a Survey Navigation System that improves the user experience by guiding respondents through different sections of a survey. The main feature of this system is a visual progress tracker, which allows users to see their progress and clearly identify unanswered sections. By marking incomplete sections with error indicators, it encourages users to complete all required questions. This structured navigation helps maintain clarity and ensures a smoother, more intuitive survey-taking process. (Appendix A Figure [A.1.](#))

The Answer Management feature is designed to handle and organize user responses throughout the survey process. Its main function is to ensure that all answers—whether selected options or written input—are properly tracked, validated, and submitted. The system helps users complete the survey accurately by flagging missing answers and guiding them to fill in required fields. This improves both the user experience and the reliability of the collected data. (Appendix A Figure [A.2](#))

User’s Answering Questions in Assessment 360

The interface provides a structured layout for users to review and answer pending assessments efficiently. Each assessment card displays essential details, including:

- Appraisee’s name
- Position
- Evaluation title
- Evaluation topic
- Total number of questions

A red “วันสุดท้าย” (Final Day) label highlights assessments nearing their deadline, prompting timely completion. The “ตอบแบบประเมิน” (Answer Assessment) button, styled in black, allows users to begin the evaluation process. Profile images are displayed when available, with default icons for missing profile pictures. The card-based design enhances readability and navigation, ensuring a smooth user experience.

The CardAssessment component (Appendix A Figure [A.3](#)) is the primary element for displaying the Assessment 360 details. It presents each question with a response interface to help users complete their assessments. The flow is as follows:

- A circular loading indicator is shown while data is being fetched.
- Once the data is available, each card is dynamically rendered using a unique identifier (item.idAppraisee).
- If no data is available, a message “ไม่มีข้อมูล” (“No Data”) is displayed.

The `updateAnswer` function (Appendix A Figure A.4) is central to handling updates to answers within the assessment form.

- Ensures responses are correctly associated with the corresponding question and appraisee.
- Accepts an index and `idAssessment_Question` to return an event handler.
- Updates the state in real time as users select a rating or enter a comment.

The `handleSubmit` function (Appendix A Figure A.5) validates user responses before submission.

- Ensures all required questions are answered.
- Prevents default browser behavior upon submission.
- If all answers are valid, it calls `handleClickOpen` for final submission.
- If any answers are missing, a snackbar message is displayed to inform the user.

Admin’s Morale Daily Question Form

The Daily Morale Question Form (Appendix A Figure A.6) is implemented as a responsive dialog component that enables administrators to create new survey questions with configurable response types and reward systems. The form leverages React Hook Form for state management and Yup for validation, ensuring data integrity throughout the submission process.

Admin’s Morale Yearly Question Form Implementation

The form is implemented as a modal dialog with a structured layout that includes distinct sections for survey configuration, question management, and publishing options. Administrators can easily set up surveys by providing basic information, selecting the survey type (quarterly or yearly), and defining the availability period using date range pickers. The form includes flexible response settings, allowing users to choose between 5 or 6-point Likert scales and upload a cover image for visual appeal. Key Features:

Dynamic Question Management System (Appendix A Figure A.7):

- Administrators can create and customize survey content flexibly, including both multiple-choice and open-ended questions.
- Each question can have a defined topic, text, and answer format.
- Descriptive labels for rating scales are available in multiple-choice questions.
- Tools to add, duplicate, or delete questions, allowing for a highly customizable survey construction process.
- This flexibility enhances the user experience by enabling precise control over the survey structure.

Publishing Workflow (Appendix A Figure A.8):

- Surveys can be saved as drafts for later editing or published immediately for active participation.

- Administrators can target specific audiences such as the entire company, certain departments, or individual employees.
- Comparison sets can be defined to benchmark results.
- A dynamic component helps administrators filter and view morale questions based on their current status, making it easier to manage multiple assessments over time.

State Management (Appendix A Figure [A.9](#) and Appendix A Figure [A.10](#)):

- The component tracks and organizes survey data efficiently through the use of several key mechanisms.
- The `selectQuestionStatus` local state variable holds the currently active tab (which defaults to 1 – drafts).
- The state is updated dynamically via the `handleQuestionStatus` function whenever users switch between tabs, ensuring real-time reflection of the selected category.
- Two helper functions, `isInProgress` and `isUnProgress`, utilize the `Day.js` library to compare each survey’s end date with the current date, categorizing surveys as either “in-progress” or “expired.”
- This enables automatic updates of survey statuses without requiring manual input.

Admin’s Morale Daily Dashboard

The dashboard calculates the average score from the provided `scoreData`. It first checks if the `dataLength` property of the `scoreData` is greater than 0, ensuring there is available data to calculate. If data exists, it computes the average score by dividing `totalScore` by `dataLength` and rounds the result to the nearest integer using `Math.round()`. If no data is present (i.e., `dataLength` is 0 or unavailable), the function returns a default value of 0, indicating that an average score cannot be calculated. (Appendix A Figure [A.11](#))

The function also calculates and visualizes sentiment data using a pie chart (Appendix A Figure [A.12](#)). It starts by determining the total number of participants by adding up the counts of responses for each sentiment category. Then, it creates an array of pie chart data, where the count, percentage, and label for each sentiment are calculated and assigned a corresponding color. Appendix A Figure [A.12](#).

`getTop3Strengths` Function (Appendix A Figure [A.13](#)):

- Processes the `filterData` array to identify and calculate the top 3 topics with the highest average scores.
- Initializes an accumulator object `scoreTopic` to store the sum of scores and the count of responses for each topic.
- The topics are sorted in descending order of their average scores, and the top 3 topics are selected.

`getBottom3Weaknesses` Function (Appendix A Figure [A.14](#)):

- Similar to the `getTop3Strengths` function, this function processes the `filterData` array to identify the bottom 3 topics with the lowest average scores.
- It calculates the average score for each topic and stores the results in the `bottomTopics` array, which is sorted in ascending order.

The department average score table is responsible for calculating the average score for each department. It goes through all the available data, tracking how many scores each department has and the total of those

scores. After reviewing all the information, it calculates the average score for every department and organizes the results in a format that can be easily shown in a table. (Appendix A Figure A.15)

Similarly, the average score table for each section calculates the average score for each section by reviewing the information, keeping track of how many scores each section received, and adding them up. Once all the data has been processed, it calculates the average for each section and prepares the results so they can be clearly presented. (Appendix A Figure A.16)

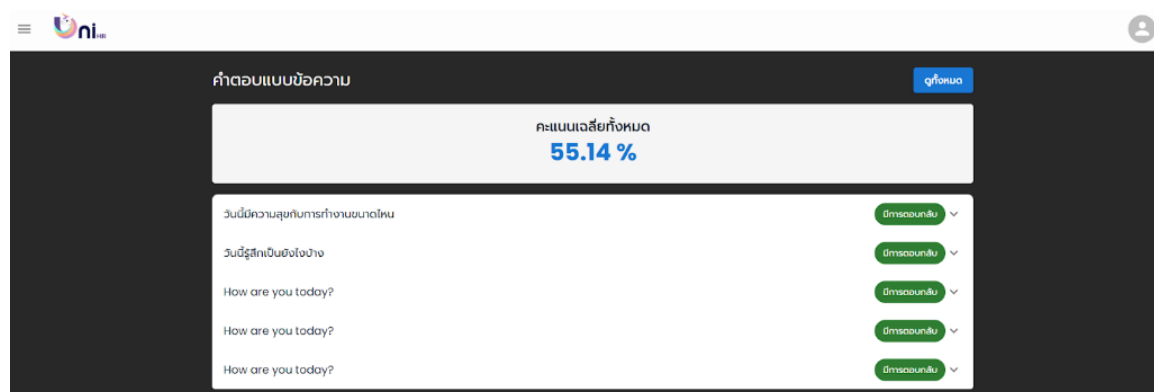


Figure 4.1 Admin Morale Daily Dashboard – Sentiment Analysis Component

In Figure 4.1, this component displays the average sentiment score of the text and the text answers from users according to the selected duration.

Total average score calculates the overall average score based on how employees responded. It first gathers all the answers together. If there are no answers, it returns a score of 0. Then, it adds up all the valid scores given by employees and calculates the average. The final result is a number rounded to two decimal places, giving a clear overview of overall sentiment. (Appendix A Figure A.17)

Admin and Manager's Morale Yearly Dashboard

The average score calculates the overall average score from a set of topic data. It helps summarize how well various topics performed by averaging their values. If no topic data is available, it simply returns 0. (Appendix A Figure A.18) Comparing average score function compares the current average score with another score to see if there has been an improvement or decline. (Appendix A Figure A.19) It helps identify trends by showing whether scores have gone up or down.

Bar chart of total average score by topic prepares data to create a bar chart showing average scores for different topics. It takes each topic's information and formats it for visualization, assigning a label, score, and color. (Appendix A Figure A.20) As well as a bar chart of total average score by department, (Appendix A Figure A.21), this function creates data for a chart showing how departments scored on average. It helps visualize how different departments performed overall.

Top strengths and weaknesses function, this function identifies the strongest and weakest areas by sorting topics based on their average scores. It highlights the top three strengths and bottom three weaknesses, making it easier to see what is going well and what needs improvement. (Appendix A Figure A.22)

Admin's Dashboard for Assessment 360

Prepare chart data function, this function is designed to organize information from employee assessments so it can be displayed clearly on a chart. It focuses on calculating the average scores for each question answered by a group of individuals. If the necessary data is missing, it returns an empty result. Otherwise, it gathers scores from the participants and calculates the average score for each question. The end result is a set of data that can be shown on a chart, helping to visualize how well each question was answered overall. (Appendix A Figure A.23)

Calculate average score function is responsible for calculating the overall average score of an individual (appraisee) based on feedback they've received. It takes into account the importance or weight of each participant's response to ensure a fair average. The function skips over any non-scored questions like open-ended text responses. It then carefully adds up the scores, adjusts them according to how much each response should count (its weight), and figures out the final average as a percentage. If there's no useful data to work with, it simply returns 0. (Appendix A Figure A.24)

4.3.2 Back-End Implementation

Morale Daily Dashboard

Get morale daily KPI function (Appendix A Figure A.25) is used to collect and organize employee morale data for a specific company, year, and month. It finds the relevant data records, gathers key employee information like name, position, and department, and even includes their profile image if available. Each employee's mood or sentiment for the day is then translated into a score to help measure overall morale. At the end, the function compiles all of this into a neat report. If anything goes wrong during the process, it safely handles the error and reports it.

Get question daily answer text function (Appendix A Figure A.26) is designed to collect and analyze written answers to daily morale questions from employees within a company for a specific year and month. It focuses only on text-based questions and gathers the responses along with each employee's name and ID. The main purpose is to understand how employees feel by analyzing their written feedback. After collecting the responses, it evaluates the mood or tone of each answer through sentiment analysis and then returns all the information in an organized format.

Morale Yearly Dashboard

The getMoraleKPI function (Appendix A Figure A.27) is responsible for gathering and organizing key performance indicators (KPIs) related to employee morale. It begins by filtering employees based on input criteria such as company, division, department, and section. If no employees match the filters, it returns an error. When matching employees are found, the function compiles a complete view of morale data, including overall scores, trends across months, departmental performance, and insights by topic. It also looks deeper into each topic by checking the types of questions used. For multiple-choice questions, it calculates average scores and trends, while for open-ended questions, it collects written responses. This comprehensive data is then sent back in a structured format.

Morale Daily Question Form

The backend implementation publishing system through the handlePublish function, which coordinates with the backend controller to distribute daily morale questions.

The addMoraleDaily function (Appendix A Figure A.28) handles the creation of daily morale entries and the assignment of participants. It takes several request parameters, including idMoraleDailyQuestion, startDate, endDate, valueTarget, selectedDepartment, and idCompany, which help define the morale data and the target

group. The function uses a database connection pool for performance efficiency and initiates a transaction to maintain data integrity throughout the process. It starts by inserting a new morale entry into the UniCulture_MoraleDaily table and retrieves the idMoraleDaily for future reference. Depending on the valueTarget, employees are fetched either by department or by company. If no employees are found, an error is raised to ensure the system does not proceed with incomplete data.

Morale Yearly Question Form

The morale creation and editing process begins when an admin selects either “Save as Draft” or “Publish” from the frontend. This choice is passed into a function that prepares the morale data like name, code, description, image, status, and target company along with its related questions. The questions are formatted to include their topic, type, and any notes for scale-based (multiple-choice) questions. Once ready, this information is sent to the backend. The addMorale controller handles saving new morale entries into the system (Appendix A Figure A.29 and A.30). It saves the main morale details, inserts each question, and, for multiple-choice questions, stores the explanation notes. Then, each question is linked to the morale entry.

If the morale status is set to “published,” the system also assigns participants to it through the addMoraleParticipant controller. This function identifies the relevant employees based on the chosen target group (company, department, or individuals) and registers them in the system under the morale entry. It also updates the morale status in the database to “published.”

For editing morale data, the updateMorale function (Appendix A Figure A.30) allows admins to change existing details such as the title, description, image, and time period. It also processes each related question, deciding whether to update or newly insert it. If any of those questions are multiple-choice, the function ensures that their notes are correctly updated or added. This entire update process is protected by transaction management to keep the data consistent, with rollback steps if anything goes wrong.

4.3.3 Machine Learning

4.3.3.1 Sentiment Analysis for UniCulture

The Thai sentiment analysis process begins with data preparation, where sentiment words are extracted from predefined lists, labeled as positive (+1) or negative (-1), and augmented by pairing words of the same sentiment. The text undergoes preprocessing, including lowercasing, special character removal, tokenization with PyThaiNLP’s “newmm” engine, and stopword elimination. The cleaned text is transformed into numerical features using TF-IDF vectorization (unigram and bigram), filtering out overly common and rare words. The dataset is split into training (70%) and testing (30%) subsets, and a Random Forest Classifier with 100 decision trees is trained, ensuring class balance and unrestricted tree growth. During inference, the analyzeSentiment function determines whether the text is in Thai or English. Thai text is sent to a FastAPI endpoint for sentiment prediction, while English text is analyzed using the VADER sentiment tool, mapping scores to a 0-100 scale. The Thai model processes input by tokenizing and vectorizing text, predicting sentiment scores for each sentence, averaging them, and identifying the top five most common words. The trained model’s performance is evaluated using standard classification metrics to ensure accuracy in career-related sentiment analysis.

4.3.3.1.1 Data Preparation

To prepare the dataset, we extracted Thai sentiment words from predefined positive and negative word lists, consisting of 365 distinct positive words and 748 distinct negative words, assigning labels (+1 for positive, -1 for negative). To enhance the dataset, we created augmented sentences by pairing words of the same sentiment,

effectively increasing data diversity. Each text entry underwent preprocessing, which included lowercasing, removing special characters, tokenizing using the newmm engine, and eliminating stopwords. The cleaned text data was then transformed into numerical features using the TF-IDF vectorizer with unigram and bigram representations, filtering out extremely common and rare words. The labeled feature vectors served as input for training a Random Forest Classifier to predict sentiment scores.

Given the time constraint, we utilized a pre-trained model for English text answers while focusing on developing a machine learning model specifically for Thai text, as existing pre-trained models lack robust Thai language support. For the English text, we employed Valence Aware Dictionary and sEntiment Reasoner (VADER), a pre-trained model known for its effectiveness in career domain-related text analysis, ensuring accurate sentiment predictions for job-related responses. This approach allowed us to leverage a high-quality model for English while dedicating resources to fine-tuning a Thai-specific sentiment analysis model using machine learning. By combining these strategies, we aimed to create a system that efficiently handles both languages while maintaining high accuracy in career-related sentiment assessment.

4.3.3.1.2 Implementation

The `analyzeSentiment` function (Appendix A Figure A.31) is an asynchronous function that determines the sentiment of a given text. The function first checks if the text is in Thai using the `isThai(text)` function. If the text is in Thai, it sends a POST request to a FastAPI endpoint (`http://localhost:8000/predict_sentiment/`), passing the text as JSON data. It processes the API response, extracting the `sentiment_score` and rounding it to the nearest integer. If the API does not return a valid sentiment score, the function logs an error and returns null as the sentiment score. If the text is not in Thai, the function uses the VADER sentiment analysis tool to compute a compound sentiment score, which it then converts to a percentage scale ranging from 0 to 100. The function returns an object containing the original text and its corresponding sentiment score. In case of any errors with the FastAPI request, the function catches the error, logs it, and returns null as the sentiment score.

4.3.3.1.3 Thai Sentiment Analysis

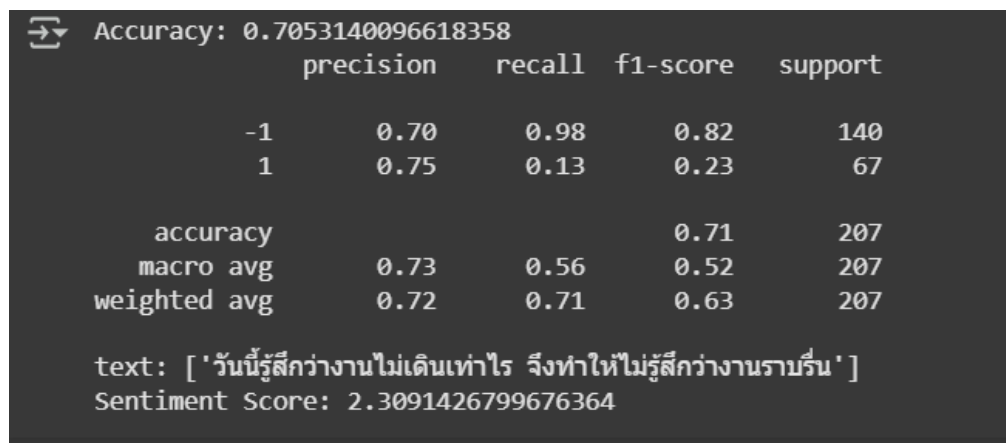
Thai sentiment analysis model using TF-IDF vectorization and a Random Forest classifier. It begins by loading Thai stopwords and predefined positive and negative sentiment words from text files, mapping them to sentiment scores (+1 for positive, -1 for negative). The dataset is augmented by combining sentiment words into sentence pairs to improve model robustness. Text preprocessing involves lowercasing, removing special characters, tokenizing using PyThaiNLP's "newmm" engine, and removing stopwords. The processed text is converted into TF-IDF vectors (using unigrams and bigrams), and the dataset is split into training and testing sets. A Random Forest classifier (with balanced class weights) is trained on the vectorized data. The `predict_sentiment` function splits an input text into sentences, processes each sentence, predicts sentiment scores (scaled to 0-100), and calculates the average sentiment score. Additionally, it tracks the most frequently occurring words in the text, returning both the average sentiment and the top five most common words. (Appendix A Figure A.32 and A.33)

4.3.3.1.4 Model Training

For model training, the preprocessed Thai text data was transformed into numerical feature representations using the TF-IDF vectorizer, which captured both unigram and bigram features while filtering out excessively common and infrequent terms. The dataset was subsequently partitioned into training (70%) and testing (30%) subsets to facilitate effective model evaluation and mitigate overfitting. A Random Forest Classifier compris-

ing 100 decision trees was employed as the predictive model, with class weights adjusted to address potential class imbalances. The model was configured with `max_depth=None` to allow unrestricted tree growth and `min_samples_split=2` to ensure an adequate number of samples for node splitting. The ensemble learning approach of the Random Forest algorithm was leveraged to enhance the model's generalization capability. Following training, the model's performance was assessed using standard classification metrics to evaluate its effectiveness in predicting sentiment within Thai-language text.

4.3.3.1.5 Model Input and Result



Accuracy: 0.7053140096618358				
	precision	recall	f1-score	support
-1	0.70	0.98	0.82	140
1	0.75	0.13	0.23	67
accuracy			0.71	207
macro avg	0.73	0.56	0.52	207
weighted avg	0.72	0.71	0.63	207
text: ['วันนี้รู้สึกทำงานไม่เดินเท่าไร จึงทำให้ไม่รู้สึกทำงานราบรื่น']				
Sentiment Score: 2.3091426799676364				

Figure 4.2 Input and output of the Thai sentiment analysis model

Figure 4.2 illustrates the input and output of the Thai sentiment analysis model. The input is a user-submitted text expressing a feeling of low productivity: “วันนี้รู้สึกทำงานไม่เดินเท่าไร จึงทำให้ไม่รู้สึกทำงานราบรื่น”. After processing this input, the model returns a sentiment score of 2.3091, which indicates a negative sentiment based on the model's scoring threshold. This score is then translated into a sentiment percentage and displayed on the dashboard, allowing administrators to monitor employee morale trends over time in a more interpretable format.

4.3.3.1.6 Model Evaluation

To assess the performance of the sentiment analysis model, we used standard evaluation metrics on a hold-out test set. The model achieved an overall accuracy of approximately 70.5%, indicating that it correctly classified about 7 out of 10 instances. However, because the dataset is imbalanced, we also examined precision, recall, and F1-score for each sentiment class to gain a clearer picture of the model's strengths and weaknesses. The model performed well on the negative sentiment class (-1), with a precision of 0.70, a high recall of 0.98, and an F1-score of 0.82, suggesting it reliably identifies negative sentiments. In contrast, its performance on the positive sentiment class (1) was weaker, with a precision of 0.75 but a low recall of 0.13, leading to a modest F1-score of 0.23. This indicates that while the model is accurate when it predicts positive sentiment, it often fails to detect actual positive cases. The macro-averaged F1-score was 0.52, and the weighted F1-score was 0.63, highlighting the model's bias toward the more frequent negative class. These results suggest the need for improvements such as rebalancing the dataset or tuning class weights to enhance performance on underrepresented sentiments. For instance, the sentence “วันนี้รู้สึกทำงานไม่เดินเท่าไร จึงทำให้ไม่รู้สึกทำงานราบรื่น” (“Today I feel that work isn't progressing much, so it doesn't feel smooth”) was correctly classified as negative, showcasing the model's strength in recognizing dissatisfaction and negative emotional tone.

4.4 UniLearning Implementation

In the first semester, our main focus was on completing the HRM features. We prioritize finishing each system before starting a new one, following our planned timeline. Therefore, this semester, the main focus has shifted to the HRD features within the UniLearning platform. We have already redesigned the relevant pages and updated the user interface design, as documented in Chapter 3.

4.4.1 Front-End

User's Online Lesson

The `init_player_events` function is designed to monitor and handle user interactions with a video player. It sets up event listeners that detect when the video is played, paused, or ended. This function ensures that the user's interaction with the video is tracked efficiently. When the video is played, it begins periodically tracking the video's progress to keep the parent component informed. When the video is paused or ends, it stops tracking and sends a final progress update. This helps ensure accurate reporting of how much of the video has been watched without overwhelming the system with too many updates. (Appendix A Figure [A.34](#))

User's Video Course Suggested by AI

The AI-suggested course page in the frontend is implemented using React.js with Material-UI for styling and Redux for state management. This component dynamically fetches and displays AI-recommended courses and skill-based training relevant to the employee, based on personalized data such as career aspirations, individual development plans, and current job roles. The page features a responsive layout, custom-styled components, and a pagination system to enhance user experience. It uses reusable card components to render course and skill suggestions, with conditional navigation for internal course details. The architecture is modular and scalable, allowing future enhancements such as course filtering, which has already been partially implemented in the code.

Admin's Rush Quiz

The Rush Quiz Management System frontend provides a smooth and organized experience for creating and managing quizzes. It uses React's `useState` to keep track of a structured list of questions, with each question containing content, an optional image, four answer choices (with correct answers marked), a time limit, and a selected visual theme. The question navigation panel allows users to easily select or delete questions using numbered buttons for each one (Appendix A Figure [A.35](#)). Editing questions is handled by the `InputForm` component, which lets users update text, upload images, and manage answers with ease (Appendix A Figure [A.36](#)).

The system also includes a dropdown for setting time limits, ranging from 5 seconds to 4 minutes, and lets users pick themes using thumbnail previews with hover effects for clear feedback (Appendix A Figure [A.37](#)). Data is saved using `FormData`, allowing both text and images to be uploaded while keeping track of existing question IDs. To prevent data loss, a navigation guard alerts users about unsaved changes before they leave the page. Users can also preview the quiz to see how it looks for participants, including all visuals and timing settings (Appendix A Figure [A.38](#)).

4.4.2 Back-End

Online lesson

The `getFinishVideoCourse` function is responsible for identifying which video courses a user has completed. It delegates the core task to another function to keep the code clean and manageable. (Appendix A Figure [A.39](#))

The `updateDuration` function handles updating the length of a course or video based on the data it receives. This helps keep course information accurate. (Appendix A Figure A.40)

The `doGetFinishVideoCourse` function is used to check if a user has completed a video course by confirming whether they've watched at least 80% of the total content. (Appendix A Figure A.41)

Another version of the `doGetFinishVideoCourse` function performs a similar task by checking if a user has watched 80% or more of a video course to determine course completion. (Appendix A Figure A.41)

Video course suggested by AI

On the backend side supporting the frontend AI-suggested course feature, this Node.js function retrieves active online courses from the MySQL database and structures them into a format suitable for AI processing. The course data, including IDs, titles, and topics, is transformed into a JSON payload and sent to a FastAPI-based recommendation engine via an HTTP POST request. This engine returns a list of recommended courses based on predefined user interests (currently hardcoded for testing). The backend then forwards these AI-generated recommendations to the frontend, enabling real-time, personalized learning suggestions. This integration showcases the seamless communication between the frontend and AI model through a RESTful architecture.

Rush Quiz

The backend of the Rush Quiz Question Management System is responsible for handling the creation, updating, and deletion of quiz questions. It supports batch processing—meaning multiple questions and their images can be uploaded or updated at once—and uses a transaction system to ensure all operations succeed together or none at all. This ensures that there are no incomplete updates or data errors. The main function, `doPostRushQuizQuestion`, starts by validating the request to confirm the quiz ID is provided (Appendix A Figure A.42). If everything is valid, the function begins a transaction to maintain data integrity (Appendix A Figure A.43).

Before saving new questions, the system deletes existing ones related to the quiz to avoid duplicates. It then processes each question individually by:

- Parsing and cleaning the input data
- Setting default values (like a 20-second timer)
- Attaching metadata such as creation date and employee ID

If a question includes an image, the image is sanitized and saved in a structured path (e.g., `course/quiz/id123/question/qid456/filename.jpg`) and stored securely (Appendix A Figure A.44). The database saves the image name for reference. To protect the system from SQL injection, the backend uses dynamic and safe SQL queries. If everything works, the transaction is committed and the system confirms success. If anything fails (such as an upload or a database insert), everything is rolled back to keep the data clean, and an error is sent back for debugging.

4.4.3 Machine Learning

4.4.3.1 Video Course Recommendation

The career path aspiration machine learning model is intended to suggest video courses based on users' skill gaps and desired career paths. However, due to time limitations, and holding requirements. The career aspiration model has not been implemented in the system (only the model itself is implemented). The video course recommendation will use data from the database and employ content-based filtering rather than a predictive model.

4.4.3.2 Data Preparation

The video recommendation system in this project relies on accurate and well-structured data to make personalized suggestions. The dataset used for training the model was retrieved directly from the company's internal training database using the following SQL query: `SELECT idcourseOnline, title, courseTopic, isActive FROM UniHR.Training_CourseOnline WHERE isActive = 1`. This query filters and extracts only active training courses, each identified by a unique course ID (`idcourseOnline`), a course title, and a list of course topics stored in the `courseTopic` column. After retrieving the data, it is loaded into a pandas DataFrame for processing. Since machine learning models require numerical input, the categorical course topics are transformed into a binary format using the `MultiLabelBinarizer` from the `scikit-learn` library. This transformation results in a feature matrix where each row represents a video and each column corresponds to a course topic, with binary values indicating the presence or absence of that topic. The resulting matrix enables the system to compute the similarity between user interests and video topics using cosine similarity.

In this project, a train-test split was not required, as the recommendation system uses content-based filtering rather than a predictive model. The system compares user interests, skill gaps, and weakness penalties with video topics in real-time using cosine similarity, without learning from historical user behavior.

4.4.3.3 Implementation

The machine learning implementation for video recommendation is built using a content-based filtering approach that leverages cosine similarity to match user interests with video content. This Python class, `VideoRecommender`, utilizes `scikit-learn`'s `MultiLabelBinarizer` to convert each video's categorical topics into a binary matrix, effectively representing each video in a multi-dimensional tag space. When a user provides their interests (e.g., "programming", "education"), these are also transformed into a binary vector to represent their preference profile. The system then calculates cosine similarity between this user profile and each video's tag vector to determine how closely they align. The top N most similar videos are recommended back to the user, complete with similarity scores, allowing the frontend to present personalized learning content. This method ensures that users receive relevant training materials based on the semantic overlap of topic tags.

4.4.3.4 Model Input and Result

```
results = suggest_courses("data science basics", "wants to be a machine learning engineer", top_n=5 )
for i, rec in enumerate(results, 1):
    print(f"{i}. {rec['course_code']} - {rec['course_title']} (Similarity: {rec['similarity']})")
```

```
1. 3 - The Science of Well-Being (Similarity: 0.52)
2. 24 - Machine Learning by Stanford University (1001) (Similarity: 0.436)
3. 2291 - Data Analytics (Similarity: 0.324)
4. 2286 - Data Visualization (Similarity: 0.324)
5. 2305 - Digital Learning (Test) (Similarity: 0.268)
```

Figure 4.3 Input and output of the video course recommendation model

In Figure 4.3, The recommend model takes two important text inputs: one is the course search query of the user (for example, "data science fundamentals"), and the other is a user goal or profile description (for example, "aspires to be a machine learning engineer"). These inputs are fed into the `suggest_courses` function, which applies cosine similarity methods to rank and fetch the most suitable video courses. The model returns a list of top n recommended courses, each with associated metadata including the course code, course title, and a similarity score that quantifies how closely the course matches the user's input. As shown in the example

output, the model successfully ranks five courses, with “The Science of Well-Being” achieving the highest similarity score of 0.52, indicating its strong relevance to the user’s learning goals. A few other top recommendations include “Machine Learning by Stanford University (1001)” and “Data Analytics”, indicating the model’s capacity to provide both foundational and specialized learning paths.

4.4.3.5 Model Evaluation

Due to the use of content-based filtering and the absence of historical user interaction data, traditional evaluation metrics (e.g., accuracy, precision) were not applicable. Instead, the model was manually evaluated by 10 participants—6 internal staff and 4 external testers.

Each participant received course recommendations based on their stated interests and skill gaps. They then assessed the relevance and usefulness of the suggested videos. Feedback was collected through ratings and comments to gauge the system’s effectiveness and alignment with user expectations.

A summary of the evaluation results is provided in Section 4.5: User Acceptance Testing (UAT) – AI-Suggested Video Courses.

4.5 UniHR Implementation

Due to time limitations and holding requirements, the UniHR platform has only completed the machine learning component; the front-end and back-end development have not yet started.

4.5.1 Machine Learning

4.5.1.1 Career Aspiration for UniHR

The CareerAspirationModel processes a dataset of NYC job postings from the internet due to the job descriptions in Thai being hard to find and the inconvenience of not having access to an office, making it necessary to use an English dataset. The model loads and preprocesses job data by combining job descriptions, required skills, and qualifications into a structured format. A TF-IDF vectorizer is trained on these career descriptions to facilitate text-based similarity matching. When a user inputs their interests, skills, and weaknesses, the model preprocesses the input and computes a cosine similarity score between the user profile and each career. A penalty is applied if the user’s weaknesses conflict with the required career skills. The model then ranks careers based on their final suitability score and presents the top recommendations along with insights into matching interests, missing skills, and potential challenges. Finally, a visualization gauge displays the suitability score, and a text summary provides details on why the career was recommended.

4.5.1.1.1 Data Preparation

The data preparation process begins by loading the NYC Jobs dataset from a CSV file and selecting relevant columns, including “Business Title”, “Job Category”, “Job Description”, “Preferred Skills”, and “Minimum Qual Requirements”. The dataset, obtained from the internet, contains a total of 3,420 rows. Missing values in these columns are filled with empty strings. A new column, “career_text”, is created by concatenating the job description, required skills, and minimum qualifications, all converted to lowercase for uniformity. This “career_text” column serves as the primary textual feature for each job. The dataset is then transformed into a list of career profiles, each containing the job title, job category (as required interests), desired skills, and the concatenated text of job-related information. Finally, the dataset is split into training and testing sets, with 70% used for training, to build a model that will generate career recommendations based on user input.

4.5.1.1.2 Feature Extraction Using TF-IDF

The `TfidfVectorizer` from `sklearn.feature_extraction.text` is used to transform text data into numerical representations based on Term Frequency-Inverse Document Frequency (TF-IDF). This method helps capture the importance of each word in the context of the entire corpus. The vectorizer is first fitted on the “career_text” field, which contains concatenated job descriptions, preferred skills, and qualifications for each career. After fitting, the vectorizer converts both the user’s input and each career’s description into TF-IDF vectors. These vectors are then used by the `cosine_similarity` function to calculate the similarity between the user’s profile and each career’s text. The use of `TfidfVectorizer` ensures that the most significant words, which are more informative and less common across the entire dataset, are given higher weight when comparing the user’s interests and skills to their careers.

4.5.1.1.3 Similarity Computation Using Cosine Similarity

The `cosine_similarity` function from `sklearn.metrics.pairwise` is used to calculate the similarity between the user’s profile (a combination of their interests and skills) and each career’s description in the dataset.

$$\textit{Similarity} = \frac{A \cdot B}{||A|| \times ||B||}$$

Where:

- A represents the TF-IDF vector of the user input.
- B represents the TF-IDF vector of a career.
- The dot product measures the overlap between the two vectors.

First, the user’s input is preprocessed and combined into a single string representing their profile. Then, the career descriptions are transformed into numerical vectors using the TF-IDF vectorizer. The `cosine_similarity` function computes the cosine similarity score between the user’s profile vector and each career’s vector. This similarity score, which ranges from 0 to 1, is then multiplied by 100 to convert it to a percentage, reflecting how closely the user’s profile matches each career. This score helps rank and recommend the most suitable career paths for the user.

4.5.1.1.4 Weakness Penalty

The model ranks career paths based on the final suitability score, which is an adjusted similarity score that takes into account both the user’s interests and skills, along with any weaknesses. The suitability score is calculated by determining how well the user’s profile matches the career description and then applying a penalty for any weaknesses that conflict with the required skills of the career. The model then presents the top three career recommendations, ranked by their suitability scores. For each recommended career, the following information is displayed: the matched interests, which are the user’s interests that align with the career’s requirements; the missing skills, which are the skills the user needs to develop in order to fit the career; and the weakness conflicts, which highlight the user’s weaknesses that could potentially create challenges in pursuing the career. This detailed feedback helps users understand both their strengths and areas for improvement in relation to each career path.

4.5.1.1.5 Model Input and Result

```

=== Career Aspiration Assessment ===
Enter your interests (comma separated): analytic, business
Enter your skills (comma separated): finance, economic, problem-solving, accounting
Enter your weaknesses (comma separated): communication, stress

```

Figure 4.4 Input of the career aspiration model

In Figure 4.4, the user is required to input their interests, current skills, and personal weaknesses. This information is then used by the machine learning model to predict the most suitable career path for them.

```

=== Top Career Recommendations ===

Career: ANALYST
Suitability Score: 20.3%

```

Figure 4.5 Text output of the career aspiration model

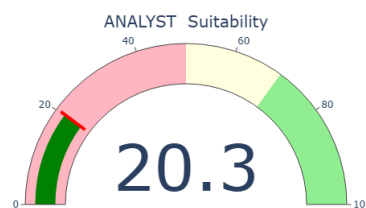


Figure 4.6 Chart output of the career aspiration model

Figures 4.5 and 4.6 illustrate one of the textual and graphical outputs of the career aspiration model. The model takes the user's input data and predicts the top three most suitable career lines based on the user's profile. The number of top career lines can also be modified in the model.

4.5.1.1.6 Model Evaluation

The career aspiration model is a predictive model; therefore, we decided to use a manual evaluation method to assess the accuracy and reliability of its results. The model was manually evaluated by a total of 10 participants, consisting of 6 internal staff members and 4 external testers. A summary of the evaluation results is presented in Section 4.6: User Acceptance Testing (UAT) – Career Aspiration.

4.6 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) for the “Employees’ Engagement and Learning Hub Website and Mobile Application” was conducted internally by users within the organization. This process involved collecting feedback through a survey designed to evaluate the functionality, usability, and overall satisfaction with each feature of the system.

The UAT survey was divided into several sections based on the key modules of the platform: Morale Daily/Yearly, Assessment 360, Quiz Management, Video Learning, Suggested Learning, Competency and Skill, and Skill Mapping. Each section of the survey consisted of 5 simple and targeted questions to assess the ease of use, UI design, and effectiveness of each feature. A total of 10 participants completed the UAT survey, including: 4 Developers, 1 Business Analyst, 1 Project Manager, 4 Testers

4.6.1 UniCulture

Morale daily and yearly question forms

For the user rating on the Morale daily and Yearly Question Forms feature which can be seen on the Appendix B Figure B.1, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings: Here's an interpretation of the ratings:

- **Form Creation and Editing (User's rating: 4.4):** Users generally found the process of creating, editing, and duplicating question forms straightforward, with ratings mostly between 4 and 5 out of 5. This indicates a high level of satisfaction with the flexibility and completeness of the form setup process, although one user rated it a 2, suggesting minor challenges or confusion in using all form options.
- **Preview Feature Usefulness (User's rating: 4.3):** The preview function received consistent positive feedback, with ratings ranging from 3 to 5. Most users rated it 4 or 5, suggesting that it helped them clearly understand how the form would appear to employees. One user rated it a 3, indicating the feature could be more intuitive or detailed for some users.
- **Filter and Search Functionality (User's rating: 4.1):** Responses for the filter usage varied slightly, with scores ranging from 2 to 5. While many users found it helpful for managing and locating forms, a few indicated some difficulty or limited usefulness, which suggests that improvements to filter visibility or filter criteria options may enhance the experience.
- **Ease of Workflow (User's rating: 4.4):** The end-to-end process from creating to assigning question forms was well-rated, with most users giving it a 4 or 5. This shows that the system's flow is intuitive for most users, but a few rated it 3–4, indicating that some users may have experienced slight complexity or extra steps during the workflow.
- **Overall Ease of Use and Satisfaction (User's rating: 4.1):** In the Figure 4.7 overall user experience with this feature was positive, with most users rating it 4 or 5. The average rating suggests that the feature is user-friendly and functional, though a small portion of users indicated room for improvement in either the interface or clarity of certain steps.

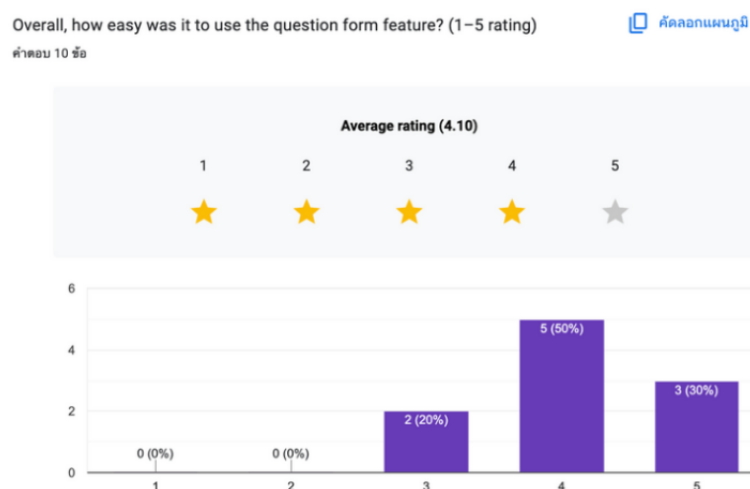


Figure 4.7 Overall User Ratings on the Morale Daily and Yearly Question Forms

Morale daily and yearly dashboard

For the user rating on the Morale daily and Yearly Dashboard which can be seen on the Appendix B Figure B.2, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Dashboard Layout Clarity (User's rating: 4.2):** Ratings for layout clarity ranged from 3 to 5 out of 5. While many users rated it 4 or 5, indicating a clean and readable layout, a few users gave it a 3, suggesting that some parts of the dashboard could be visually improved or made more intuitive.
- **Chart and Graph Readability (User's rating: 3.5):** Responses for this area showed mixed experiences, with ratings ranging from 2 to 5. While some users found the graphs easy to interpret at a glance (rated 4–5), others gave lower ratings (2–3), which suggests that certain visualizations may need clearer labeling, simplified design, or better use of color and spacing.
- **Ease of Finding “Happy/Unhappy” Status (User's rating: 4.3):** This function received varied feedback, with most users rating it between 3 and 5. Some found it straightforward to locate the status for each company, while others rated it a 3, indicating a need for more prominent or centralized display of this information.
- **Element Responsiveness and Performance (User's rating: 4.0):** Ratings for the responsiveness and functionality of dashboard components like buttons and graphs were generally high, with most users giving it a 4 or 5. A few ratings at 3 suggest that occasional delays or minor UI issues were experienced by some users.
- **Overall Ease of Navigation and Use (User's rating: 4.5):** In the Figure 4.8, Overall ratings for dashboard usability were strong, with many users giving a 4 or 5 out of 5. This indicates a smooth and functional experience for most users, though the few lower ratings imply that minor improvements to navigation or design consistency could further enhance user satisfaction.

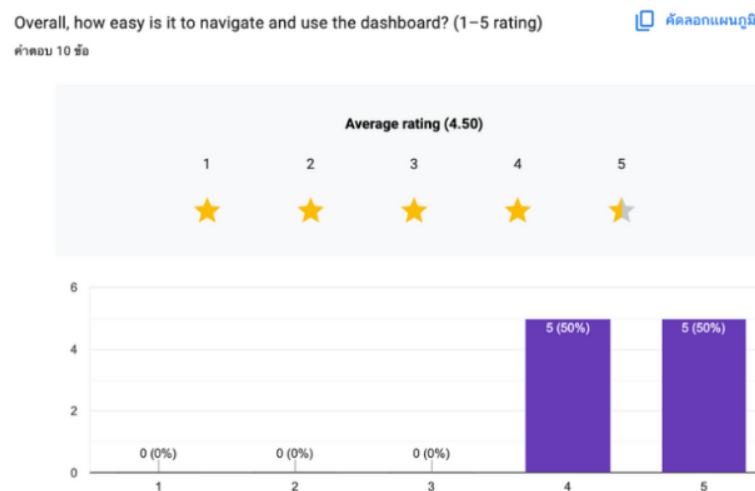


Figure 4.8 Overall User Ratings on the Morale Daily and Yearly Dashboard

360 Assessment

For the user rating on the 360 Assessment which can be seen on the Appendix B Figure B.3, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Ease of Finding and Tracking Assessments (User's rating: 4.0):** Ratings for this feature ranged from 3 to 5 out of 5, with many users rating it 4 or 5. This suggests that most users found it easy to locate and track assessments, though a few users rated it lower (3), implying that there may be a need for clearer navigation or better visibility for certain assessments.
- **Visibility and Positioning of Progress Bars and Charts (User's rating: 4.5):** Most users gave ratings of 4 or 5, indicating that the progress bars and charts are generally clear and well-positioned. However, a few users rated it lower (3), suggesting that these elements might not be as easily noticeable for some or could benefit from better placement for improved visibility.
- **Ability to See Who Has Completed Assessments (User's rating: 4.2):** This feature received high ratings, with many users rating it 4 or 5, indicating that the system successfully allows users to easily identify who has completed the assessments. A few lower ratings (3) suggest that, for some users, it might not be immediately obvious or accessible.
- **Interface Smoothness and User-Friendliness in Reviewing Responses (User's rating: 4.3):** Users gave mixed feedback on the smoothness and user-friendliness of reviewing responses, with ratings ranging from 3 to 5. While some found the interface easy to navigate and smooth to interact with, others (rated 3) experienced minor issues, possibly with loading times or the complexity of reviewing multiple responses.
- **Overall Interaction with Assessment Features (User's rating: 3.9):** In the figure 4.9, The overall ease of interacting with the assessment features received varied ratings, from 3 to 5. Many users rated it 4 or 5, indicating a generally positive experience. However, a few users gave lower ratings (3), suggesting that improvements could be made to streamline interactions or enhance the clarity of certain aspects within the assessment feature.

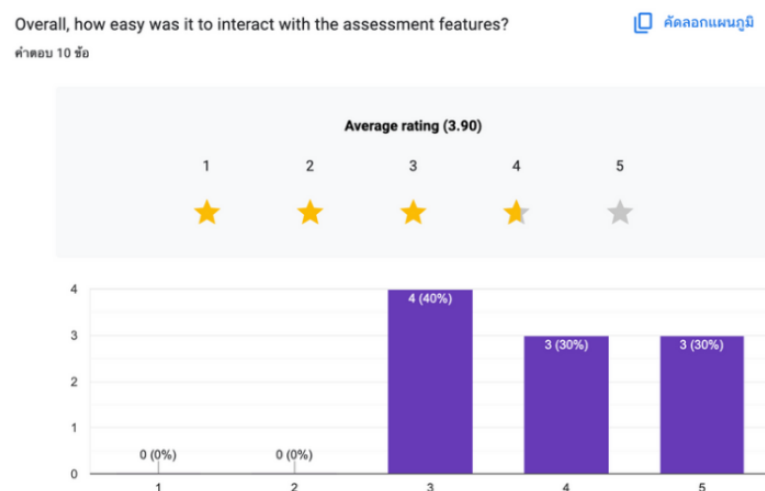


Figure 4.9 Overall User Ratings on the 360 Assessment

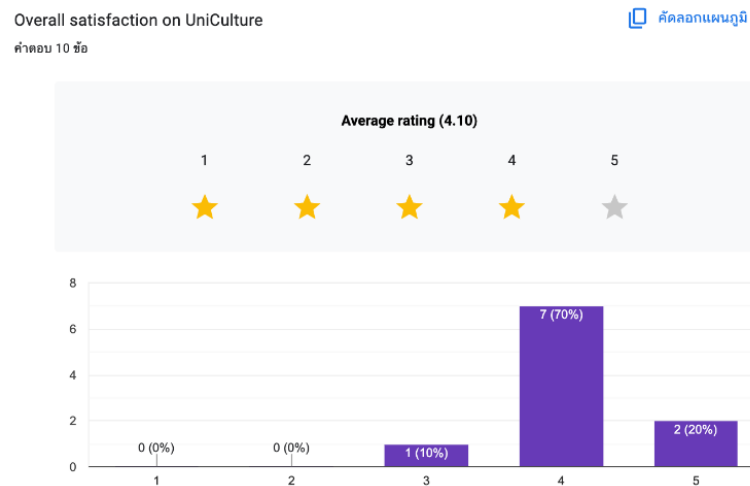


Figure 4.10 Overall User Ratings on UniCulture Platforms

The satisfaction ratings shown in Figure 4.10 show the user's rating to be 4.1 for UniCulture were generally positive, with most users providing scores of 4 or 5. This suggests that the platform is well-received, though there are areas that could benefit from further improvement.

Do you have suggestions or comments on features for further improvements on UniCulture

คำตอบ 3 ข้อ

the ui is easy to use

Overall is good, but there should be easier to assign the assessment. However, the dumping excel file function is good.

there's still the issue on the loading time

Figure 4.11 User Suggestions on UniCulture Platforms

In the Figure 4.11 shown overall, users had a positive experience with UniCulture, but there are a few areas for improvement. One user suggested making it easier to assign assessments, especially when assigning to multiple people. The ability to export data to Excel was appreciated and seen as useful. Some users pointed out that the loading times could be faster, as it's still a bit slow. The user interface (UI) was generally liked, with users finding it easy to use and navigate.

4.6.2 UniLearning

Quiz management

For the user rating on the Quiz Management which can be seen on the Appendix B Figure B.4, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Ease of Finding and Starting Assigned Quizzes (User's rating: 4.4):** Ratings for this feature ranged from 3 to 5 out of 5, with several users rating it 5, indicating that it is generally easy to find and start quizzes. However, a few users rated it 3, suggesting some users may have faced difficulties or experienced delays in accessing their assigned quizzes.

- **Clarity of Quiz Results (User's rating: 4.2):** The ratings for the clarity of quiz results were mixed, with some users rating it 3 and others giving it 4 or 5. Those who rated it 3 indicated that the results may not have been as easy to understand or lacked sufficient explanation, while higher ratings suggest that for most users, the results were clear and easy to interpret.
- **Navigation Through Quiz Questions (User's rating: 4.1):** Navigation through quiz questions received generally positive feedback, with ratings ranging from 4 to 5, indicating that users found it relatively easy to move between questions. However, one user rated it 2, implying that there may have been some confusion or difficulty in navigating the quiz for certain users.
- **Visibility of Quiz Progress or Remaining Time (User's rating: 4.1):** This feature received varied ratings, with most users rating it 4 or 5, indicating that the quiz progress and remaining time were generally visible and helpful. However, some users (rated 3) noted that it could have been more prominent or easier to track.
- **Overall Ease of Taking and Managing Quizzes (User's rating: 4.3):** In the figure 4.12, The overall experience of taking and managing quizzes had a wide range of ratings, from 3 to 5. While many users found it easy to manage and complete quizzes (rated 4 or 5), a few users rated it 3, suggesting that some improvements might be needed to enhance the overall quiz-taking experience, particularly in terms of clarity or accessibility.

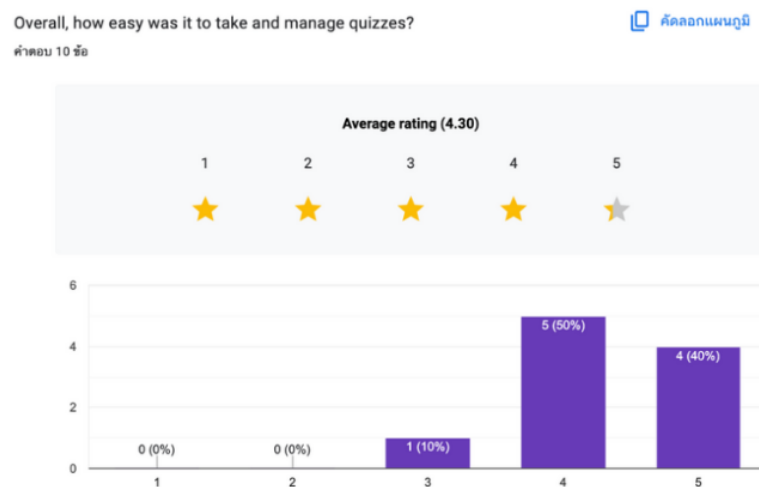


Figure 4.12 Overall User Ratings on the Quiz Management

Video learning

For the user rating on the Video Learning which can be seen on the Appendix B Figure B.5, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Clarity and Simplicity of the Video Player Interface (User's rating: 3.6):** The video player interface received a range of ratings from 2 to 5, with most users rating it 4 or 5, indicating that the interface is generally clear and easy to use. However, one user rated it 2, suggesting that they experienced difficulty or confusion with the interface, which may require improvements for certain users.
- **Ease of Finding Videos (User's rating: 4.1):** Ratings for this feature ranged from 3 to 5. A few users rated it 3, indicating some difficulty in finding the videos they wanted to watch. However, higher ratings suggest that for most users, finding videos was straightforward and efficient.

- **Visibility and Usability of the Progress Bar (User's rating: 3.9):** Most users rated this feature positively (4 or 5), indicating that the progress bar for tracking watched videos is easy to spot and use. However, one user rated it 3, suggesting that it may not have been as intuitive or visible for everyone.
- **Clarity and Helpfulness of Video Categories and Sorting (User's rating: 4.4):** Ratings for the clarity and helpfulness of video categories and sorting were mixed, with ratings ranging from 3 to 5. A few users rated it 3, suggesting that the categories and sorting could be more intuitive or useful. However, higher ratings indicate that for many users, the system's organization of videos was clear and beneficial.
- **Overall Ease of Watching and Navigating Through the Video Learning System (User's rating: 4.4):** In the figure 4.13, Overall, the video learning system was rated positively, with most users rating it 4 or 5, indicating that they found it easy to navigate and watch videos. However, a few users rated it 3, implying that there were areas where the system could be improved to enhance the experience.

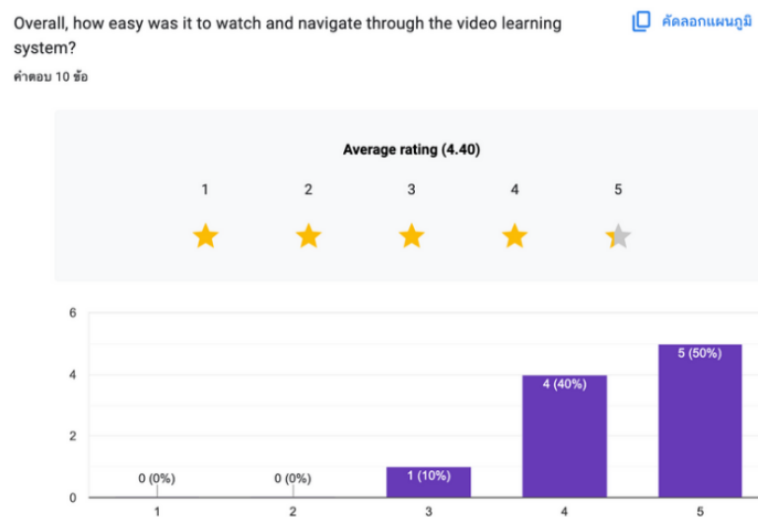


Figure 4.13 Overall User Ratings on the Video Learning

AI-Suggested Video Courses

For the user rating on the AI-Suggested Video Courses which can be seen in Appendix B Figure B.6, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Ease of Access and Use of the Recommendation Feature (User's rating: 4.3):** The recommendation feature was rated highly by most users, with ratings between 3 and 5. The majority of users found it easy to access and use, with several giving it the highest rating of 5. However, a few users rated it 3, indicating that some users might have experienced difficulty accessing or using the feature.
- **Clarity and Understandability of Recommended Courses (User's rating: 4.4):** Most users found the list of recommended courses clear and easy to understand, with ratings mostly between 4 and 5. One user rated it 3, suggesting that there may have been some confusion or lack of clarity for certain courses.
- **Explanation for Course Recommendations (User's rating: 4.4):** The feature explaining why certain courses are suggested received more mixed ratings, ranging from 3 to 5. A few users rated it 3, implying that the reasoning behind the recommendations wasn't always clear. However, the majority of users rated it 4 or 5, indicating that most found the suggestions logical and helpful.

- **Ease of Exploring or Searching for More Courses (User's rating: 3.9):** This aspect received a strong overall rating, with most users rating it 4 or 5. This suggests that users found it easy to explore or search for more courses when needed, with one user rating it 3, which may suggest some challenges with searching for additional courses.
- **Overall Usability of the AI-Suggested Video Courses Feature (User's rating: 4.4):** In the figure 4.14, Overall, the feature was rated highly, with most users giving it a 4 or 5. This indicates that users generally found the AI-suggested video courses feature easy to use. However, one user gave a rating of 2, which may indicate dissatisfaction or issues that could be improved for certain users.

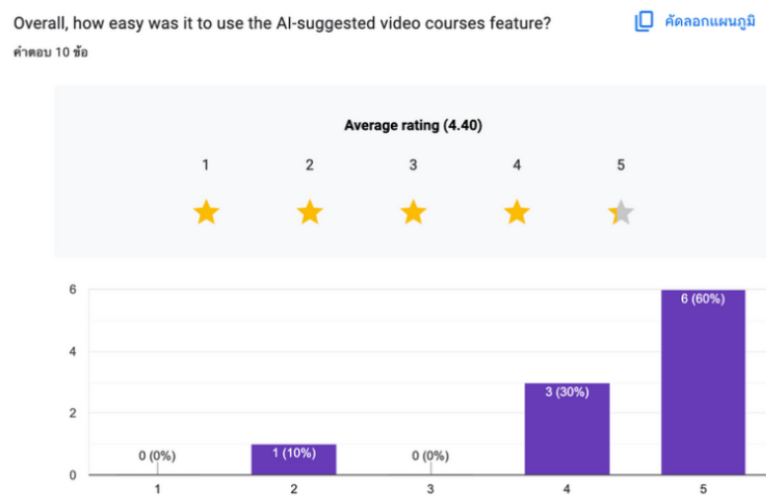


Figure 4.14 Overall User Ratings on the AI-Suggested Video Courses

Competency and Skill

For the user rating on the Competency and Skill which can be seen in Appendix B Figure B.7, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Clarity of Competency and Skill Progress (User's rating: 4.4):** Users rated the clarity of competency and skill progress highly, with most users giving it a 4 or 5. This suggests that the majority of users found the progress clearly shown. One user rated it 3, indicating that there may be some confusion or areas for improvement in how the progress is displayed.
- **Ease of Comparing Skills Across Different Categories (User's rating: 4.0):** Most users found it easy to compare their skills across different categories, with ratings ranging from 3 to 5. The majority rated it 4 or 5, indicating that the comparison feature is clear. However, one user rated it 3, implying some difficulty in making comparisons.
- **Intuitiveness of the Layout for Finding Ratings and Feedback (User's rating: 4.5):** The layout received generally positive feedback, with ratings mostly between 4 and 5. Users found it easy to navigate and locate their ratings and feedback. However, one user rated it 2, suggesting that for this user, the layout may not have been intuitive or easy to use.
- **Understanding of Charts and Ratings at a Glance (User's rating: 4.1):** The charts and ratings were mostly easy to understand at a glance, with ratings predominantly between 4 and 5. However, one user

rated it 3, which might suggest that the chart or rating system could benefit from clearer presentation for some users.

- **Overall Ease of Viewing and Understanding Competencies and Skills (User's rating: 4.0):** In the figure 4.15, Overall, users found it relatively easy to view and understand their competencies and skills, with most users giving it a 4 or 5 rating. One user gave it a 2, indicating that for some users, the feature might require further refinement to improve ease of use and understanding.



Figure 4.15 Overall User Ratings on the Competency and Skill

Career Aspiration

For the user rating on the Career Aspiration which can be seen in the Appendix B Figure B.8, the user evaluated various aspects and provided ratings. Here's an interpretation of the ratings:

- **Relevance to User Interests (User's rating: 4.6):** Most participants agreed that the recommended careers matched their interests, with ratings predominantly at 4 or 5. This suggests that the model was generally successful in aligning suggestions with users' personal preferences. A few users rated this aspect a 3 or 4, indicating minor gaps in perceived relevance for some individuals.
- **Alignment with Skills or Strengths (User's rating: 4.1):** The recommended careers were viewed as reasonably aligned with the users' current strengths, with all participants rating this item between 4 and 5. This indicates a positive perception that the suggestions reflected users' self-identified capabilities, though there is still room for improvement.
- **Helpfulness in Exploring Career Options (User's rating: 3.8):** When asked whether the recommendations helped them understand what careers they could pursue, responses were slightly more mixed. Ratings ranged from 3 to 5, with several users giving a score of 3, suggesting that while the model provided insight, it may not have been detailed or personalized enough for all users.
- **Discovery of New Career Options (User's rating: 3.5):** This aspect received the lowest average rating, indicating that not all users felt they discovered significantly new or unexpected career paths. Multiple users rated it a 3, which suggests that the model may benefit from broader or more exploratory suggestions to increase novelty and user learning.

- **Overall Usefulness of the Career Recommendation (User's rating: 4.3):** In the Figure 4.16, Overall, users found the recommendation results useful, with most ratings at 4 or 5. The feedback reflects general satisfaction, although a few users indicated a moderate level of usefulness by rating it a 3 or 4. These responses imply that while the system offers value, further enhancement of the underlying recommendation logic could improve its overall impact.

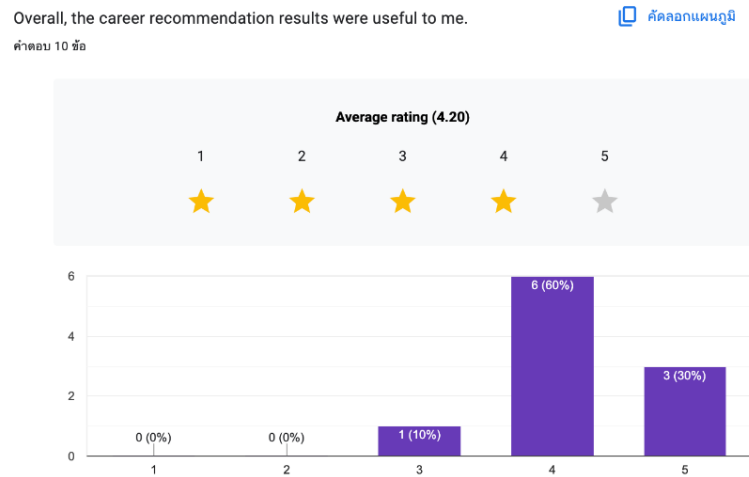


Figure 4.16 Overall User Ratings on the Career Aspiration

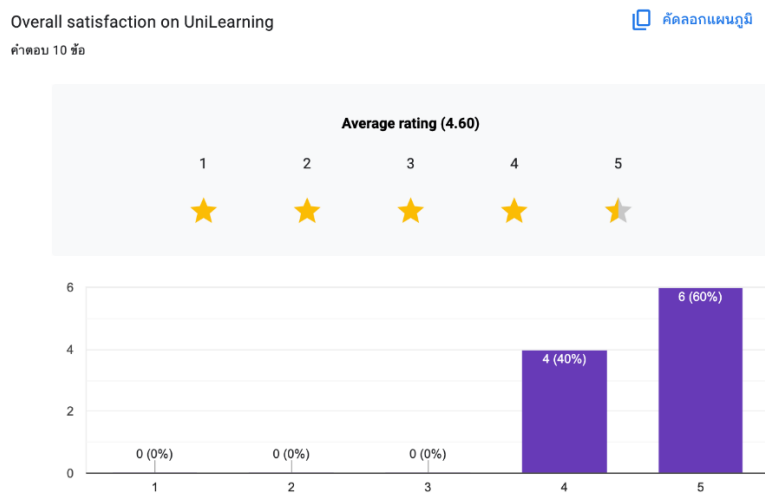


Figure 4.17 Overall User Ratings on UniLearning Platforms

Figures 4.17 showed the user's rating of satisfaction ratings for UniLearning to be 4.6 were mostly positive, with most users giving scores of 4 or 5. This indicates that the platform is well-received, although there are some aspects that could still be improved.

Do you have suggestions or comments on features for further improvements on UniLearning

คำตอบ 4 ข้อ

- the learning platform is good, still need to be more responsive
- it's a good idea to have learning hub like this so that i can learn in one platforms, love the ai reccomendation system. and having like a rush quiz can help me improve my skills on the topics as well.
-
- it's the gap time before the completed tag is showed on the video learning page, i think there need to be updated immediately on the page

Figure 4.18 User Suggestions on UniLearning Platforms

Figure 4.18 showed the users generally provided positive feedback on UniLearning, but there are some areas for improvement. Several users mentioned that the platform could be more responsive, with improvements in speed and overall performance being beneficial. There was also a suggestion to address the delay in updating the “completed” tag on the video learning page, as immediate updates would enhance the user experience. On a positive note, the learning hub concept was well-received, with users appreciating the ability to access multiple resources in one place. The AI recommendation system was highlighted for its effectiveness in suggesting relevant courses, and the addition of a “rush quiz” feature was seen as a helpful way to improve skills on specific topics.

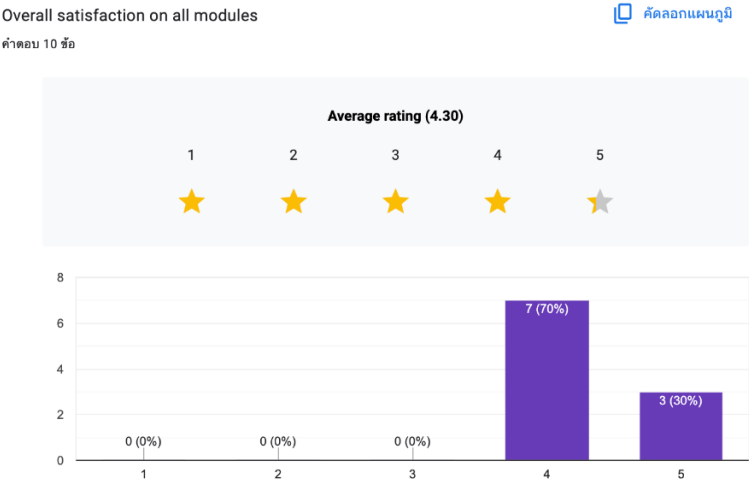


Figure 4.19 Overall User Ratings on Platforms

In Figure 4.19, the satisfaction ratings for all modules were generally positive, with most users providing scores of 4 or 5. This indicates that the modules are well-received and meet the expectations of the users. The consistent ratings of 4 and 5 reflect a solid user experience across the platform, though there may still be areas for further refinement to enhance overall satisfaction.

4.7 Challenges and Resolutions

4.7.1 Uploading Excel Files and Automating Email Sending to Outsiders

One major challenge was implementing the feature to upload Excel files and automatically send emails to outsider participants. This was a completely new functionality for the system, with no prior implementation.

Resolution:

We used the `xlsx` library (or `exceljs`) to handle Excel file parsing and `Nodemailer` to automate email sending. This allowed us to read participant data directly from the uploaded Excel file and trigger email notifications efficiently.

4.7.2 Limited Thai Dataset for Model Training

Another challenge involved training a sentiment analysis model for morale responses. Suitable Thai-language datasets were difficult to find, and our internal test user base was too small to provide sufficient training data.

Resolution:

To overcome this, we used a sentiment dictionary consisting of predefined positive and negative sentiment words. Text augmentation techniques were applied to increase data volume. We also explored external datasets and considered using transfer learning to boost model accuracy despite limited native data.

4.7.3 Dependency Conflicts Preventing Library Installation

Several required libraries failed to install due to version conflicts between dependencies. These issues slowed development and prevented necessary tools from being used.

Resolution:

We resolved this by using `pip check`, virtual environments, and specifying exact versions in a `requirements.txt` file. We also applied the `--use-deprecated=legacy-resolver` flag when needed and considered replacing conflicting libraries with alternatives.

4.7.4 Frequent Changes Due to Client Testing

Active client involvement in testing led to frequent and sometimes disruptive changes, affecting feature stability and system consistency.

Resolution:

We adopted an agile workflow, creating separate staging and production environments. All changes were tested thoroughly before being merged. Automated testing helped catch regressions quickly, maintaining stability despite constant feedback.

4.7.5 Error Bugs

Various bugs were reported by clients during testing phases, degrading the user experience. These issues ranged from UI glitches to logic errors in business processes.

Resolution:

We implemented a structured bug tracking system and prioritized fixes based on severity. Continuous integration tools were introduced to run automated tests, and frequent deployments were used to validate bug fixes. Client feedback loops ensured resolutions were complete and effective.

4.7.6 Understanding New Code and Database Structure after Transition to UniCulture

Migrating to the new UniCulture platform caused initial delays as developers needed time to understand the updated architecture and data models.

Resolution:

To accelerate onboarding, we conducted team-wide knowledge-sharing sessions, maintained updated documentation, and paired junior developers with seniors. This helped the team adapt quickly and ensured consistent development across the platform.

4.7.7 System Failures Due to High Concurrent Users in the Rush Quiz Function

The “Rush Quiz” feature crashed during high user load, as the system couldn’t handle many users joining simultaneously.

Resolution:

We optimized server-side logic, implemented caching, and introduced load balancing to support higher traffic. Stress testing was conducted regularly to monitor system performance under heavy loads, leading to a more robust and scalable quiz system.

4.8 Conclusions

Up to now, we have completed nearly all planned tasks, including the development of two machine learning models for the HRM features in the UniCulture module. We have also been actively working on the HRD features in the UniLearning module, following our project plan. However, there are still a few pages that remain unfinished, such as the Competency and Skill pages. Additionally, we need to ensure that the Rush Quiz feature is stable and capable of supporting multiple users simultaneously.

For the machine learning component of the HRD module, the career video suggestion model, development is still in progress. We are continuing to refine the model to achieve the highest possible accuracy.

CHAPTER 5 CONCLUSIONS

5.1 Conclusion

This project aimed to enhance employee development and satisfaction through the creation of the Employees' Engagement and Learning Hub, available on both web and mobile platforms. The system enables employees to participate in learning activities anytime and anywhere, meeting the goal of accessibility. In the HRD section, we developed an adaptive learning hub and quizzes that respond to individual needs, supported by a machine learning model to help improve skills. For the HRM section, the platform incorporates features that promote employee engagement and recognize satisfaction, in line with company culture. Interactive learning modules and competency assessments were also integrated to make learning experiences more engaging and measurable.

Initial feedback from testing showed increased interest in learning features and positive user responses to the engagement functions. The system has the potential to be further enhanced with advanced analytics and broader integration. Through this project, we also gained practical experience in solving real-world problems, balancing technical requirements with user needs. Overall, the system effectively supports personal growth and contributes to long-term organizational success.

5.2 Completion Status Table

In this section, we provide a comprehensive overview of the completion status of major features within the platform. The completion status for each major feature is detailed in Tables 5.1 through 5.7. Each feature is meticulously dissected into specific functionalities, accompanied by its current status. This information serves as a progress report, detailing which aspects of the platform have been fully implemented, are still in progress, or have not yet been started.

Completion Status Definitions:

- **Done:** This status indicates that the feature or functionality has been fully implemented and meets the specified requirements. Users can utilize these features without any major issues.
- **Partially Complete:** This means that significant progress has been made, but some aspects or functionalities are still under development or require further refinement. Users may encounter minor issues with these features.

From table 5.1, the project has been completed with the exception of the Career Aspiration module and Mobile application. The Career Aspiration module currently has only the machine learning model developed; both the front-end and back-end implementations have not yet been started due to time constraints and pending requirement clarifications. Based on the project manager's decision, the integration of the machine learning model is temporarily on hold. This module has been deprioritized in favor of completing the AI-suggested video course feature, which has been identified as a higher priority.

Table 5.1 Completion Status Table

Feature/Task	Description	Status
HRD Modules	Learning modules, quizzes, and competency assessments	Completed
HRM Modules	Morale surveys, morale dashboard, assessment 360	Completed
Machine Learning	1. Sentiment textual analysis 2. Video course suggestion	Completed
Machine Learning	Career aspiration	Partially Complete
Web Application	React.js front-end with Node.js backend	Completed
Mobile Application	Flutter-based cross-platform mobile app	Completed
Dashboard Development	Real-time visual analytics for HR data	Completed
User Acceptance Testing	Testing with end-users for functionality & usability	Completed
Final Report & Presentation	Project documentation and formal presentation	Completed

5.3 Discussion of Problems and Solutions

Throughout development, we encountered various challenges that required adaptive problem-solving and teamwork:

- **Time Management:** Balancing academic responsibilities and development work was a key issue. We mitigated this by using agile methodologies and Jira to break the work into manageable sprints.
- **Machine Learning Integration:** Integrating ML into HR features, especially with limited real-world data, was difficult. To overcome this, we relied on simulated data during testing and focused on building scalable ML models that can evolve with real usage.
- **Cross-platform Design Consistency:** Ensuring UI consistency across web and mobile interfaces was another challenge. We addressed this using Figma for shared design systems and Flutter's widget library for mobile responsiveness.
- **Data Privacy:** With sensitive employee data, securing access and ensuring compliance with best practices was vital. We solved this using secure Google Cloud hosting, strict authentication protocols, and encryption strategies.

5.4 Lessons Learned

This project taught us far more than technical skills:

- We learned the importance of clear communication and consistent documentation in collaborative projects.
- Gained experience in full-stack development with real-world frameworks like React, Flutter, and Node.js.
- Understanding the significance of empathy in design — developing tools that genuinely improve user experience requires thinking from the user's perspective.
- Learned to be flexible and adaptive when faced with incomplete data or changing project requirements, especially during ML model training and UI design feedback.

5.5 Future Extensions and Enhancements

While the current system is fully functional, there are still opportunities for future development:

- **Real Data Integration for ML Models:** As more user data becomes available, the accuracy and depth of our predictive models can be improved significantly.
- **Gamification Features:** Adding more game-like elements such as badges, leaderboards, and challenges could further boost engagement.
- **Chatbot Support:** An intelligent chatbot for HR FAQs, course recommendations, and morale check-ins would enhance usability.
- **API Integration:** Providing APIs for integration with other enterprise systems like payroll or attendance tracking could increase platform value.
- **Multi-language Support:** As the platform grows, adding language options would support diverse workforces in global organisations.

REFERENCES

1. Forbes Advisor, 2024, "What is Human Resources?," Available at <https://www.forbes.com/advisor/business/what-is-human-resources/>, [Online; accessed 20-September-2024].
2. AIHR, 2024, "What is Human Resources?," Available at <https://www.aihr.com/blog/what-is-human-resources/>, [Online; accessed 20-September-2024].
3. Wellhub, 2024, "Organisational Life Cycle," Available at <https://wellhub.com/en-us/blog/organizational-development/organizational-life-cycle/>, [Online; accessed 20-September-2024].
4. What is Human Resources, 2024, "Human Resource Development (HRD)," Available at <https://www.whatishumanresource.com/human-resource-development>, [Online; accessed 06-October-2024].
5. GeeksforGeeks, 2024, "Human Resource Development (HRD)," Available at <https://www.geeksforgeeks.org/introduction-of-dbms-database-management-system-set-1/>, [Online; accessed 20-September-2024].
6. Coursera, 2024, "What Is Data Analysis? (With Examples)," Available at <https://www.coursera.org/articles/what-is-data-analysis-with-examples>, [Online; accessed 20-September-2024].
7. IBM, 2024, "What is Machine Learning?," Available at <https://www.ibm.com/topics/machine-learning>, [Online; accessed 20-September-2024].
8. Jaspersoft, 2024, "What is a Performance Dashboard?," Available at <https://www.jaspersoft.com/articles/what-is-a-performance-dashboard>, [Online; accessed 20-September-2024].
9. CareerFoundry, 2024, "What is UI Design? A Guide to User Interface Design," Available at <https://careerfoundry.com/en/blog/ui-design/what-is-ui-design-guide/#what-is-ui-design>, [Online; accessed 30-September-2024].
10. Rwanda Education Board, 2024, "Types of Human Resources," Available at <https://elearning.reb.rw/course/view.php?id=1071§ion=2>, [Online; accessed 30-September-2024].
11. TechTarget, 2024, "Mobile app," Available at <https://www.techtarget.com/whatis/definition/mobile-app>, [Online; accessed 30-September-2024].
12. React Official Website, 2024, "React - A JavaScript library for building user interfaces," Available at <https://react.dev/>, [Online; accessed 30-September-2024].
13. React Hook Form, 2024, "React Hook Form Documentation," Available at <https://react-hook-form.com/>, [Online; accessed 30-September-2024].
14. Yup GitHub, 2024, "Yup Validation Schema," Available at <https://github.com/jquense/yup>, [Online; accessed 30-September-2024].
15. Google Developers Blog, 2024, "Why Choose Flutter for Cross-Platform Development," Available at <https://developers.googleblog.com/en/making-development-across-platforms-easier-for-developers>, [Online; accessed 30-September-2024].
16. GeeksforGeeks, 2024, "What is SQL?," Available at https://www.w3schools.com/sql/sql_intro.asp, [Online; accessed 30-September-2024].
17. MySQL Official Website, 2024, "MySQL Workbench Documentation," Available at <https://dev.mysql.com/doc/>, [Online; accessed 30-September-2024].
18. Mozilla Developer Network, 2024, "Introduction to Node.js," Available at https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/Introduction, [Online; accessed 30-September-2024].
19. Express Official Website, 2024, "Express - Node.js web application framework," Available at <https://expressjs.com/>, [Online; accessed 30-September-2024].
20. Google Cloud Documentation, 2024, "Google Cloud SQL," Available at <https://cloud.google.com/sql/docs>, [Online; accessed 30-September-2024].

21. W3Schools, 2024, “What is GitHub?,” Available at https://www.w3schools.com/whatis/whatis_github.asp, [Online; accessed 30-September-2024].
22. UX Academy, 2024, “What is Figma and its Advantages?,” Available at <https://myuxacademy.com/what-is-figma/>, [Online; accessed 30-September-2024].
23. Technia, 2024, “Jira Software,” Available at <https://www.technia.com/software/jira/>, [Online; accessed 30-September-2024].
24. Happily.ai Blog, 2024, “What Is Happily Application?,” Available at <https://blog.happily.ai/th/what-is-happily-application/>, [Online; accessed 30-September-2024].
25. People Managing People, 2024, “Bonusly Review,” Available at <https://peoplemanagingpeople.com/tools/bonusly-review/>, [Online; accessed 30-September-2024].
26. People Managing People, 2024, “Motivosity In-Depth Software Review for 2024,” Available at <https://peoplemanagingpeople.com/tools/motivosity-review/>, [Online; accessed 30-September-2024].

APPENDIX A
IMPLEMENTATIONS OF PLATFORMS

UniCulture

Front-end

- User's Morale yearly question

```
<Stepper alternativeLabel activeStep={activeStep}>
  {moraleQuestionItem.questionTopic.map((label, index) => {
    const stepProps = {};
    const labelProps = {};
    const isIncomplete = incompleteSteps.includes(index);
    if (isIncomplete) {labelProps.error = true;}
    labelProps.optional = (
      <Typography>{`${label.questionList.length} / ${a}`}</Typography>
    );
    return (
      <Step key={index} {...stepProps}>
        <StyledStepLabel {...labelProps}>{label.topicName}</StyledStepLabel>
      </Step>
    );
  })}
</Stepper>
```

Figure A.1 Survey Navigation System

Figure A.1, the system uses a Stepper component that dynamically maps through each survey topic. It supports various Question Types, such as multiple choice (commonly displayed in a Likert scale format) and open-ended responses. Depending on the question type, the appropriate input components are rendered—for example, a RadioGroup for multiple choice questions or a TextField for open-ended ones. This dynamic rendering allows flexibility while preserving design consistency. Visual feedback, such as highlighting selected options, further enhances clarity and interaction for the user.

```
const updateAnswer = (idMoraleQuestion) => (event) => {
  const selectedParticipant = moraleQuestionItem.participantList.find(
    (item) => item.idEmployees === userProfile.idEmployees
  );
  const newValue = { ...answer };
  newValue[event.target.name] = {
    idMoraleQuestion: idMoraleQuestion,
    idMoraleParticipant: selectedParticipant.idMoraleParticipant,
    answer: event.target.value.trim(),
  };
  setAnswer(newValue);
};
```

Figure A.2 Answer Management

Figure A.2, the implementation uses centralized state management with the useState hook to store responses in a structured object format, where each answer is linked to its corresponding morale question. This design supports both radio button selections and text input fields, managing them consistently. Validation logic is integrated to check for required fields before submission, ensuring no incomplete data is sent. This structured and validated approach maintains data integrity while streamlining the survey workflow.

• User's Answering Questions in Assessment 360

```
<StyledRoot className={"page"}>
  <HeaderPage textLabel={"ประเมิน 360 องศา"} icon={iconHeader} />
  <StyledDivider />
  {isFetching ? (
    <CircularProgress />
  ) : appraiseeUser?.length === 0 ? (
    <Typography variant="h5">{"ไม่มีข้อมูล"}</Typography>
  ) : (
    <Container>
      <Stack spacing={4}>
        {appraiseeUser?.map((item) => (
          <CardAssessment key={item.idAppraisee} appraisee={item} />
        ))}
      </Stack>
    </Container>
  )}
</StyledRoot>
```

Figure A.3 Fetching Card Style of Pending Assessment 360 Questions

```
const updateAnswer = (index, idAssessment_Question) => (event) => {
  const selectSampleSize = appraiseeItem.sampleSize.find(
    (item) => item.idAppraiser === userProfile.idEmployees
  );

  const currentQuestion = appraiseeItem.questionList.find(
    (question) => question.idAssessment_Question === idAssessment_Question
  );

  let newValue = {
    idAppraisee_SampleSize: selectSampleSize.idAppraisee_SampleSize,
    idAssessment_Question: idAssessment_Question,
  };

  if (event.target.name === "value") {
    newValue.value = event.target.value;
  } else if (
    event.target.name === "comment" &&
    currentQuestion?.questiontype === "choice"
  ) {
    newValue.comment = event.target.value;
  }

  const existingAnswerIndex = answer.findIndex(
    (ans) => ans.idAssessment_Question === idAssessment_Question
  );

  if (existingAnswerIndex === -1) {
    setAnswer([...answer, newValue]);
  } else {
    const updatedAnswers = answer.map((ans) =>
      ans.idAssessment_Question === idAssessment_Question
        ? { ...ans, [event.target.name]: event.target.value }
        : ans
    );
    setAnswer(updatedAnswers);
  }
};
```

Figure A.4 Assessment 360 updateAnswer Function

Figure A.4, the function first locates the relevant appraisee (from the sample size array) and identifies the current question based on the appraiserId and questionId. It then creates a newValue object containing basic identifiers like idAssessment_Question, idAppraisee, and type. Depending on the question type and the name of the event target (e.g., “value” or “comment”), it updates the corresponding field in newValue. If no existing answer for that question is found in the answers array, the function adds the newValue as a new entry. Otherwise, it updates the existing answer with the new input. Finally, it calls setAnswer to update the React state, effectively managing both new submissions and edits to existing responses.

```

const handleSubmit = (e) => {
  e.preventDefault();

  const answerPass =
    answer.filter((item) => {
      const currentQuestion = appraiseeItem.questionList.find(
        (question) =>
          question.idAssessment_Question === item.idAssessment_Question
      );

      if (currentQuestion?.questionType === "choice") {
        return (
          item.value &&
          item.value.trim().length > 0 &&
          (item.comment === undefined || item.comment.trim().length > 0)
        );
      } else {
        return item.value && item.value.trim().length > 0;
      }
    }).length === appraiseeItem.questionList.length;

  if (answerPass) {
    handleClickOpen();
  } else {
    setOpenSnackbar(true);
  }
};

```

Figure A.5 Assessment 360 handleSubmit Function

Figure A.5, the function filters through the answers array and validates each item based on its related question from `appraiseeItem.questionList`, matched using `idAssessment_Question`. For questions of type “choice”, it checks that a value is selected and allows the comment to be either empty or filled. For other question types, it simply ensures that a value exists. The validation passes only if the total number of valid answers matches the total number of questions. This logic ensures data integrity before submission and provides a responsive feedback mechanism using state-controlled UI components like a snackbar.

- **Admin’s Morale Daily Question Form**

```

const { control, watch, handleSubmit, formState } = useForm({
  defaultValues: {
    questionType: "",
    scale: "",
    idQuestionTopic: "",
    question: "",
    reward: 1
  },
  resolver: yupResolver(validationSchema) // Yup validation
});

// Watching values for live preview
const question = watch("question");
const reward = watch("reward");
const questionType = watch("questionType");

```

Figure A.6 Form State Management (React Hook Form)

Figure A.6, implements the state management using React Hook Form’s Controller components that manage all form inputs, with `watch()` methods enabling reactive preview updates. Local state handles the anonymous response checkbox and scale selection. The validation schema ensures question text doesn’t exceed 200 characters and that scale selection is required for multiple-choice questions. On submission, dispatched actions create the question and refresh the question list, with toast notifications providing user feedback.

- **Admin’s Morale Yearly Question Form Implementation**

```

{questions.map((question, index) => (
  <Paper key={index} elevation={3} sx={{ mt: 3, p: 4 }}>
    <Typography variant="h6">Question {index + 1}</Typography>
    <TextField
      placeholder="Question text"
      value={question.questionText}
      onChange={(e) => handleQuestionChange(index, "questionText", e.target.value)}
      fullWidth
    />
    <TextField
      select
      label="Question type"
      value={question.questionStyle}
      onChange={(e) => handleQuestionChange(index, "questionStyle", e.target.value)}
    >
      {questionTypes.map((type) => (
        <MenuItem key={type} value={type}>{type}</MenuItem>
      ))}
    </TextField>
  </Paper>
)}}

```

Figure A.7 Dynamic Question Rendering Code

Figure A.7, the interface uses interactive form components tied to state management for real-time updates as users modify question content. For multiple-choice questions, the system supports dynamic rendering of scale options, where labels and values are customizable. Each question block is uniquely identified and rendered using keys, ensuring React's efficient re-rendering process. Client-side validation logic is implemented to check for missing or incomplete fields before submission, with visual cues like error messages or highlights guiding users to correct issues. This ensures data completeness and prevents submission of incomplete assessments.

```

const [openStatusDialog, setOpenStatusDialog] = useState(false);
const [valueTarget, setValueTarget] = useState("company");

<Dialog open={openStatusDialog}>
  <DialogTitle>Select Status</DialogTitle>
  <DialogContent>
    <RadioGroup value={valueTarget} onChange={(e) => setValueTarget(e.target.value)}>
      <FormControlLabel value="company" control={<Radio />} label="Company-wide" />
      <FormControlLabel value="department" control={<Radio />} label="By Department" />
    </RadioGroup>
    {valueTarget === "department" && (
      <Autocomplete
        multiple
        options={departments}
        getOptionLabel={(dept) => dept.name}
        renderInput={(params) => <TextField {...params} label="Select departments" />}
      />
    )}
  </DialogContent>
  <DialogActions>
    <Button onClick={() => setOpenStatusDialog(false)}>Cancel</Button>
    <Button onClick={handlePublish}>Publish</Button>
  </DialogActions>
</Dialog>

```

Figure A.8 Publishing Workflow Code

Figure A.8, the component receives three key props: value (an array of survey questions), year (the selected year filter), and selectedCompany (the active company ID). It categorizes survey data into four tabs:

- In Progress: Surveys that are currently active (end date is in the future).
- Draft: Surveys saved as drafts but not yet published.
- History: All published surveys, regardless of date.
- Expired: Published surveys whose end dates have passed.

based on each survey's publishing and end date status. The system uses complex state management to handle nested data structures for survey content and configurations, while ensuring that UI interactions remain

responsive. Built with a consistent design system, the layout adapts to different screen sizes and provides a seamless user experience through accessible elements and visual indicators that guide users in managing their surveys effectively.

```
const [selectQuestionStatus, setSelectQuestionStatus] = React.useState(1);
const handleQuestionStatus = (event, newQuestionStatus) => {
  if (newQuestionStatus !== null) {
    setSelectQuestionStatus(newQuestionStatus);
  }
};
const isInProgress = (endDate) => {
  const currentDate = dayjs();
  return dayjs(endDate).isAfter(currentDate);
};
const isUnProgress = (endDate) => {
  const currentDate = dayjs();
  return dayjs(endDate).isBefore(currentDate);
};
```

Figure A.9 Derived State (Date Validation)

```
<Tabs
  value={selectQuestionStatus}
  onChange={handleQuestionStatus}
  exclusive
  aria-label="text alignment"
  textColor="primary"
>
  <Tab label="กำลังดำเนินการ" value={2} aria-label="active" />
  <Tab label="กำลังรอส่ง" value={1} aria-label="active" />
  <Tab label="ประวัติ" value={3} aria-label="achieve" />
  <Tab label="หมายเหตุ" value={0} aria-label="achieve" />
</Tabs>
```

Figure A.10 Tab State Management

- **Admin's Morale Daily Dashboard**

```
const totalAverageScore =
  scoreData?.dataLength > 0
    ? Math.round(scoreData.totalScore / scoreData.dataLength)
    : 0;
```

Figure A.11 The totalAverageScore function

Figure A.11, the function leverages basic conditional logic to check for the presence of data before performing the calculation. The formula used for the average is $\text{totalScore} / \text{dataLength}$, with the result rounded using `Math.round()`. This ensures that the computed average is a whole number. If the condition of `dataLength > 0` is not met, the function provides a fallback value of 0 to handle cases where no data is available, preventing errors or undefined behavior.

```

// Calculate total participants
const totalParticipants = feelings.reduce(
  (total, feeling) => total + (scoreData[feeling] || 0),
  0
);

// Generate pie chart data
const pieData = feelings.map((feeling) => {
  const count = scoreData[feeling] || 0;
  const percentage = totalParticipants
    ? Math.round((count / totalParticipants) * 100)
    : 0;

  return {
    label: `${feelingLabels[feeling]} (${percentage}%, ${count})`,
    value: count,
    color: feelingColors[feeling],
  };
});

```

Figure A.12 Data preparation

```

const getTop3Strengths = (filterData) => {
  const scoreTopic = filterData?.reduce((acc, cur) => {
    for (const x of cur.questionTopics) {
      if (!acc[x.questionTopic]) {
        acc[x.questionTopic] = { sumScore: 0, dataLength: 0 };
      }
      if (x.averageScore) {
        acc[x.questionTopic].sumScore += Number(x.averageScore);
        acc[x.questionTopic].dataLength++;
      }
    }
    return acc;
  }, {});

  const topTopics = Object.keys(scoreTopic).map((topic) => {
    const data = scoreTopic[topic];
    return {
      topic,
      averageScore: data.dataLength > 0 ? data.sumScore / data.dataLength : 0,
    };
  });

  const top3Topics = topTopics
    .sort((a, b) => b.averageScore - a.averageScore)
    .slice(0, 3);

  return top3Topics;
};

```

Figure A.13 Top3StrengthDailyTable data preparation function

Figure A.13, the function starts by creating the scoreTopic object to accumulate the scores and response counts for each topic. It iterates through the filterData array, updating the sum and count for each topic. After the loop, the average score for each topic is calculated by dividing the sum by the count of responses. The results are stored in the topTopics array, which is then sorted in descending order based on the average scores. Finally, the top 3 topics with the highest scores are selected and returned as the output.

```

const getBottom3Weaknesses = (filterData) => {
  const scoreTopic = filterData?.reduce((acc, cur) => {
    for (const x of cur.questionTopics) {
      if (!acc[x.questionTopic]) {
        acc[x.questionTopic] = { sumScore: 0, dataLength: 0 };
      }
      if (x.averageScore) {
        acc[x.questionTopic].sumScore += Number(x.averageScore);
        acc[x.questionTopic].dataLength++;
      }
    }
    return acc;
  }, {});

  const bottomTopics = Object.keys(scoreTopic).map((topic) => {
    const data = scoreTopic[topic];
    return {
      topic,
      averageScore: data.dataLength > 0 ? data.sumScore / data.dataLength : 0,
    };
  });

  const bottom3Topics = bottomTopics
    .sort((a, b) => a.averageScore - b.averageScore)
    .slice(0, 3);

  return bottom3Topics;
};

```

Figure A.14 Top3WeaknessDailyTable Data Preparation Function

Figure A.14, the function first initializes the scoreTopic object to track the sum of scores and response counts for each topic. It then iterates over the filterData array, updating the sum and count for each topic. After the iteration, it calculates the average score for each topic by dividing the sum by the count of responses. The results are stored in the bottomTopics array, which is then sorted in ascending order based on the average scores. Finally, the bottom 3 topics with the lowest average scores are selected and returned as the output.

```

const departmentScores = filterData.reduce((acc, cur) => {
  const deptKey = cur.departmentName;
  if (!acc[deptKey]) {
    acc[deptKey] = { sumScore: 0, count: 0 };
  }
  if (cur.averageScore) {
    acc[deptKey].sumScore += cur.averageScore;
    acc[deptKey].count += 1;
  }
  return acc;
}, {});

const departmentScoresArray = Object.keys(departmentScores).map((key) => ({
  department: key,
  averageScore: Math.round(
    departmentScores[key].sumScore / departmentScores[key].count
  ),
}));

```

Figure A.15 Department Average Score Table Data Preparation Function

```
const sectionScores = filterData.reduce((acc, cur) => {
  const sectionKey = cur.sectionName;
  if (!acc[sectionKey]) {
    acc[sectionKey] = { sumScore: 0, count: 0 };
  }
  if (cur.averageScore) {
    acc[sectionKey].sumScore += cur.averageScore;
    acc[sectionKey].count += 1;
  }
  return acc;
}, {});

const sectionScoresArray = Object.keys(sectionScores).map((key) => ({
  section: key,
  averageScore: Math.round(
    sectionScores[key].sumScore / sectionScores[key].count
  ),
}));
```

Figure A.16 Section Average Score Table Data Preparation Function

```
const calculateTotalAverageScore = () => {
  const allAnswers = answers.flatMap((item) => item.employeesAnswer || []);
  if (allAnswers.length === 0) return 0;
  const totalScore = allAnswers.reduce(
    (sum, answer) =>
      sum +
      (answer.sentimentScore !== null && answer.sentimentScore !== undefined
        ? answer.sentimentScore
        : 0),
    0
  );
  return (totalScore / allAnswers.length).toFixed(2);
};
```

Figure A.17 calculateTotalAverageScore Function

- **Admin and Manager's Morale Yearly Dashboard**

```
const avgScore = () => {
  if (!moraleKPI?.moraleTopicAvgAndBest?.length) return 0;
  const totalScore = moraleKPI.moraleTopicAvgAndBest.reduce(
    (sum, topic) => sum + topic.average,
    0
  );
  return Math.ceil(totalScore / moraleKPI.moraleTopicAvgAndBest.length);
};
```

Figure A.18 avgScore Function

Figure A.18, it works by checking if the moraleTopicAvgAndBest array exists and is not empty. If it's empty, it returns 0. Otherwise, it uses the reduce method to sum all the average values from the array, then divides the total by the number of items. The result is rounded up to the nearest whole number using Math.ceil() before being returned.

```
const compareAverageScore = () => {
  if (!moraleKPI?.moraleTopicAvgAndBest?.length || !moraleCompareData) {
    return null;
  }

  const currentAvg = avgScore();
  const selectedAvg = calculateAverageScore(moraleCompareData);
  const difference = selectedAvg !== null ? currentAvg - selectedAvg : null;

  return {
    difference: difference !== null ? Math.abs(difference.toFixed(2)) : null,
    isIncrease: difference !== null ? difference > 0 : null,
    isDecrease: difference !== null ? difference < 0 : null,
  };
};

const scoreComparison = compareAverageScore();
```

Figure A.19 compareAverageScore Function

Figure A.19, the function first confirms that the necessary data exists. It then calculates the difference between the current average (from avgScore()) and a previous score (moraleCompareData). The result is an object that includes the difference and flags indicating if the score increased (isIncrease) or decreased (isDecrease).

```
const dataTopic2 = (moraleKPI?.moraleTopicAvgAndBest || []).map(
  (topic, index) => ({
    questionTopic: topic.questionTopic || "Unknown",
    average: parseFloat(topic.average?.toFixed(2)) || 0,
    color: colorSet[index % colorSet.length],
  })
);
```

Figure A.20 Bar Chart of Total Average Score According to Question Topic Data Preparation

Figure A.20, it loops through the moraleTopicAvgAndBest array and maps each item into a new object with three properties: questionTopic (or “Unknown” if missing), average (parsed and rounded to 2 decimal places), and color (cycled from colorSet based on the index). The result is stored in dataTopic2 for use in the chart.

```
const dataDepartment2 = (moraleKPI?.moraleDepAvgAndBest || []).map((dep) => ({
  departmentName: dep.departmentName || "Unknown",
  averageDepartment: parseFloat(dep.average?.toFixed(2)) || 0,
  color: "#00b81e",
}));
```

Figure A.21 Bar Chart of Total Average Score According to Section Data Preparation

Figure A.21, it processes the moraleDepAvgAndBest array, generating a new array called dataDepartment2. Each department entry includes the name (defaulting to “Unknown” if not present), the average score (rounded to two decimal places or 0 if missing), and a green color #00b81e. Each formatted entry is ready for use in visual displays.


```
const getTopStrengthsAndWeaknesses = () => {
  const sortedTopics = moraleKPI.moraleTopicAvgAndBest
    .slice()
    .sort((a, b) => b.average - a.average);

  const topStrengths = sortedTopics
    .slice(0, 3)
    .map((topic) => ({ ...topic, type: "Strength" }));

  const topWeaknesses = sortedTopics
    .slice(-3)
    .map((topic) => ({ ...topic, type: "Weakness" }));

  return [...topStrengths, ...topWeaknesses];
};

const topTopics = getTopStrengthsAndWeaknesses();
```

Figure A.22 getTopStrengthsAndWeaknesses Function

Figure A.22, the function sorts the moraleTopicAvgAndBest array in descending order based on average scores. It selects the top 3 as strengths and the bottom 3 as weaknesses. Each topic is given a type label either “Strength” or “Weakness” and the two groups are combined into one array named topTopics for display or further analysis.

- **Admin’s Dashboard for Assessment 360**

```
const prepareChartData = (appraisee) => {
  if (!appraisee || !appraisee.sampleSize || !appraisee.questionList)
    return { series: [], xAxis: [] };

  const questions = appraisee.questionList.map((item) => item.question);
  const scores = new Array(questions.length).fill(0);
  const counts = new Array(questions.length).fill(0);

  appraisee.sampleSize.forEach((sample) => {
    if (sample.answerList) {
      sample.answerList.forEach((answer) => {
        const questionIndex = appraisee.questionList.findIndex(
          (question) =>
            question.idAssessment_Question === answer.idAssessment_Question
        );

        if (questionIndex >= 0) {
          const score = Number(answer.answer);
          if (!isNaN(score)) {
            scores[questionIndex] += score;
            counts[questionIndex] += 1;
          }
        }
      });
    }
  });

  const series = [
    {
      data: scores.map((score, index) =>
        counts[index] > 0 ? score / counts[index] : 0
      ),
    },
  ];

  const xAxis = [{ data: questions, scaleType: "band" }];

  return { series, xAxis };
};
```

Figure A.23 prepareChartData Function

Figure A.23, the prepareChartData function checks if sampleSize and questionList exist in the appraisee object. If not, it returns empty arrays for series and xAxis. It then creates scores and counts arrays to keep track of total scores and the number of responses for each question. For each participant in sampleSize, the function loops through their answerList, and matches answers with questions using idAssessment_Question. Valid numeric scores are added to the scores array and their counts are incremented. Once all data is processed, average scores are calculated by dividing totals by their counts, and these are stored in series. The question texts are stored in xAxis, and both are returned as a result object for use in a chart component.

```

const calculateAverageScore = (appraisee) => {
  if (
    !appraisee ||
    appraisee.questionList.every((q) => q.questionType === "text")
  )
    return 0;

  let weightedTotalScore = 0;
  let fullScore = 0;

  appraisee.sampleSize.forEach((sample, sampleIndex) => {
    const sampleWeight = sample.ratio;
    const questionLength = sample.answerList?.length || 0;

    if (questionLength > 0 && !isNaN(sampleWeight)) {
      sample.answerList.forEach((answer, answerIndex) => {
        const score = Number(answer.answer);

        if (!isNaN(score)) {
          const weightedScore = score * (sampleWeight / 100);
          weightedTotalScore += weightedScore;
        }
      });

      fullScore += questionLength * 100 * (sampleWeight / 100);
    } else if (isNaN(sampleWeight)) {
      console.warn('Invalid ratio for sample:', sample);
    }
  });

  return fullScore > 0 ? (weightedTotalScore / fullScore) * 100 : 0;
};

```

Figure A.24 calculateAverageScore Function

Figure A.24, the calculateAverageScore function begins by validating the appraisee object and filtering out any questions from questionList that are of type “text”. It sets weightedTotalScore to sum up all valid weighted scores, and fullScore to calculate the total possible score. It loops through the sampleSize array, retrieving each sample’s ratio (used as a percentage weight). For each sample that has an answerList, it multiplies each answer’s score by the weight (ratio / 100) and adds it to weightedTotalScore. The function also updates fullScore by considering the number of valid questions and the sample’s weight. If the sample weight is not a valid number (NaN), a warning is logged. Finally, the function computes the weighted average score using the formula $(\text{weightedTotalScore} / \text{fullScore}) * 100$, and returns 0 if fullScore is zero.

Front-end

• Morale Daily Dashboard

```

exports.getMoraleDailyKPI = async (req, res) => {
  try {
    const { idCompany, year, month } = req.query;
    const moraleDaily = await pool.query(
      `*Confidential Query Code*`,
      [idCompany, year, month]
    );

    const idMoraleDailyList = moraleDaily.map((item) => item.idMoraleDaily);
    console.log("idMoraleDailyList", idMoraleDailyList);

    let result = [];
    if (idMoraleDailyList.length > 0) {
      result = await pool.query(
        `*Confidential Query Code*`,
        [idMoraleDailyList, idCompany]
      );
    }

    for (let e of result) {
      e.imageUrl = e.imageName
      ? await getSignedUrlUniHR(
        `employees/profile/${e.idCompany}/${e.imageName}`
      )
      : null;

      delete e.imageName;
      delete e.idCompany;

      const questionTopic = await pool.query(
        `*Confidential Query Code*`,
        [e.idEmployees, idMoraleDailyList, idCompany]
      );
      e.questionTopics = questionTopic;

      const [averageScore] = await pool.query(
        `*Confidential Query Code*`,
        [e.idEmployees, idMoraleDailyList]
      );
      Object.assign(e, averageScore);
    }

    res.status(200).send(result);
  } catch (error) {
    console.error("Error in getMoraleDailyKPI:", error.message);
    res.status(500).send({ message: error.message });
  }
};

```

Figure A.25 The getMoraleDailyKPI Function

Figure A.25, the getMoraleDailyKPI function starts by filtering morale records using the provided companyId, year, and month. It extracts relevant employee information including employeeId, firstname_TH, lastname_TH, position, department, and profileImage. If a profileImage exists, it generates a signed URL for secure access. The function retrieves the set of questions or topics from morale assessments and calculates the average score per employee by mapping text-based sentiments (like “Very Happy”) to numerical values (e.g., 100 for “Very Happy”, 0 for “Very Bad”). These values are used to compute each employee’s daily morale score. All processed data is then returned in a structured format; any server errors are caught and returned with appropriate status and logging.

```
async function dogetQuestionDailyAnswerText(idCompany, year, month) {
  const moraleDaily = await pool.query(
    `*Confidential Query Code*`,
    [idCompany, year, month]
  );

  for (const e of moraleDaily) {
    const employees = await pool.query(
      `*Confidential Query Code*`,
      [e.idMoraleDaily]
    );

    for (const emp of employees) {
      try {
        const sentiment = await analyzeSentiment(emp.answer);
        emp.sentimentScore = sentiment.sentimentScore;
      } catch (error) {
        emp.sentimentScore = null;
      }
    }

    e.employeesAnswer = employees;
  }

  return moraleDaily;
}
```

Figure A.26 The dogetQuestionDailyAnswerText Function

- Morale Yearly Dashboard

```

exports.getMoraleDailyKPI = async (req, res) => {
  try {
    const { idCompany, year, month } = req.query;
    const moraleDaily = await pool.query(
      '*Confidential Query Code*',
      [idCompany, year, month]
    );

    const idMoraleDailyList = moraleDaily.map((item) => item.idMoraleDaily);
    console.log("idMoraleDailyList", idMoraleDailyList);

    let result = [];
    if (idMoraleDailyList.length > 0) {
      result = await pool.query(
        '*Confidential Query Code*',
        [idMoraleDailyList, idCompany]
      );
    }

    for (let e of result) {
      e.imageUrl = e.imageName
      ? await getSignedUrlUniHR(
          'employees/profile/${e.idCompany}/${e.imageName}'
        )
      : null;

      delete e.imageName;
      delete e.idCompany;

      const questionTopic = await pool.query(
        '*Confidential Query Code*',
        [e.idEmployees, idMoraleDailyList, idCompany]
      );
      e.questionTopics = questionTopic;

      const [averageScore] = await pool.query(
        '*Confidential Query Code*',
        [e.idEmployees, idMoraleDailyList]
      );
      Object.assign(e, averageScore);
    }

    res.status(200).send(result);
  } catch (error) {
    console.error("Error in getMoraleDailyKPI:", error.message);
    res.status(500).send({ message: error.message });
  }
};

```

Figure A.27 The getMoraleKPI function

Figure A.27, the getMoraleKPI function accepts parameters including idMorale, companyId, divisionId, departmentId, and sectionId. It applies these filters to query the database and retrieve a list of applicable employees. If no employees are found, it sends a 400 error response. When employees are available, it proceeds to fetch:

- moraleKPI (overall indicators)
- moraleMonthly (monthly sentiment breakdown)
- moraleDepAvgAndBest (average and best-performing departments)
- moraleTopicAvgAndBest (topic-based breakdowns)

For each topic, it queries related questions. If a question is choice-based, it uses methods to compute averages and score distributions. If the question is open-ended, it gathers the responses and associates sentiment analysis or raw text data. All results are returned in a JSON structure for use in dashboards or reports.

- **Morale Daily Question Form**

```
exports.addMoraleDaily = async (req, res) => {
  try {
    // Parameter extraction and initialization
    const connection = await pool.promise().getConnection();
    await connection.beginTransaction();

    // Survey scheduling
    const [{ insertId: idMoraleDaily }] = await connection.query(
      "INSERT INTO UniCulture_MoraleDaily (startDate, endDate, idMoraledailyQuestion) VALUES (?, ?, ?)",
      [startDate, endDate, idMoraledailyQuestion]
    );

    // Participant resolution
    let idEmployees = [];
    if(valueTarget === 'department') {
      // Department-specific employee query
    } else if(valueTarget === 'company') {
      // Company-wide employee query
    }

    // Bulk participant insertion
    const participantValues = idEmployees.map(idEmp => [idEmp, idMoraleDaily]);
    await connection.query(
      "INSERT INTO UniCulture_MoraleDailyParticipant VALUES ?",
      [participantValues]
    );

    await connection.commit();
    res.status(201).json({ message: "Create morale daily success" });
  } catch (err) {
    // Error handling
  }
};
```

Figure A.28 The addMoraleDaily Function

Figure A.28, implementation includes a transaction that commits only if all operations are successful, or rolls back if any errors occur. To efficiently insert participant data, the function uses bulk insertion to add multiple entries into the UniCulture_MoraleDailyParticipant table. On the frontend side, the handlePublish function collects the form data and dispatches it to the API, handling both success and failure scenarios. This design focuses on transactional control, bulk data insertion efficiency, and robust error management to ensure the system can handle morale data management reliably and at scale.

- Morale Yearly Question Form

```

export.addMorale = async (req, res) => {
  const { moraleName, moraleCode, description, image, status, idCompany, questions } = req.body;
  const connection = await db.getConnection();

  try {
    await connection.beginTransaction();
    const [moraleResult] = await connection.query(
      "INSERT INTO UniCulture_Morale (moraleName, moraleCode, description, image, status, idCompany) VALUES (?, ?, ?, ?, ?, ?)",
      [moraleName, moraleCode, description, image, status, idCompany]
    );

    const moraleId = moraleResult.insertId;
    for (const question of questions) {
      const [questionResult] = await connection.query(
        "INSERT INTO UniCulture_Question (topic, type, text) VALUES (?, ?, ?)",
        [question.topic, question.type, question.text]
      );

      const questionId = questionResult.insertId;
      await connection.query(
        "INSERT INTO UniCulture_MoraleQuestion (idMorale, idQuestion) VALUES (?, ?)",
        [moraleId, questionId]
      );

      if (question.type === "Multiple choice") {
        await connection.query(
          "INSERT INTO UniCulture_QuestionScaleNotes (idQuestion, scaleNotes) VALUES (?, ?)",
          [questionId, question.scaleNotes]
        );
      }
    }

    await connection.commit();
    res.status(201).json({ message: "Morale added successfully." });
  } catch (error) {
    await connection.rollback();
    res.status(500).json({ error: "Failed to add morale." });
  } finally {
    connection.release();
  }
};

```

Figure A.29 The addMorale Function

Figure A.29, The addMorale controller employs parameterized SQL queries to prevent SQL injection and ensure safe data handling. It inserts main morale data into the UniCulture_Morale table and handles each question individually by determining its associated topic and inserting it into the UniCulture_Question table. For multiple-choice questions, the controller inserts detailed scale notes into the UniCulture_QuestionScaleNotes table. Once the question processing is complete, the controller maps the questions to the UniCulture_MoraleQuestion table to link them with the corresponding morale entry.

The addMoraleParticipant controller focuses on participant assignment based on the target group. It retrieves employee data from relevant tables and inserts participant records into the UniCulture_MoraleParticipant table. Upon publishing the morale entry, the UniCulture_Morale table is updated to reflect the published status.

```

exports.updateMorale = async (req, res) => {
  const idMorale = req.body.idMorale;
  const questions = req.body.questions;
  let connection = null;
  try {
    connection = await pool.promise().getConnection();
    await connection.query(
      `UPDATE UniCulture_Morale SET moraleName = ?, code = ?, description = ?, picture = ?, startDate = ?,
      endDate = ?, moraleStatus = ?, moraleType = ?, scale = ?, idMoraleCompare = ? WHERE idMorale = ?`
      [req.body.moraleName, req.body.moraleCode, req.body.descriptionText, req.body.image, req.body.startDate,
      req.body.endDate, req.body.moraleStatus, req.body.moraleType, req.body.scale, req.body.idMoraleCompare,
      idMorale],
    );

    for (const question of questions) {
      const { idQuestion, questionText, questionType, questionTopic, scaleNotes } = question;
      const [topicResult] = await connection.query(
        "SELECT idQuestionTopic FROM UniCulture_QuestionTopic WHERE questionTopic = ?", [questionTopic]
      );
      const idQuestionTopic = topicResult[0].idQuestionTopic;

      if (idQuestion) {
        await connection.query(
          "UPDATE UniCulture_Question SET question = ?, questionType = ?, idQuestionTopic = ? WHERE idQuestion = ?",
          [questionText, questionType, idQuestionTopic, idQuestion]
        );
      } else {
        const [questionResult] = await connection.query(
          "INSERT INTO UniCulture_Question (question, questionType, idQuestionTopic) VALUES (?, ?, ?)",
          [questionText, questionType, idQuestionTopic]
        );
        await connection.query(
          "INSERT INTO UniCulture_MoraleQuestion (idMorale, idQuestion) VALUES (?, ?)",
          [idMorale, questionResult.insertId]
        );
      }
    }

    await connection.commit();
    res.status(200).send({ message: "Morale and questions updated successfully." });
  } catch (err) {
    if (connection) await connection.rollback();
    res.status(500).send({ message: err.message });
  } finally {
    if (connection) await connection.release();
  }
};

```

Figure A.30 The updateMorale Controller

- Machine Learning

```

async function analyzeSentiment(text) {
  if (isThai(text)) {
    try {
      const response = await axios.post(
        "http://localhost:8000/predict_sentiment/",
        { text }
      );

      if (response.data && typeof response.data.sentiment_score === "number") {
        const sentimentScore = Math.round(response.data.sentiment_score);
        return {
          sentence: text,
          sentimentScore: sentimentScore,
        };
      } else {
        console.error("Unexpected FastAPI response format:", response.data);
        return {
          sentence: text,
          sentimentScore: null,
        };
      }
    } catch (error) {
      console.error("Error communicating with FastAPI:", error.message);
      return {
        sentence: text,
        sentimentScore: null,
      };
    }
  } else {
    const analysis = vader.SentimentIntensityAnalyzer.polarity_scores(text);
    const sentimentScore = Math.round((analysis.compound + 1) * 50);
    return {
      sentence: text,
      sentimentScore: sentimentScore,
    };
  }
}

```

Figure A.31 The analyzeSentiment Function

Figure A.31, the analyzeSentiment function is implemented asynchronously to handle real-time sentiment analysis requests. If the text is in Thai, the function sends a POST request to a FastAPI endpoint and processes the response to extract and round the sentiment_score. If no valid score is found, it logs an error and returns null. For non-Thai text, the function uses the VADER sentiment analysis tool (vader.SentimentIntensityAnalyzer.polarity_scores) to compute a compound sentiment score, which is then normalized to a percentage scale from 0 to 100, where -1 maps to 0 and 1 maps to 100. The function wraps up by returning an object with the original text and its sentiment score. Any issues with the FastAPI request are caught, logged, and result in a null sentiment score.

```

# Load Thai stopwords
stop_words = list(thai_stopwords())

# Load sentiment words
negative_file_path = "./data/expanded-negative-sentiment-words.txt"
positive_file_path = "./data/expanded-positive-sentiment-words.txt"

with open(negative_file_path, "r", encoding="utf-8") as f:
    negative_words = set(f.read().splitlines())

with open(positive_file_path, "r", encoding="utf-8") as f:
    positive_words = set(f.read().splitlines())

# Create a sentiment dictionary
sentiment_dict = {word: -1 for word in negative_words}
sentiment_dict.update({word: 1 for word in positive_words})

# Preprocess text
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^\u0E00-\u0E7Fa-zA-Z\s]', '', text) # Remove special characters
    tokens = word_tokenize(text, engine="newmm")
    tokens = [token for token in tokens if token not in stop_words] # Remove stopwords
    return ' '.join(tokens)

# Prepare dataset
def prepare_dataset():
    texts = list(negative_words) + list(positive_words)
    labels = [sentiment_dict[word] for word in texts]

    augmented_texts = []
    augmented_labels = []

    for sentiment in [-1, 1]:
        words = [w for w in texts if sentiment_dict[w] == sentiment]
        for i in range(len(words)):
            for j in range(i + 1, min(i + 5, len(words))): # Create sentence pairs
                augmented_texts.append(f"{words[i]} {words[j]}")
                augmented_labels.append(sentiment)

    all_texts = texts + augmented_texts
    all_labels = labels + augmented_labels

    return all_texts, all_labels

# Create and train model
texts, labels = prepare_dataset()
processed_texts = [preprocess_text(text) for text in texts]

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
)

X = vectorizer.fit_transform(processed_texts)
y = np.array(labels)

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    class_weight='balanced'
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=101)
model.fit(X_train, y_train)

```

Figure A.32 Thai Sentiment Analysis Machine Learning

```

# Function to predict sentiment
def predict_sentiment(text):
    sentences = re.split(r'(?<=[\u0E47\u0E48\u0E49\u0E4A\u0E4B\u0E4C\u0E4D\u0E4E.!])\s+', text)
    if not sentences:
        return 0, []

    sentiments = []
    word_frequencies = Counter()

    for sentence in sentences:
        processed_sentence = preprocess_text(sentence)
        X_new = vectorizer.transform([processed_sentence])
        probability_positive = model.predict_proba(X_new)[0][1]
        scaled_prediction = int(round(probability_positive * 100))
        sentiments.append(scaled_prediction)

        # Update word frequencies
        tokens = word_tokenize(processed_sentence, engine="newmm")
        word_frequencies.update(tokens)

    # Calculate average sentiment
    avg_sentiment = sum(sentiments) / len(sentiments) if sentiments else 0

    # Get most repeated words
    most_repeated_words = word_frequencies.most_common(5)

    return avg_sentiment, most_repeated_words

```

Figure A.33 Thai Sentiment Analysis Machine Learning

UniLearning

Front-end

- User's Online Lesson

```

const init_player_events = (player) => {
    let currentTime = null;
    let previousTime = 0;
    let position = 0;
    let latestUpdate = null
    let cooldown = 10 * 1000; //10s

    player.on("play", () => {
        // props.onPlay(player.currentTime());
        intervalRef.current = player.setInterval(function() {
            let getDate = new Date().getTime()
            if(!(getDate - latestUpdate < cooldown)){
                console.log("UPDATE BY play");
                let playedDuration = getTimeRange(player.played())
                props.onPlay(playedDuration)
                latestUpdate = getDate
            }else{
                console.log("Update is CoolDown (play)");
            }
        }, cooldown);
    });

    player.on("pause", () => {
        player.clearInterval(intervalRef.current)
        let getDate = new Date().getTime()
        if(!(getDate - latestUpdate < cooldown)){
            console.log("UPDATE BY pause");
            console.log(player.played());

            let playedDuration = getTimeRange(player.played())
            props.onPause(playedDuration)
            latestUpdate = getDate
        }else{
            console.log("Update is CoolDown (pause)");
        }
        // props.onPause(player.currentTime());
    });

    player.on("ended", () => {
        console.log("Video Ended");
        player.clearInterval(intervalRef.current)
        let playedDuration = getTimeRange(player.played())
        props.onEnd(playedDuration)
        // props.onEnd();
    });
};

```

Figure A.34 Fetching The Video Progress

Figure A.34 shows when the video is played, a timed interval is initiated to call a progress tracking function every 10 seconds. If the video is paused, this interval is cleared to stop further updates. A progress update is still sent on pause, but only if enough time has elapsed since the last update, avoiding excessive reporting. When the video ends, a final progress report is sent. These updates are communicated using callback functions provided as props—props.onPlay, props.onPause, and props.onEnd—allowing the parent component to respond appropriately to each event.

- **Admin’s Rush Quiz**

```
<StylePaperList>
  <List component="div" disablePadding>
    {questions.map( (_, index) => (
      <ListItemButton
        key={index}
        selected={selectedQuizIndex === index}
        onClick={() => setSelectedQuizIndex(index)}
      >
        {index + 1}
        {questions.length > 1 && (
          <IconButton onClick={(e) => {removeQuestion(index); e.stopPropagation();}}>
            <DeleteIcon />
          </IconButton>
        )}
      </ListItemButton>
    )}
  </List>
</StylePaperList>
```

Figure A.35 Question Navigation Panel

```
<InputForm
  idQuizQA={selectedQuizIndex}
  data={questions[selectedQuizIndex]}
  onChangeDirty={setIsDirty}
  onSubmit={(_, formData) => {
    const updatedQuestion = {};
    for (let [key, value] of formData.entries()) {
      updatedQuestion[key] = JSON.parse(value);
    }
    updateQuestionData(updatedQuestion);
  }}
/>
```

Figure A.36 Question Editing System

```
const handleSaveAllQuestions = async () => {
  const formDataArray = new FormData();

  questions.forEach((question, index) => {
    Object.keys(question).forEach(key => {
      if (key !== 'questionImg' || question[key] instanceof File) {
        formDataArray.append(`questions[${index}][${key}]`,
          typeof question[key] === 'object'
            ? JSON.stringify(question[key])
            : question[key]
        );
      }
    });
  });

  if (question.questionImg instanceof File) {
    formDataArray.append(`questionImg_${index}`, question.questionImg);
  }
});

try {
  const result = await dispatch(postRushQuizQuestion(state.idQuizRush, formDataArray));
  if (result?.status === 200) {
    // Handle successful save
  }
} catch (error) {
  // Handle error
}
};
```

Figure A.37 Data Saving Function

```
const handlePreviewClick = () => setShowPreview(true);

{showPreview && (
  <QuizPreview
    questions={questions}
    onClose={() => setShowPreview(false)}
  />
)}
```

Figure A.38 Preview Function

Front-end

```
exports.getFinishVideoCourse = (req, res) => {
  try {
    doGetFinishVideoCourse(req, res);
  } catch (error) {
    res.status(500).send({ message: error.message });
  }
};
```

Figure A.39 getFinishVideoCourse Function

Figure A.39, the `getFinishVideoCourse` function uses Express route handling to manage HTTP requests. It invokes `doGetFinishVideoCourse` for the core logic, passing the request and response objects. The function is wrapped in a try-catch block to handle any unexpected runtime errors. If an exception is caught, it returns a response with HTTP status code 500, which indicates an internal server error, and sends back the error message to inform the client of what went wrong. This ensures robust error handling and better user experience in case of backend failures.

```
exports.updateDuration = (req, res) => {
  try {
    doUpdateDuration(req, res);
  } catch (error) {
    res.status(500).send({ message: error.message });
  }
};
```

Figure A.40 updateDuration Function

Figure A.40, the `updateDuration` function uses a try-catch block to manage errors effectively. If an exception occurs during the execution of `doUpdateDuration`, the error is caught, and the server responds with an HTTP status code 500, indicating an internal server error. The error message is included in the response to aid in debugging or informing the client. This structure ensures that the application can handle backend failures gracefully and provide informative feedback when something goes wrong during the update process.

```

async function doGetFinishVideoCourse(req, res) {
  console.log("-----GetFinishVideoCourse-----");
  const { idEmployees } = req.params;
  console.log(req.params.id);

  try {
    const documents = await pool.query(
      'SELECT idOnlineDocuments, IFNULL(lengthVideo, 0) as lengthVideo
      FROM UniHR.Training_OnlineDocuments
      LEFT JOIN UniHR.Training_OnlineChapter ON Training_OnlineChapter.idOnlineChapter =
      Training_OnlineDocuments.idOnlineChapter
      LEFT JOIN UniHR.Training_OnlineSections ON Training_OnlineSections.idOnlineSection =
      Training_OnlineChapter.idOnlineSection
      WHERE Training_OnlineSections.idcourseOnline = ? AND Training_OnlineDocuments.documentsType = "video" ',
      [parseInt(req.params.id)]
    );

    console.log(documents);

    if (!documents || documents.length === 0) {
      return res.status(200).send({ isFinished: false });
    }

    const totalLengthVideo = documents.reduce((total, item) => total + item.lengthVideo, 0);

    const watchDurations = await pool.query(
      'SELECT idOnlineDocuments, start, end FROM Training_OnlineVideoDuration WHERE idEmployee = ? AND idOnlineDocuments
      IN (${documents.map(doc => doc.idOnlineDocuments).join(',')})',
      [idEmployees]
    );

    if (!watchDurations || watchDurations.length === 0) {
      return res.status(200).send({ isFinished: false });
    }

    const totalWatchedDuration = watchDurations.reduce((total, duration) => {
      const watchedTime = Math.max(new Date(duration.end) - new Date(duration.start), 0);
      return total + watchedTime;
    }, 0);

    const watchedPercentage = totalWatchedDuration === 0 ? 0 : (totalWatchedDuration / totalLengthVideo) * 100;
    const isFinished = watchedPercentage >= 80;

    res.status(200).send({ isFinished: isFinished });
  } catch (err) {
    console.log(err);
    res.status(500).send({ message: err.sqlMessage || err.message });
  }
}

```

Figure A.41 `doGetFinishVideoCourse` Function

Figure A.41, the function works by joining several database tables to collect relevant video records using SQL queries. It sums the `lengthVideo` fields to compute the total course duration. To assess user progress, it queries the user's watch records, calculates the cumulative watch time, and derives the percentage watched. A threshold comparison is then applied: if the user has viewed 80% or more, they are marked as having completed the course. The function also uses standard error handling; if any exception arises during these steps, it responds with an HTTP 500 status code along with the error message. This structure ensures accurate progress tracking and robust handling of incomplete or missing data.

The function performs several key operations involving database queries and calculations. It joins multiple tables to filter and fetch only video-type course documents. Using SQL aggregation, it sums the `lengthVideo` fields to get the total duration. The user's viewing activity is then fetched from another table, and the watched time is aggregated. The function calculates the watched percentage using the formula: $(\text{total watched time} / \text{total video length}) * 100$. It uses a condition to check if this percentage is at least 80%. Additionally, the function includes error handling to catch and return a 500 status code along with an error message if any issues occur during execution, ensuring reliability and proper client feedback.

- Rush quiz

```
const { questions } = req.body; // Array of question objects
const files = req.files;        // Uploaded files (images)

// Check if quiz ID exists
if (!req.params.idQuizRush) {
  return res.status(500).send({ message: "NOT FOUND ID_QUIZ_RUSH" });
}
```

Figure A.42 Request Validation & Initial Setup

```
pool.getConnection(function (err, connection) {
  if (err) throw err;

  connection.beginTransaction(async function (err) {
    if (err) {
      connection.rollback(() => connection.release());
      throw err;
    }

    try {
      // Main logic here...
      connection.commit(() => { ... });
    } catch (error) {
      connection.rollback();
      console.error("Rollback due to error:", error);
      res.status(500).send({ success: false, message: error.message });
    }
  });
});
```

Figure A.43 Database Connection & Transaction Management

```
const keys = Object.keys(formData);
const values = Object.values(formData);

const result = await connection.promise().query(
  `INSERT INTO UniHR.Training_Quiz_Rush_QA (${keys.join(',')}) VALUE (?)`,
  [values]
);
```

Figure A.44 Inserting Into Database

APPENDIX B
USER RATING ON MODULE

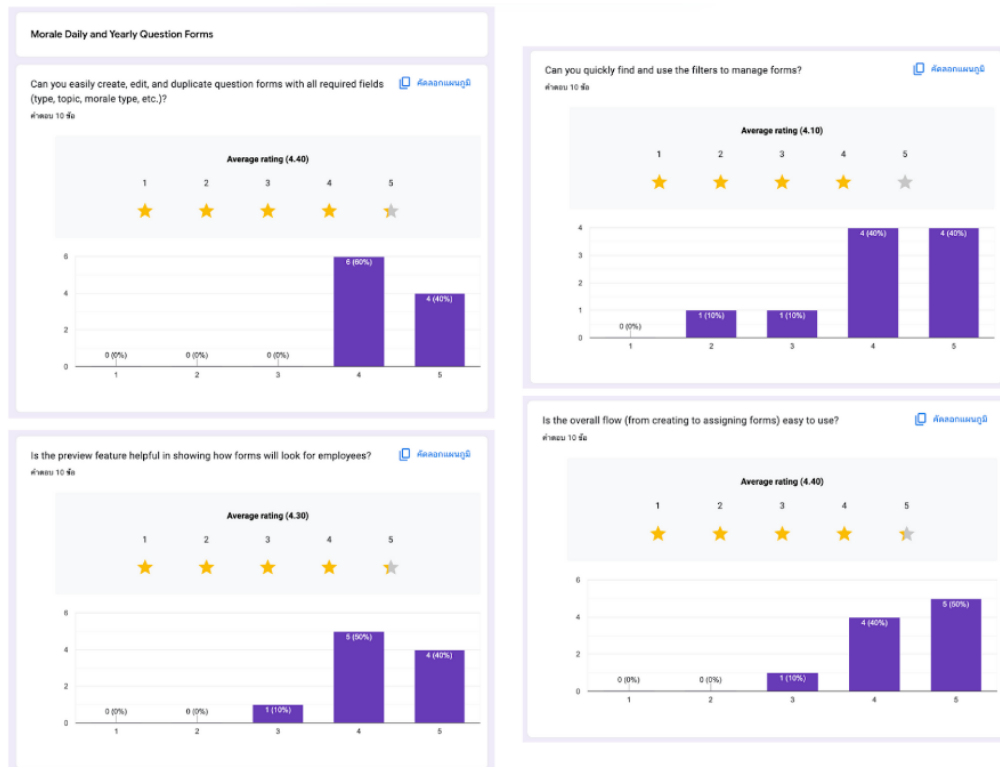


Figure B.1 User Ratings on the Morale Daily and Yearly Question Forms

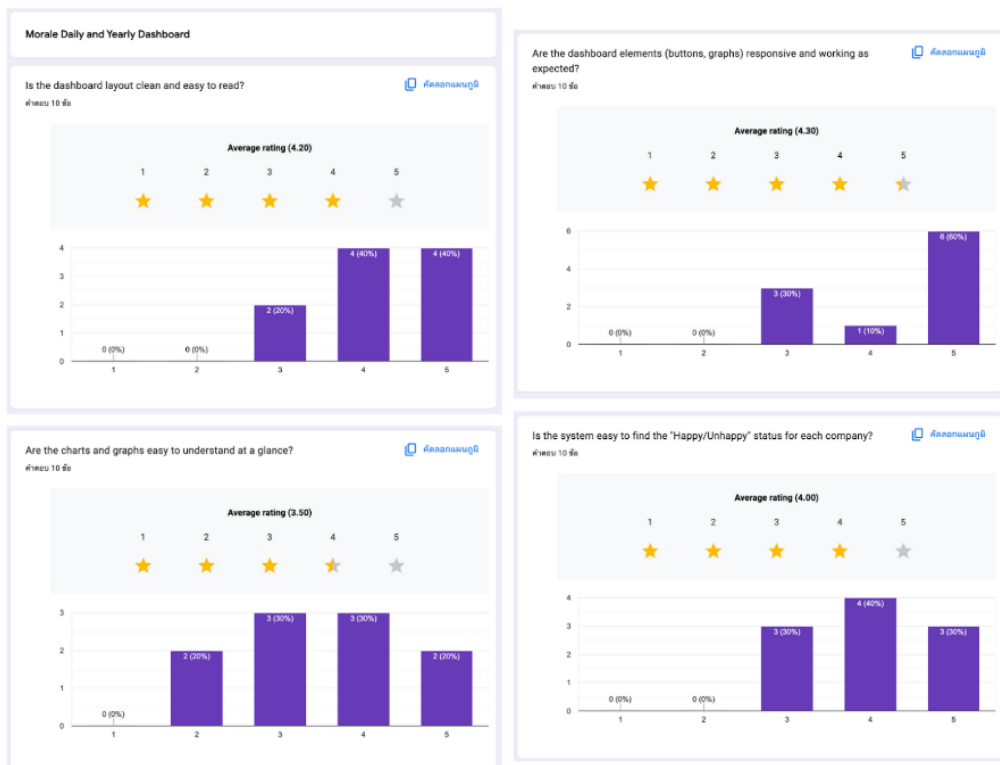


Figure B.2 User Ratings on the Morale Daily and Yearly Dashboard

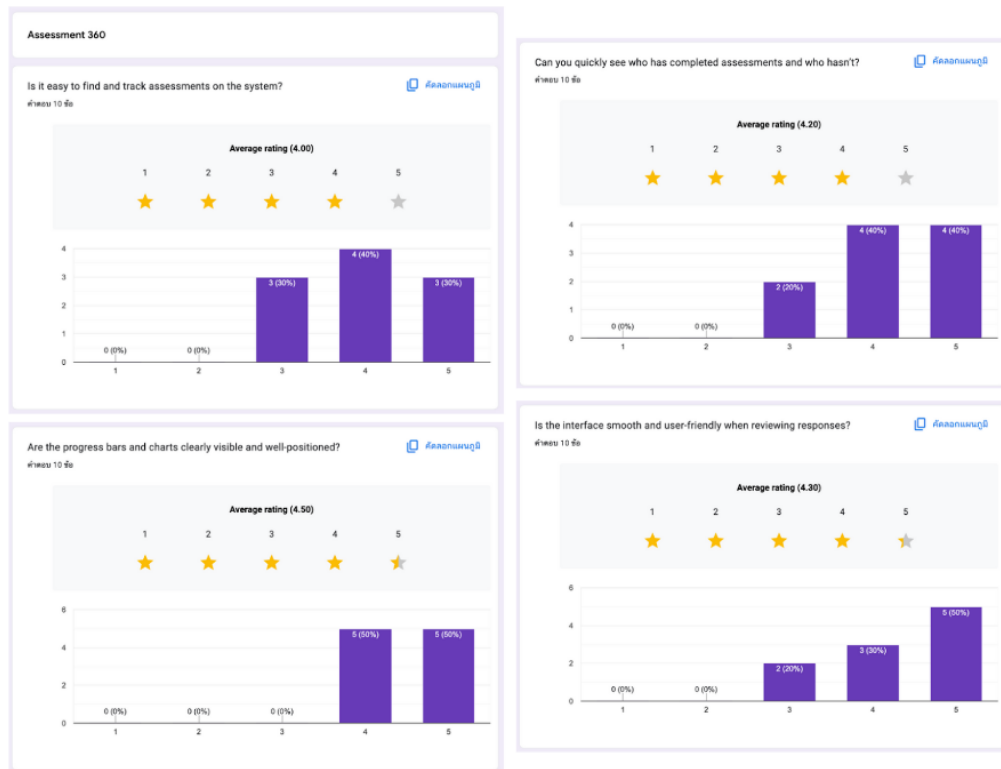


Figure B.3 User Ratings on the 360 Assessment

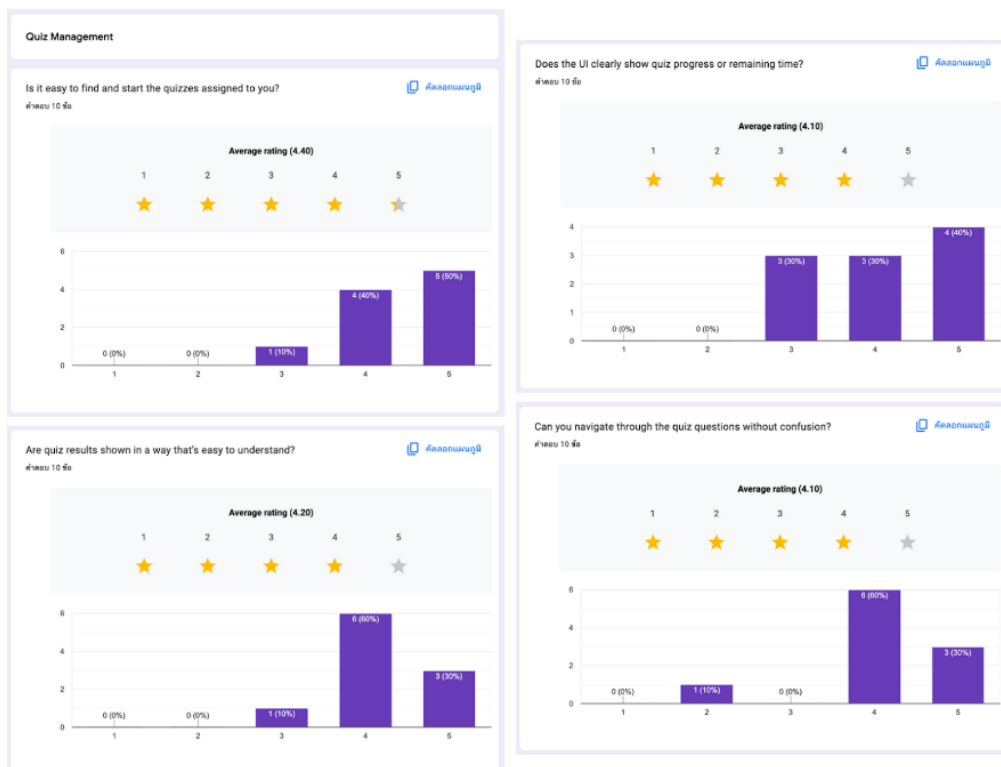


Figure B.4 User Ratings on the Quiz Management

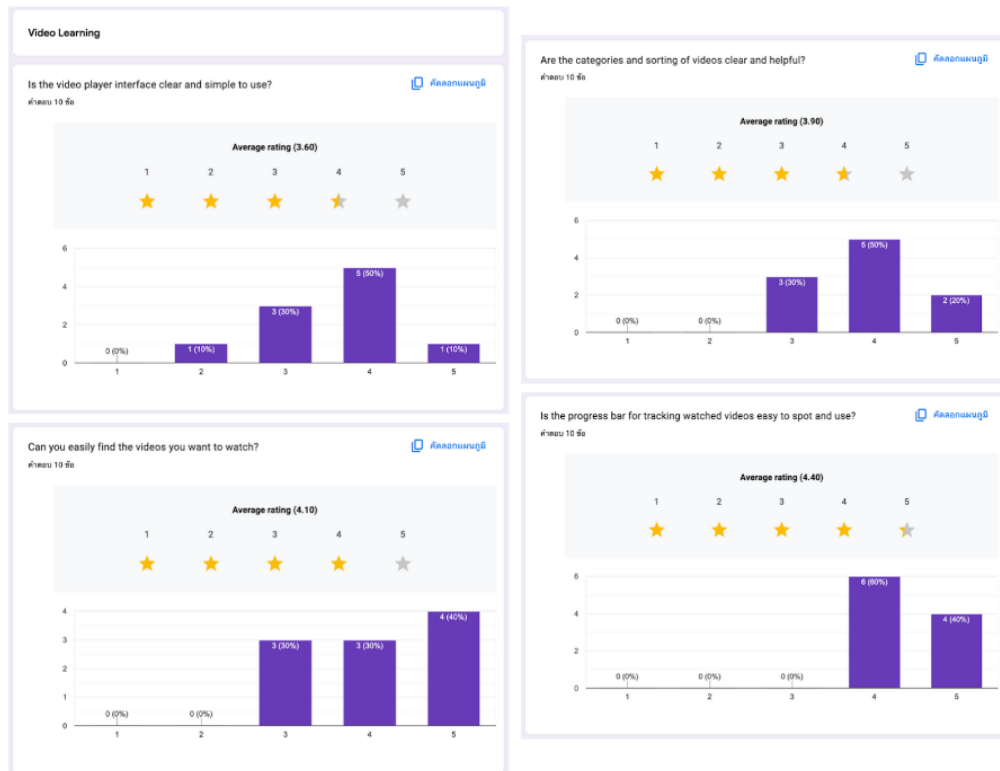


Figure B.5 User Ratings on the Video Learning

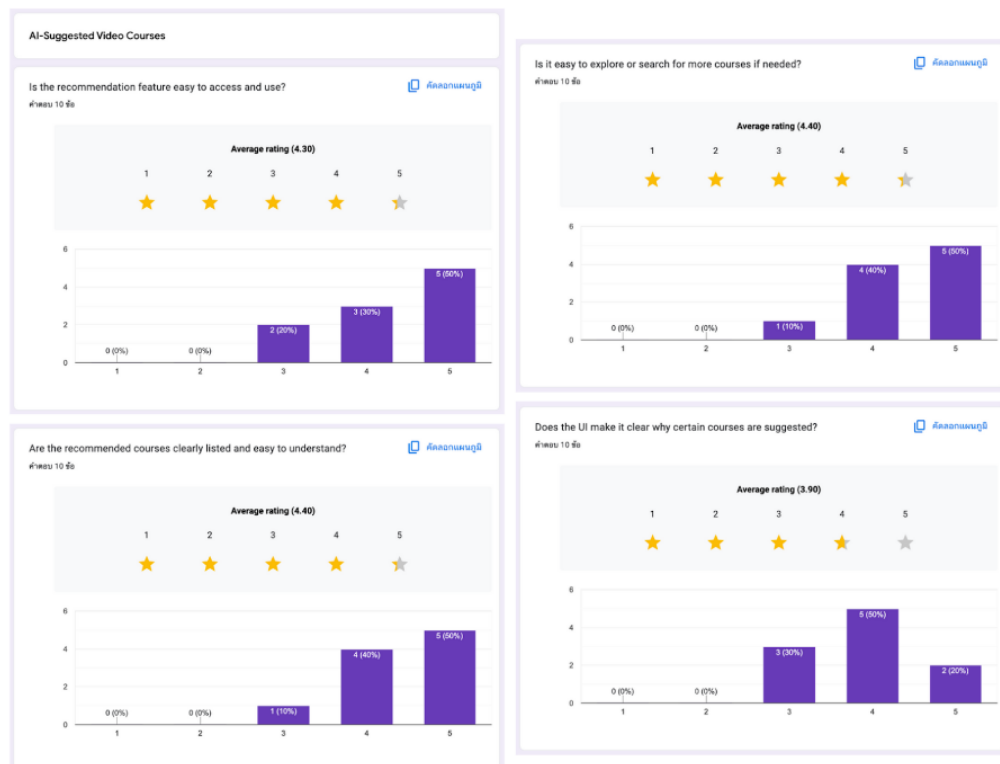


Figure B.6 User Ratings on the AI-Suggested Video Courses

Career Aspiration

The recommended careers are relevant to my interests.

Анкету 10 68

Average rating (4.60)

Rating	Count	Percentage
1	0	0%
2	0	0%
3	0	0%
4	4	40%
5	6	60%

The recommendations helped me understand what careers I could pursue.

Анкету 10 68

Average rating (3.70)

Rating	Count	Percentage
1	0	0%
2	0	0%
3	5	50%
4	3	30%
5	2	20%

The recommended careers align with my current skills or strengths.

Анкету 10 68

Average rating (4.10)

Rating	Count	Percentage
1	0	0%
2	0	0%
3	0	0%
4	9	90%
5	1	10%

I discovered new career options that I hadn't considered before.

Анкету 10 68

Average rating (3.50)

Rating	Count	Percentage
1	0	0%
2	0	0%
3	5	50%
4	5	50%
5	0	0%

Figure B.8 User Ratings on the Career Aspiration